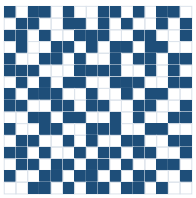
Classic McEliece

El cifrado de clave pública basado en códigos de Goppa



→ $C = \mathbf{He}$
cifrado compacto
(síndrome)

De McEliece (1978) al estándar ISO/IEC 18033-2 (2026): teoría, laboratorio numérico y estado del arte

Documento didáctico con ejemplos verificables, figuras y ejercicios.

Dedicado a la memoria de Robert J. McEliece (1942–2019).

Prefacio: cómo usar estos apuntes

Estas notas explican, con el mayor rigor que hemos podido alcanzar, **cómo funciona Classic McEliece**: el cifrado de clave pública basado en *códigos binarios de Goppa*, propuesto por Robert J. McEliece en 1978 y estandarizado por ISO/IEC en junio de 2026 (enmienda a la norma ISO/IEC 18033-2). No es un manual histórico: es un recorrido completo que empieza en la teoría de códigos, construye el esquema pieza a pieza, lo ejecuta sobre un ejemplo miniatura *totalmente verificable*, y termina analizando el estado del arte criptoanalítico **a fecha de octubre de 2026** — que incluye una sacudida reciente que ningún texto de hace un año recoge.

Para quién es este documento

- Estudiantes de matemáticas, informática o ingeniería con interés en criptografía.
- Profesionales que necesitan *entender* (de verdad) qué se despliega en sus sistemas, más allá de la casilla «post-cuántico».
- Docentes que busquen material listo para clase: figuras, tablas, ejemplos numéricos y problemas con solución.

Se asume una base de álgebra lineal (matrices, rango, espacios vectoriales sobre $\mathbb{F}_2$) y cierta soltura con notación matemática. Todo lo demás — cuerpos finitos, códigos, decodificación — se desarrolla desde cero. No se requieren conocimientos previos de criptografía.

Tres recorridos posibles

- **Exprés (2–3 horas):** capítulos 1, 4, 6, 8, 9, 10 y 16, más el laboratorio del capítulo 11.
- **Estándar (un curso corto):** el documento en orden, saltando los apartados marcados como «avanzado» y las letras pequeñas.
- **Profundo (investigación):** todo, incluyendo la parte IV, el debate metodológico de costes del capítulo 14 y las referencias.

Convenciones y avisos de rigor

- **Las fuentes primarias mandan.** Todo dato cuantitativo se contrasta con: el artículo original de McEliece (1978) [56]; la especificación oficial de Classic McEliece del 23 de octubre de 2022 [10]; su documento de racional de diseño [28] y su guía para evaluadores de seguridad [29]; el informe NIST IR 8545 (marzo de 2025) [2]; y la literatura de ataques de 2026 citada en el capítulo 16.
- **Los números llevan fecha.** En 2026 la criptoanalítica de McEliece se movió más rápido que en las dos décadas anteriores. Cada estimación de seguridad de este documento indica su fuente, su modelo de coste y su fecha. Donde hay debate abierto (y lo hay), se muestra el debate, no una conclusión empaquetada.

- **Distinguimos siempre** entre resultados *demostrados*, *heurísticos* y *estimaciones concretas* — y entre ataques *ejecutables* y *teóricos*. Los recuadros ⚖ «Rigor» señalan precisamente esos puntos.
- **Convención de vectores.** En las partes I y II los vectores son fila (como en el artículo de 1978 y en casi todos los libros de códigos). La especificación de Classic McEliece usa vectores *columna*; el capítulo 8 explica la traducción y a partir de ahí seguimos fielmente la especificación. Este cambio está señalizado y no es un descuido.
- **El laboratorio miniatura no es seguro.** Los códigos de los capítulos 5 y 11 (longitud 16, "clave" de 8 bytes) existen para enseñar los mecanismos con números reales; cualquier parecido con la seguridad real es casualidad. No los use fuera de estas notas. El código Python que reproduce *todas* las tablas y cifras aparece en el apéndice A.
- **Bit vs byte.** Los tamaños de claves y cifrados se dan en bytes (redondeando hacia arriba cuando hay bits de relleno); los tamaños criptográficos y de seguridad, en bits de operación o bits de seguridad, según se indique. Usamos el espacio fino como separador de millares (p. ej. 261 120).

Mapa del documento

La **parte I** construye las herramientas (códigos lineales, cuerpos finitos, códigos de Goppa) y termina con un laboratorio sobre un código de Goppa real de juguete. La **parte II** presenta el esquema de 1978, su versión dual de Niederreiter y la transformación a KEM que se estandariza. La **parte III** recorre la especificación de Classic McEliece al detalle (generación de claves, encapsulación, desencapsulación, parámetros y rendimiento) con un segundo laboratorio *de extremo a extremo*. La **parte IV** analiza la seguridad: reducciones, decodificación por conjuntos de información, ataques estructurales y el terremoto criptoanalítico de 2026. La **parte V** cierra con preguntas frecuentes, ejercicios, soluciones, glosario y referencias.

Índice general

Prefacio: cómo usar estos apuntes . . . **1**
Para quién es este documento . . . 1
Tres recorridos posibles . . . 1
Convenciones y avisos de rigor . . . 1

I Fundamentos: de los bits a los códigos de Goppa **13**

1 La amenaza cuántica y el giro post-cuántico . . . **14**
1.1 La amenaza: Shor, Grover y el adversario que espera . . . 14
1.2 Qué se está estandarizando: el proceso del NIST . . . 15
1.3 Dónde encaja Classic McEliece . . . 16
1.4 Cómo leer el resto del documento . . . 17
1.5 Ejercicios de calentamiento . . . 18

2 Códigos lineales: el lenguaje de la corrección de errores . . . **19**
2.1 El espacio de Hamming . . . 19
2.2 Decodificación: síndromes . . . 21
2.3 El problema de la decodificación por síndrome . . . 22
2.4 La idea de McEliece: esconder un decodificador . . . 23
2.5 Ejercicios . . . 24

3 Cuerpos finitos: el terreno donde vive Goppa . . . **25**
3.1 ¿Por qué cuerpos finitos? . . . 25
3.2 Construcción de $\mathbb{F}_{2^m}$ . . . 25
3.3 Frobenius, traza y conjugados . . . 28
3.4 Raíces cuadradas: cuadrar es fácil, y también su inversa . . . 29
3.5 Cómo se muestrean cuerpos finitos en la práctica . . . 29
3.6 Ejercicios . . . 30

4 Códigos de Goppa: el corazón del esquema . . . **31**
4.1 La definición de Goppa . . . 31
4.2 La matriz de paridad: evaluar en los α_i . . . 31
4.3 Parámetros: dimensión y distancia . . . 32
4.4 ¿Por qué Goppa binario y no otra cosa? . . . 34
4.5 Ejercicios . . . 34

5 Laboratorio 1: el juguete completo . . . **36**
5.1 El montaje: un código de Goppa de longitud 16 . . . 36
5.2 Las matrices: $\tilde{\mathbf{H}}$, $\hat{\mathbf{H}}$ y la forma sistemática . . . 37
5.3 El síndrome de un error de dos posiciones . . . 38
5.4 Decodificación de Patterson, paso a paso . . . 39
5.5 Vista previa: el juguete cifra (McEliece 1978 en miniatura) . . . 42
5.6 Ejercicios . . . 43

II El criptosistema de McEliece **45**

6 El McEliece de 1978 . . . **46**
6.1 La idea: una trampa amable . . . 46
6.2 El criptosistema, paso a paso . . . 47
6.3 El juguete del capítulo 5 es este esquema . . . 48
6.4 El argumento de seguridad de 1978 . . . 48
6.5 Los números del artículo, recalculados . . . 49
6.6 Dónde falla (y dónde no) . . . 50
6.7 Ejercicios . . . 51

7 La reformulación de Niederreiter . . . **52**
7.1 La idea dual: mensajes de peso fijo . . . 52
7.2 El criptosistema (1986), en nuestras convenciones . . . 53
7.3 Equivalencia con el esquema de 1978 . . . 54
7.4 Los números: cuándo gana la forma dual . . . 55
7.5 Por qué Classic McEliece es niederreiteriano . . . 55
7.6 Ejercicios . . . 55

8 Del cifrado a un KEM . . . **57**
8.1 De filas a columnas (y por qué) . . . 57
8.2 Qué es un KEM y qué le pedimos . . . 58
8.3 Por qué el esquema desnudo no basta . . . 58
8.4 La transformación tipo FO con rechazo implícito . . . 59
8.5 El KEM de Classic McEliece: tres simplificaciones . . . 61
8.6 Ejercicios . . . 62

III Classic McEliece **63**

9 La generación de claves . . . **64**
9.1 La idea: una semilla manda . . . 64
9.2 FieldOrdering: los α_i . . . 65
9.3 Irreducible: el polinomio g . . . 66
9.4 MatGen: de Γ a la clave pública . . . 67
9.5 Los tamaños reales . . . 68
9.6 Ejercicios . . . 69

10 Cifrado y descifrado . . . **70**
10.1 FixedWeight: la semilla del cifrador . . . 70
10.2 Encode: el criptograma es un síndrome . . . 71
10.3 Decode: Patterson con el soporte secreto . . . 71
10.4 Una vuelta completa . . . 72
10.5 ¿Cuánto cuesta? . . . 73
10.6 Ejercicios . . . 73

11 Laboratorio 2: el KEM completo . . . **75**
11.1 La clave del juguete (KeyGen en miniatura) . . . 75
11.2 Encap: de la semilla al criptograma . . . 75
11.3 Decap: la vuelta, paso a paso . . . 76
11.4 Un criptograma manipulado . . . 76
11.5 Del juguete al estándar (y qué no prueba el juguete) . . . 78
11.6 Ejercicios . . . 78

12 Los conjuntos y las variantes . . . **80**
12.1 Los cinco tamaños . . . 80

12.2 Los dieciséis conjuntos de ISO . . . 81
12.3 La variante `f`: forma semi-sistemática . . . 82
12.4 La variante `pc`: confirmación de texto claro . . . 83
12.5 Simply Decoded y Narrowly Decoded: el relleno . . . 84
12.6 ¿Cuánta seguridad? . . . 85
12.7 Elegir un conjunto . . . 86
12.8 Ejercicios . . . 87

IV Seguridad **88**

13 ¿Qué significa romper McEliece? . . . **89**
13.1 Tres objetivos (y no todos rompen) . . . 89
13.2 El juego OW-CPA y el problema SD . . . 90
13.3 La cadena de reducciones y la conjetura central . . . 91
13.4 Las familias de ataque de esta parte . . . 91
13.5 Cómo se leen las cifras de seguridad . . . 92
13.6 Fuera del modelo: la implementación . . . 93
13.7 Ejercicios . . . 93

14 Decodificación por conjuntos de información . . . **95**
14.1 La idea: un conjunto de información . . . 95
14.2 Sesenta años de mejoras . . . 96
14.3 Los números: memoria libre y memoria con coste . . . 97
14.4 Lo que las mejoras no cambian . . . 99
14.5 El debate metodológico: qué cuenta un 2^b . . . 99
14.6 Criptoanálisis cuántico . . . 100
14.7 Ejercicios . . . 101

15 Ataques estructurales: recuperar la clave privada . . . **102**
15.1 Dos cajas fuertes: la anatomía de la clave privada . . . 102
15.2 La división del soporte . . . 102
15.3 Distinguidores: saben que es Goppa, ¿y qué? . . . 104
15.4 El balance de 2022 . . . 105
15.5 Ejercicios . . . 106

16 El terremoto: 2025–2026 . . . **107**
16.1 La primera grieta (2024–2025) . . . 107
16.2 La primavera de 2026: de CFS a los retos TII . . . 108
16.3 El método *holdout* (agosto de 2026) . . . 109
16.4 La cascada de septiembre . . . 110
16.5 El debate de los costes . . . 111
16.6 La respuesta institucional . . . 112
16.7 Lectura final . . . 113
16.8 Ejercicios . . . 114

V Cierre **115**

17 Preguntas frecuentes . . . **116**
17.1 El esquema, en preguntas cortas . . . 116
17.2 Ecosistema y estándares . . . 117
17.3 Seguridad: qué se sabe y qué no . . . 117
17.4 Práctica: elegir, desplegar, verificar . . . 118

18 Repaso y soluciones **119**
18.1 Repaso: doce preguntas 119
18.2 Soluciones del repaso 120
18.3 Soluciones de los ejercicios de los capítulos 1–16 121

19 Glosario **132**

A El programa del juguete: `toy.py` **139**
A.1 Qué es y cómo se ejecuta 139
A.2 El listado completo 139
A.3 La salida 146
A.4 Simplificaciones del juguete 147

B Cronología: de Goppa (1970) a octubre de 2026 . . . . . **148**

C GF(16) de bolsillo **151**
C.1 Potencias y logaritmos 151
C.2 Inversos, cuadrados, raíces y traza 152
C.3 La tabla de multiplicar 152
C.4 Cómo usar estas tablas 152

Bibliografía **153**

Índice de figuras

1.1 De 1978 a octubre de 2026. En sepia, la prehistoria del cifrado con códigos; en azul, los algoritmos cuánticos que motivaron el giro; en verde, el proceso del NIST; en violeta, la estandarización ISO; en rojo, los acontecimientos de 2026 que se analizan en el capítulo 16. 15

1.2 Tamaño de clave pública y de cifrado en tres familias estandarizadas (octubre de 2026). La escala es logarítmica: cada división vertical es un factor 10. Classic McEliece intercambia claves de centenares de KiB por cifrados de un centenar de bytes; los esquemas de retículos y HQC son más equilibrados. Datos: FIPS 203, especificación de HQC y especificación de Classic McEliece [34, 1, 10]. 17

2.1 Las bolas de radio $t = \lfloor (d-1)/2 \rfloor$ alrededor de cada codeword no se solapan: cualquier palabra recibida suficientemente cerca de un codeword se corrige de forma *única*. Fuera de esos radios, la «mejor» corrección puede ser ambigua. . . . 21

2.2 La cadena completa: codificar, atravesar el canal, calcular el síndrome y decodificar. El público ve y y, en su caso, la matriz $\mathbf{H}$; la dificultad no está en el álgebra lineal del síndrome, sino en encontrar un error de peso pequeño que lo explique. 23

3.1 El grupo multiplicativo de $\mathbb{F}_{2^4}$ es cíclico de orden 15: las potencias de z recorren todos los elementos no nulos, y $z^{15} = 1$. En naranja, la órbita de z bajo el automorfismo de Frobenius $x \mapsto x^2$ (véase la sección 3.3). 27

4.1 El código de Goppa como «máquina de verificación»: c pertenece al código si su función racional asociada es divisible por g. La proposición 4.2 convierte esta condición en t ecuaciones lineales sobre $\mathbb{F}_q$ (mt binarias). 32

5.1 La matriz de paridad binaria $\hat{\mathbf{H}}$ del juguete, en dos mitades (la fila (i, ℓ) contiene el bit ℓ —el coeficiente de z^ℓ— de la entrada $\alpha_j^i / g(\alpha_j)$), y su forma sistemática $(\mathrm{I}_8 \,|\, \mathbf{T})$ obtenida por operaciones de fila, que no cambian el núcleo. La mitad derecha $\mathbf{T}$ (8×8) es la misma que aparecerá como clave pública en el laboratorio 2. 38

5.2 Histograma de pesos de los 256 codewords del juguete (65 536 palabras examinadas). El hueco de pesos 1–4 refleja $d_{\mín} = 5 = 2t + 1$: el código corrige $t = 2$ errores. . 38

5.3 La decodificación de Patterson del error $e_{10} + e_{13}$, paso a paso, con los números del juguete. Cada caja lleva su certificado exacto (múltiplo de g) en letra pequeña. 41

5.4 La matriz aleatoria invertible S (8×8) y la clave pública del juguete $\mathbf{G}' = S\,\mathbf{G}\,P$ (8×16). La línea gruesa separa las 8 primeras coordenadas de las 8 últimas. . . . 42

5.5 El cifrado en tiras: en azul, las posiciones con bit 1; en rojo, las posiciones alteradas por el error. De arriba abajo: $\mathbf{uG}'$ (codeword sin error), el error $\mathbf{z}$, el criptograma $\mathbf{x} = \mathbf{uG}' + \mathbf{z}$ y, tras deshacer P, la palabra recibida $\mathbf{x}P^{-1}$, con el error ya en las posiciones 0 y 10. 43

6.1 El circuito del esquema de 1978. Arriba, la generación de claves: la clave pública es la matriz de Goppa disfrazada $G' = SGP$. Abajo, el uso: el emisor suma un error de peso t a un codeword y el receptor deshace el disfraz y decodifica. Los recuadros naranjas contienen material secreto. 48
6.2 Las dos familias de ataques que ya identifica el artículo de 1978. La seguridad práctica moderna se discute toda dentro de este dibujo, con dos precisiones que veremos en la parte IV: los ataques estructurales también *distinguen* códigos de Goppa escondidos de códigos aleatorios, y los decodificadores también pueden llegar a recuperar la clave. 49

7.1 Las dos miradas al mismo código. En McEliece se publica G' y se envían n bits (codeword con ruido); en Niederreiter se publica H' y se envían $n-k$ bits (síndrome del mensaje-error). Classic McEliece usará la segunda columna. 53

8.1 La transformación, en una imagen. Encap sortea la semilla e, la cifra y deriva la clave de sesión del par (e, C). Decap descifra; si el criptograma es válido reconstruye la misma clave (verde); si no, devuelve una clave pseudoaleatoria derivada de s (rojo), indistinguible de una clave legítima: el *rechazo implícito*. El adversario ve (pk, C, K) y puede consultar Decap con cualquier $C' \neq C$. 60

9.1 La expansión de la semilla. De δ sale todo: s, la materia prima de α y g, y la semilla δ' de un eventual reintento. Los cuerpos son $\mathbb{F}_q = \mathbb{F}_{2^m}$ con $q = 2^m$; el campo de Goppa es $\mathbb{F}_q[y]/F(y)$ con F fijado por los parámetros (tabla 9.1). . . . 65
9.2 MatGen en tres actos. La matriz de Goppa $t \times n$ sobre $\mathbb{F}_q$ se expande bit a bit a una matriz binaria $mt \times n$; la eliminación gaussiana la lleva a la forma sistemática $(\mathrm{I}_{mt} \,|\, T)$ y publica el bloque T. La permutación secreta del capítulo 7 es aquí el ordenamiento α: las columnas de $\tilde{H}$ están etiquetadas por elementos del cuerpo, en un orden que solo conoce la clave secreta. 68

10.1 FixedWeight: de bits al azar a un soporte de tamaño t exacto. El lote de τ enteros abarata el muestreo y facilita la vectorización; el rechazo garantiza uniformidad exacta (proposición 10.1). Los valores τ son $128, 192, 256, 238, 128$ para los cinco conjuntos de la tabla 9.1. 71
10.2 El recorrido completo. En azul, lo que hace Encap con la clave pública; en naranja, lo que hace Decap con la secreta; en verde y rojo, las dos formas de terminar. La especificación unifica el hash final como $K = H(b, v, C)$: con $b = 1$ y $v = e'$ si el descifrado valida, y con $b = 0$ y $v = s$ (los n bits secretos) si no. 72

11.1 El KEM del juguete, con todos los números. A la izquierda, Encap: del error al criptograma y a la clave. A la derecha, Decap: del criptograma al error y a la misma clave (el signo = verde). Abajo, el caso manipulado: un bit distinto y una clave completamente distinta, indistinguible de una legítima. 77

12.1 Los dieciséis conjuntos de ISO, como familia. Los cuatro tamaños (filas) y las cuatro variantes (columnas). La confirmación de texto claro (`pc`, `pcf`) añade 32 bytes al criptograma; la variante `f` no cambia ningún tamaño, solo la manera de generar la clave (forma semi-sistemática). 82
12.2 La forma (μ, ν)-semi-sistemática, en esquema reducido ($\mu = 4$, $\nu = 6$; en el estándar, $(32, 64)$). Izquierda: la matriz escalonada; los primeros $mt - \mu$ pivotes ya están en la diagonal y los últimos μ viven dentro de una ventana de ν columnas. Basta con $\leq \mu$ intercambios de columnas para alcanzar la forma sistemática de la derecha. 83

12.3 El relleno de `mceliece6960119`: el criptograma son 1547 bits dentro de 194 bytes, y a los 5 bits finales la especificación les da dos lecturas. Una implementación u otra debe elegirse de forma consistente entre las dos partes del protocolo; mezclarlas convierte en divergencia lo que es una maleabilidad benigna. 85

13.1 Los tres objetivos de un atacante, con la dirección demostrada (clave ⇒ mensaje) y los interrogantes abiertos. La seguridad del KEM descansa, en último término, en el objetivo 1: los teoremas del capítulo 8 reducen su IND-CCA2 a la OW-CPA del esquema desnudo, que es exactamente este problema. 90
13.2 La cadena de reducciones de Classic McEliece. Arriba, la conjetura dura; en medio, la propiedad que se mide con el criptoanálisis (capítulos 14 y 15); abajo, la garantía del KEM, que además necesita el hash y la PRF. La implementación real queda fuera del modelo, y se vigila aparte. 91

14.1 Prange (1962): elegir k columnas, resolver el sistema y repetir hasta que el azar deje el error fuera del conjunto de información. Los números son los de `mceliece348864` ($n = 3488$, $k = 2720$, $t = 64$) y reproducen el $2^{173{,}4}$ de la guía de seguridad [29]. . 96
14.2 La idea de Stern y de toda la familia moderna: en vez de exigir un conjunto de información perfecto, enumerar errores parciales en dos listas y cruzarlas. Las mejoras de 1991–2018 refinan cada caja: cómo se construyen las listas, cómo se cruzan y qué se reutiliza entre permutaciones. 97
14.3 La escalera de los ataques contra `mceliece348864`: de Prange (1962) al mejor ataque de 2022, con memoria libre (arriba) y pagando la memoria con raíz cuadrada (abajo). El primer salto —el cruce de listas de Stern— es el grande; el resto es afinar. BJMM aparece en ambos carriles para mostrar el aplanamiento: 141,9 con memoria libre se convierte en 162,2 pagándola. 98

15.1 Las rutas conocidas hacia la clave privada, a fecha de 2022. Arriba, la ruta estructural: adivinar g y dividir el soporte con el algoritmo de Sendrier [67]; con $n = q$ basta adivinar g, y con $n < q$ la puerta exige además las posiciones $(\alpha_0, \ldots, \alpha_{n-k})$ (ataque de Kirshanova–May, 2022, con dato parcial [48]). Abajo, la ruta genérica de decodificación, que no usa estructura alguna. Los números son los de las fuentes: 2^{762} y $2^{9116{,}8}$ para `mceliece348864`, $2^{140{,}8}$ de la tabla 14.1 [29]. 103
15.2 El balance de 2022 para `mceliece348864`, en escala de exponentes (bits). Decodificar cuesta $2^{140{,}8}$; recuperar la clave por la vía de g cuesta 2^{762} (y esta vía requiere haber resuelto antes el conjunto, o $n = q$); recuperarla por la vía del dato parcial cuesta $2^{9116{,}8}$. Datos: tablas 15.1 y 14.1. 106

16.1 El test *holdout* de GIJS [38]: retirar una columna y buscar polinomios de bajo grado que se anulen con multiplicidad alta en todas las demás. Sobre una clave real el polinomio se anula también en la retirada (estructura de curva); sobre una matriz aleatoria, no. La extensión a recuperación de clave usa la geometría local de la curva en la columna retirada [72, 73]. 110

Índice de cuadros

1.1 Efecto de los algoritmos cuánticos conocidos sobre las familias criptográficas clásicas. «Seguridad efectiva» es una estimación de libro mayor; los detalles de los modelos de coste importan (véase el capítulo 14). 15
1.2 Estado de la estandarización post-cuántica en octubre de 2026 (KEMs y firmas principales). Fuentes: FIPS 203–205, NIST IR 8545 y la página ISO del equipo Classic McEliece [24]. 16

2.1 Diccionario de decodificación del Hamming $[7, 4, 3]$: cada error de peso uno tiene por síndrome la columna correspondiente de $\mathbf{H}$. Los siete síndromes agotan los vectores no nulos de $\mathbb{F}_2^3$: es un código *perfecto*. 22
2.2 Notación básica de códigos lineales. 24

3.1 Potencias de z en $\mathbb{F}_{2^4}$, con $z^4 = z + 1$. Cada elemento se escribe como polinomio, como vector de bits $(b_3b_2b_1b_0)$ (el bit i-ésimo es el coeficiente de z^i, como manda la especificación [10]) y como dígito hexadecimal. 27

5.1 Las 16 columnas del código de Goppa del juguete: valores $g(\alpha_j)$ y los dos síndromes sobre $\mathbb{F}_{16}$, $1/g(\alpha_j)$ y $\alpha_j/g(\alpha_j)$. Todas las cifras provienen de `toy.py`. 37
5.2 Evaluación del localizador $\sigma(x) = 5 + ex + 2x^2$ en los 16 puntos de L. Sus raíces (en rojo) señalan las posiciones del error: 10 y 13. 41

6.1 Reparto de material y tamaños en el McEliece de 1978. La clave pública domina; reducirla será el primer problema a resolver (capítulos 7 y 9). 47
6.2 Estimaciones del artículo de 1978 para $n = 1024$, $t = 50$, $k = 524$, y recálculo con aritmética exacta (caja siguiente). Coinciden en lo esencial y las tres favorecen al defensor. 49

7.1 Los cinco tamaños de Classic McEliece (sin las variantes `f` y `pc`; detalles en el capítulo 12) frente al juguete. La clave pública guarda la matriz T, de $mt \times k$ bits, en $mt\lceil k/8 \rceil$ bytes; el criptograma es un síndrome de $\lceil mt/8 \rceil$ bytes (en `mceliece6960119`, $mt = 1547$ no es múltiplo de 8 y ambas cosas se rellenan). A modo de comparación: una matriz generadora completa de `mceliece348864` ocuparía $k\,n/8 = 1\,185\,920$ bytes, más de cuatro veces lo que ocupa T. 55

8.1 Diccionario entre las dos convenciones. Transponer no cambia ningún resultado: es la misma álgebra con las transposiciones explícitas. La matriz de un código de Goppa (las t filas de $\alpha_i^j/g(\alpha_i)$, $j = 0, \ldots, t-1$) funciona igual en ambos mundos. 57
8.2 La transformación genérica y el KEM real, lado a lado. La distribución de e y la corrección del esquema son idénticas; lo que cambia es cuánto material hay que guardar y recalcular. 62

9.1 Los cinco conjuntos sin sufijo (véanse las variantes `f`, `pc` y `pcf` en el capítulo 12). f define el cuerpo $\mathbb{F}_q = \mathbb{F}_2[z]/f(z)$; F define $\mathbb{F}_q[y]/F(y)$, el cuerpo donde vive β para Irreducible. 68
9.2 Tamaños de las claves. La pública ocupa $mt\lceil k/8\rceil$ bytes: fila a fila, la matriz T con $\lceil k/8\rceil$ bytes por fila. La secreta suma la semilla δ (32), la lista c (8 en ambos tipos de conjunto: $\lceil \nu/8\rceil$ o la codificación fija de $2^{32}-1$ para $(\mu,\nu)=(0,0)$), el polinomio g ($t\lceil m/8\rceil$), el ordenamiento α (la red de Benes, $\lceil(2m-1)2^{m-4}\rceil$ bytes: $2m-1$ etapas de 2^{m-1} conmutadores de un bit) y la clave de rechazo s ($\lceil n/8\rceil$). 69

11.1 El juguete y el primer conjunto real. Los elementos criptográficos «gruesos» — semilla y clave de sesión— ya tienen el tamaño definitivo; lo que crece es la matemática: código, matriz pública y criptograma. 78

12.1 Los cinco conjuntos base, con $ek = mt\lceil k/8\rceil$, $ct = \lceil mt/8\rceil$ y sk como en la tabla 9.2. La columna «¿ISO?» anticipa la sección siguiente: el estándar ISO incluye los cuatro tamaños mayores, cada uno con cuatro variantes; `mceliece348864` queda fuera. 80
12.2 Los dieciséis conjuntos de ISO, cada fila con sus cuatro variantes. 81
12.3 Costos en bits ($\log_2$) de los mejores ataques conocidos según el estimador de Esser–Bellini: contra Prange (1962) y contra May–Ozerov (MO), con memoria de acceso gratis y con costo de acceso en raíz cuadrada. Las dos últimas columnas: la categoría que el equipo asigna contando costos realistas de memoria, y la que correspondería si la memoria se contara como gratis. 85
12.4 Tamaños de los KEM post-cuánticos relevantes (ML-KEM según FIPS 203 [34]; HQC según el informe del NIST [2]; Classic McEliece según la tabla 12.1). El criptograma de McEliece es el más pequeño con diferencia; la clave pública, la más grande. 87

13.1 Las familias de ataque contra Classic McEliece. El capítulo 16 añade la cuarta dimensión: el tiempo, con los resultados de 2025 y 2026 y las respuestas del equipo. 92
13.2 Tres lenguajes de coste. Toda comparación honesta de cifras exige fijar las unidades y el modelo de memoria; la sección 12.6 ya dio las advertencias para la tabla 12.3. 92

14.1 El mejor ataque conocido contra cada conjunto (en bits, modelo de memoria libre), según el estimador de Esser–Bellini, en la tabla 1 de la guía de seguridad [29]. BC: ball-collision (Bernstein–Lange–Peters, 2011); pdw: variante de BJMM con peso parcialmente disjunto; MO: May–Ozerov, 2015; BM: Both–May, 2018. El mínimo de cada fila está en negrita: May–Ozerov gana en los cinco conjuntos. 97
14.2 Los mismos ataques con *costes de acceso a memoria* (modelo 1/3: coste de raíz cúbica; modelo 1/2: coste de raíz cuadrada), tabla 1 de [29]. Compárese una fila con la otra: las mejoras combinatorias valen mucho menos cuando la memoria se paga (para `mceliece348864`, de 178,3 a 158,6 frente a de 173,4 a 140,8). 98

15.1 Los espacios de búsqueda de los ataques estructurales, con los ejemplos que citan las fuentes. La última fila no es un ataque estructural: es el rival genérico del capítulo 14, incluido para que la comparación de la figura 15.2 tenga a mano sus dos extremos. 104

15.2 Los distinguidores conocidos y sus márgenes, según la rationale del equipo [28]. La segunda columna dice qué región del espacio de parámetros explota cada ataque; la tercera, hasta dónde llega; la cuarta, dónde viven los conjuntos de Classic McEliece. El ataque de 2013 es el llamado «distinguidor de alta tasa» de Faugère–Gauthier-Umaña–Otmani–Perret–Tillich; el de Couvreur–Otmani–Tillich ataca las variantes «wild». 105

16.1 La fotografía de octubre de 2026, en operaciones de bits y sin cargo de memoria. La columna ISD es el mejor ataque genérico sin trucos de memoria (guía de seguridad; capítulos 12 y 14). Las columnas de 2026 son estimaciones *condicionales*, apoyadas en supuestos heurísticos, de ataques de recuperación de clave: GIJS (revisión del 27 de agosto, su tabla 3), Saarinen (revisión del 15 de septiembre) y Weis (versión 0.5 de octubre, tabla 1) [38, 64, 73]. La última columna es la referencia de NIST para las categorías 1/3/5 (búsqueda de clave sobre el AES correspondiente). Ninguna de las cifras de 2026 corresponde a un ataque ejecutado ni ejecutable en hardware real. 111

C.1 Las potencias z^i, $i = 0, \ldots, 14$: el valor de z^i es el dígito hex y su expresión polinómica, la de al lado. El cuerpo tiene 15 elementos no nulos: z es primitivo y su ciclo los agota todos (figura 3.1). 151
C.2 Logaritmos en base z (los exponentes, también en hex: por ejemplo $\log_z \mathtt{6} = \mathtt{5}$, pues $z^5 = z + z^2$, y el 0 no tiene logaritmo). Para multiplicar: $ab = z^{\log a + \log b}$ con el exponente módulo 15. 151
C.3 Inversos multiplicativos. Comprobación rápida: $\mathtt{3} \cdot \mathtt{e} = z^4 \cdot z^{11} = z^{15} = 1$. 152
C.4 Cuadrados, raíces cuadradas ($\sqrt{x} = x^8$, la mitad de 2^{m-1} porque la característica es 2) y traza absoluta ($\mathrm{Tr}(x) = x + x^2 + x^4 + x^8 \in \mathbb{F}_2$). Exactamente 8 elementos tienen traza 1: el núcleo de Tr es un subespacio de dimensión 3. 152
C.5 Producto $a \cdot b$ en $\mathbb{F}_{2^4}$, con a en la fila y b en la columna (como en todas las tablas de este apéndice). Cada fila no nula es una permutación de $\{1, \ldots, \mathtt{f}\}$: la tabla lo muestra y la estructura de cuerpo lo garantiza. 153

PARTE I

Fundamentos: de los bits a los códigos de Goppa

CAPÍTULO 1

La amenaza cuántica y el giro post-cuántico

En este capítulo

Este capítulo sitúa a Classic McEliece en su contexto y responde a tres preguntas: *por qué* hace falta criptografía post-cuántica, *qué* se está estandarizando y *dónde encaja* un esquema de 1978 en los sistemas de 2026. Es un capítulo de orientación: la maquinaria matemática empieza en el capítulo 2.

Toda la criptografía de clave pública que protege internet hoy —RSA, Diffie–Hellman, las curvas elípticas— descansa sobre dos problemas que llevamos medio siglo sin resolver con métodos clásicos: factorizar enteros grandes y calcular logaritmos discretos. En 1994, Peter Shor demostró que un ordenador cuántico suficientemente grande resolvería *ambos* en tiempo polinómico [68]. Dos años después, Lov Grover mostró una aceleración cuadrática para la búsqueda no estructurada [41], que debilita —que no anula— la criptografía simétrica. La conclusión de aquellos dos resultados define el campo: **necesitamos cifrado de clave pública que siga siendo seguro en un mundo con ordenadores cuánticos**, y que además funcione hoy en nuestros ordenadores clásicos. Eso es la criptografía *post-cuántica* (o *crypto resiliente ante lo cuántico*).

Dos matices de vocabulario

«Post-cuántico» no significa «cuántico»: los algoritmos post-cuánticos son clásicos (corren en máquinas normales); lo que cambia es *contra qué* adversario prometen seguridad. Y «seguro contra Shor» no es el único criterio: un esquema debe resistir a toda la criptoanalítica clásica conocida, que para los esquemas de códigos lleva acumulada casi medio siglo de trabajo.

1.1 La amenaza: Shor, Grover y el adversario que espera

Dos algoritmos que cambian las reglas

Shor (1994) resuelve en tiempo polinómico la factorización y el logaritmo discreto —dos problemas que sostienen RSA, Diffie–Hellman y ECC—, con una única herramienta: la transformada cuántica de Fourier. En cambio, no conoce ningún atajo análogo para los problemas *NP-duros* típicos de la criptografía basada en códigos o retículos: frente a ellos solo ofrece la aceleración de Grover, cuadrática y genérica. Para cifrado simétrico eso implica que AES-128 protege, en el mejor modelo cuántico, como AES de 64 bits; de ahí la recomendación de claves simétricas de 256 bits (seguridad efectiva $\approx$ 128 bits). La tabla 1.1 resume el efecto.

¿Cuándo llega el ordenador cuántico?

Nadie ha construido un ordenador cuántico tolerante a fallos capaz de ejecutar Shor sobre claves reales, y *no* es este documento el lugar para zanjar cuándo llegará. Lo que sí está documentado: los recursos estimados crecen y se han refinado a la baja (un análisis de 2021

Cuadro 1.1: Efecto de los algoritmos cuánticos conocidos sobre las familias criptográficas clásicas. «Seguridad efectiva» es una estimación de libro mayor; los detalles de los modelos de coste importan (véase el capítulo 14).

Primitiva	**Problema**	**Algoritmo cuántico**	**Efecto**
RSA	factorización	Shor (1994)	**rota** (polinómico)
Diffie–Hellman / ECC	log. discreto	Shor (1994)	**rota**
AES / hashes	búsqueda, colisiones	Grover (1996), BHT	clave de 2λ bits para λ de seguridad
Códigos (McEliece)	decodificación por síndrome	Grover / BHT	sin atajo conocido; exponente a lo sumo a la mitad (cap. 14)

para RSA-2048 habla de ≈ 20 millones de qubits físicos ruidosos y ≈ 8 horas [39]); los criptogramas *de hoy* pueden grabarse y descifrarse *mañana* (*harvest now, decrypt later*), fenómeno decisivo para datos con larga vida útil; y migrar infraestructura criptográfica lleva una década o más, como ilustra el propio calendario de NIST (IR 8547): retirada de los algoritmos vulnerables recomendada para 2030 y prohibición desde 2035 [57]. La prudencia se apoya en esos tres hechos, no en una profecía.

Treinta segundos de historia hasta hoy

La figura 1.1 recorre los hitos que desembocan en estas notas. La cronología no está a escala: los años recientes se han espaciado para poder rotularlos.

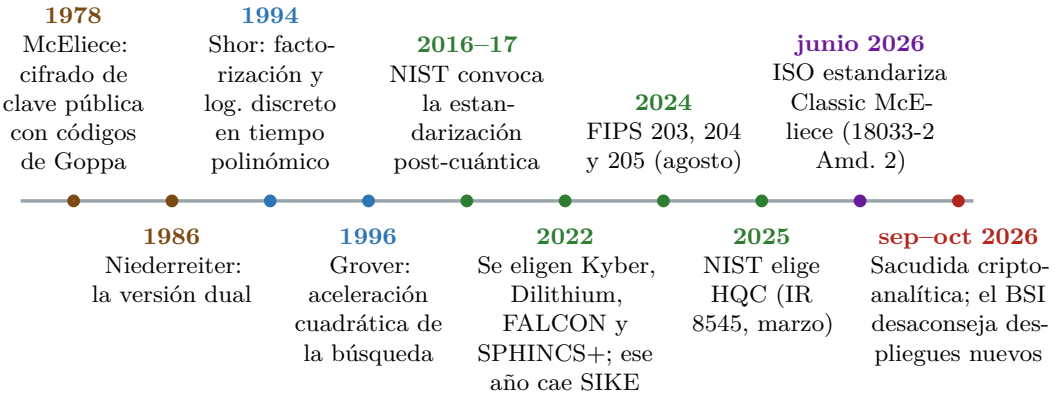


Figura 1.1: De 1978 a octubre de 2026. En sepia, la prehistoria del cifrado con códigos; en azul, los algoritmos cuánticos que motivaron el giro; en verde, el proceso del NIST; en violeta, la estandarización ISO; en rojo, los acontecimientos de 2026 que se analizan en el capítulo 16.

1.2 Qué se está estandarizando: el proceso del NIST

En 2016 el NIST abrió una convocatoria pública para elegir algoritmos post-cuánticos. Recibió 82 propuestas (69 aceptadas como completas y correctas), que fueron cribándose en rondas sucesivas: 26 candidatos en la segunda ronda (2019) y 15 en la tercera (2020: 7 finalistas y 8 alternativos). En julio de 2022 seleccionó cuatro algoritmos y en agosto de 2024 publicó las tres primeras normas federales: FIPS 203 (ML-KEM, ex CRYSTALS-Kyber), FIPS 204 (ML-DSA,

ex Dilithium) y FIPS 205 (SLH-DSA, ex SPHINCS+) [34, 35, 36]. A la ronda final de KEMs llegaron tres candidatos alternativos: BIKE, HQC y Classic McEliece. En marzo de 2025, tras la cuarta ronda de análisis, el NIST eligió *solo* a HQC [2]. Classic McEliece quedó fuera de la selección del NIST —no por un fallo criptoanalítico, sino porque sus claves públicas de kilobytes a megabytes encajan mal en los protocolos de internet— y tomó otro camino: ISO/IEC.

Cuadro 1.2: Estado de la estandarización post-cuántica en octubre de 2026 (KEMs y firmas principales). Fuentes: FIPS 203–205, NIST IR 8545 y la página ISO del equipo Classic McEliece [24].

Nombre estándar	**Antecesor**	**Familia**	**Estado (oct. 2026)**
ML-KEM	CRYSTALS-Kyber	retículos	norma (FIPS 203, ago. 2024)
ML-DSA	CRYSTALS-Dilithium	retículos	norma (FIPS 204, ago. 2024)
SLH-DSA	SPHINCS+	funciones hash	norma (FIPS 205, ago. 2024)
FN-DSA	FALCON	retículos	borrador en curso
HQC	HQC	códigos	elegido (mar. 2025); norma prevista para 2027
Classic McEliece	McEliece (1978)	códigos	**ISO/IEC 18033-2 Amd. 2 (jun. 2026)**

SIKE: cuando la diversidad de hipótesis importa

En agosto de 2022, semanas después de que el NIST anunciara su primera selección, Castryck y Decru rompieron SIKE —candidato basado en isogenias— con un ataque que ejecutaba en minutos sobre una clave que la comunidad consideraba sana. El episodio explica por qué las instituciones recomiendan *híbridos* (combinar un KEM post-cuántico con ECDH) y por qué la diversidad de hipótesis matemáticas —retículos, códigos, hashes— se considera un valor en sí misma.

1.3 Dónde encaja Classic McEliece

El esquema de estas notas es, cronológicamente, el más veterano de todos los que aparecen en la tabla 1.2: McEliece lo publicó en 1978 [56] y, con la disciplina de usar siempre código binario de Goppa y parámetros muy holgados, ha sobrevivido sin rupturas a *cuarenta y ocho años* de criptoanálisis clásico. Sus señas de identidad:

- **Suposición muy conservadora.** No depende de estructuras nuevas ni de problemáticas recientes: su dureza se apoya en el problema de la decodificación por síndrome (capítulo 2), para el cual no se ha encontrado ni un atajo estructural en código binario de Goppa (capítulo 15).
- **Claves públicas enormes, cifrados diminutos.** La clave pública es de 255 KiB a 1.3 MiB según el conjunto de parámetros, pero un cifrado ocupa apenas 96–208 bytes: el esquema «comprime» el mensaje de forma agresiva. La figura 1.2 compara estos tamaños con los de sus vecinos estandarizados.
- **Rendimiento asimétrico pero rápido.** La generación de claves es costosa (décimas de segundo a segundos); la encapsulación y desencapsulación tardan microsegundos o decenas de microsegundos. En cifrado de sesiones, donde la clave pública se reutiliza, el coste amortizado es bueno.
- **Es un KEM, no un cifrado.** Desde 2019 el esquema se especifica como mecanismo de encapsulación de claves con rechazo implícito (capítulo 8), que es como se estandarizó en [10] y, en junio de 2026, en el apéndice a la norma ISO/IEC 18033-2 [45, 24].

- **Qué conjuntos estandariza ISO.** La enmienda ISO recoge los conjuntos `mceliece460896`, `mceliece6688128`, `mceliece6960119` y `mceliece8192128`, cada uno en cuatro variantes (`f`, `pc` y `pcf` además de la básica); el conjunto más pequeño (`mceliece348864`) quedó fuera: no alcanza los 128 bits de seguridad en el modelo cuántico (con aceleración de Grover) que exige ISO [24]. Los detalles, en el capítulo 12.

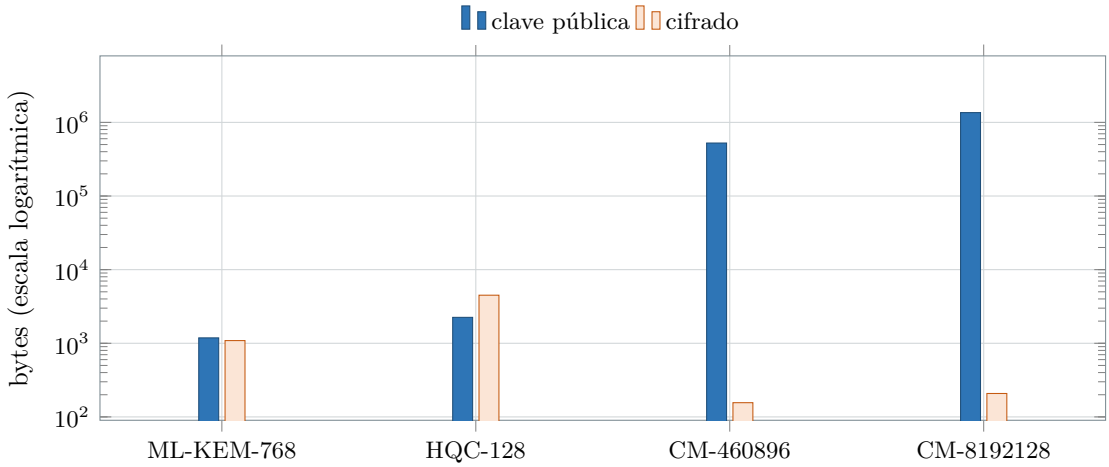


Figura 1.2: Tamaño de clave pública y de cifrado en tres familias estandarizadas (octubre de 2026). La escala es logarítmica: cada división vertical es un factor 10. Classic McEliece intercambia claves de centenares de KiB por cifrados de un centenar de bytes; los esquemas de retículos y HQC son más equilibrados. Datos: FIPS 203, especificación de HQC y especificación de Classic McEliece [34, 1, 10].

Híbridos: cómo se despliega esto mañana

Un KEM post-cuántico rara vez sustituye a ECDH en solitario: lo habitual es el modo híbrido (X25519 + ML-KEM-768, por ejemplo), que sobrevive si *cualquiera* de los dos resultara roto. Classic McEliece no es el candidato natural para el apretón de manos de TLS (su clave pública no cabe en un datagrama), pero sí encaja donde el almacenamiento es barato y la criptografía es de larga vida: firmware firmado, archivos cifrados, raíces de confianza. Los tamaños de la figura 1.2 se entienden mejor con esa lógica de despliegue.

1.4 Cómo leer el resto del documento

La **parte I** construye las tres herramientas previas: códigos lineales (cap. 2), cuerpos finitos (cap. 3) y códigos de Goppa (cap. 4), con un laboratorio numérico totalmente verificable al final (cap. 5). La **parte II** presenta el criptosistema original de 1978 (cap. 6), la variante dual de Niederreiter (cap. 7) y la transformación a KEM (cap. 8). La **parte III** recorre la especificación de Classic McEliece al detalle (caps. 9–12) y ejecuta un mini-KEM de extremo a extremo (cap. 11). La **parte IV** analiza qué significa romper McEliece: reducciones (cap. 13), decodificación por conjuntos de información (cap. 14), ataques estructurales (cap. 15) y el terremoto criptoanalítico de 2026 (cap. 16). La **parte V** reúne preguntas frecuentes, ejercicios, soluciones, glosario y referencias.

El hilo conductor

Todo el documento responde a una sola pregunta: *¿por qué publicar una matriz generadora «barajada» de un código con decodificador eficiente da un criptosistema?* Veremos la idea, su historia, su estandarización y —en 2026— sus grietas y sus límites.

1.5 Ejercicios de calentamiento

1. Sin mirar la tabla 1.1: ¿por qué AES-256 se considera suficiente en el mundo post-cuántico y AES-128 no?
2. Ordene por tamaño de *cifrado* (de menor a mayor): ML-KEM-768, HQC-128, mceliece460896. Después, ordene por tamaño de *clave pública*.
3. Explique en dos frases por qué «harvest now, decrypt later» obliga a actuar antes de que exista un ordenador cuántico.
4. (Investigación) Busque el comunicado del BSI del 1 de octubre de 2026 citado en el capítulo 16 y resuma en cinco líneas sus dos argumentos principales.

CAPÍTULO 2

Códigos lineales: el lenguaje de la corrección de errores

En este capítulo

Este capítulo introduce el vocabulario mínimo imprescindible: códigos lineales binarios, matrices generadora y de paridad, síndromes y decodificación. Al final aparecen las dos piezas que conectan la teoría de códigos con la criptografía: el problema de la decodificación por síndrome y la idea de esconder un decodificador eficiente bajo una matriz «barajada». Todo lo demás del documento se apoya en este capítulo.

2.1 El espacio de Hamming

La corrección de errores empieza con una decisión de modelado: los datos son palabras de n bits y el canal puede *cambiar* algunos bits. Medimos el daño contando posiciones distintas.

Definición 2.1: Peso y distancia de Hamming

Para $x, y \in \mathbb{F}_2^n$:

- El **peso de Hamming** $\mathrm{wt}(x)$ es el número de posiciones no nulas de x.
- La **distancia de Hamming** $\mathrm{d}(x, y) = \mathrm{wt}(x - y)$ (en $\mathbb{F}_2$, $x - y = x + y$) es el número de posiciones en que difieren.

d es una distancia en el sentido métrico: simétrica, nula solo entre iguales y con desigualdad triangular.

Ejemplo 2.2: A mano

En $\mathbb{F}_2^5$: $x = 10110$, $y = 00111$; $x + y = 10001$, luego $\mathrm{d}(x, y) = \mathrm{wt}(10001) = 2$.

Definición 2.3: Código lineal, parámetros

Un **código lineal binario** C de longitud n es un subespacio vectorial de $\mathbb{F}_2^n$. Si $\dim C = k$ y la distancia mínima $d = \min\{\mathrm{d}(c, c') : c \neq c' \in C\}$ vale d, se dice que C es un código $[n, k, d]$. La **tasa** del código es k/n.

En un código lineal la distancia mínima se lee sin recorrer todos los pares:

Proposición 2.4: Distancia mínima y pesos

Si C es lineal y no nulo, $d = \min_{c \in C,\, c \neq 0} \mathrm{wt}(c)$.

Demostración. $\mathrm{d}(c, c') = \mathrm{wt}(c - c')$ y $c - c' \in C$; recíprocamente todo $c \in C$ es $c - 0$. □

Definición 2.5: Matriz generadora

Una **matriz generadora** de C es una matriz $G \in \mathbb{F}_2^{k\times n}$ cuyas filas forman una base de C; entonces $C = \{uG : u \in \mathbb{F}_2^k\}$. El mapa $u \mapsto uG$ es inyectivo y codifica k bits en n bits. Cuando $G = (\mathrm{I}_k \mid A)$ se dice que el código es **sistemático**: los k bits de mensaje aparecen tal cual en las primeras k posiciones de la palabra.

Definición 2.6: Matriz de paridad

Una **matriz de paridad** de C es una matriz $\mathbf{H} \in \mathbb{F}_2^{r\times n}$ cuyas filas generan el ortogonal $C^\perp = \{v \in \mathbb{F}_2^n : v \cdot c = 0 \ \forall c \in C\}$; entonces $C = \{\mathbf{x} \in \mathbb{F}_2^n : \mathbf{H}\,\mathbf{x}^\top = 0\}$ y $r = n - k$ (si las filas de $\mathbf{H}$ son independientes).

Generar y verificar son las dos caras de la misma moneda:

Proposición 2.7: $G\mathbf{H}^\top = 0$

Con la convención de vectores *fila*: $G\mathbf{H}^\top = 0$, y C es exactamente el núcleo de $\mathbf{H}$ (visto como aplicación lineal $\mathbb{F}_2^n \to \mathbb{F}_2^{n-k}$, $\mathbf{x} \mapsto \mathbf{x}\mathbf{H}^\top$).

Demostración. Cada fila de G es un elemento de C, ortogonal a cada fila de $\mathbf{H}$, luego $G\mathbf{H}^\top = 0$. El núcleo tiene dimensión $n - \operatorname{rank}(\mathbf{H}) = k$ y contiene a las filas de G (dimensión k), así que coincide con C. □

La matriz de paridad también certifica la distancia mínima, con un criterio que usaremos constantemente:

Teorema 2.8: Columnas de H y distancia mínima

d es el menor número de columnas de $\mathbf{H}$ linealmente dependientes. En particular $d \geq d_0$ si y solo si cualesquiera $d_0 - 1$ columnas de $\mathbf{H}$ son independientes.

Demostración. Si $c \in C$ con $\operatorname{supp}(c) = \{i_1, \ldots, i_w\}$, entonces $0 = \mathbf{H}c^\top = \sum_j \mathrm{col}_{i_j}(\mathbf{H})$: las columnas del soporte son dependientes, luego $d \geq$ (menor tamaño dependiente). Recíprocamente, una dependencia $\sum_{j\in J} \mathrm{col}_j = 0$ con $|J| = w$ produce el codeword $c = \sum_{j\in J} e_j$ de peso w (o menor si hay cancelaciones), luego (menor tamaño dependiente) $\geq d$ para el mínimo. □

Ejemplo 2.9: El código de repetición

$C = \{000, 111\} \subset \mathbb{F}_2^3$ es un $[3, 1, 3]$: una matriz generadora es $G = (1\ 1\ 1)$ y una de paridad $\mathbf{H} = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}$. Toda palabra con a lo sumo un error se corrige por unanimidad: $y = 101$ se «vota» a 111.

Ejemplo 2.10: El código de Hamming $[7, 4, 3]$

Con

$$G = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{pmatrix}, \qquad \mathbf{H} = \begin{pmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 1 \end{pmatrix},$$

las columnas de $\mathbf{H}$ son todos los vectores no nulos de $\mathbb{F}_2^3$ (¡las representaciones binarias de $1, \ldots, 7$, en algún orden!): no hay columnas nulas ni repetidas, luego cualquier suma parcial se anula solo en el conjunto vacío. Por el teorema 2.8, $d \geq 3$; como la fila 3 de G tiene peso 3, $d = 3$.

2.2 Decodificación: síndromes

Enviar un codeword c por un canal ruidoso produce $y = c + e$, donde e es el *vector de error*. Decodificar es, dado y, recuperar un codeword válido «lo más cercana posible». El instrumento central es el síndrome.

Definición 2.11: Síndrome

El **síndrome** de $y \in \mathbb{F}_2^n$ (respecto de $\mathbf{H}$) es $\mathrm{Synd}(y) = y\,\mathbf{H}^\top \in \mathbb{F}_2^{n-k}$.

Proposición 2.12: Propiedades del síndrome

1. Synd es lineal: $\mathrm{Synd}(y + y') = \mathrm{Synd}(y) + \mathrm{Synd}(y')$.
2. $\mathrm{Synd}(y) = \mathrm{Synd}(y')$ si y solo si $y - y' \in C$.
3. $\mathrm{Synd}(y) = \mathrm{Synd}(e)$: el síndrome solo ve el error.
4. y admite corrección con error e de peso w si y solo si $\mathrm{Synd}(y) = \mathrm{Synd}(e)$ para algún e con $\mathrm{wt}(e) = w$.

Demostración. (1) y (3) salen de la linealidad de $x \mapsto x\mathbf{H}^\top$. Para (2), Synd tiene núcleo C (proposición 2.7); para (4), si $\mathrm{Synd}(y) = \mathrm{Synd}(e)$ entonces $y - e \in C$, es decir, y es el codeword $y - e$ más el error e. □

El conjunto de vectores con un síndrome dado es una *clase (lateral)* $y + C$; el representante de peso mínimo se llama **líder de clase** y es la mejor corrección posible en el sentido de máxima verosimilitud (canal simétrico binario). Con d como distancia mínima, si $\mathrm{wt}(e) \le t$ con

$$t = \left\lfloor \frac{d-1}{2} \right\rfloor, \tag{2.1}$$

la corrección es además *única*: las bolas de radio t centradas en codewords no se solapan (figura 2.1). A t se le llama capacidad de corrección del código.

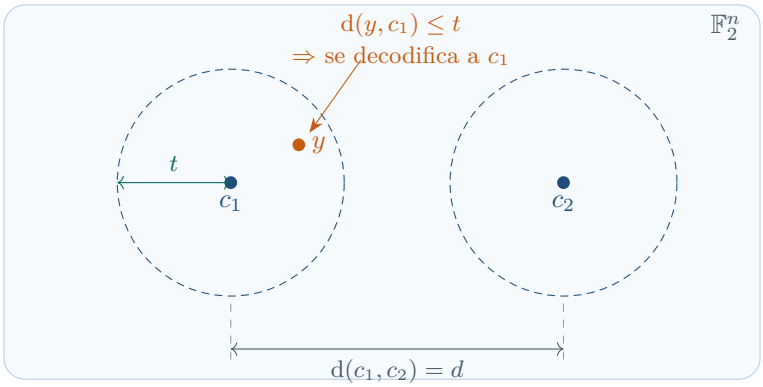


Figura 2.1: Las bolas de radio $t = \lfloor (d-1)/2 \rfloor$ alrededor de cada codeword no se solapan: cualquier palabra recibida suficientemente cerca de un codeword se corrige de forma *única*. Fuera de esos radios, la «mejor» corrección puede ser ambigua.

Ejemplo 2.13: Decodificación por síndrome en Hamming $[7,4,3]$

Con las matrices del ejemplo 2.10, $t = 1$. Supongamos que se recibe $y = 1110001$. Su síndrome es

$$y\,\mathbf{H}^\top = (1,1,1,0,0,0,1)\begin{pmatrix}1&1&0\\0&1&1\\1&1&1\\1&0&1\\1&0&0\\0&1&0\\0&0&1\end{pmatrix} = (0,1,1),$$

que es exactamente la *segunda columna* de $\mathbf{H}$. El diccionario «síndrome = columna j» identifica el error $\hat{e} = e_2$ y devuelve $\hat{c} = y + \hat{e} = 1010001$. Como el código es sistemático, el mensaje es $\hat{u} = 1010$. La tabla 2.1 muestra el diccionario completo.

Cuadro 2.1: Diccionario de decodificación del Hamming $[7,4,3]$: cada error de peso uno tiene por síndrome la columna correspondiente de $\mathbf{H}$. Los siete síndromes agotan los vectores no nulos de $\mathbb{F}_2^3$: es un código *perfecto*.

posición j	síndrome	posición j	síndrome	posición j	síndrome	posición j	síndrome
1	110	2	011	3	111	4	101
5	100	6	010	7	001	—	—

2.3 El problema de la decodificación por síndrome

Hasta aquí todo es álgebra lineal elemental. La dificultad aparece cuando *no* se conoce la matriz de paridad secreta que hace fácil corregir, y solo se dispone de un código «como objeto combinatorio». Formalizamos el problema central de la criptografía basada en códigos, en sus dos versiones (seguimos la formulación de la especificación de Classic McEliece [10]):

Definición 2.14: Problema de decodificación por síndrome, SD

Sea $\mathbf{H} \in \mathbb{F}_2^{r\times n}$, $s \in \mathbb{F}_2^r$ y w un entero.

- **SD de búsqueda (search SD):** encontrar $e \in \mathbb{F}_2^n$ con $\mathrm{wt}(e) \le w$ y $e\,\mathbf{H}^\top = s$ (si existe).
- **SD de decisión (decision SD):** distinguir con ventaja no despreciable los pares $(\mathbf{H}, s)$ en los que tal e existe de los pares con $\mathbf{H}$ uniforme y s uniforme.

De Prange a la NP-completitud

Eugene Prange ya planteó en 1962 la decodificación mediante conjuntos de información [62]; en 1978, el mismo año del artículo de McEliece, Berlekamp, McEliece y van Tilborg demostraron que la versión de decisión de SD es NP-completa [6]. Es decir: el problema en el que se apoya McEliece es de los que la teoría de la complejidad considera duros... en el peor caso. Volvemos sobre este matiz justo debajo.

«NP-duro» no es una demostración de seguridad

NP-duro es una garantía de *peor caso*: dice que no habrá un algoritmo polinómico que resuelva *todas* las instancias (salvo que P=NP). Un criptosistema necesita algo más fuerte: que las instancias *de la distribución concreta que genera las claves* sean duras *en media* y *con el tamaño real*. Nadie ha demostrado que SD sea duro en media para las distribuciones relevantes; la confianza en McEliece es *empírica e histórica*: casi medio siglo de ataques no ha doblado la respuesta. Además, los códigos de un criptosistema no son aleatorios (tienen estructura algebraica, la que permite decodificar con la clave secreta), y esa estructura es justo lo que atacan los ataques «estructurales» de la parte IV. Por último, NP-duro no detiene a Shor: los algoritmos cuánticos conocidos para SD (Grover, paseos cuánticos) recuperan a lo sumo una raíz cuadrada del exponente, con sobrecostes grandes; véase la sección 14.6.

Ejemplo 2.15: Resolver SD «a lo bruto»

Para el código de Hamming $[7, 4, 3]$, dado un síndrome se prueban los vectores de peso 0 y 1 hasta acertar: a lo sumo 8 candidatos (tabla 2.1). Para el conjunto `mceliece460896` ($n = 4608$, $t = 96$), en cambio, probar errores de peso ≤ 96 a lo bruto es del orden de $\binom{4608}{96} \approx 2^{668.6}$ candidatos. Ese abismo —8 frente a 2^{668}— es, comprimido, todo el argumento de seguridad de este documento.

2.4 La idea de McEliece: esconder un decodificador

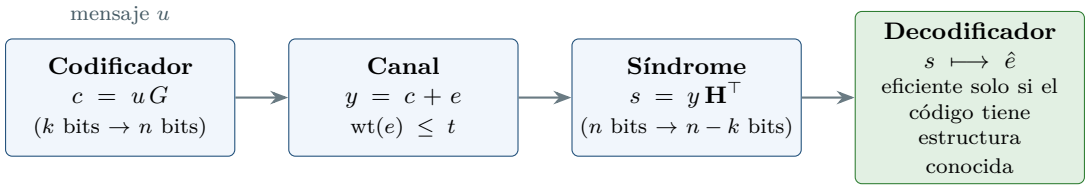


Figura 2.2: La cadena completa: codificar, atravesar el canal, calcular el síndrome y decodificar. El público ve y y, en su caso, la matriz $\mathbf{H}$; la dificultad no está en el álgebra lineal del síndrome, sino en encontrar un error de peso pequeño que lo explique.

Un código con decodificador eficiente es una *puerta secreta*: quien conoce el código puede corregir errores de peso hasta t en tiempo polinómico; quien solo ve su matriz generadora, no. La idea fundacional de McEliece (1978) es:

La puerta secreta de McEliece

Tómese un código C con un decodificador eficiente —p. ej. un código de Goppa, capítulo 4— y publíquese una versión *barajada* de sus matrices:

$$G' = S\,G\,P,$$

con S invertible y P una matriz de permutación. La matriz G' genera un código *equivalente* a C (el mismo código salvo permutar las coordenadas y cambiar de base), pero sin la estructura que hace eficiente la decodificación. Cifrar es codificar con G' y sumar un error de peso $\leq t$; descifrar exige deshacer P, decodificar con la puerta secreta y deshacer S. Para un adversario sin la clave, no queda más que resolver SD para un código que parece aleatorio.

Los tres capítulos siguientes construyen ese código especial (los *códigos de Goppa*); la parte II convierte la idea en criptosistema y la parte IV pregunta con dureza qué pasa cuando el adversario intenta *deshacer el barajado* en lugar de decodificar.

Por qué elegir una permutación «cara»

Reordenar las coordenadas de un código no cambia su distancia mínima ni sus propiedades algebraicas, pero destruye la relación directa entre posiciones y elementos del cuerpo finito que hace eficiente la decodificación. Esa es toda la «trampa»: un isomorfismo oculto. La contrapartida es que el atacante puede intentar recuperar el isomorfismo (support splitting, ataques algebraicos, ...), y ahí se libra la batalla real de Classic McEliece (capítulos 15 y 16).

Notación a partir de aquí

La tabla 2.2 fija los símbolos que usaremos. En las partes I y II los vectores son *fila*, como en el artículo original de 1978; en el capítulo 8 la especificación de Classic McEliece impone vectores *columna* y señalaremos la traducción sin drama.

Cuadro 2.2: Notación básica de códigos lineales.

Símbolo	Significado	Símbolo	Significado
n	longitud de las palabras	u	mensaje (k bits)
k	dimensión del código	c	codeword (n bits)
d	distancia mínima	e	vector de error
t	$\lfloor (d-1)/2 \rfloor$: errores corregibles	y	palabra recibida $c+e$
G	matriz generadora	s	síndrome
$\mathbf{H}$	matriz de paridad	Synd	aplicación síndrome
C	el código	$C^{\perp}$	código dual (ortogonal)

2.5 Ejercicios

1. Demuestre la desigualdad triangular para la distancia de Hamming.
2. Sea $C = \{0000, 1100, 0011, 1111\}$. Calcule una matriz G, una matriz $\mathbf{H}$ y la distancia mínima d.
3. Para el Hamming $[7, 4, 3]$ del ejemplo 2.10, decodifique $y = 0101010$ y $y' = 1100110$ e indique el mensaje de 4 bits en cada caso.
4. ¿Cuántas palabras de peso exactamente t tiene el cubo $\mathbb{F}_2^n$? Compare $n = 7, t = 1$ con $n = 3488, t = 64$ dando el exponente en base 2 del segundo caso.
5. Explique por qué $\mathrm{Synd}(y) = \mathrm{Synd}(e)$ implica que todo codeword de la clase $y + C$ dista de y como mucho $\mathrm{wt}(e'')$, donde e'' es el mínimo peso de un vector con ese síndrome.
6. (Síntesis) Formule en tres líneas qué propiedad debe tener un código para que la construcción «publicar $G' = SGP$» sea un cifrado razonable, y qué dos recursos (uno computacional, uno algebraico) hacen falta para descifrar.

CAPÍTULO 3

Cuerpos finitos: el terreno donde vive Goppa

En este capítulo

Los códigos de Goppa se construyen sobre cuerpos finitos: el polinomio g y los puntos de evaluación α_i viven en $\mathbb{F}_q$ con $q = 2^m$. Este capítulo construye $\mathbb{F}_{2^m}$ desde cero —aritmética, potencias, traza, raíces cuadradas— y fija el modelo de representación que usa la especificación de Classic McEliece. El ejemplo de referencia es $\mathbb{F}_{2^4} = \mathbb{F}_2[z]/(z^4 + z + 1)$, el mismo del laboratorio.

3.1 ¿Por qué cuerpos finitos?

En el capítulo anterior, todo ocurría sobre $\mathbb{F}_2$: bits, matrices, síndromes. El salto de McEliece consiste en construir un código binario *a través* de un cuerpo mayor. La definición de los códigos de Goppa (capítulo 4) necesita dos ingredientes:

- un polinomio g de grado t con coeficientes en un cuerpo *grande* (no en $\mathbb{F}_2$), y
- una lista de n elementos distintos $\alpha_0, \ldots, \alpha_{n-1}$ de ese cuerpo, con $g(\alpha_i) \neq 0$.

El código resultante es binario, pero sus ecuaciones de paridad están escritas sobre $\mathbb{F}_q$, $q = 2^m$. Por eso necesitamos saber calcular en $\mathbb{F}_{2^m}$ con soltura: sumar (trivial), multiplicar, invertir, elevar al cuadrado, sacar raíces cuadradas y evaluar polinomios.

3.2 Construcción de $\mathbb{F}_{2^m}$

De la aritmética modular a los cuerpos finitos

Para quien no haya estudiado teoría de anillos, la construcción de $\mathbb{F}_{2^m}$ es exactamente análoga a la aritmética modular de enteros ($\mathbb{Z}/p\mathbb{Z}$): en $\mathbb{Z}/p\mathbb{Z}$ los elementos son los restos posibles $\{0, 1, \ldots, p-1\}$ de dividir entre un número primo p, y la irreducibilidad de p asegura que todo elemento no nulo tenga inverso. En $\mathbb{F}_{2^m}$, el papel de p lo asume un polinomio *irreducible* $f(z)$ de grado m: los elementos del cuerpo son todos los polinomios con coeficientes en $\mathbb{F}_2$ de grado menor que m (los posibles restos al dividir entre $f(z)$), y «módulo $f(z)$» significa simplemente quedarse con el resto de la división euclídea de polinomios.

Definición 3.1: Cuerpo finito $\mathbb{F}_{2^m}$

Sea $f(z)$ un polinomio *irreducible* de grado m con coeficientes en $\mathbb{F}_2$. El anillo cociente

$$\mathbb{F}_{2^m} \;=\; \mathbb{F}_2[z]/(f(z))$$

es un cuerpo con 2^m elementos. Cada elemento se escribe de forma única como un polinomio

de grado $< m$,

$$a \;=\; c_0 + c_1 z + \cdots + c_{m-1} z^{m-1}, \qquad c_i \in \mathbb{F}_2,$$

y se representa como el vector de bits $(c_0, c_1, \ldots, c_{m-1})$ *(el bit i es el coeficiente de z^i)*. La suma es el XOR de bits; el producto es la multiplicación de polinomios seguida de reducción módulo $f(z)$.

Observación 3.2: Existencia y unicidad

Para cada $m \geq 1$ existe un f irreducible de grado m sobre $\mathbb{F}_2$, y todos los cuerpos de 2^m elementos son isomorfos entre sí (se denotan $\mathbb{F}_{2^m}$ sin ambigüedad). La irreducibilidad es esencial: si f se factoriza, el anillo tiene divisores de cero y no es un cuerpo. La demostración estándar (que $\mathbb{F}_2[z]/(f)$ es cuerpo si y solo si f es irreducible, y que $\mathbb{F}_{2^m}$ existe) puede verse en Lidl y Niederreiter [52].

Ejemplo 3.3: El cuerpo más pequeño con extensión: $\mathbb{F}_4$

Con $f(z) = z^2 + z + 1$ (el único polinomio irreducible de grado 2 sobre $\mathbb{F}_2$), los elementos son $0, 1, z, 1+z$. Multiplicación: $z \cdot z = z^2 \equiv z + 1$ (pues $z^2 = z + 1$ módulo f), y por tanto $(1+z)^2 = (1+z) + (z+z^2) = 1 + z^2 = 1 + (z+1) = z$. Todos los productos no nulos están en $\{1, z, 1+z\}$: no hay divisores de cero.

El elemento primitivo y los logaritmos

Un hecho estructural clave: el grupo multiplicativo $\mathbb{F}_{2^m}^{\times} = \mathbb{F}_{2^m} \setminus \{0\}$ es *cíclico* de orden $2^m - 1$ [52]. Un generador z se llama *elemento primitivo*: sus potencias $z^0, z^1, \ldots, z^{2^m-2}$ recorren todo el cuerpo salvo el cero. Multiplicar se vuelve sumar exponentes (tablas de logaritmos), que es como trabajaremos.

Ejemplo 3.4: $\mathbb{F}_{2^4}$ de referencia

Tomamos en toda la parte I el cuerpo

$$\mathbb{F}_{2^4} = \mathbb{F}_2[z]/(z^4 + z + 1),$$

que es la construcción usada por el laboratorio y por los conjuntos de parámetros reales con $m = 12, 13$ (allí, $z^4 + z + 1$ se sustituye por un polinomio irreducible de grado 12 o 13). Aquí z es primitivo: sus potencias recorren los 15 elementos no nulos (tabla 3.1 y figura 3.1). La suma de bits es XOR; el producto, polinómico con reducción $z^4 = z + 1$.

Ejemplo 3.5: Aritmética en $\mathbb{F}_{2^4}$ con logaritmos

Producto: $(z^2 + z)(1 + z^3)$. Con la tabla: $z^2 + z = z^5$ y $1 + z^3 = z^{14}$; luego el producto es $z^{19} = z^4 = 1 + z$. *Inverso:* $(1+z)^{-1}$: como $1 + z = z^4$, el inverso es $z^{15-4} = z^{11} = z + z^2 + z^3$, y en efecto $(1+z)(z + z^2 + z^3) = z^{15} = 1$.

⚠ Sobre la representación y el orden de bits

Conviene distinguir con cuidado dos formas de escribir un elemento $a = c_0 + c_1 z + \cdots + c_{m-1} z^{m-1} \in \mathbb{F}_{2^m}$:

- **Como vector de coordenadas:** $(c_0, c_1, \ldots, c_{m-1})$, con el término independiente c_0 en la primera posición (índice 0), que es como se disponen las columnas y filas de las matrices

Cuadro 3.1: Potencias de z en $\mathbb{F}_{2^4}$, con $z^4 = z+1$. Cada elemento se escribe como polinomio, como vector de bits $(b_3b_2b_1b_0)$ (el bit i-ésimo es el coeficiente de z^i, como manda la especificación [10]) y como dígito hexadecimal.

i	z^i	bits $b_3b_2b_1b_0$	hex
0	1	0001	0x1
1	z	0010	0x2
2	z^2	0100	0x4
3	z^3	1000	0x8
4	$1+z$	0011	0x3
5	$z+z^2$	0110	0x6
6	z^2+z^3	1100	0xC
7	$1+z+z^3$	1011	0xB

i	z^i	bits $b_3b_2b_1b_0$	hex
8	$1+z^2$	0101	0x5
9	$z+z^3$	1010	0xA
10	$1+z+z^2$	0111	0x7
11	$z+z^2+z^3$	1110	0xE
12	$1+z+z^2+z^3$	1111	0xF
13	$1+z^2+z^3$	1101	0xD
14	$1+z^3$	1001	0x9
	0	0000	0x0

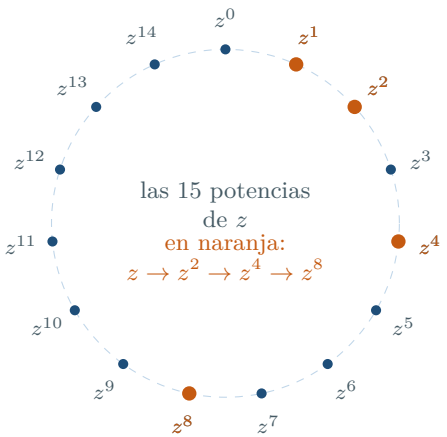


Figura 3.1: El grupo multiplicativo de $\mathbb{F}_{2^4}$ es cíclico de orden 15: las potencias de z recorren todos los elementos no nulos, y $z^{15} = 1$. En naranja, la órbita de z bajo el automorfismo de Frobenius $x \mapsto x^2$ (véase la sección 3.3).

en los capítulos 5 y 11.

- **Como entero o hexadecimal:** $b_{m-1}\dots b_1 b_0$ con el bit más significativo a la izquierda ($b_i = c_i$). Por ejemplo, $z = 0\cdot z^3 + 0\cdot z^2 + 1\cdot z + 0\cdot 1$ se escribe en binario como `0010` (`2`), y su cuadrado z^2 como `0100` (`4`). Al elevar un monomio al cuadrado, su bit se desplaza hacia la izquierda en la notación numérica, pero hacia la derecha en el vector de coordenadas.

Toda la especificación de Classic McEliece [10] y los códigos de este libro respetan estrictamente esta correspondencia.

3.3 Frobenius, traza y conjugados

Proposición 3.6: El automorfismo de Frobenius

La aplicación $\varphi\colon \mathbb{F}_{2^m} \to \mathbb{F}_{2^m}$, $\varphi(x) = x^2$, es un automorfismo de cuerpos que fija exactamente a $\mathbb{F}_2$. Los *conjugados* de un elemento β son $\beta, \beta^2, \beta^4, \dots, \beta^{2^{m-1}}$ (con repeticiones si la órbita es más corta), y $\varphi^m(\beta) = \beta^{2^m} = \beta$: tras m pasos el ciclo se cierra. El producto de los m conjugados (con multiplicidades) es la norma $\beta^{1+2+\cdots+2^{m-1}} = \beta^{2^m-1}$, que vale 1 para $\beta \neq 0$.

Demostración. $(x+y)^2 = x^2 + y^2$ y $(xy)^2 = x^2y^2$ en característica 2, luego φ es morfismo inyectivo y, por ser finito, biyectivo. Los elementos fijos de φ son raíces de $X^2 - X$, es decir, 0 y 1. La órbita de $\beta \neq 0$ bajo φ es un ciclo porque $\beta^{2^m} = \beta$ (grupo multiplicativo de orden $2^m - 1$). □

La traza linealiza el cuerpo hasta $\mathbb{F}_2$ y será la herramienta para decidir irreducibilidad de cuadráticas:

Definición 3.7: Traza

$$\mathrm{Tr}(\beta) = \beta + \beta^2 + \beta^4 + \cdots + \beta^{2^{m-1}} \in \mathbb{F}_2.$$

Proposición 3.8: Propiedades de la traza

Tr es $\mathbb{F}_2$-lineal; $\mathrm{Tr}(\beta^2) = \mathrm{Tr}(\beta)$; y para $a \in \mathbb{F}_2$, $\mathrm{Tr}(a) = ma$ mód 2.

Demostración. Ejercicio para el lector; la linealidad sale de que cada término $\beta \mapsto \beta^{2^i}$ es lineal en $\mathbb{F}_2$. □

Teorema 3.9: Criterio de irreducibilidad para cuadráticas

Sea $\beta \in \mathbb{F}_{2^m}$. El polinomio $x^2 + x + \beta$ es irreducible sobre $\mathbb{F}_{2^m}$ si y solo si $\mathrm{Tr}(\beta) = 1$ (si $\beta = 0$ es reducible: $x(x+1)$).

Bosquejo. Si existe raíz γ (es decir, $\gamma^2 + \gamma = \beta$), entonces $\mathrm{Tr}(\beta) = \mathrm{Tr}(\gamma^2) + \mathrm{Tr}(\gamma) = 0$ por 3.8. Recíprocamente, si $\mathrm{Tr}(\beta) = 0$ y $\beta \neq 0$, la ecuación $x^2 + x = \beta$ tiene solución en $\mathbb{F}_{2^m}$ (argumento clásico de conteo: la traza es la suma de los conjugados y el mapa $x \mapsto x^2 + x$ tiene núcleo $\mathbb{F}_2$; véase [52]). □

Ejemplo 3.10: Traza en $\mathbb{F}_{2^4}$

$\mathrm{Tr}(z^3) = z^3 + z^6 + z^{12} + z^9$; con la tabla, z^3=0x8, z^6=0xC, z^{12}=0xF, z^9=0xA, y 0x8 $\oplus$ 0xC $\oplus$ 0xF $\oplus$ 0xA = 0x1 = 1. Por el teorema 3.9, $x^2 + x + z^3$ es irreducible: es el g del juguete.

3.4 Raíces cuadradas: cuadrar es fácil, y también su inversa

Proposición 3.11: Raíz cuadrada en característica 2

El mapa $x \mapsto x^2$ es $\mathbb{F}_2$-lineal y biyectivo. Su inversa es la raíz cuadrada, que se calcula como

$$\sqrt{x} \;=\; x^{\,2^{m-1}}.$$

Demostración. $(x^2)^{2^{m-1}} = x^{2^m} = x$ para todo x (el grupo multiplicativo tiene orden $2^m - 1$, y el cero se comprueba aparte). □

Ejemplo 3.12: Raíz cuadrada en $\mathbb{F}_{2^4}$

$\sqrt{1+z}$: como $1+z = z^4$, la raíz es z^2, y $(z^2)^2 = z^4 = 1+z$. Con la fórmula: $(z^4)^8 = z^{32} = z^2$ (los exponentes se reducen módulo 15).

Este detalle, que parece anecdótico, es el corazón del *decodificador de Patterson* (capítulo 5): en característica 2 sacar raíces cuadradas de polinomios es tan barato como calcular cuadrados, y eso hace que la decodificación de Goppa sea eficiente.

3.5 Cómo se muestrean cuerpos finitos en la práctica

La semántica abstracta («elige un polinomio irreducible al azar, un orden al azar de los elementos») se implementa en Classic McEliece con reglas precisas que conviene conocer, porque reaparecen en la generación de claves (capítulo 9):

- **Elementos a partir de bits.** Los m bits de la convención anterior se extraen directamente del flujo pseudoaleatorio (PRG basado en SHAKE256 y la semilla δ).
- **Polinomio g (algoritmo Irreducible).** Se genera un elemento β de una extensión de grado t y se calcula su polinomio mínimo sobre $\mathbb{F}_q$; si el grado es t, es irreducible y se devuelve g. Si no, se reintenta. La probabilidad de éxito en cada intento es mayor que 0,99 para $t \geq 16$ (proposición 9.2), así que en la práctica se acierta al primer intento.
- **Orden de la lista L (algoritmo FieldOrdering).** Se sortean q enteros de $\sigma_2 = 32$ bits (uno por elemento), se ordenan los pares (a_i, i) y resulta la permutación π con $a_{\pi(0)} < \cdots < a_{\pi(q-1)}$; se define $\alpha_i = \sum_j \pi(i)_j \, z^{m-1-j}$, donde $\pi(i)_j$ es el bit j-ésimo del índice $\pi(i)$. La permutación se guarda comprimida como los bits de control de una red de Benes (sección 9.2, en el capítulo 9).

Observación 3.13: El sentido del «exponente invertido»

El uso de z^{m-1-j} (bit j del entero $\mapsto$ exponente $m-1-j$) hace que α_i *no* sea la representación numérica del índice $\pi(i)$: los bits se invierten (si π fuera la identidad, L empezaría en el cero y seguiría con el elemento de representación 2^{m-1}). Este detalle está en la especificación [10] y afecta a la reproducibilidad exacta de las claves; en nuestros ejemplos de juguete usamos el orden natural, lo que señalaremos donde importe.

Por qué importa el muestreo

Dos implementaciones que «hagan lo mismo» pero muestreen el orden de L de forma distinta producen claves *incompatibles*. La especificación dedica secciones enteras (representación

de polinomios, orden de cuerpos, redes de Benes) a eliminar cualquier ambigüedad: la reproducibilidad byte a byte de las claves es un requisito de interoperabilidad, no una manía.

3.6 Ejercicios

1. Demuestre que en $\mathbb{F}_{2^4}$, $z^5 = z + z^2$ y calcule z^{-5}.
2. Verifique con la tabla que $\mathrm{Tr}(1) = 0$ y $\mathrm{Tr}(z) = z + z^2 + z^4 + z^8$. ¿Cuántos elementos de $\mathbb{F}_{2^4}$ tienen traza 1?
3. Calcule $\sqrt{z^3 + z^2}$ y $\sqrt{z^2 + z + 1}$ en $\mathbb{F}_{2^4}$.
4. Demuestre que $\mathrm{Tr}(\beta^2) = \mathrm{Tr}(\beta)$ y que Tr es $\mathbb{F}_2$-lineal, usando la definición.
5. Explique por qué $x^2 + x + 1$ es irreducible sobre $\mathbb{F}_2$ pero $x^2 + x$ no. ¿Qué dice el criterio de la traza en cada caso?
6. (Síntesis) Enumere las cuatro operaciones que Patterson necesitará sobre $\mathbb{F}_{2^m}$: suma, producto, inverso y raíz cuadrada. ¿Cuál es más cara de implementar en hardware y por qué?

CAPÍTULO 4

Códigos de Goppa: el corazón del esquema

En este capítulo

Este capítulo define los códigos de Goppa binarios, demuestra que la matriz de evaluación $t \times n$ sobre $\mathbb{F}_{2^m}$ es una matriz de paridad, fija los parámetros ($k \geq n - mt$, $d \geq 2t+1$ para g libre de cuadrados) y explica por qué esta familia concreta —y no otras— ha resistido desde 1970. El capítulo cierra con el puente hacia el criptosistema: la matriz sistemática T, que será la clave pública.

4.1 La definición de Goppa

La idea de V. D. Goppa (1970) fue construir un código binario mediante condiciones de divisibilidad sobre un cuerpo mayor. Fijemos:

Definición 4.1: Código de Goppa

Sea $q = 2^m$, sea $g \in \mathbb{F}_q[x]$ mónico, irreducible de grado t, y sea $L = (\alpha_0, \ldots, \alpha_{n-1})$ una lista de $n \leq q$ elementos *distintos* de $\mathbb{F}_q$ con $g(\alpha_i) \neq 0$ para todo i. Sea

$$A(x) = \prod_{j=0}^{n-1} (x - \alpha_j).$$

El **código de Goppa q-ario** es

$$\Gamma_q(L, g) = \Big\{ c \in \mathbb{F}_q^n \;:\; \sum_{i=0}^{n-1} c_i \frac{A(x)}{x - \alpha_i} \;\equiv\; 0 \pmod{g(x)} \Big\},$$

y el **código de Goppa binario** es la parte binaria, $\Gamma_2(L, g) = \Gamma_q(L, g) \cap \mathbb{F}_2^n$.

Cómo leer la definición

El polinomio $A(x)/(x-\alpha_i)$ es el «selector» de la posición i: vale $A'(\alpha_i) \neq 0$ en α_i y 0 en todos los demás α_j. La suma $\sum_i c_i A/(x-\alpha_i)$ es, por tanto, un polinomio de grado $\leq n-1$ que codifica c; la condición dice que g lo divide. Dividir por $x - \alpha_i$ es legítimo porque $g(\alpha_i) \neq 0$: el denominador nunca se anula *módulo* g.

4.2 La matriz de paridad: evaluar en los α_i

La definición anterior es elegante pero incómoda para calcular. El primer resultado sustituye la condición polinómica por t ecuaciones lineales concretas sobre $\mathbb{F}_q$.

Proposición 4.2: La matriz de evaluación es de paridad

Con las hipótesis de la definición 4.1, el código Γ_q es el núcleo de la matriz $t \times n$

$$\tilde{\mathbf{H}} = \left(\tilde{h}_{j,i}\right)_{0\le j<t,\ 0\le i<n}, \qquad \tilde{h}_{j,i} = \frac{\alpha_i^j}{g(\alpha_i)}.$$

En particular, expandiendo cada entrada $\tilde{h}_{j,i} \in \mathbb{F}_q$ en sus m bits se obtiene una matriz binaria $\hat{\mathbf{H}}$ $(mt \times n)$ de paridad para Γ_2.

Demostración. Como $\gcd(A, g) = 1$ (los α_i no son raíces de g), multiplicar por A es invertible módulo g, así que la condición de la definición 4.1 equivale a

$$R_c(x) := \sum_{i=0}^{n-1} \frac{c_i}{x - \alpha_i} \equiv 0 \pmod{g(x)}. \tag{4.1}$$

Para cada i, define $Q_i(x) = \dfrac{g(x) - g(\alpha_i)}{x - \alpha_i} = \sum_{k=1}^{t} g_k (x^{k-1} + x^{k-2}\alpha_i + \cdots + \alpha_i^{k-1})$, polinomio de grado $\le t-1$. Entonces $(x - \alpha_i)Q_i(x) \equiv -g(\alpha_i) \pmod{g(x)}$ y, por tanto,

$$\frac{1}{x - \alpha_i} \equiv -\frac{Q_i(x)}{g(\alpha_i)} \pmod{g(x)}.$$

Sustituyendo en (4.1): $R_c \equiv 0$ si y solo si el polinomio $\sum_i c_i Q_i(x)/g(\alpha_i)$ de grado $\le t-1$ es *idénticamente* cero. Mirando el coeficiente de x^{t-1-j} se obtiene la forma lineal $\sum_i c_i\, v_j(\alpha_i)/g(\alpha_i)$, donde $v_j(y) = \sum_{l=0}^{j} g_{t-j+l}\, y^l$ es un polinomio mónico de grado exactamente j (en particular $v_0 = g_t = 1$ y $v_{t-1} = (g(y) - g(0))/y$). El sistema en las t formas v_j es triangular respecto de las potencias $\alpha_i^0, \ldots, \alpha_i^{t-1}$ (porque $g_t = 1$), luego ambas familias de ecuaciones tienen el mismo núcleo. La última afirmación es inmediata: cada ecuación sobre $\mathbb{F}_q$ son m ecuaciones binarias una vez elegida la base (la representación en bits de la definición 3.1). □

La figura 4.1 resume el mecanismo, y la matriz del juguete (capítulo 5) muestra los números.

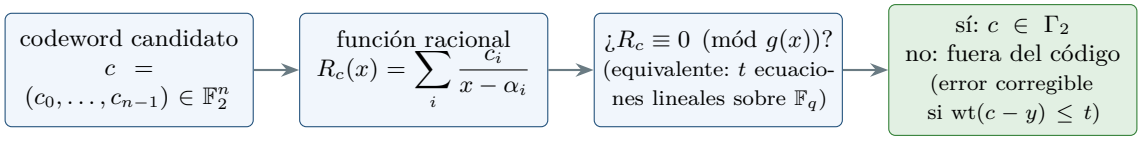


Figura 4.1: El código de Goppa como «máquina de verificación»: c pertenece al código si su función racional asociada es divisible por g. La proposición 4.2 convierte esta condición en t ecuaciones lineales sobre $\mathbb{F}_q$ (mt binarias).

4.3 Parámetros: dimensión y distancia

Teorema 4.3: Parámetros de un código de Goppa binario

Sea $\Gamma_2(L, g) = [n, k, d]$ como en la definición 4.1, con g libre de cuadrados (en particular, g irreducible como en Classic McEliece). Entonces

$$k \ge n - mt, \qquad d \ge 2t + 1.$$

Presentamos la cota de dimensión y la mitad «fácil» de la cota de distancia, y explicamos con honestidad qué es clásico y qué se demuestra.

Demostración(dimensión). $\hat{\mathbf{H}}$ tiene mt filas, luego su núcleo tiene dimensión al menos $n-mt$; ese núcleo *es* Γ_2 por la proposición 4.2. (Para $n=q$ y g irreducible, se tiene de hecho $k=n-mt$; en el juguete, $8=16-2\cdot 4$.) □

Demostración(distancia, paso $t+1$). Sea $c\in\Gamma_2$ no nula, $I=\operatorname{supp}(c)$, $w=|I|$, y $V_I(x)=\prod_{i\in I}(x-\alpha_i)$. Multiplicando (4.1) por V_I,

$$\sum_{i\in I}\frac{V_I(x)}{x-\alpha_i}=V_I'(x)\ \equiv\ 0\pmod{g(x)},$$

donde V_I' es la derivada formal (definida algebraicamente por $(x^k)'=kx^{k-1}$, cumpliendo la regla del producto), es decir, $g\mid V_I'$. Pero V_I es un producto de factores simples, así que $V_I'\neq 0$ y $\deg V_I'\le w-1$ (en característica 2, la derivada de un producto simple baja un grado más). Si $w\le t$ tendríamos $0\neq\deg V_I'<t=\deg g$, imposible que g lo divida. Luego $w\ge t+1$ para todo codeword no nulo: $d\ge t+1$. □

Teorema 4.4: La versión clásica completa

Con las hipótesis del teorema 4.3, de hecho $d\ge 2t+1$.

La mejora de $t+1$ a $2t+1$ es el hecho profundo de la teoría de códigos de Goppa binarios. La razón es una identidad clásica de característica 2 *(identidad de van Lint)*: si g es libre de cuadrados,

$$\Gamma_2(L,g)=\Gamma_2(L,g^2). \tag{4.2}$$

El código de la derecha tiene módulo de grado $2t$, y el «paso fácil» aplicado a g^2 —una vez verificado que g^2 factoriza en factores simples *para este propósito*— da $d\ge 2t+1$. La identidad no es trivial: expresa que en característica 2 el cuadrado del síndrome racional de un codeword binario vuelve a ser un síndrome del mismo código con g reemplazado por g^2. En el laboratorio la comprobaremos exhaustivamente (es exacta para el juguete: 256 palabras por cada lado); la demostración estándar puede verse en MacWilliams y Sloane [53] o van Lint [71], y la usamos tal cual.

Corolario 4.5: Se corrige cualquier error de peso $\le t$

Todo vector de $\mathbb{F}_2^n$ está a distancia $\le t$ de, a lo sumo, un codeword de $\Gamma_2(L,g)$.

Demostración. Si $\mathrm{d}(y,c_1),\mathrm{d}(y,c_2)\le t$ con $c_1\neq c_2\in\Gamma_2$, entonces $\mathrm{wt}(c_1-c_2)\le 2t<d$, contradicción con $d\ge 2t+1$. □

⚖ Qué es teorema y qué es costumbre

La cota $d\ge 2t+1$ es un teorema con 50 años de antigüedad (Goppa, van Lint; véanse los libros citados), no una conjetura. La afirmación «$k=n-mt$» en cambio es *típica*, no universal: depende de que $\hat{\mathbf{H}}$ tenga rango máximo, cosa que la especificación maneja explícitamente en MatGen (y que en la práctica siempre ocurre con los parámetros elegidos). Distinguimos ambas cosas porque en la parte IV algunos ataques explotan precisamente irregularidades estructurales de los códigos, no violaciones de estas cotas.

Ejemplo 4.6: El juguete, anunciado

Con $m = 4$ ($q = 16$), $t = 2$, $g = x^2 + x + z^3$ y $L = (0, 1, z, 1 + z, \dots)$:

$$n = 16, \qquad k = 16 - 2 \cdot 4 = 8, \qquad d \geq 5,$$

y una comprobación exhaustiva da $d = 5$ exactamente, $|\Gamma_2| = 256$ y unicidad de la decodificación para todos los errores de peso ≤ 2. Es el código que estudiaremos pieza a pieza en el capítulo 5.

4.4 ¿Por qué Goppa binario y no otra cosa?

Conviene decirlo pronto, porque es la pregunta que todo el mundo hace: existen muchas familias de códigos con decodificador eficiente, y casi todas las que se probaron en esquemas tipo McEliece acabaron rompiéndose o perdiendo sus ventajas (Reed–Solomon, códigos AG, quasi-cíclicos, rango, ...). Los códigos de Goppa *binarios* tienen tres propiedades que explican su resistencia:

1. **Densidad de códigos buenos.** Para $n = q$ y casi cualquier g irreducible, los parámetros alcanzan la cota de Gilbert–Varshamov: hay muchos códigos con distancia relativa excelente. El atacante no puede «adivinar» una estructura débil: no la hay.
2. **La identidad (4.2) y la rigidez en característica 2.** Los ataques algebraicos que derriban variantes sobre $\mathbb{F}_q$ con $q > 2$ (capítulo 15) se estrellan en el caso binario: el «cuadrado de Schur» del código no escapa de la familia de Goppa, así que no delata el polinomio secreto.
3. **Casi 50 años de escrutinio.** Desde 1978 se han publicado cientos de artículos intentando romper McEliece con Goppa binario; ninguno ha degradado la seguridad más allá de factores polinómicos o mejoras de decodificación (capítulos 14 y 16).

Goppa, antes de McEliece

Los códigos de Goppa (1970) son un poco mayores que el criptosistema: V. D. Goppa los introdujo como códigos correctores «algebraicos» con una cota de distancia inusualmente buena para su tasa [40]. McEliece tuvo la ocurrencia, ocho años después, de que un código con decodificador eficiente *y* matriz generadora indistinguible de aleatoria valía como clave pública. Los códigos de Goppa cumplían ambas cosas entonces y las siguen cumpliendo hoy; el resto de la historia es de la parte II.

4.5 Ejercicios

1. ¿Por qué la definición exige $g(\alpha_i) \neq 0$? Muestre que si algún $g(\alpha_i) = 0$, la definición de Γ_q se *degenera* (todas las condiciones se anulan en la posición i).
2. Compruebe que $A(x)/(x - \alpha_i)$ evaluado en α_i vale $A'(\alpha_i) = \prod_{j \neq i}(\alpha_i - \alpha_j)$.
3. Para el juguete ($t = 2$), escriba la matriz $\tilde{\mathbf{H}}$ de 2×16 sobre $\mathbb{F}_{2^4}$ como en la especificación: filas $\alpha_i^0/g(\alpha_i)$ y $\alpha_i^1/g(\alpha_i)$. (La solución está en la salida del laboratorio, pero intente calcular la columna $i = 0$ y la columna $i = 1$ a mano.)

4. Deduzca del teorema 4.3 que *ningún* codeword no nulo tiene peso $\leq 2t$. Explique por qué esto no contradice que d pueda ser mayor que $2t+1$.

5. (Conceptual) Un estudiante propone cambiar g irreducible por g con raíces repetidas ($g = h^2$). ¿Qué le respondió el teorema 4.4 y por qué McEliece nunca lo intentó?

6. (Síntesis) Escriba en cinco líneas la ruta completa: bits $\to g, L \to \Gamma_2 \to \hat{\mathbf{H}} \to$ forma sistemática $(\mathrm{I} \,|\, T)$. ¿Qué pieza de todas ellas se publica y cuál se mantiene secreta?

CAPÍTULO 5

Laboratorio 1: el juguete completo

En este capítulo

En este capítulo ejecutamos, con todos sus números, el código de Goppa miniatura anunciado en los capítulos 3 y 4: $m = 4$, $n = 16$, $t = 2$ y $g(x) = x^2 + x + z^3$. Construimos las matrices $\tilde{\mathbf{H}}$, $\hat{\mathbf{H}}$ y la forma sistemática $(\mathrm{I}_8 \,|\, \mathbf{T})$; calculamos el síndrome de un error de dos posiciones; lo decodificamos paso a paso con Patterson; verificamos exhaustivamente la corrección (120/120 errores de peso 2) y la identidad de van Lint sobre las 65 536 palabras de $\mathbb{F}_2^{16}$; y terminamos cifrando y descifrando un mensaje con el McEliece de 1978 en miniatura. Cada cifra de este capítulo es reproducible con el script del apéndice A.

Lectura y reproducibilidad

El script `toy.py` (apéndice A) imprime todos los números de este capítulo, en el mismo orden en que aparecen. Tres convenciones de lectura, avisadas una sola vez:

- Los **elementos de** $\mathbb{F}_{16}$ se escriben con un *dígito* hexadecimal, como en la tabla de potencias del capítulo 3: $9 = 1 + z^3$, $a = z + z^3$, $d = 1 + z^2 + z^3$, $f = 1 + z + z^2 + z^3$.
- Los **vectores de bits** se escriben con la *posición* 0 *a la izquierda*, en el mismo orden que las columnas j de las matrices y que la lista L: el error con unos en las posiciones 10 y 13 es `0000000000100100`.
- Los **polinomios** sobre $\mathbb{F}_{16}$ se escriben con sus coeficientes en ese mismo dígito hexadecimal: $S(x) = \mathtt{4} + \mathtt{9}x$ significa $S(x) = z^2 + (1 + z^3)x$.

5.1 El montaje: un código de Goppa de longitud 16

El escenario es el del capítulo 3:

$$\mathbb{F}_{16} = \mathbb{F}_2[z]/(z^4 + z + 1),$$

con z primitivo: sus potencias recorren el grupo multiplicativo, de orden 15. El polinomio de Goppa del juguete es

$$g(x) = x^2 + x + z^3,$$

irreducible porque $\mathrm{Tr}(z^3) = z^3 + z^6 + z^{12} + z^9 = 1$ (ejemplo 3.10 y criterio 3.9). El script comprueba además que $g(\alpha) \neq 0$ para todo $\alpha \in \mathbb{F}_{16}$, que es la condición de la definición 4.1. Como puntos de evaluación tomamos la lista completa del cuerpo en orden natural,

$$L = (0,\ 1,\ z,\ 1+z,\ z^2,\ 1+z^2,\ z+z^2,\ 1+z+z^2,\ z^3,\ \ldots,\ 1+z+z^2+z^3),$$

y los parámetros son

$$n = q = 16, \qquad t = 2, \qquad mt = 8, \qquad k = n - mt = 8, \qquad d \geq 2t + 1 = 5.$$

La tabla 5.1 contiene, columna a columna, todo lo necesario para reconstruir las matrices: cada fila es una columna de $\tilde{\mathbf{H}}$, es decir, el par $(1/g(\alpha_j),\ \alpha_j/g(\alpha_j))$.

j	α_j	$g(\alpha_j)$	$(1/g,\ \alpha_j/g)$
0	0	z^3	(f, 0)
1	1	z^3	(f, f)
2	z	$z+z^2+z^3$	(3, 6)
3	$1+z$	$z+z^2+z^3$	(3, 5)
4	z^2	$1+z+z^2+z^3$	(8, 6)
5	$1+z^2$	$1+z+z^2+z^3$	(8, e)
6	$z+z^2$	$1+z^3$	(2, c)
7	$1+z+z^2$	$1+z^3$	(2, e)

j	α_j	$g(\alpha_j)$	$(1/g,\ \alpha_j/g)$
8	z^3	z^2+z^3	(a, f)
9	$1+z^3$	z^2+z^3	(a, 5)
10	$z+z^3$	$z+z^3$	(c, 1)
11	$1+z+z^3$	$z+z^3$	(c, d)
12	z^2+z^3	$1+z+z^3$	(5, 9)
13	$1+z^2+z^3$	$1+z+z^3$	(5, c)
14	$z+z^2+z^3$	$1+z^2+z^3$	(4, d)
15	$1+z+z^2+z^3$	$1+z^2+z^3$	(4, 9)

Cuadro 5.1: Las 16 columnas del código de Goppa del juguete: valores $g(\alpha_j)$ y los dos síndromes sobre $\mathbb{F}_{16}$, $1/g(\alpha_j)$ y $\alpha_j/g(\alpha_j)$. Todas las cifras provienen de `toy.py`.

Ejemplo 5.1: Dos columnas a mano

Comprobemos la columna $j=0$ ($\alpha_0=0$) y la $j=1$ ($\alpha_1=1$), como pedía el ejercicio del capítulo 4. Para $j=0$:

$$g(0)=z^3, \qquad \frac{1}{g(0)}=z^{-3}=z^{12}=1+z+z^2+z^3=\mathtt{f}, \qquad \frac{\alpha_0}{g(\alpha_0)}=0.$$

Para $j=1$, en característica 2, $g(1)=1+1+z^3=z^3$, así que $\frac{1}{g(1)}=\mathtt{f}$ y $\frac{\alpha_1}{g(\alpha_1)}=1\cdot\mathtt{f}=\mathtt{f}$. La tabla 5.1 muestra, en efecto, primero la columna (f, 0) y después (f, f).

Observación 5.2: Una simetría de la tabla

Como $g(x+1)=(x+1)^2+(x+1)+z^3=x^2+x+z^3=g(x)$, los pares $\alpha_{2k},\alpha_{2k+1}$ (que difieren en 1) comparten g y sus columnas difieren solo en la segunda fila: $\frac{\alpha+1}{g(\alpha)}=\frac{\alpha}{g(\alpha)}+\frac{1}{g(\alpha)}$. Compruébelo en la tabla, por ejemplo en $j=2,3$: $\mathtt{6}\oplus\mathtt{3}=\mathtt{5}$.

5.2 Las matrices: $\tilde{\mathbf{H}}$, $\hat{\mathbf{H}}$ y la forma sistemática

Con la convención de la especificación, la fila i de $\tilde{\mathbf{H}}$ ($i=0,1$) tiene entradas $\alpha_j^i/g(\alpha_j)$: exactamente lo que lista la tabla 5.1. En hexadecimal (columnas $j=0,\dots,15$):

$\tilde{\mathbf{H}}=$

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$i=0$	f	f	3	3	8	8	2	2	a	a	c	c	5	5	4	4
$i=1$	0	f	6	5	6	e	c	e	f	5	1	d	9	c	d	9

Cada entrada es un elemento de $\mathbb{F}_{16}$ y por tanto cuatro bits, con el bit ℓ = coeficiente de z^ℓ (la convención del capítulo 3). Expandiendo cada columna en sus bits y apilando las filas de $i=0$ encima de las de $i=1$ se obtiene $\hat{\mathbf{H}}$, la matriz binaria de 8×16 que muestra la figura 5.1. El rango de $\hat{\mathbf{H}}$ es 8: el código $\Gamma_2=\ker\hat{\mathbf{H}}$ tiene dimensión $k=8$, como manda el teorema 4.3.

De aquí en adelante identificamos el código con su versión sistemática: un vector $\mathbf{c}=(w\,|\,v)$ con $w,v\in\mathbb{F}_2^8$ pertenece a Γ_2 si y solo si $w=\mathbf{T}v$ (vectores columna), o, en convención de fila, si $\mathbf{c}=\mathbf{u}\,\mathbf{G}$ con $\mathbf{G}=(\mathbf{T}^\top\,|\,\mathrm{I}_8)$ (ecuación (5.4) más abajo).

Para terminar de conocer el juguete, el script verifica por fuerza bruta las 65 536 palabras de $\mathbb{F}_2^{16}$:

- $|\Gamma_2|=256=2^8$: la dimensión efectiva es $k=8$.
- $d_{\text{mín}}=5$ exactamente, igual que la cota $2t+1$; la figura 5.2 muestra el histograma completo de pesos. No hay palabras de peso $1,\dots,4$, y el peso mínimo se alcanza 24 veces.

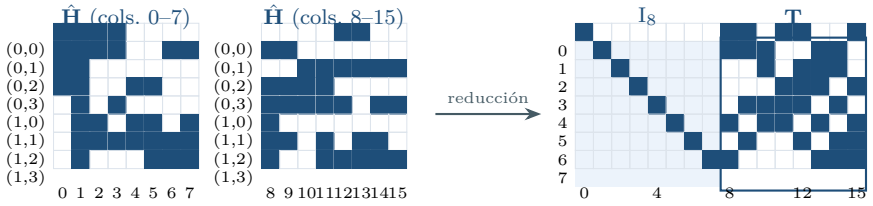


Figura 5.1: La matriz de paridad binaria $\hat{\mathbf{H}}$ del juguete, en dos mitades (la fila (i, ℓ) contiene el bit ℓ —el coeficiente de z^ℓ— de la entrada $\alpha_j^i / g(\alpha_j)$), y su forma sistemática $(\mathrm{I}_8 \,|\, \mathbf{T})$ obtenida por operaciones de fila, que no cambian el núcleo. La mitad derecha $\mathbf{T}$ (8×8) es la misma que aparecerá como clave pública en el laboratorio 2.

- Los síndromes de los $1 + 16 + 120 = 137$ errores de peso ≤ 2 son todos distintos entre sí: en este código, cada error de peso $\leq t$ es la *única* solución de su síndrome. Solo 137 de los 256 síndromes posibles vienen de un error de peso ≤ 2 (esta asimetría reaparecerá en el laboratorio 2).

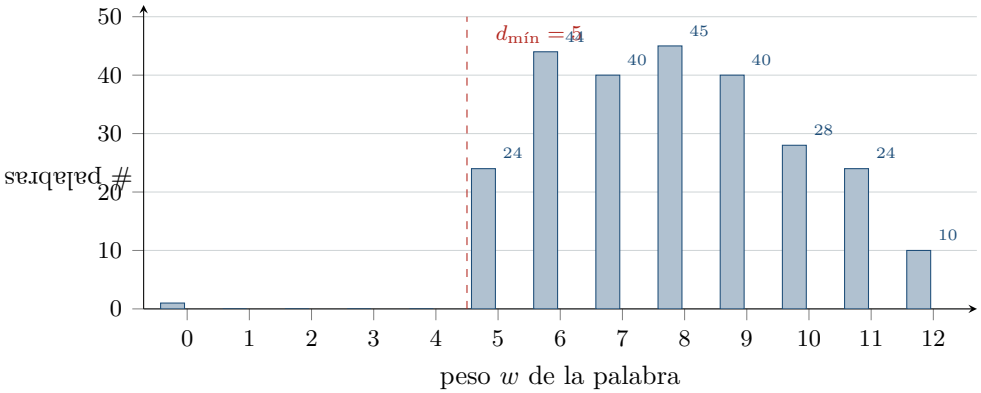


Figura 5.2: Histograma de pesos de los 256 codewords del juguete (65 536 palabras examinadas). El hueco de pesos 1–4 refleja $d_{\text{mín}} = 5 = 2t + 1$: el código corrige $t = 2$ errores.

5.3 El síndrome de un error de dos posiciones

Elegimos el error que nos acompañará durante todo el capítulo:

$$e = e_{10} + e_{13} = \texttt{0000000000100100} \qquad \text{(unos en las posiciones 10 y 13).}$$

Sus dos columnas en $\tilde{\mathbf{H}}$ son, según la tabla 5.1,

$$\mathrm{col}_{10} = \begin{pmatrix} \texttt{c} \\ \texttt{1} \end{pmatrix}, \qquad \mathrm{col}_{13} = \begin{pmatrix} \texttt{5} \\ \texttt{c} \end{pmatrix},$$

y sumando coordenada a coordenada sobre $\mathbb{F}_{16}$:

$$\tilde{\mathbf{H}}e = \begin{pmatrix} \texttt{c} \oplus \texttt{5} \\ \texttt{1} \oplus \texttt{c} \end{pmatrix} = \begin{pmatrix} \texttt{9} \\ \texttt{d} \end{pmatrix} = (1 + z^3,\ 1 + z^2 + z^3).$$

En bits, descomponiendo según la base $\{1, z, z^2, z^3\}$ de menor a mayor grado $(1 + z^3 \mapsto (1, 0, 0, 1)$ y $1 + z^2 + z^3 \mapsto (1, 0, 1, 1))$, el síndrome binario es el vector `10011011`, en el orden de filas

$(0,0),(0,1),(0,2),(0,3),(1,0),\ldots$ que fija la figura 5.1 (nótese que al escribir el entero `d` en binario habitual con el bit más significativo a la izquierda se lee 1101_2, mientras que como vector de coordenadas el coeficiente de z^0 ocupa la posición inicial $(1,0,1,1)$).

Para decodificar, Patterson no usa $\tilde{\mathbf{H}}e$ directamente sino el *síndrome racional*

$$S(x) = \sum_{j\in\{10,13\}} \frac{1}{x+\alpha_j} \pmod{g(x)},$$

la misma cantidad de la ecuación (4.1) (en característica 2, $x-\alpha = x+\alpha$). Para calcularla, obsérvese que

$$g(x)+g(\alpha) = (x^2+x+z^3)+(\alpha^2+\alpha+z^3) = (x+\alpha)^2+(x+\alpha) = (x+\alpha)(x+\alpha+1),$$

y como $g(x)\equiv 0 \pmod{g}$, resulta $g(\alpha)\equiv (x+\alpha)(x+\alpha+1) \pmod{g}$; dividiendo por $(x+\alpha)\,g(\alpha)$:

$$\frac{1}{x+\alpha} \equiv \frac{x+\alpha+1}{g(\alpha)} \pmod{g(x)}. \tag{5.1}$$

Sumando sobre las dos posiciones del error:

$$S(x) = x\underbrace{\sum_{j\in E}\frac{1}{g(\alpha_j)}}_{=:\,s_0} + \underbrace{\sum_{j\in E}\frac{\alpha_j+1}{g(\alpha_j)}}_{=:\,s_0+s_1},$$

es decir, con $s_0 = \mathtt{9}$ y $s_1 = \mathtt{d}$ (las componentes de $\tilde{\mathbf{H}}e$):

$$S(x) = s_0x + (s_0+s_1) = \mathtt{9}x + (\mathtt{9}\oplus\mathtt{d}) = \mathtt{9}x+\mathtt{4} = z^2+(1+z^3)x. \tag{5.2}$$

El síndrome no es $S(x)$

$\tilde{\mathbf{H}}e = (s_0,s_1) = (\mathtt{9},\mathtt{d})$ y $S(x) = \mathtt{4}+\mathtt{9}x$ son dos representaciones del mismo dato. La primera es el vector de síndrome (8 bits; en el laboratorio 2 será el criptograma); la segunda es el polinomio módulo g que necesita Patterson. Se relacionan por una transformación lineal fija —aquí, $S(x) = s_0x+(s_0+s_1)$—, de modo que los coeficientes de S *no* son (s_0,s_1): confundirlos daría una decodificación sin sentido.

5.4 Decodificación de Patterson, paso a paso

La derivada formal en característica 2 y la descomposición A^2+xB^2

Sobre cualquier cuerpo finito, la derivada de un polinomio se define formalmente mediante la regla algebraica $(x^k)' = kx^{k-1}$, extendida por linealidad. En característica 2 ($2=0$), las potencias pares se anulan al derivar: $(x^{2k})' = 2kx^{2k-1} = 0$, mientras que las impares dan $(x^{2k+1})' = (2k+1)x^{2k} = x^{2k} = (x^k)^2$.

Además, por el «sueño del novato» en característica 2, elevar al cuadrado conmuta con la suma: $(u+v)^2 = u^2+v^2$. Agrupando cualquier polinomio $\sigma(x)$ en sus términos de grado par y de grado impar, y extrayendo la raíz cuadrada de sus coeficientes (que en $\mathbb{F}_{2^m}$ siempre existe y es única por la proposición 3.11), se obtienen dos polinomios únicos $A(x)$ y $B(x)$ tales que $\sigma(x) = A(x)^2 + xB(x)^2$. Al derivar:

$$\sigma'(x) \;=\; (A(x)^2)' + (xB(x)^2)' \;=\; 0 + 1\cdot B(x)^2 + x\cdot(B(x)^2)' \;=\; B(x)^2.$$

Esta notable simplificación es la base del algoritmo de Patterson.

Sea $E = \{10, 13\}$ y sea $\sigma(x) = \prod_{j\in E}(x + \alpha_j)$ el *localizador del error*. En característica 2, derivar un producto elimina cada factor y deja el producto de los demás:

$$\sigma'(x) = \sum_{j\in E} \prod_{k\neq j} (x + \alpha_k) = \sum_{j\in E} \frac{\sigma(x)}{x + \alpha_j}.$$

Multiplicando la definición de S por σ y reduciendo módulo g aparece la *ecuación clave*:

$$\sigma(x)\, S(x) \equiv \sigma'(x) \pmod{g(x)}. \tag{5.3}$$

Patterson la resuelve así: en característica 2 todo polinomio se escribe como $\sigma = A^2 + xB^2$ (separando coeficientes de grado par e impar con la proposición 3.11), y entonces $\sigma' = B^2$; con $R := S^{-1}$, la ecuación (5.3) equivale a

$$A^2 + xB^2 \equiv R\, B^2 \implies A^2 \equiv B^2\,(x + R) \implies A \equiv B\, W,$$

donde $W^2 \equiv x+R \pmod{g}$. Con $\tau := W^{-1}$, queda por encontrar el par (A, B) tal que $\deg A \leq t/2$, $\deg B \leq (t-1)/2$ y $B \equiv A\,\tau \pmod{g}$; el algoritmo de Euclides parcial lo produce (véase [60]). En el juguete todo se reduce a siete operaciones, resumidas en la figura 5.3 y detalladas aquí con *certificados exactos*: los múltiplos de g que el script calcula sin dejar resto.

Observación 5.3: Patterson general: raíces cuadradas y Euclides con parada temprana

En el juguete ($t = 2$) el proceso termina en una sola división, pero en Classic McEliece ($t = 64, 96, 128$) las etapas clave operan así:

- **Raíz cuadrada $W = \sqrt{x + R}$ mód $g(x)$:** Como g es irreducible de grado t sobre $\mathbb{F}_q$, el anillo $\mathbb{F}_q[x]/(g(x))$ es el cuerpo $\mathbb{F}_{q^t}$. En característica 2, la aplicación $P \mapsto P^2$ mód g es un automorfismo lineal invertible; por tanto, la raíz cuadrada módulo g siempre existe y es única. En las implementaciones se calcula mediante una multiplicación matriz–vector precalculada o por exponenciación.
- **Euclides extendido con parada temprana:** Para hallar (A, B) con $\deg A \leq t/2$ y $\deg B \leq (t-1)/2$ tales que $A \equiv BW \pmod{g}$, se aplica el algoritmo de Euclides extendido a $g(x)$ y $W(x)$. Se detiene en el primer resto r_k cuyo grado cumple $\deg r_k \leq t/2$. El resto r_k y su correspondiente coeficiente de Bézout proporcionan directamente los polinomios A y B.

1. $S(x) = 4 + 9x$ (ecuación (5.2)).
2. $R = S^{-1} = a + 7x$; certificado $SR = 1 + (z + z^3)\, g$.
3. $x + R = a + 6x$.
4. $W = \sqrt{x + R} = c + 7x$; certificado $W^2 = (x + R) + (z + z^2)\, g$.
5. $\tau = W^{-1} = 3 + bx$; certificado $W\tau = 1 + z^2\, g$.
6. Dividiendo g entre τ: cociente $A = 3 + 5x$ y resto $B = d$, de modo que $g = \tau A + B$ *exactamente*; y $\deg A = 1 \leq t/2$, $\deg B = 0 \leq (t-1)/2$.
7. $\sigma = A^2 + xB^2 = (3 + 5x)^2 + x\,(d)^2 = (5 + 2x^2) + ex = 5 + ex + 2x^2$, usando $3^2 = 5$, $5^2 = 2$ y $d^2 = e$: cuadrar en característica 2 duplica los exponentes y eleva al cuadrado los coeficientes.

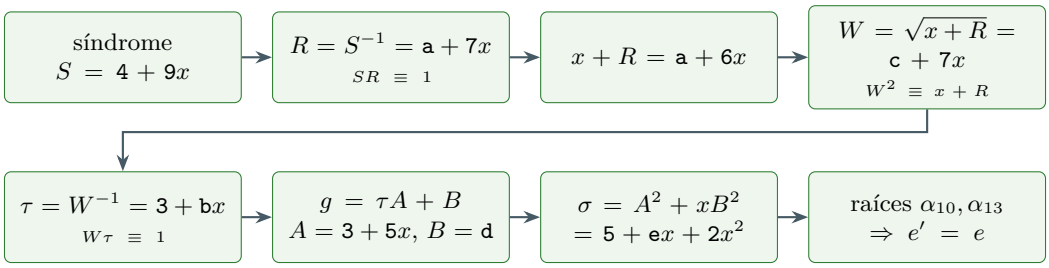


Figura 5.3: La decodificación de Patterson del error $e_{10} + e_{13}$, paso a paso, con los números del juguete. Cada caja lleva su certificado exacto (múltiplo de g) en letra pequeña.

La comprobación final es la ecuación (5.3), que se cumple con holgura:

$$\sigma S = \sigma' + (x + \mathtt{e})\, g, \qquad \sigma' = \mathtt{e}.$$

Evaluando σ en la lista L (tabla 5.2), los únicos ceros son $\sigma(\alpha_{10}) = \sigma(\alpha_{13}) = 0$; el error reconstruido es

$$e' = e_{10} + e_{13} = 0000000000100100 = e.$$

j	α_j	$\sigma(\alpha_j)$
0	0	$1 + z^2$
1	1	$1 + z^3$
2	z	z
3	$1 + z$	$z + z^2 + z^3$
4	z^2	$z + z^2 + z^3$
5	$1 + z^2$	z
6	$z + z^2$	$1 + z^3$
7	$1 + z + z^2$	$1 + z^2$

j	α_j	$\sigma(\alpha_j)$
8	z^3	$1 + z + z^2$
9	$1 + z^3$	$1 + z + z^3$
10	$z + z^3$	0
11	$1 + z + z^3$	$z^2 + z^3$
12	$z^2 + z^3$	$z^2 + z^3$
13	$1 + z^2 + z^3$	0
14	$z + z^2 + z^3$	$1 + z + z^3$
15	$1 + z + z^2 + z^3$	$1 + z + z^2$

Cuadro 5.2: Evaluación del localizador $\sigma(x) = 5 + \mathtt{e}x + 2x^2$ en los 16 puntos de L. Sus raíces (en rojo) señalan las posiciones del error: 10 y 13.

Observación 5.4: La simetría de la tabla de σ

Los valores $\sigma(\alpha_j)$ se repiten en espejo dentro de cada mitad de la tabla 5.2. No es casualidad: la suma de las raíces es $\alpha_{10} + \alpha_{13} = 1 + z + z^2 = \alpha_7$, así que el apoyo del error, $\{10, 13\} = \{10,\ 10 \oplus 7\}$, es invariante bajo $j \mapsto j \oplus 7$; y como en esta lista $\alpha_{j\oplus 7} = \alpha_j + \alpha_7$, resulta $\sigma(x + \alpha_7) = \sigma(x)$ (compruébelo con los coeficientes: $2c^2 + \mathtt{e}\,c = c + c = 0$ para $c = \alpha_7$). Por eso $\sigma(\alpha_j) = \sigma(\alpha_{j\oplus 7})$ y la tabla es palindrómica.

Con esto queda verificado un error. El script repite el procedimiento sobre los $\binom{16}{2} = 120$ errores de peso 2 posibles y verifica el resultado: 120/120 correctos. Los casos límite también están claros: si $S = 0$ no hay error ($e' = 0$), y si el vector recibido no está a distancia $\leq t$ de ningún codeword, las raíces no reconstruyen un error consistente; el decodificador del esquema real lo detecta verificando el resultado (capítulo 10).

La promesa del capítulo 4: van Lint

Para comprobar la identidad $\Gamma_2(L, g) = \Gamma_2(L, g^2)$ —la ecuación (4.2)— el script construye la matriz sobre $\mathbb{F}_{16}$ de 4 filas con entradas $\alpha_j^i / g(\alpha_j)^2$ ($i = 0, \ldots, 3$; 16 filas al expandir en bits) y contrasta las dos pertenencias sobre las 65 536 palabras de $\mathbb{F}_2^{16}$: ambas dan exactamente 256 palabras y coinciden palabra a palabra. Es una corroboración finita de un teorema clásico

(véanse [53, 71]), no una demostración alternativa; lo que sí muestra es que la construcción del juguete es coherente con la teoría del capítulo 4.

5.5 Vista previa: el juguete cifra (McEliece 1978 en miniatura)

Para cerrar la parte I, montamos con estas piezas el criptosistema de 1978 en miniatura (el esquema general se estudia en el capítulo 6). El código es el núcleo de $(\mathrm{I}_8 \,|\, \mathbf{T})$: un vector $\mathbf{c} = (w \,|\, v)$ pertenece a Γ_2 si y solo si $w = \mathbf{T}v$ (vectores columna). En convención de fila, esto es $\mathbf{c} = \mathbf{u}\,\mathbf{G}$ con

$$\mathbf{G} = (\,\mathbf{T}^{\top} \,|\, \mathrm{I}_8\,), \tag{5.4}$$

cuyas filas son $\mathbf{G}_i = (\text{columna } i \text{ de } \mathbf{T},\ \mathbf{e}_i)$:

$$\mathbf{G} = \begin{pmatrix} 1100010110000000 \\ 1100101001000000 \\ 0110110000100000 \\ 1001110000010000 \\ 1011101000001000 \\ 0111010100000100 \\ 0110101100000010 \\ 1001011100000001 \end{pmatrix}.$$

A continuación se eligen una matriz S invertible aleatoria y una permutación σ de las posiciones:

j	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$\sigma(j)$	14	2	7	10	4	8	12	6	9	15	11	5	0	3	13	1

con la convención $(\mathbf{v}P)_j = \mathbf{v}_{\sigma(j)}$: la coordenada antigua $\sigma(j)$ pasa a la posición j. La clave pública del juguete es $\mathbf{G}' = S\,\mathbf{G}\,P$, en la figura 5.4.

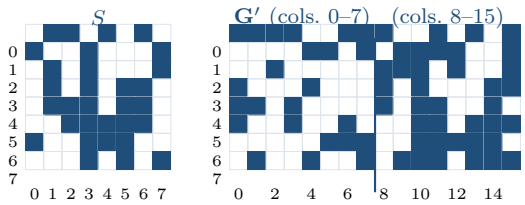


Figura 5.4: La matriz aleatoria invertible S (8×8) y la clave pública del juguete $\mathbf{G}' = S\,\mathbf{G}\,P$ (8×16). La línea gruesa separa las 8 primeras coordenadas de las 8 últimas.

Cifrado. Tomamos el mensaje $\mathbf{u} = 01001101$ (unos en $i = 1, 4, 5, 7$) y el error $\mathbf{z}$ de peso 2 con apoyo $\{3, 12\}$:

$$\mathbf{z} = 0001000000001000.$$

La primera mitad es una suma de filas de $\mathbf{G}'$, $\mathbf{u}\mathbf{G}' = \bigoplus_{i\in\{1,4,5,7\}} \mathbf{G}'_i$:

$$\mathbf{G}'_1 = 0000110101111001, \qquad \mathbf{G}'_4 = 1101000010110010,$$

$$\mathbf{G}'_5 = 1001001000110111, \qquad \mathbf{G}'_7 = 0100100101110111,$$

y sumando las cuatro (equivalentemente, sumando por columnas) más el error:

$$\mathbf{u}\mathbf{G}' = 0000011010001011, \qquad \mathbf{x} = \mathbf{u}\mathbf{G}' + \mathbf{z} = 0001011010000011.$$

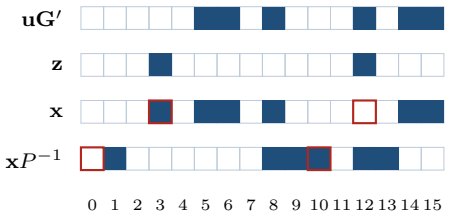


Figura 5.5: El cifrado en tiras: en azul, las posiciones con bit 1; en rojo, las posiciones alteradas por el error. De arriba abajo: $\mathbf{uG'}$ (codeword sin error), el error $\mathbf{z}$, el criptograma $\mathbf{x} = \mathbf{uG'} + \mathbf{z}$ y, tras deshacer P, la palabra recibida $\mathbf{x}P^{-1}$, con el error ya en las posiciones 0 y 10.

La figura 5.5 muestra las tiras de bits de cada paso.

Descifrado. Se deshace la permutación, $(\mathbf{v}P^{-1})_j = \mathbf{v}_{\sigma^{-1}(j)}$:

$$\mathbf{x}P^{-1} = \texttt{0100000011101100}, \qquad \operatorname{supp}(\mathbf{z}P^{-1}) = \{\sigma(3), \sigma(12)\} = \{10, 0\}.$$

Ahora el problema es el de decodificar una palabra a distancia 2 de un codeword. Patterson (la misma rutina de la sección anterior; aquí $S(x) = \texttt{2} + \texttt{3}x$ y $\sigma(x) = \texttt{c}\,x + \texttt{f}x^2$) devuelve

$$\mathbf{z}' = \texttt{1000000000100000} \qquad (\text{apoyo } \{0, 10\}), \qquad \mathbf{c} = \mathbf{x}P^{-1} + \mathbf{z}' = \texttt{1100000011001100}.$$

Los últimos 8 bits de $\mathbf{c}$ son el mensaje «escalado» $\mathbf{u}S = \texttt{11001100}$; deshaciendo S:

$$\mathbf{u} = (\mathbf{u}S)\,S^{-1} = \texttt{01001101},$$

el mensaje original. Fin del juguete: en el laboratorio 2 (capítulo 11) el mismo montaje se convierte en un KEM completo con SHAKE256, y en el capítulo 6 el esquema de 1978 se presenta en toda su generalidad. Advertencia: que este juguete cifre *no* significa que proteja algo; cuantificar por qué es el ejercicio 6 y, en serio, la parte IV.

Del juguete al esquema real

La tabla mental del capítulo: $\mathbb{F}_{16} \leftrightarrow \mathbb{F}_{2^m}$ con $m = 12$ o 13; $\deg g = 2 \leftrightarrow t = 64, \ldots, 128$; los 16 puntos de $L \leftrightarrow$ las $n \approx 8000$ posiciones del orden de cuerpo (*FieldOrdering*); $\tilde{\mathbf{H}}, \hat{\mathbf{H}}$ de 2×16 y $8 \times 16 \leftrightarrow t \times n$ y $mt \times n$; la forma sistemática $(\mathrm{I}_8 \,|\, \mathbf{T}) \leftrightarrow (\mathrm{I}_{mt} \,|\, \mathbf{T})$ con $\mathbf{T}$ de $mt \times k$; Patterson, idéntico; el error de peso 2 de esta sección $\leftrightarrow$ un error de peso t entre $\binom{n}{t} \approx 2^{456}$ posibilidades para `mceliece348864`; y el mini-esquema $\leftrightarrow$ el KEM completo de los capítulos 8–10, con rechazo implícito y confirmación de texto claro.

5.6 Ejercicios

1. **La columna $j = 2$ a mano.** Con $z^4 = z + 1$ y la tabla de potencias del capítulo 3, verifique que $g(z) = z + z^2 + z^3$, que $1/g(z) = \texttt{3}$ y que $z/g(z) = \texttt{6}$; compare con la tabla 5.1.

2. **Linealidad del síndrome.** Calcule $\tilde{\mathbf{H}}e_5$, $\tilde{\mathbf{H}}e_9$ y $\tilde{\mathbf{H}}(e_5 + e_9)$: debe obtener $(\texttt{2}, \texttt{b})$. ¿Por qué $e_5 + e_9$ también se decodifica de forma única?

3. **Un certificado a mano.** Compruebe, usando una y otra vez $z^4 = z+1$, que $(\texttt{4}+\texttt{9}x)(\texttt{a}+\texttt{7}x) = 1 + (z + z^3)\,g(x)$; es el certificado del paso 2 de Patterson.

4. **El caso de un solo error.** Si $E = \{j\}$, demuestre que $\sigma(x) = x + \alpha_j$ y compruebe la ecuación (5.3) (aquí $\sigma' = 1$). ¿Qué devuelve Patterson?

5. **La permutación, paso a paso.** Con $(\mathbf{v}P)_j = \mathbf{v}_{\sigma(j)}$, compruebe que $(\mathbf{x}P^{-1})_1 = \mathbf{x}_{\sigma^{-1}(1)} = \mathbf{x}_{15} = 1$ y que $(\mathbf{x}P^{-1})_3 = \mathbf{x}_{\sigma^{-1}(3)} = \mathbf{x}_{13} = 0$. ¿En qué posiciones quedan los dos errores de $\mathbf{z}$?

6. **¿Por qué el juguete no es seguro?** Dé dos ataques concretos: uno enumerando palabras de $\mathbb{F}_2^{16}$ y otro usando los 256 síndromes. Explique en una línea qué cambia al pasar a $n \approx 8000$ (parte IV).

Cierre de la parte I. Con esto termina la parte de fundamentos: los códigos lineales y sus parámetros (capítulos 1 y 2), la aritmética de cuerpos finitos (capítulo 3), los códigos de Goppa (capítulo 4) y un juguete completo, verificado número a número, que ya construye matrices, decodifica con Patterson y cifra. La parte II convierte todo esto en el criptosistema de clave pública: el McEliece de 1978, su dual de Niederreiter y la transformación a KEM que lleva a Classic McEliece.

PARTE II

El criptosistema de McEliece

CAPÍTULO 6

El McEliece de 1978

En este capítulo

Este capítulo presenta el criptosistema original de Robert McEliece (1978), con el que empezó todo: la construcción, la demostración de que el descifrado funciona, el argumento de seguridad que lo acompaña y sus tres debilidades prácticas (tamaño de clave, cifrado sin garantías modernas y ausencia de firmas). El lector reconocerá de inmediato el juguete del capítulo 5: este capítulo es aquel laboratorio con parámetros grandes, y el resto de la parte II convierte el esquema en un KEM.

Dos páginas y media que esperaron cuarenta años

El esquema apareció en el *DSN Progress Report 42-44* del Jet Propulsion Laboratory (enero–febrero de 1978, pp. 114–116), casi al mismo tiempo que RSA, y McEliece lo cierra con dos observaciones que la historia confirmó palabra por palabra: permite «tasas de comunicación cercanas a 10^6 bits/s o más», y su algoritmo de descifrado *no* sirve para firmar. Durante décadas fue una curiosidad teórica, sin estándares ni implementaciones masivas; la amenaza cuántica (capítulo 1) lo devolvió al primer plano, y su descendiente directo, Classic McEliece, es el esquema que ISO/IEC estandarizó en junio de 2026 como enmienda a la norma 18033-2 [45, 24].

6.1 La idea: una trampa amable

En 1976, Diffie y Hellman habían enseñado a construir canales seguros sin clave compartida, y en 1978 existían ya dos «trampas» concretas —problemas fáciles con información secreta y difíciles sin ella—: la factorización de enteros (RSA) y el problema del mochilero (Merkle–Hellman). McEliece propuso una tercera, que no viene de la teoría de números sino de los *códigos correctores de errores*. La asimetría que explota está demostrada en las partes anteriores:

- **Con** la descripción secreta —un polinomio de Goppa g de grado t y su soporte L— existe un decodificador eficiente: el algoritmo de Patterson corrige hasta t errores en tiempo $O(nt)$ [60].
- **Sin** ella, el problema de decodificar un código lineal arbitrario a partir de una matriz generadora es NP-completo (Berlekamp, McEliece y van Tilborg, 1978) [6].

El truco consiste en esconder el código de Goppa a la vista de todos: se multiplica su matriz generadora por una matriz invertible densa S y se permutan las coordenadas con P. Ninguna de las dos operaciones cambia la dimensión, la distancia ni la decodificabilidad del código, pero $G' = SGP$ pierde toda la apariencia de «matriz de Goppa»: el adversario solo ve lo que parece una matriz aleatoria $k \times n$.

La asimetría en una línea

Con la clave secreta, descifrar es decodificar un código de Goppa (fácil). Sin ella, es decodificar un código lineal cualquiera (conjeturadamente difícil).

6.2 El criptosistema, paso a paso

Definición 6.1: El criptosistema de McEliece (1978)

Parámetros: enteros m y t; longitud $n = 2^m$; dimensión $k = n - mt$. Tres algoritmos:

A. Generación de claves. Elíjase $g \in \mathbb{F}_{2^m}[x]$ irreducible de grado t y tómese como soporte la lista L de todos los elementos de $\mathbb{F}_{2^m}$. Sea $\Gamma(L, g)$ el código de Goppa correspondiente y sea G una matriz generadora *canónica* ($k \times n$; por ejemplo, en forma sistemática). Sortéense una matriz invertible $S \in \mathrm{GL}(k, \mathbb{F}_2)$ y una permutación P de las n coordenadas. La **clave pública** es

$$G' = S\,G\,P \qquad \text{junto con el número } t; \tag{6.1}$$

la **clave secreta** es (g, L, S, P).

B. Cifrado. Para cifrar $\mathbf{u} \in \mathbb{F}_2^k$ se sortea un error $\mathbf{z} \in \mathbb{F}_2^n$ de peso *exactamente* t y se envía

$$\mathbf{x} = \mathbf{u}\,G' + \mathbf{z}. \tag{6.2}$$

C. Descifrado. Dado $\mathbf{x}$: (1) deshágase la permutación, $\mathbf{x}P^{-1}$; (2) decodifíquese con Patterson, lo que produce el codeword $(\mathbf{u}S)\,G$; (3) léase $\mathbf{u}S$ en las coordenadas sistemáticas de G y devuélvase $\mathbf{u} = (\mathbf{u}S)\,S^{-1}$.

Proposición 6.2: El descifrado recupera el mensaje

Con la notación de la definición 6.1, si $\mathbf{x} = \mathbf{u}G' + \mathbf{z}$ y $\mathrm{wt}(\mathbf{z}) \le t$, el descifrado devuelve $\mathbf{u}$.

Demostración. Como $G' = SGP$ y $PP^{-1} = \mathrm{I}_n$,

$$\mathbf{x}P^{-1} = \mathbf{u}S\,G\,(\underbrace{P\,P^{-1}}_{\mathrm{I}_n}) + \mathbf{z}P^{-1} = \underbrace{(\mathbf{u}S)\,G}_{\text{codeword de } \Gamma_2} + \mathbf{z}P^{-1}, \tag{6.3}$$

y $\mathrm{wt}(\mathbf{z}P^{-1}) = \mathrm{wt}(\mathbf{z}) \le t$, porque permutar coordenadas no cambia el peso. El corolario 4.5 garantiza que ese codeword es el único a distancia $\le t$, y Patterson lo encuentra. La matriz G es canónica, así que sus coordenadas sistemáticas son $\mathbf{u}S$; multiplicando por S^{-1} se recupera $\mathbf{u}$. □

Quien cifra solo necesita (G', t); quien descifra necesita la clave secreta. La figura 6.1 resume el circuito completo y la tabla 6.1 separa lo público de lo secreto, con sus tamaños para los parámetros originales.

	Contenido	**Tamaño (parámetros originales: $n = 1024$, $k = 524$, $t = 50$)**
Público	$G' = SGP$, de $k \times n$	$524 \cdot 1024 = 536\,576$ bits $= 64$ KiB
	t	despreciable
Secreto	g (grado t)	$mt = 500$ bits
	L	implícita ($L = \mathbb{F}_{2^{10}}$ completo)
	S	$k^2 = 274\,576$ bits (≈ 34 KiB)
	P	$n \log_2 n = 10\,240$ bits ($\approx 1{,}3$ KiB)

Cuadro 6.1: Reparto de material y tamaños en el McEliece de 1978. La clave pública domina; reducirla será el primer problema a resolver (capítulos 7 y 9).

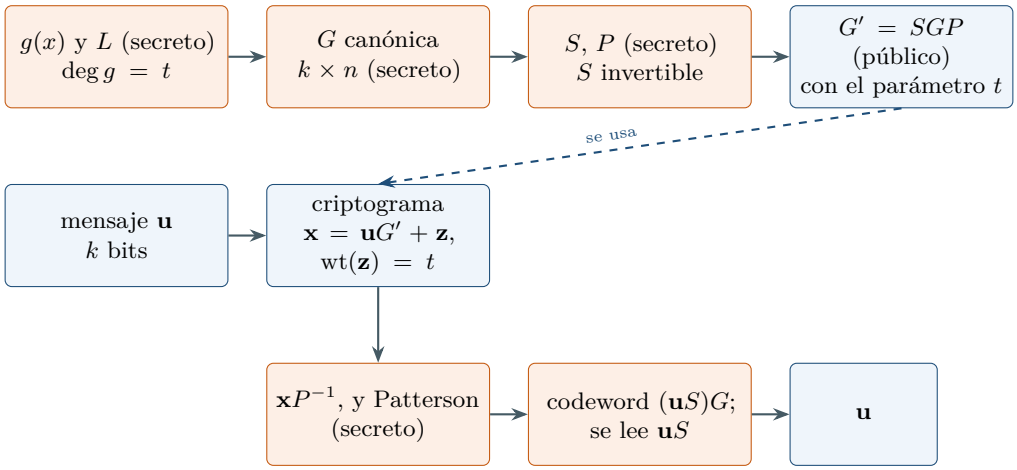


Figura 6.1: El circuito del esquema de 1978. Arriba, la generación de claves: la clave pública es la matriz de Goppa disfrazada $G' = SGP$. Abajo, el uso: el emisor suma un error de peso t a un codeword y el receptor deshace el disfraz y decodifica. Los recuadros naranjas contienen material secreto.

6.3 El juguete del capítulo 5 es este esquema

El capítulo 5 fue, literalmente, una ejecución de la definición 6.1 con $m = 4$, $t = 2$: allí $n = 16$, $k = 8$ y $g(x) = x^2 + x + z^3$. La matriz S de 8×8 y la permutación están en la figura 5.4; la clave pública $G' = SGP$ también; el criptograma $\mathbf{x} = \texttt{0001011010000011}$ en la figura 5.5; y el descifrado con Patterson, la lectura de $\mathbf{u}S$ y el producto por S^{-1}, con nombres y números, en todo el capítulo. La única diferencia es la escala —y en criptografía la escala lo es todo: el capítulo 14 mostrará por qué $n = 16$ se rompe en microsegundos y $n = 3488$ resiste a todos los algoritmos conocidos.

6.4 El argumento de seguridad de 1978

Un adversario que conoce G' (y t) y ve $\mathbf{x}$ tiene, como el propio artículo señala, dos caminos (figura 6.2):

1. **Ataques estructurales:** recuperar (g, L, S, P), o al menos *una* descripción equivalente del código, para decodificar a gusto. El artículo lo considera «desesperado» por el número astronómico de descomposiciones $S^{-1}G'P^{-1}$ posibles; la parte IV mostrará que la intuición era correcta en el recuento pero incompleta: la especialidad de Goppa deja huellas.

2. **Ataques decodificadores:** resolver directamente

$$\text{dados } G' \text{ y } \mathbf{x}, \quad \text{hallar } \mathbf{u} \text{ con } \mathrm{wt}(\mathbf{x} - \mathbf{u}G') \le t, \tag{6.4}$$

   sin aprender nada de la estructura. Es el problema de la decodificación de máxima verosimilitud (MLD) restringido a la clase de códigos que G' genera.

El segundo camino se apoya en un teorema: el MLD para códigos lineales binarios arbitrarios es NP-completo [6]. Pero un teorema así no cierra la cuestión.

⚖ Qué prueba y qué no la NP-dureza

La NP-dureza es un enunciado del *peor caso*: «existen instancias difíciles». La criptografía necesita más: que las instancias que el esquema *genera* sean difíciles. Y la instancia de McEliece no es un código lineal arbitrario, sino un código de Goppa disfrazado, una subclase muy especial sobre la que el teorema no dice nada. La seguridad de McEliece es, desde 1978, una **conjetura** cuantificada una y otra vez por la comunidad: el coste del mejor algoritmo conocido (capítulos 14 y 16) acota la confianza, pero no la demuestra.

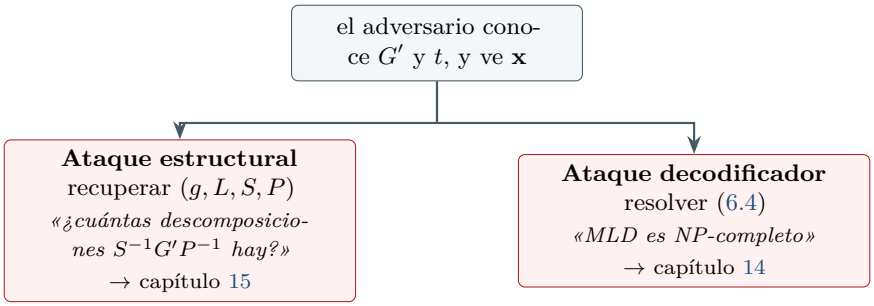


Figura 6.2: Las dos familias de ataques que ya identifica el artículo de 1978. La seguridad práctica moderna se discute toda dentro de este dibujo, con dos precisiones que veremos en la parte IV: los ataques estructurales también *distinguen* códigos de Goppa escondidos de códigos aleatorios, y los decodificadores también pueden llegar a recuperar la clave.

6.5 Los números del artículo, recalculados

Con $(m, t) = (10, 50)$ se tiene $n = 1024$, $k = 524$ y $d \geq 2t + 1 = 101$. McEliece sopesó los ataques evidentes; la tabla 6.2 compara sus cifras con el recálculo de hoy.

Magnitud	Artículo (1978)	Recálculo exacto
Codewords posibles (2^k)	$2^{524} \approx 10^{158}$	igual
Clases laterales (2^{n-k})	—	$2^{500} \approx 10^{150.5}$
Ataque de las k coordenadas	10^{19} ($\approx 2^{63}$)	$2^{80.7} \approx 2 \cdot 10^{24}$
Pr[decodificar un vector al azar]	$2^{-215.59}$	$2^{-215.89}$

Cuadro 6.2: Estimaciones del artículo de 1978 para $n = 1024$, $t = 50$, $k = 524$, y recálculo con aritmética exacta (caja siguiente). Coinciden en lo esencial y las tres favorecen al defensor.

⚖ Recalculando un artículo de 1978

Dos de las cifras del artículo se pueden recalcular hoy sin margen de interpretación.

(1) **Ataque de las k coordenadas.** La probabilidad de que k coordenadas elegidas al azar eviten las t posiciones del error (sin reemplazo) es

$$\frac{\binom{n-t}{k}}{\binom{n}{k}} = \prod_{i=0}^{k-1} \frac{974 - i}{1024 - i} = 2^{-53.61},$$

de modo que el trabajo esperado es $2^{53.61} \cdot k^3 \approx 2^{80.7} \approx 2 \cdot 10^{24}$. El artículo usó la

aproximación *con* reemplazo $(1-t/n)^k = 2^{-37.8}$, que da $1{,}9 \cdot 10^{19}$. La corrección *aumenta* el coste del ataque: la estimación de 1978 era conservadora a favor del atacante.

(2) **La (no) firma.** El número de vectores de longitud n que están a distancia $\leq t$ de algún codeword es $2^k \cdot \sum_{i=0}^{t} \binom{n}{i}$, y

$$\sum_{i=0}^{50} \binom{1024}{i} = 2^{284.11}, \qquad \Pr = \frac{2^{524} \cdot 2^{284.11}}{2^{1024}} = 2^{-215.89}.$$

El artículo escribe $2^{284.41}$ y $2^{-215.59}$: una diferencia de 0,3 bits sin consecuencias (firmar exigiría acertar un suceso de probabilidad 2^{-215}).

En ambos casos la discrepancia es de aritmética de la época, no de concepto, y las dos conclusiones —atacar es prohibitivamente caro, «firmar» es imposible— quedan intactas.

La velocidad seguía siendo el argumento de venta: «cercanas a 10^6 bits/s o más». Sigue siéndolo: cifrar es una multiplicación de bits $k \times n$ más un xor, y descifrar añade Patterson. Cifrar es prácticamente gratis; descifrar cuesta del orden de un milisegundo en software actual, con aritmética de cuerpos finitos en vez de aritmética modular sobre enteros enormes.

6.6 Dónde falla (y dónde no)

Tres grietas prácticas —todas arreglables o evitables— y una conjetura que, por ahora, aguanta.

1. No hay firmas. El propio autor lo advierte: el descifrado solo está definido para vectores a distancia $\leq t$ de un codeword, y esa fracción es $2^{-215.89}$ (caja anterior). Un esquema de firma necesita que «descifrar» esté definido en todo el espacio; «firmar» cifrando con la clave secreta produce basura casi siempre. Existen esquemas de firma basados en códigos desde 2001, pero son otra familia: Classic McEliece, como veremos, es *solo* un KEM.

2. Cifrado sin garantías modernas. El cifrado de 1978 no aspira a IND-CPA ni a IND-CCA: es la capa que reduce la recuperación del mensaje a un problema de decodificación. Dos observaciones elementales lo muestran:

- Dado $\mathbf{x} = \mathbf{u}G' + \mathbf{z}$, el error $\mathbf{z}$ está *unívocamente* determinado (ejercicio 3): el cifrado no esconde «ruido», esconde un problema MLD.
- El cifrado es *maleable*: si se modifica $\mathbf{x}$ en posiciones del error, el descifrado sigue devolviendo el mismo $\mathbf{u}$: $\mathbf{x} \oplus \mathbf{e}'$ con $\mathrm{supp}(\mathbf{e}') \subseteq \mathrm{supp}(\mathbf{z})$ decodifica a $\mathbf{u}$.

Un adversario con acceso al oráculo de descifrado explota lo segundo como *ataque de reacción*: pregunta por $\mathbf{x} \oplus \mathbf{e}_j$ para cada j y aprende, por las reacciones, dónde está el error (ejercicio 6). El remedio moderno —rechazo implícito— llega con Classic McEliece (capítulos 8 y 10).

⚠ «El descifrado siempre devuelve el mensaje»

Falso: Patterson solo garantiza la decodificación si $\mathrm{wt}(\mathbf{z}) \leq t$. Si el canal introduce tres errores en el juguete ($t = 2$), o si un adversario retoca un bit de más, el descifrador produce basura o falla; y en el esquema de 1978 ese *fallo es observable*. En un KEM moderno el fallo se oculta (rechazo implícito, capítulo 10).

3. El tamaño de la clave pública. G' tiene $k \times n$ bits: en 1978, $524 \cdot 1024$ bits $= 64$ KiB; con parámetros actuales, $2720 \cdot 3488$ bits $\approx 1{,}13$ MB. Dos remedios históricos: publicar la matriz de paridad del código dual (Niederreiter, capítulo 7) —más pequeña cuando $k > n/2$— o publicar solo la parte sistemática $(\mathrm{I}_{mt} \mid \mathbf{T})$ y regenerar $\mathbf{T}$ de una semilla corta, que es lo que hace Classic McEliece (capítulos 9 y 12).

4. Lo que no ha fallado. En casi medio siglo no se ha roto la conjetura central: no se conoce ningún ataque que recupere el mensaje o la clave de los parámetros recomendados en tiempo factible. Pero todo lo demás ha cambiado: los parámetros de 1978 están hoy por debajo de cualquier margen razonable; la seguridad se mide con algoritmos ISD cada vez mejores (capítulo 14); hay ataques estructurales que explotan la especialidad de las variantes mal elegidas (capítulo 15); y 2025–2026 trajo resultados nuevos que obligan a revisar supuestos (capítulo 16). La próxima parada es la reformulación dual de Niederreiter, capítulo 7.

6.7 Ejercicios

1. **Corrección.** Repase la proposición 6.2 comprobando cada igualdad. ¿Dónde se usa $\mathrm{wt}(\mathbf{z}) \le t$ (y no la igualdad)? ¿Qué fallaría si el emisor eligiera errores de peso $t+1$ «a veces»?

2. **Tamaños.** Calcule los bits de G' para $(n, k) = (1024, 524)$ y para `mceliece348864` ($k = 2720$, $n = 3488$, clave pública de 261 120 bytes). ¿Cuántos bytes ocupa la parte sistemática $(\mathrm{I}_{2720} \mid \mathbf{T})$? ¿Qué factor de ahorro supone frente a G' completa?

3. **El error es único.** Pruebe que, con $\mathbf{x} = \mathbf{u}G' + \mathbf{z}$ y $\mathrm{wt}(\mathbf{z}) \le t$, no existe otro par $(\mathbf{u}', \mathbf{z}')$ con $\mathbf{u}' \neq \mathbf{u}$, $\mathrm{wt}(\mathbf{z}') \le t$ y $\mathbf{x} = \mathbf{u}'G' + \mathbf{z}'$. (Use $d \ge 2t+1$.)

4. **Los números del artículo.** Con una calculadora o Python, reproduzca los tres valores no triviales de la tabla 6.2: 2^{500}, el cociente $\binom{974}{524}/\binom{1024}{524}$ y $\sum_{i \le 50} \binom{1024}{i}$. Pista: trabaje con logaritmos en base 2.

5. **Por qué no basta la NP-dureza.** Explique en cinco líneas por qué «MLD es NP-completo» no demuestra que McEliece sea seguro.

6. **Ataque de reacción.** Suponga que el descifrado «avisa» cuando su entrada no es decodificable. Demuestre que enviando $\mathbf{x} \oplus \mathbf{e}_j$ para cada posición j se aprenden todas las posiciones del error y, con ellas, $\mathbf{u}$: ¿cuántas consultas hacen falta? ¿Cómo se comporta el KEM real ante esa consulta (capítulo 10)?

CAPÍTULO 7

La reformulación de Niederreiter

En este capítulo

Este capítulo da la vuelta al esquema: en lugar de publicar una matriz generadora y enviar un codeword con ruido, publicaremos una *matriz de paridad* y enviaremos el *síndrome* de un vector de peso fijo. El resultado (Niederreiter, 1986) tiene criptogramas más cortos y un espacio de mensajes distinto, con la misma dificultad de fondo. No es un capítulo de arqueología: el KEM real, Classic McEliece, es niederreiteriano de principio a fin.

1986: publicar la otra matriz

Harald Niederreiter propuso en 1986 [58] cambiar *qué* se publica y *qué* se transmite: en vez de la matriz generadora de 1978, la matriz de paridad; y en vez de un codeword con error, el síndrome del error. Casi cuarenta años después, la especificación de Classic McEliece hace exactamente esto: su clave pública T define la matriz de paridad $H = (\mathrm{I}_{mt} \,|\, T)$ y sus criptogramas son síndromes de mt bits [10].

7.1 La idea dual: mensajes de peso fijo

Recordemos el cuadro de los capítulos 2 y 4: un código lineal Γ de parámetros $[n, k, d]$ se describe indiferentemente por una matriz generadora G $(k \times n)$ o por una matriz de paridad H_0 $((n-k) \times n)$, con

$$\mathbf{c} \in \Gamma \iff \mathbf{c}\, H_0^{\top} = \mathbf{0}.$$

En el McEliece de 1978 se publica $G' = SGP$ disfrazada y se envía $\mathbf{x} = \mathbf{u}G' + \mathbf{z}$: n bits, de los que k son mensaje. Niederreiter observó que se puede cifrar con el código «al revés»:

- El **mensaje** es un vector $\mathbf{e} \in \mathbb{F}_2^n$ de peso *exactamente* t (no una palabra arbitraria). Hay $\binom{n}{t}$ mensajes posibles: para $(n, t) = (1024, 50)$, $\binom{1024}{50} \approx 2^{284}$, es decir ≈ 284 bits útiles por bloque. Generar un mensaje es «codificar en peso fijo» (*FixedWeight* en la especificación; algoritmos en el capítulo 9).
- El **criptograma** es su síndrome: $\mathbf{c} = \mathbf{e}\,(H')^{\top}$, un vector de $n - k = mt$ bits, donde H' es la matriz de paridad pública.

Con los parámetros de 1978, $n = 1024$ y $t = 50$: el criptograma baja de 1024 a $n - k = 500$ bits, mientras que el mensaje baja de $k = 524$ bits libres a $\log_2 \binom{1024}{50} \approx 284$ bits. La ganancia es real pero modesta; con los parámetros de hoy, en cambio, la balanza se inclina: $k \approx 0{,}78\, n$ y el criptograma cae por debajo de la cuarta parte de n (tabla 7.1). Esa es una de las razones por las que la forma dual esperó a los KEM: su ventaja se materializa cuando $n - k \ll n$.

7.2 El criptosistema (1986), en nuestras convenciones

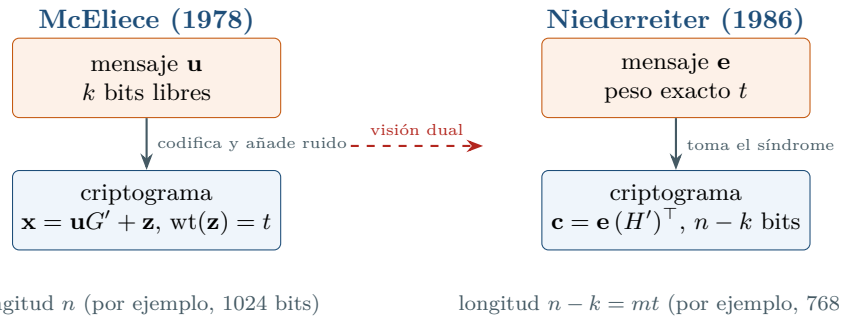


Figura 7.1: Las dos miradas al mismo código. En McEliece se publica G' y se envían n bits (codeword con ruido); en Niederreiter se publica H' y se envían $n-k$ bits (síndrome del mensaje-error). Classic McEliece usará la segunda columna.

Definición 7.1: El criptosistema de Niederreiter (1986)

Parámetros m, t; $n = 2^m$; $k = n - mt$. Sea $\Gamma(L, g)$ el código de Goppa secreto y sea H_0 una matriz de paridad suya de rango completo. Las operaciones de fila sobre H_0 no cambian el código que comprueba, y una permutación de columnas se absorbe en P; así pues, sin pérdida de generalidad suponemos H_0 reducida: $H_0 = (\mathrm{I}_{n-k} \,|\, \mathbf{T}_0)$. (Es la misma forma $(\mathrm{I}_8 \,|\, \mathbf{T})$ que el juguete destiló en la figura 5.1.) Tres algoritmos:

A. Generación de claves. Se sortean $S \in \mathrm{GL}(n-k, \mathbb{F}_2)$ y una permutación P de las n coordenadas. La **clave pública** es

$$H' = S\,H_0\,P \qquad \text{(y el número } t\text{)}, \tag{7.1}$$

y la **clave secreta** es (g, L, S, P).

B. Cifrado. Para cifrar $\mathbf{e} \in \mathbb{F}_2^n$ con $\mathrm{wt}(\mathbf{e}) = t$, envíese

$$\mathbf{c} = \mathbf{e}\,(H')^\top \in \mathbb{F}_2^{n-k} \qquad \text{(en notación de columnas: } \mathbf{c}^\top = H'\mathbf{e}^\top\text{)}. \tag{7.2}$$

C. Descifrado. Dado $\mathbf{c}$: (1) calcúlese $\gamma = \mathbf{c}\,(S^{-1})^\top$; (2) encuéntrese el único $\mathbf{f} \in \mathbb{F}_2^n$ con $\mathbf{f}H_0^\top = \gamma$ y $\mathrm{wt}(\mathbf{f}) \le t$ (con Patterson: basta decodificar la palabra $(\gamma, \mathbf{0})$, cuyo síndrome es γ); (3) devuélvase $\mathbf{e} = \mathbf{f}P$.

La convención de permutación es la del capítulo 5: $(\mathbf{v}P)_j = \mathbf{v}_{\sigma(j)}$, la coordenada antigua $\sigma(j)$ pasa a la posición j; recordemos también que $P^\top = P^{-1}$.

Proposición 7.2: El descifrado recupera el mensaje-error

Con la notación de la definición 7.1, si $\mathbf{c} = \mathbf{e}(H')^\top$ y $\mathrm{wt}(\mathbf{e}) = t$, el descifrado devuelve $\mathbf{e}$.

Demostración. Como $H' = SH_0P$,

$$\mathbf{c}\,(S^{-1})^\top = \mathbf{e}\,(SH_0P)^\top(S^{-1})^\top = \mathbf{e}\,P^\top H_0^\top S^\top (S^{-1})^\top = \underbrace{(\mathbf{e}P^{-1})}_{\text{peso } t} H_0^\top =: \gamma.$$

Por tanto $\mathbf{f} = \mathbf{e}P^{-1}$ es una solución de peso t del paso (2) (de hecho, $(\mathbf{e}P^{-1})H_0^\top = \gamma$ es la identidad anterior), y es la única: si $\mathbf{f}'$ fuera otra con $\mathrm{wt}(\mathbf{f}') \le t$, entonces $\mathbf{f} + \mathbf{f}' \in \ker H_0^\top = \Gamma$,

con $\mathrm{wt}(\mathbf{f}+\mathbf{f}') \le 2t < d$, imposible. Patterson encuentra ese único $\mathbf{f}$ (el capítulo 5 mostró el mecanismo con todas sus cuentas). Y $\mathbf{f}P = \mathbf{e}P^{-1}P = \mathbf{e}$. □

Observación 7.3: El mensaje es el error

En McEliece, el mensaje $\mathbf{u}$ viaja «escalado» dentro de $\mathbf{u}S$ y hay que deshacer S; el ruido $\mathbf{z}$ es solo eso, ruido. En Niederreiter no hay mensaje separado del error: el vector de peso t *es* el mensaje, y el descifrado de un síndrome es recuperarlo. La frase «el descifrado de Niederreiter es un problema de decodificación por síndromes» es literal.

7.3 Equivalencia con el esquema de 1978

Los dos esquemas resuelven el mismo problema con distinto embalaje. Formalmente:

Proposición 7.4: Un ataque niederreiteriano rompe McEliece

Sean $G' = S_1GP$ y $H' = S_2H_0P$ dos disfraces del mismo código secreto, con la *misma* permutación P en ambos (S_1 y S_2 pueden ser distintas). Un algoritmo que, dados H' y $\mathbf{c} = \mathbf{e}(H')^\top$ con $\mathrm{wt}(\mathbf{e}) = t$, recupera $\mathbf{e}$, recupera también el mensaje de todo criptograma de McEliece $\mathbf{x} = \mathbf{u}G' + \mathbf{z}$.

Demostración. Sea $\mathbf{x} = \mathbf{u}G' + \mathbf{z}$ un criptograma de McEliece. Lo primero es que G' y H' son ortogonales aunque los disfraces S_1, S_2 difieran:

$$G'\,(H')^\top = S_1\,G\,\underbrace{P\,P^\top}_{\mathrm{I}_n}\,H_0^\top\,S_2^\top = S_1\,\underbrace{(G\,H_0^\top)}_{=0}\,S_2^\top = 0,$$

porque G genera el mismo código que H_0 comprueba. Por tanto

$$\mathbf{x}\,(H')^\top = \mathbf{u}\,G'(H')^\top + \mathbf{z}\,(H')^\top = \mathbf{z}\,(H')^\top,$$

es decir: el síndrome público de $\mathbf{x}$ es el síndrome de $\mathbf{z}$, y se calcula sin ninguna clave. Se lo damos al recuperador niederreiteriano: devuelve el único $\mathbf{e}$ de peso t con $\mathbf{e}(H')^\top = \mathbf{z}(H')^\top$. Pero entonces $(\mathbf{z}+\mathbf{e})(H')^\top = 0$, así que $\mathbf{z}+\mathbf{e}$ es un codeword de un código equivalente a Γ (sus coordenadas solo están permutadas, y eso no cambia el peso) y tiene peso $\le 2t < d$: la única posibilidad es $\mathbf{e} = \mathbf{z}$, por el mismo argumento de unicidad de la proposición 7.2. Recuperado el error, $\mathbf{x} + \mathbf{z} = \mathbf{u}G'$, y $\mathbf{u}$ se despeja por álgebra lineal (o se lee en las coordenadas sistemáticas de G'). □

La dirección demostrada es la que se usa: los ataques modernos contra Classic McEliece se describen casi siempre como decodificación por síndromes contra H' (parte IV). El recíproco —que un atacante capaz de descifrar McEliece rompa también Niederreiter— fue establecido por Li, Deng y Wang en 1994 [51], que probaron la equivalencia polinómica de ambos esquemas con parámetros correspondientes. La guía de los diseñadores lo resume así: la variante de Niederreiter «cambia la seguridad de forma insignificante» [28]. Equivalencia, con matices: los *problemas de inversión* son esencialmente el mismo; los esquemas completos (tamaños de clave, longitud de criptograma y las transformaciones a KEM del capítulo 8) no son idénticos, y ahí está toda la diferencia práctica.

7.4 Los números: cuándo gana la forma dual

En 1978 se publicaba una matriz $k \times n$; en la forma dual se publica una matriz $(n-k) \times n$. El factor de ahorro es $k/(n-k)$: con los parámetros originales, $k \approx n/2$ y el ahorro resultaba mísero (5 %); con los parámetros reales, $k \approx 0{,}78\,n$ y el ahorro es de 3,5 veces. La tabla 7.1 muestra los números del estándar, que ya usan la compresión sistemática que veremos en el capítulo 9.

Conjunto	m	n	t	k	mt	clave pública	criptograma
juguete (cap. 5)	4	16	2	8	8	8 B	1 B
`mceliece348864`	12	3488	64	2720	768	261 120 B	96 B
`mceliece460896`	13	4608	96	3360	1248	524 160 B	156 B
`mceliece6688128`	13	6688	128	5024	1664	1 044 992 B	208 B
`mceliece6960119`	13	6960	119	5413	1547	1 047 319 B	194 B
`mceliece8192128`	13	8192	128	6528	1664	1 357 824 B	208 B

Cuadro 7.1: Los cinco tamaños de Classic McEliece (sin las variantes `f` y `pc`; detalles en el capítulo 12) frente al juguete. La clave pública guarda la matriz T, de $mt \times k$ bits, en $mt\lceil k/8 \rceil$ bytes; el criptograma es un síndrome de $\lceil mt/8 \rceil$ bytes (en `mceliece6960119`, $mt = 1547$ no es múltiplo de 8 y ambas cosas se rellenan). A modo de comparación: una matriz generadora completa de `mceliece348864` ocuparía $k\,n/8 = 1\,185\,920$ bytes, más de cuatro veces lo que ocupa T.

7.5 Por qué Classic McEliece es niederreiteriano

En la especificación, la función Encode recibe el vector $\mathbf{e}$ y la clave pública T —una matriz de $mt \times k$ bits— y devuelve

$$C = (\mathrm{I}_{mt} \,|\, T)\,\mathbf{e}^{\top},$$

un síndrome de $n-k = mt$ bits; la función Decode reconstruye la palabra $v = (C, \mathbf{0})$ (los mt bits del síndrome y k ceros) y busca el codeword único a distancia $\leq t$, del que recupera $\mathbf{e} = v + c$ [10]. Son los pasos (2) y (3) de la definición 7.1, con dos simplificaciones de ingeniería que estudiaremos en los capítulos 9 y 10: (i) la clave pública no lleva una S explícita, porque T ya está en forma canónica y las operaciones de fila quedan absorbidas en ella; (ii) la permutación P no se sortea como matriz aparte: el soporte de Goppa se elige en un orden aleatorio (la *FieldOrdering* de la especificación) y ese orden vive en la clave secreta. La matemática es la del capítulo; el formato, el del KEM: clave pública más de cuatro veces más corta y criptogramas de mt bits.
El capítulo 8 convierte esta forma dual en un KEM; los capítulos 9 y 10 abren la especificación línea a línea.

7.6 Ejercicios

1. **El truco del relleno de ceros.** Verifique que, con $H_0 = (\mathrm{I}_{n-k} \,|\, \mathbf{T}_0)$, la palabra $v = (\gamma, \mathbf{0})$ tiene síndrome $vH_0^{\top} = \gamma$; explique por qué eso convierte el paso (2) del descifrado en exactamente el problema de decodificación del capítulo 4, con Patterson aplicable tal cual.

2. **Unicidad del síndrome.** Pruebe que si $\mathbf{f}H_0^{\top} = \mathbf{f}'H_0^{\top}$ con $\mathrm{wt}(\mathbf{f}), \mathrm{wt}(\mathbf{f}') \leq t$, entonces $\mathbf{f} = \mathbf{f}'$. (Pista: la diferencia está en Γ y su peso es $\leq 2t < d$.)

3. **La reducción, con lápiz.** Reproduzca la demostración de la proposición 7.4 e indique en qué paso se usa $PP^{\top} = \mathrm{I}_n$ y en cuál la ortogonalidad $GH_0^{\top} = 0$. ¿Por qué el argumento falla si G' y H' usaran permutaciones distintas?

4. **Factores de ahorro.** Calcule el factor $k/(n-k)$ de los cinco conjuntos de la tabla 7.1 y compárelo con el del juguete (donde vale 1). ¿Por qué el esquema original de $(1024, 524)$ apenas ganaba?

5. **Peso exacto.** El descifrado también funcionaría con $\mathrm{wt}(\mathbf{e}) < t$; dé dos razones por las que la especificación fija el peso en t exacto. (Pista: tamaño del espacio de mensajes y uniformidad: las transformaciones a KEM del capítulo 8 exigen que $\mathbf{e}$ sea uniforme en el conjunto de vectores de peso t.)

6. **Mensaje y error.** Escriba los dos problemas de inversión uno al lado del otro,

$$\text{(McEliece)} \quad \mathbf{x} = \mathbf{u}G' + \mathbf{z} \mapsto \mathbf{u}; \qquad \text{(Niederreiter)} \quad \mathbf{c} = \mathbf{e}(H')^{\top} \mapsto \mathbf{e},$$

   y explique en qué sentido son «el mismo problema» y en qué se parecen a la decodificación por síndromes del capítulo 4.

CAPÍTULO 8

Del cifrado a un KEM

En este capítulo

Los esquemas de los capítulos 6 y 7 cierran la parte II: sabemos cifrar, y sabemos que romper el cifrado es decodificar. Pero un protocolo real no necesita «cifrar mensajes»: necesita *acordar claves de sesión* que un adversario activo no pueda distinguir. Este capítulo presenta ese objetivo (un KEM IND-CCA2), muestra por qué el esquema desnudo no lo alcanza y describe la transformación —tipo Fujisaki–Okamoto, con *rechazo implícito*— que adoptó Classic McEliece. Es el puente conceptual hacia la especificación de la parte III.

De «descifrar» a «no distinguir»

Los objetivos de seguridad evolucionaron junto con los esquemas: recuperar el mensaje era el listón de 1978; a mediados de los noventa llegaron la indistinguibilidad y, con ella, el cifrado elegido (IND-CCA2). En 1999 Fujisaki y Okamoto publicaron su transformación clásica [37]; en 2001 Shoup propuso empaquetar las claves de sesión en KEMs [69]; el rechazo implícito apareció en 2012, en la tesis de Persichetti, para esquemas basados en códigos [61], y Hofheinz, Hövelmanns y Kiltz le dieron una prueba modular en 2017 [44]. Classic McEliece usa la versión de 2018 de Bernstein y Persichetti [12], la adapta y añade mejoras propias.

8.1 De filas a columnas (y por qué)

Hasta aquí escribimos los vectores como *filas*; la especificación de Classic McEliece los escribe como *columnas*. Es un cambio puramente tipográfico —transponer—, pero conviene fijarlo antes de seguir, porque las fórmulas de los capítulos 9 a 11 estarán en la segunda convención. El diccionario completo es la tabla 8.1; la matemática no cambia en absoluto.

Objeto	en filas (caps. 1–7)	en columnas (desde aquí)
vector	v de tamaño $1 \times n$	v de tamaño $n \times 1$
código	$c\,H_0^\top = 0$	$H_0\,c = 0$
cifrado de McEliece	$x = uG' + z$	$x = (G')^\top u + z$
cifrado de Niederreiter	$C = e\,(H')^\top$	$C = H'\,e$
decodificación	Patterson, igual	Patterson, igual

Cuadro 8.1: Diccionario entre las dos convenciones. Transponer no cambia ningún resultado: es la misma álgebra con las transposiciones explícitas. La matriz de un código de Goppa (las t filas de $\alpha_i^j / g(\alpha_i)$, $j = 0, \ldots, t-1$) funciona igual en ambos mundos.

8.2 Qué es un KEM y qué le pedimos

Un *mecanismo de encapsulación de claves* (KEM) no cifra mensajes: produce un criptograma y, a la vez, una clave de sesión. Formalmente:

Definición 8.1: KEM: sintaxis y corrección

Un KEM es una terna de algoritmos:

- Gen: sin entrada más que los parámetros, produce una pareja $(\mathrm{pk}, \mathrm{sk})$;
- $\mathsf{Encap}(\mathrm{pk})$: con la clave pública, produce un criptograma C y una clave de sesión K (en Classic McEliece, 32 bytes);
- $\mathsf{Decap}(\mathrm{sk}, C)$: con la clave secreta y el criptograma, reproduce la clave K (o, en diseños con *rechazo explícito*, el símbolo $\perp$).

El KEM es *correcto* si para toda pareja $(\mathrm{pk}, \mathrm{sk})$ y todo $(C, K) \leftarrow \mathsf{Encap}(\mathrm{pk})$ se tiene $\mathsf{Decap}(\mathrm{sk}, C) = K$.

Definición 8.2: Seguridad IND-CCA2 de un KEM

El juego entre un adversario $\mathcal{A}$ y un retador:

1. el retador calcula $(\mathrm{pk}, \mathrm{sk}) \leftarrow \mathsf{Gen}$ y $(C^*, K_0^*) \leftarrow \mathsf{Encap}(\mathrm{pk})$; sortea $b \leftarrow \{0, 1\}$ y una clave uniforme K_1^*;
2. $\mathcal{A}$ recibe $(\mathrm{pk}, C^*, K_b^*)$ y puede consultar un oráculo de descifrado que, para cualquier $C \neq C^*$, responde $\mathsf{Decap}(\mathrm{sk}, C)$;
3. $\mathcal{A}$ termina con un bit b' y gana si $b' = b$.

La *ventaja* de $\mathcal{A}$ es $\left|\Pr[b' = b] - \frac{1}{2}\right|$. El KEM es IND-CCA2 si todo adversario eficiente tiene ventaja despreciable.

Los objetivos de seguridad se ordenan: OW-CPA (recuperar el mensaje), IND-CPA (distinguirlo sin oráculo) e IND-CCA2 (distinguirlo con oráculo). En este libro el trabajo duro se concentra en OW-CPA: la parte IV mide, muy en serio, la dificultad de recuperar el mensaje; todo lo demás lo aporta la transformación de este capítulo. ¿Y por qué KEMs y no cifrados de clave pública? Porque los protocolos de transporte (TLS, SSH) necesitan exactamente esto: una clave simétrica mutuamente acordada. El patrón estándar es KEM–DEM [69]: el KEM encapsula la clave K y un cifrado simétrico autenticado (AES-GCM, por ejemplo) cifra los datos con ella.

«Un KEM no cifra»

La confusión inversa también es un error: Encap *no* recibe un mensaje, y Decap *no* devuelve un mensaje. Si su protocolo necesita mandar datos, no intente «pasar el mensaje por el KEM»: encapsule una clave y cifre los datos con un DEM. Todos los KEMs modernos (ML-KEM, HQC, Classic McEliece) funcionan así.

8.3 Por qué el esquema desnudo no basta

Lo que el esquema de Niederreiter *sí* ofrece es OW-CPA: recuperar e de $C = H'e$ (o u de $x = uG' + z$) es exactamente el problema de decodificación que estudia la parte IV. Pero como

interfaz criptográfica falla ante un adversario activo:

- **Relaciones entre criptogramas.** En McEliece, dos cifrados del mismo mensaje difieren en $z_1 + z_2$, un vector de peso $\leq 2t$: un atacante que recibe dos reenvíos ya no ve dos problemas independientes, sino uno con un vector de peso pequeño conocido de antemano (ataque de reenvío de Berson, 1997 [13]). En Niederreiter, repetir la aleatoriedad e repite el criptograma exacto.
- **Oráculos de descifrado.** En el capítulo 6 (ejercicio 6) vimos que, si un descifrador responde de forma distinguible al éxito y al fracaso, cada consulta sobre $C + \delta$ revela información sobre el error y el atacante acaba reconstruyendo z entero, consulta a consulta.
- **Ciclos del generador aleatorio.** Si la aleatoriedad se repite entre usuarios o entre sesiones (ciclos de RNG, claves mal generadas), los criptogramas se relacionan y el esquema se rompe de forma trivial [29].

La conclusión es importante y sutil: *el problema matemático no es el culpable*; lo que hay que blindar es la interfaz. Eso es, exactamente, lo que hace la transformación que viene a continuación.

8.4 La transformación tipo FO con rechazo implícito

La idea, depurada por veinte años de literatura, cabe en tres frases: (1) que la clave de sesión dependa de la semilla aleatoria e y del criptograma C; (2) que el descifrado verifique, sin ambigüedad, que el criptograma es legítimo; (3) que el fallo no sea distinguible: en vez de $\perp$, se devuelve una clave pseudoaleatoria derivada de una clave secreta s.

Definición 8.3: Transformación tipo FO con rechazo implícito

Sea $(\mathsf{Gen}', \mathsf{Enc}, \mathsf{Dec})$ una PKE correcta y *determinista* con espacio de mensajes P —aquí, los vectores de peso t— y sea H una función de hash con separación de dominio. El KEM se construye así:

1. Gen: $(\mathrm{pk}', \mathrm{sk}') \leftarrow \mathsf{Gen}'$; se sortea la clave de rechazo s; la clave pública es $\mathrm{pk} = \mathrm{pk}'$ y la clave secreta es $(\mathrm{sk}', \mathrm{pk}', s)$. (La versión genérica guarda pk' para reencriptar; veremos que Classic McEliece puede prescindir de ella.)
2. $\mathsf{Encap}(\mathrm{pk})$: se sortea $e \leftarrow P$ uniforme y se calcula
$$C = \mathsf{Enc}(\mathrm{pk}, e), \qquad K = H(1, e, C).$$
3. $\mathsf{Decap}(\mathrm{sk}, C)$: se descifra $e' \leftarrow \mathsf{Dec}(\mathrm{sk}', C)$ y la clave es
$$K = \begin{cases} H(1, e', C), & \text{si } \mathsf{Enc}(\mathrm{pk}, e') = C, \\ H(0, s, C), & \text{en caso contrario.} \end{cases}$$

La seguridad de esta construcción está demostrada en dos modelos:

Teorema 8.4: IND-CCA2 en el modelo del oráculo aleatorio (ROM)

Sea $\mathcal{A}$ un adversario IND-CCA2 contra el KEM de la definición 8.3 que hace q_D consultas al oráculo de descifrado y q_H a H. Existe un adversario B contra la propiedad OW-CPA de la PKE, con tiempo y recursos comparables a los de $\mathcal{A}$ (más el costo de reencriptar cada

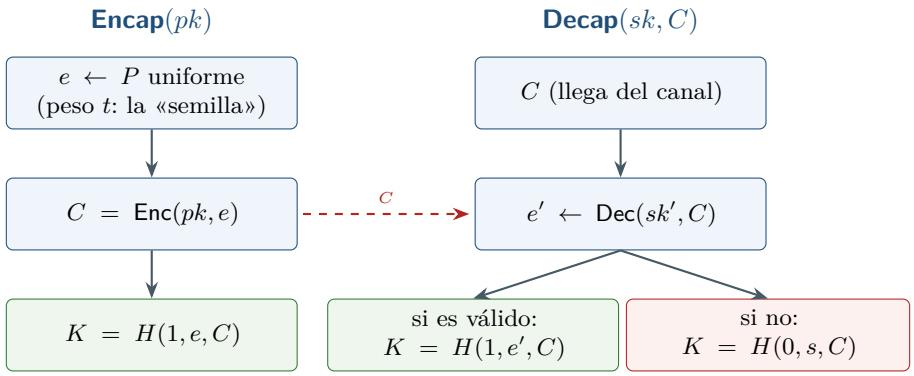


Figura 8.1: La transformación, en una imagen. Encap sortea la semilla e, la cifra y deriva la clave de sesión del par (e, C). Decap descifra; si el criptograma es válido reconstruye la misma clave (verde); si no, devuelve una clave pseudoaleatoria derivada de s (rojo), indistinguible de una clave legítima: el *rechazo implícito*. El adversario ve (pk, C, K) y puede consultar Decap con cualquier $C' \neq C$.

consulta a H), tal que

$$|\Pr[b' = b] - \tfrac{1}{2}| \;\leq\; \Pr[B \text{ gana OW-CPA}] \;+\; \frac{q_D}{|P|} \;+\; \frac{2\,q_H}{|S|},$$

donde $|P|$ es el tamaño del espacio de mensajes de la PKE y $|S|$ el de la clave de rechazo. *(Bernstein–Persichetti, 2018 [12], teorema 14.3.)*

Los tres términos admiten lectura directa: el primero es la conjetura dura (la que mide la parte IV); el segundo es la ventaja de adivinar un criptograma válido y consultarlo al oráculo; el tercero, la probabilidad de adivinar la clave de rechazo. Con los números reales del capítulo 12 son minúsculos: para `mceliece348864`, $|P| > 2^{456}$, mientras que la clave de rechazo s mide $n = 3488$ bits ($|S| = 2^{3488}$, con entropía efectiva acotada por la semilla δ de 256 bits, $|S|_{\mathrm{ef}} \geq 2^{256}$). Incluso con la cota conservadora $|S| = 2^{256}$, con $q_D = q_H = 2^{64}$ consultas los dos últimos términos no pasan de 2^{-392} y 2^{-191}.

Teorema 8.5: IND-CCA2 en el modelo cuántico (QROM)

En el modelo del oráculo aleatorio cuántico, con las mismas notaciones,

$$|\Pr[b' = b] - \tfrac{1}{2}| \;\leq\; 2\sqrt{\Pr[B \text{ gana OW-CPA}]} \;+\; 2\,\varepsilon_{\mathrm{PRF}} \;+\; \varepsilon_{\mathrm{iny}},$$

donde $\varepsilon_{\mathrm{iny}} = 0$ para una PKE correcta («inyectividad») y $\varepsilon_{\mathrm{PRF}}$ es la ventaja contra la pseudoaleatoriedad de $H(0, s, \cdot)$. *(Bindel–Hamburg–Hövelmanns–Hülsing–Persichetti, 2019 [14], teorema 2.)*

El término $\varepsilon_{\mathrm{PRF}}$ corresponde a una búsqueda tipo Grover sobre s: con $|S| = 2^{256}$ es inalcanzable. La novedad del QROM es la *raíz cuadrada* del primer término: si se quiere mantener la ventaja IND-CCA2 por debajo de $1/1000$, el teorema exige que ningún ataque OW-CPA con recursos comparables tenga éxito por encima de $1/4\,000\,000$. Dicho de otro modo: *en el QROM, la seguridad probada de un KEM tipo FO es, en bits, la mitad de la seguridad OW-CPA subyacente.* Es una pérdida de ajuste real (la "tightness"), bien conocida, y una de las razones por las que los parámetros de Classic McEliece se eligen con márgenes holgados [29].

La seguridad demostrable tiene grietas y errores

La propia guía de seguridad de Classic McEliece lo advierte con honestidad ejemplar: «la seguridad demostrable, como el criptoanálisis, puede tener lagunas y errores». Bernstein y Persichetti documentan contraejemplos a teoremas de seguridad IND-CCA2 publicados antes de 2018 [12]: conviene tratar estos teoremas como herramientas a revisar, no como cajas negras. Además, una prueba en el QROM no es una prueba en el mundo real, y *ninguna* prueba descarta ataques mucho más rápidos contra OW-CPA: la confianza última en Classic McEliece no viene de estos teoremas, sino del criptoanálisis de la parte IV.

8.5 El KEM de Classic McEliece: tres simplificaciones

Classic McEliece no usa la transformación genérica tal cual: usa una versión específica "para mayor eficiencia", en palabras de sus diseñadores [28], con tres simplificaciones. En la práctica, su Encap y su Decap se leen así (pk= T, la clave pública de $mt \times k$ bits; sk= $(\delta, c, g, \alpha, s)$, los datos del código y la clave de rechazo; capítulos 9–10):

El KEM de Classic McEliece, en dos columnas

Encap(pk)

1. $e \leftarrow$ vector uniforme de peso t (FixedWeight).
2. $C \leftarrow (\mathrm{I}_{mt} \,|\, T)\, e$.
3. $K \leftarrow H(1, e, C)$.
4. salida: (C, K).

Decap(sk, C)

1. $e' \leftarrow$ Decode(C) (Patterson; puede fallar).
2. si e' es válido, $\mathrm{wt}(e') = t$ y $(\mathrm{I}_{mt} \,|\, T)\, e' = C$: $K \leftarrow H(1, e', C)$.
3. si no: $K \leftarrow H(0, s, C)$ (rechazo implícito).
4. salida: K.

Las tres diferencias con la definición 8.3 son:

1. **Sin reencriptar.** La comprobación $\mathsf{Enc}(\mathrm{pk}, e') = C$ no hace falta: el propio Decode ya exige $\mathrm{wt}(e') = t$ y $(\mathrm{I}_{mt}|T)e' = C$. La guía de seguridad llama a esto la *rigidez* de la PKE: si un criptograma se descifra, se descifra a un único e' verificado [29].
2. **Sin clave pública dentro de la clave secreta.** Como el descifrado no reencripta, no necesita pk′; se elimina [29].
3. **La clave de rechazo s es una cadena uniforme de n bits** (no un vector de peso t): es más simple de generar, y los teoremas posteriores de 2017–2019 permiten claves de rechazo más grandes que el espacio de mensajes [44, 12, 14].

Queda un detalle de la especificación que usaremos en el capítulo 10: el hash H tiene *separación de dominio* por etiquetas. La etiqueta 1 deriva claves legítimas, la 0 claves de rechazo y la 2 se reserva para la confirmación de texto claro de las variantes pc (capítulo 12). Es la versión concreta de la idea del capítulo: distintas entradas, distintos espacios de salida, sin ambigüedades.

8.6 Ejercicios

	transformación genérica (definición 8.3)	Classic McEliece (especificación)
clave secreta del KEM	$(\mathrm{sk}', \mathrm{pk}', s)$	(sk', s), sin pk'
verificación al descifrar	reencriptar: $\mathsf{Enc}(\mathrm{pk}, e') = C$	ya la hace Decode: $\mathrm{wt}(e') = t$ y $He' = C$
clave de rechazo s	256 bits uniformes	n bits uniformes ($n \geq 3488$)
semilla e	uniforme en P	FixedWeight, uniforme en peso t

Cuadro 8.2: La transformación genérica y el KEM real, lado a lado. La distribución de e y la corrección del esquema son idénticas; lo que cambia es cuánto material hay que guardar y recalcular.

1. **Corrección del KEM.** Recorra $\mathsf{Decap}(\mathrm{sk}, \mathsf{Encap}(\mathrm{pk}))$ en la definición 8.3 y compruebe que las dos partes calculan la misma clave. Señale el único punto donde se usa que la PKE es *correcta*.

2. **¿Por qué $H(1, e, C)$ y no $H(e)$?** En una implementación real, dos cadenas de bytes distintas pueden decodificar al mismo criptograma C (bits de relleno; véase el capítulo 12): el oráculo de descifrado acepta ambas. Demuestre que un esquema con $K = H(e)$ sería vulnerable en ese escenario: el adversario consulta el oráculo con la segunda codificación de C^* y obtiene K^*. ¿Por qué la misma consulta es inútil con $K = H(1, e, C)$?

3. **Rechazo implícito.** En muchos protocolos, un Decap que devuelve $\perp$ obliga a manejar el error (alertas TLS, reintentos…), y esa reacción es observable. Explique por qué devolver siempre una clave —pseudoaleatoria si hace falta— es una ventaja de seguridad, y en qué parte de los teoremas 8.4 y 8.5 aparece la clave s.

4. **Los números de la cota ROM.** Para `mceliece348864` se tiene $|P| > 2^{456}$ y la entropía efectiva de la clave de rechazo $s \in \mathbb{F}_2^n$ está fijada por la semilla δ ($|S| \geq 2^{256}$). Con $q_D = q_H = 2^{64}$, calcule los valores de los términos segundo y tercero del teorema 8.4. ¿Qué haría falta para que el término $\frac{2q_H}{|S|}$ dejara de ser despreciable?

5. **La raíz cuadrada del QROM.** Usando el teorema 8.5 (ignore los términos de s): (a) ¿qué éxito OW-CPA máximo tolera el teorema si se quiere ventaja IND-CCA2 menor que $1/1000$?; (b) ¿y si se quieren 128 bits de ventaja, es decir, 2^{-128}? Comente por qué esto obliga a elegir parámetros con margen amplio.

6. **¿Por qué no Fujisaki–Okamoto a secas?** El rationale dice que la conversión de Classic McEliece «no es FO genérico» [28]. Explique, con la tabla 8.2 delante, qué ahorra cada una de las simplificaciones (material de clave, reencriptaciones) y por qué la verificación de Decode bastaría para justificarlas.

Cierre de la parte II. La idea de McEliece —esconder un código con decodificador eficiente tras un isomorfismo— quedó convertida en criptosistema (capítulo 6), reformulada en su forma dual con mensajes de peso fijo (capítulo 7) y transformada en un KEM con rechazo implícito (capítulo 8), con sus reducciones a la dureza de la decodificación y sus dos modelos, ROM y QROM. La parte III abre ahora la especificación de Classic McEliece palabra por palabra: semillas, matrices, laboratorios y parámetros.

PARTE III

Classic McEliece

CAPÍTULO 9

La generación de claves

En este capítulo

Aquí empieza el Classic McEliece real. Este capítulo abre la especificación por su primera mitad —la generación de claves— y muestra algo notable: todo el proceso es determinista a partir de una semilla de 256 bits. Veremos cómo se sortea el orden del soporte (FieldOrdering), cómo se sortea el polinomio de Goppa (Irreducible) y cómo se reduce la matriz de paridad a forma sistemática (MatGen), con sus probabilidades de éxito y sus tamaños exactos. Es el capítulo que convierte la teoría de la parte I en ingeniería auditable.

De una propuesta a un estándar

Classic McEliece se presentó al concurso del NIST en 2017 y llegó a la ronda final de KEMs; en 2022 el NIST eligió Kyber (ML-KEM) y en marzo de 2025 cerró su proceso con HQC: Classic McEliece quedó fuera de la selección (capítulo 12). El veredicto no fue un juicio sobre su seguridad —el criptoanálisis no encontró grietas en sus parámetros— sino sobre sus claves de cientos de kilobytes, incómodas para los usos que allí se buscaban. En junio de 2026, ISO/IEC publicó su estandarización en la enmienda 2 de la norma de cifrado asimétrico (ISO/IEC 18033-2:2006/Amd. 2:2026), junto a ML-KEM y FrodoKEM, con los 16 conjuntos de parámetros que veremos en el capítulo 12.

9.1 La idea: una semilla manda

La especificación define la generación de claves en dos niveles. El algoritmo KeyGen no recibe más que los parámetros: sortea una cadena uniforme δ de $\ell = 256$ bits —la *semilla*— y llama a SeededKeyGen(δ). Y SeededKeyGen es completamente *determinista*: la misma δ produce siempre la misma clave. Sus pasos son los siguientes:

1. Sea $E = G(\delta)$ la cadena de $n + \sigma_2 q + \sigma_1 t + \ell$ bits que resulta de aplicar el generador $G = \mathsf{SHAKE256}(64, \delta)$ (primero la etiqueta 64, que lo separa del resto de usos del hash).
2. Los primeros n bits de E son s, la clave de rechazo implícito del capítulo 8.
3. Con los siguientes $\sigma_2 q = 32q$ bits, FieldOrdering produce q elementos distintos $\alpha_0, \ldots, \alpha_{q-1}$ de $\mathbb{F}_q$. Si falla, se reinicia con $\delta \leftarrow \delta'$.
4. Con los siguientes $\sigma_1 t = 16t$ bits, Irreducible produce un polinomio mónico irreducible g de grado t. Si falla, se reinicia con $\delta \leftarrow \delta'$.
5. Sea $\Gamma = (g, \alpha_0, \alpha_1, \ldots, \alpha_{n-1})$: el soporte del código son los primeros n elementos del ordenamiento ($\alpha_n, \ldots, \alpha_{q-1}$ no se usan).
6. Con MatGen(Γ) se calculan la clave pública T y los datos Γ_0 y c (la lista de columnas permutadas, para las variantes `f`). Si falla, se reinicia con $\delta \leftarrow \delta'$.
7. La clave pública es T; la clave secreta es $(\delta, c, g, \alpha, s)$, donde α es el ordenamiento $\alpha_0, \ldots, \alpha_{n-1}$ tras los intercambios de MatGen, seguido de $\alpha_n, \ldots, \alpha_{q-1}$.

El truco del reinicio merece un comentario: los 256 bits finales de E son δ', la semilla del intento siguiente. *El azar no se pide de nuevo al generador externo*: todo el proceso, incluidos los reintentos, es una función determinista de la semilla original. Por eso una clave privada de Classic McEliece podría comprimirse a sus 32 bytes de semilla y regenerarse por completo; la especificación guarda la versión expandida solo por eficiencia.

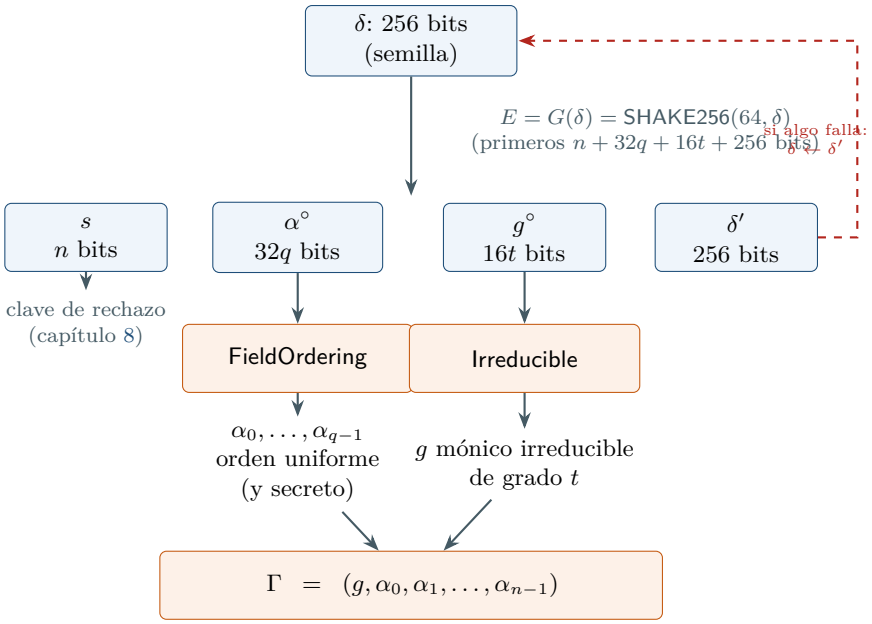


Figura 9.1: La expansión de la semilla. De δ sale todo: s, la materia prima de α y g, y la semilla δ' de un eventual reintento. Los cuerpos son $\mathbb{F}_q = \mathbb{F}_{2^m}$ con $q = 2^m$; el campo de Goppa es $\mathbb{F}_q[y]/F(y)$ con F fijado por los parámetros (tabla 9.1).

9.2 FieldOrdering: los α_i

El soporte de un código de Goppa es una lista de q elementos distintos de $\mathbb{F}_q$: ordenados de una manera y no de otra, definen —como vimos en el capítulo 7— la permutación secreta que esconde la estructura del código. La especificación lo sortea así:

1. Los $\sigma_2 q = 32q$ bits de entrada se leen como q enteros de 32 bits $a_0, \ldots, a_{q-1}$.
2. Si los a_i no son todos distintos, se falla ($\perp$).
3. Se ordenan los pares (a_i, i) lexicográficamente; resulta una permutación π con $a_{\pi(0)} < a_{\pi(1)} < \cdots < a_{\pi(q-1)}$.
4. Se define $\alpha_i = \sum_{j=0}^{m-1} \pi(i)_j \, z^{m-1-j}$, donde $\pi(i)_j$ es el j-ésimo bit menos significativo de $\pi(i)$.

Proposición 9.1: FieldOrdering produce un orden uniforme

Si los $32q$ bits de entrada son uniformes e independientes y el algoritmo no falla, $(\alpha_0, \ldots, \alpha_{q-1})$ es un ordenamiento uniforme de $\mathbb{F}_q$. La probabilidad de no fallar es

$$\prod_{i=1}^{q-1} \left(1 - \frac{i}{2^{32}}\right),$$

que supera 0,99 para $q \le 2^{13}$ y 0,6 para $q \le 2^{16}$.

Demostración. Condicionados a ser distintos, los a_i son un vector uniforme de enteros distintos (simetría de las $q!$ posibilidades). Ordenarlos da una permutación uniforme π, y el mapa $\pi(i) \mapsto \alpha_i$ es una biyección fija de $\{0, \ldots, q-1\}$ en $\mathbb{F}_q$; componer una biyección con una permutación uniforme da un ordenamiento uniforme. La probabilidad de que q enteros uniformes sean distintos es el producto de las probabilidades condicionales $(1 - \frac{i}{2^{32}})$; las cotas numéricas están en la razón de diseño [28]. □

Un ejemplo minúsculo: con $m = 2$ ($q = 4$) y enteros $(7, 1, 9, 3)$, el orden de π es $1 < 3 < 7 < 9$, luego $\pi = (1, 3, 0, 2)$ y, leyendo los bits en el orden de la definición, $(\alpha_0, \alpha_1, \alpha_2, \alpha_3) = (z,\ z{+}1,\ 0,\ 1)$. Con todos los a_i iguales, el algoritmo fallaría: la distinción estricta es lo que garantiza que π sea una permutación de verdad.

La clave privada no guarda la lista como q enteros de m bits, sino la permutación π comprimida como los bits de control de una *red de Beneš*: una red de interconexión con $2m - 1$ etapas de 2^{m-1} conmutadores elementales de 2×2. Cada conmutador toma dos entradas y, según un bit de control $b \in \{0, 1\}$, las deja pasar directas ($b = 0$) o las intercambia ($b = 1$). En total se requieren $(2m - 1)2^{m-1}$ bits de control, empaquetados en $\lceil (2m - 1)2^{m-4} \rceil$ bytes. Para $m = 12$, α ocupa 5888 bytes (la columna α de la tabla 9.2); decapsular recorre la red en sentido inverso y reconstruye el orden exacto. El formato es canónico —los mismos bytes en toda implementación— porque la reproducibilidad de la clave es un requisito de interoperabilidad del estándar, no una manía de sus autores.

¿Por qué una red de Beneš en vez de una tabla?

Si la desencapsulación reconstruyera el orden secreto consultando una tabla $\pi[i]$ en memoria, los accesos con índices secretos provocarían variaciones temporales medibles a través de la memoria caché (ataques de canal lateral). En una red de Beneš, la permutación se aplica mediante una secuencia fija de operaciones lógicas sobre los cables guiadas por los bits de control: las direcciones de memoria y el tiempo de ejecución son *estrictamente independientes* del secreto, garantizando tiempo constante por diseño.

9.3 Irreducible: el polinomio g

El polinomio de Goppa debe ser mónico, irreducible y de grado t; y, para que el sistema sea seguro, debe ser *impredecible*. La especificación lo genera de una forma elegante:

1. Los $\sigma_1 t = 16t$ bits de entrada se parten en t bloques de 16 bits; de cada bloque se usan los primeros m bits para formar $\beta_j = \sum_{i=0}^{m-1} d_{\sigma_1 j + i} z^i$, y se ignora el resto.
2. Se construye $\beta = \beta_0 + \beta_1 y + \cdots + \beta_{t-1} y^{t-1}$ en el cuerpo $\mathbb{F}_q[y]/F(y)$ (de tamaño q^t).
3. Se calcula el polinomio minimal g de β sobre $\mathbb{F}_q$. Si su grado es t, se devuelve; si no, se falla ($\perp$).

El polinomio minimal vía álgebra lineal

El *polinomio minimal* de un elemento $\beta \in \mathbb{F}_{q^t}$ sobre $\mathbb{F}_q$ es el polinomio mónico de menor grado no nulo con coeficientes en $\mathbb{F}_q$ que satisface $g(\beta) = 0$. Dado que $\mathbb{F}_{q^t}$ tiene dimensión t como espacio vectorial sobre $\mathbb{F}_q$, los $t + 1$ vectores $\{1, \beta, \beta^2, \ldots, \beta^t\}$ son necesariamente linealmente dependientes. Encontrar la combinación lineal mónica $c_0 + c_1\beta + \cdots + \beta^t = 0$ es simplemente resolver un sistema lineal homogéneo sobre $\mathbb{F}_q$ mediante eliminación gaussiana. Si no existe dependencia lineal de grado menor que t, el polinomio resultante g tiene grado t

y es automáticamente irreducible sobre $\mathbb{F}_q$.

Proposición 9.2: Irreducible produce un polinomio uniforme

Si los $16t$ bits de entrada son uniformes, la salida g (cuando no se falla) es un polinomio mónico irreducible de grado t uniforme, y la probabilidad de éxito es mayor que 0,99 para $t \geq 16$.

Demostración. β es un elemento uniforme del cuerpo $\mathbb{F}_{q^t}$. Cada polinomio mónico irreducible g de grado t tiene exactamente t raíces en $\mathbb{F}_{q^t}$ (divide a $y^{q^t} - y$, que se escinde con raíces simples), y son exactamente los β cuyo minimal es g: cada g aparece por tanto t de las q^t elecciones de β, es decir, con la misma probabilidad. El algoritmo falla únicamente cuando β cae en un subcuerpo propio de $\mathbb{F}_{q^t}$; el número de tales elementos está dominado por $q^{t/2+\varepsilon}$ (suma sobre divisores propios de t), de modo que, para $t \geq 16$, el fallo es inferior al 1 % (en nuestros parámetros, muchísimo menos) [28]. □

9.4 MatGen: de Γ a la clave pública

Con g y el ordenamiento α a mano, MatGen fabrica la clave pública. En el caso sistemático ($(\mu, \nu) = (0, 0)$, los conjuntos sin sufijo `f`):

1. Se construye la matriz $\tilde{H}$, de $t \times n$ sobre $\mathbb{F}_q$, con entradas $h_{i,j} = \alpha_j^i / g(\alpha_j)$ para $i = 0, \ldots, t-1$ y $j = 0, \ldots, n-1$. Es la matriz de paridad clásica del código de Goppa de la definición del capítulo 4.
2. Se expande cada entrada (un elemento de $\mathbb{F}_q$, es decir, m bits $u_0, \ldots, u_{m-1}$) a una columna de m bits: resulta $\hat{H}$, de $mt \times n$ sobre $\mathbb{F}_2$.
3. Se reduce $\hat{H}$ a forma sistemática $(\mathrm{I}_{mt} \,|\, T)$ mediante eliminación gaussiana. Si no es posible, se falla ($\perp$).
4. Se devuelve T: la clave pública, de $mt \times k$ bits.

¿Por qué puede fallar el paso 3? La eliminación por filas no puede mover columnas: existe la forma $(\mathrm{I}_{mt} \,|\, T)$ exactamente cuando las primeras mt columnas de $\hat{H}$ son linealmente independientes. Para una matriz binaria «genérica» eso ocurre con probabilidad

$$\prod_{i=0}^{mt-1} \left(1 - 2^{\,i-mt}\right) \approx 0{,}289,$$

es decir, falla en torno al 71 % de los intentos: se necesitan $\sim 3{,}4$ de media, cada uno con su semilla nueva $\delta \leftarrow \delta'$ [28]. Las variantes `f` admiten la forma *semi-sistemática* $(\mu, \nu) = (32, 64)$: se permiten hasta $\mu = 32$ intercambios de columna dentro de una ventana de $\nu = 64$, lo que reduce el fallo por intento a menos de 2^{-30} (prácticamente, un solo intento); los intercambios quedan registrados en la lista c de la clave secreta [28]. La conversión cambia la seguridad en, a lo sumo, 2 bits, y la estudiaremos en el capítulo 12.

Observación 9.3: El código es público; el secreto es la descripción

Tras MatGen, cualquiera puede calcular el código $\ker H$ con $H = (\mathrm{I}_{mt}|T)$: el subespacio no es secreto. Lo que guarda la clave privada es la *descripción eficiente* del código —el polinomio g y el orden de las α_i—, la única que permite decodificar con Patterson. Esta distinción se

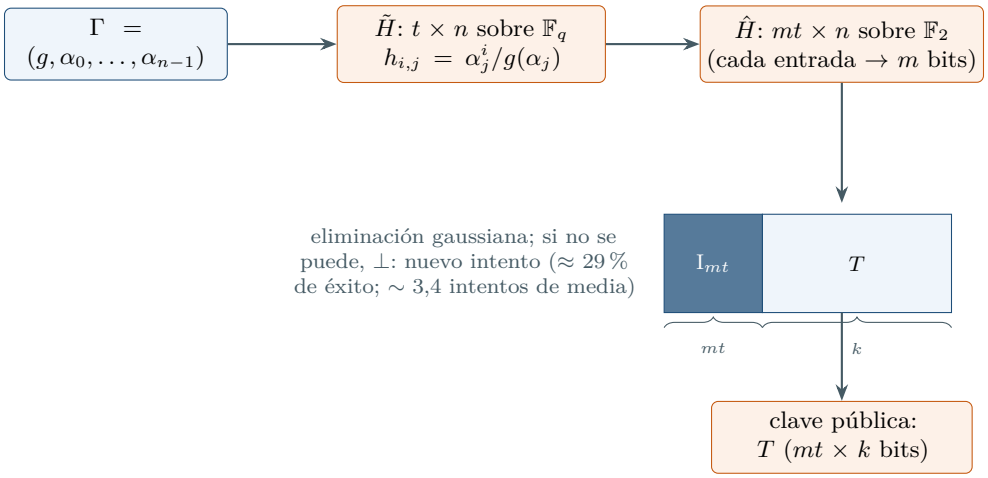


Figura 9.2: MatGen en tres actos. La matriz de Goppa $t \times n$ sobre $\mathbb{F}_q$ se expande bit a bit a una matriz binaria $mt \times n$; la eliminación gaussiana la lleva a la forma sistemática $(\mathrm{I}_{mt} \,|\, T)$ y publica el bloque T. La permutación secreta del capítulo 7 es aquí el ordenamiento α: las columnas de $\tilde{H}$ están etiquetadas por elementos del cuerpo, en un orden que solo conoce la clave secreta.

> confunde a menudo y es esencial: un atacante que descubriera g y el ordenamiento tendría un decodificador; un atacante con solo el código en la mano tiene el problema duro de la parte IV.

9.5 Los tamaños reales

Los cinco conjuntos principales fijan el cuerpo y los polinomios de la tabla 9.1; la tabla 9.2 da los tamaños de las claves.

Conjunto	m	n	t	$k = n - mt$	polinomios $f(z)$ (cuerpo) y $F(y)$ (Goppa)
`mceliece348864`	12	3488	64	2720	$z^{12}+z^3+1$; $y^{64}+y^3+y+z$
`mceliece460896`	13	4608	96	3360	$z^{13}+z^4+z^3+z+1$; $y^{96}+y^{10}+y^9+y^6+1$
`mceliece6688128`	13	6688	128	5024	$z^{13}+z^4+z^3+z+1$; $y^{128}+y^7+y^2+y+1$
`mceliece6960119`	13	6960	119	5413	$z^{13}+z^4+z^3+z+1$; $y^{119}+y^8+1$
`mceliece8192128`	13	8192	128	6528	$z^{13}+z^4+z^3+z+1$; $y^{128}+y^7+y^2+y+1$

Cuadro 9.1: Los cinco conjuntos sin sufijo (véanse las variantes `f`, `pc` y `pcf` en el capítulo 12). f define el cuerpo $\mathbb{F}_q = \mathbb{F}_2[z]/f(z)$; F define $\mathbb{F}_q[y]/F(y)$, el cuerpo donde vive β para Irreducible.

Dos observaciones sobre la tabla. La primera: la clave pública tiene el tamaño $k(n-k)$ que prometía el capítulo 7 ($mt \cdot k$ bits); en los conjuntos `mceliece6*` ronda el megabyte, y el equipo de Classic McEliece recomienda no bajar de $n = 6688$ para seguridad a largo plazo. La segunda: en la clave secreta, el ordenamiento α pesa más que todo lo demás junto —es una permutación de q elementos—, y aun así podemos regenerarlo entero desde los 32 bytes de δ.

9.6 Ejercicios

Conjunto	pk (bytes)	δ (bytes)	c (bytes)	g (bytes)	α (bytes)	s (bytes)	sk (bytes)
`mceliece348864`	261 120	32	8	128	5888	436	6492
`mceliece460896`	524 160	32	8	192	12800	576	13 608
`mceliece6688128`	1 044 992	32	8	256	12800	836	13 932
`mceliece6960119`	1 047 319	32	8	238	12800	870	13 948
`mceliece8192128`	1 357 824	32	8	256	12800	1024	14 120

Cuadro 9.2: Tamaños de las claves. La pública ocupa $mt\lceil k/8\rceil$ bytes: fila a fila, la matriz T con $\lceil k/8\rceil$ bytes por fila. La secreta suma la semilla δ (32), la lista c (8 en ambos tipos de conjunto: $\lceil \nu/8\rceil$ o la codificación fija de $2^{32}-1$ para $(\mu,\nu)=(0,0)$), el polinomio g ($t\lceil m/8\rceil$), el ordenamiento α (la red de Benes, $\lceil(2m-1)2^{m-4}\rceil$ bytes: $2m-1$ etapas de 2^{m-1} conmutadores de un bit) y la clave de rechazo s ($\lceil n/8\rceil$).

1. **Reproducibilidad.** Suponga que, en el paso 3, FieldOrdering falla por un choque entre enteros. Explique por qué el proceso completo sigue siendo determinista y cómo las pruebas de vectores de la especificación pueden ejecutar (y comparar) los reintentos sin generar azar nuevo.

2. **FieldOrdering con el juguete.** Con $m=2$ ($q=4$) y el vector de enteros de 32 bits $(7,1,9,3)$, reproduzca π y deduzca $(\alpha_0,\ldots,\alpha_3)$. Repita con $(0,1,2,3)$ y con $(0,0,0,0)$; explique qué ocurre en el último caso.

3. **Uniformidad de Irreducible.** Demuestre con detalle que cada polinomio mónico irreducible de grado t aparece con exactamente t de las q^t elecciones de β; concluya la uniformidad de g.

4. **La probabilidad del 29 %.** Deduzca $\prod_{i=0}^{mt-1}(1-2^{i-mt})\approx 0{,}289$ como probabilidad de que una matriz binaria $mt\times n$ «genérica» tenga sus primeras mt columnas independientes. ¿Cuántos intentos de media se necesitan? ¿Qué compra la forma semi-sistemática $(\mu,\nu)=(32,64)$?

5. **Tamaños.** Verifique con las fórmulas del pie de la tabla 9.2 los números de `mceliece8192128`: $pk=1357824$ bytes y $sk=14120$ bytes. ¿Qué componente domina la clave secreta, y cuál la pública?

6. **El secreto es la descripción.** Argumente que el código $\ker(\mathrm{I}_{mt}|T)$ es público y calcule, para `mceliece348864`, su dimensión y su número de codewords. ¿Qué gana un atacante que descubra g y el ordenamiento α? Relacione la respuesta con la observación 9.3 y con la proposición 7.4 del capítulo 7.

CAPÍTULO 10

Cifrado y descifrado

En este capítulo

Este capítulo recorre las rutinas que ejecuta el KEM en cada operación —FixedWeight, Encode, Decode y el hash final— en el orden exacto de la especificación. Veremos por qué el mensaje de un KEM niederreiteriano es un vector de peso exacto, cómo se descifra con Patterson sobre el soporte secreto y por qué un último chequeo (peso y síndrome) es imprescindible para que la seguridad IND-CCA2 sea la del capítulo 8. Al terminar, el lector podrá leer la especificación línea a línea sin sorpresas.

Por qué «Classic»

El nombre no es nostalgia. A partir de 2017, el NIST recibió docenas de propuestas basadas en códigos —cuasicíclicas, de paridad moderada, convolucionales...— que ahorraban bytes añadiendo estructura. El equipo de McEliece presentó lo contrario: el McEliece clásico, con códigos de Goppa binarios, de 1978, sin estructura adicional. Frente a los trucos que ahorran espacio a costa de nueva superficie de ataque, la apuesta fue pagar claves grandes y conservar casi medio siglo de criptoanálisis limpio. La apuesta le valió un estándar ISO en 2026 y sigue sin roturas algorítmicas conocidas.

10.1 FixedWeight: la semilla del cifrador

En un cifrado de ElGamal se sortea un exponente; en un KEM niederreiteriano, el azar que hay que sortear es un *vector de peso exacto*. La especificación genera e con FixedWeight, que no recibe nada y devuelve $e \in \mathbb{F}_2^n$ con $\mathrm{wt}(e) = t$ uniforme. El algoritmo usa lotes de τ enteros, con $\tau = t$ si $n = q$ y $\tau = 2t$ si $q/2 \le n < q$ (todos nuestros conjuntos cumplen una de las dos cosas):

1. Se generan $\sigma_1\tau = 16\tau$ bits uniformes $b_0, \ldots, b_{16\tau-1}$.
2. Cada bloque de 16 bits produce un entero $d_j = \sum_{i=0}^{m-1} b_{16j+i} 2^i$ para $j = 0, \ldots, \tau - 1$ (solo se usan los primeros m bits de cada bloque).
3. De la lista $d_0, \ldots, d_{\tau-1}$ se toman las primeras t entradas *en el rango* $\{0, \ldots, n-1\}$. Si hay menos de t, se reinicia.
4. Si esas t entradas no son todas distintas, se reinicia.
5. Se define e como el vector de peso t cuyo soporte son esas t posiciones.

Proposición 10.1: FixedWeight es exactamente uniforme

La salida e de FixedWeight (cuando no se reinicia) está uniformemente distribuida sobre los $\binom{n}{t}$ vectores de peso t. La probabilidad de que cada intento supere la prueba de distinción es $\prod_{i=1}^{t-1}(1 - \frac{i}{n})$, mayor que $1/4$ en todos los conjuntos; y la de que el lote aporte t entradas en rango supera 0,96 en el peor caso, `mceliece460896` [28].

Demostración. Los d_j son independientes y uniformes en $\{0,\dots,q-1\}$; la pertenencia al rango $\{0,\dots,n-1\}$ es así independiente para cada j, y condicionada a qué posiciones caen en rango, sus valores son uniformes en el rango (el submuestreo de una sucesión uniforme por un criterio independiente de los valores lo es). La primera vez que aparecen t entradas en rango no es distinto: se toma la subsucesión de entradas en rango, que hereda la uniformidad. Finalmente, exigir que las t entradas sean distintas, por rechazo, deja una sucesión ordenada uniforme de t valores distintos, es decir, un soporte uniforme entre los $\binom{n}{t}$ posibles. Las probabilidades son las del argumento de cumpleaños, con las cotas de la razón de diseño [28]. □

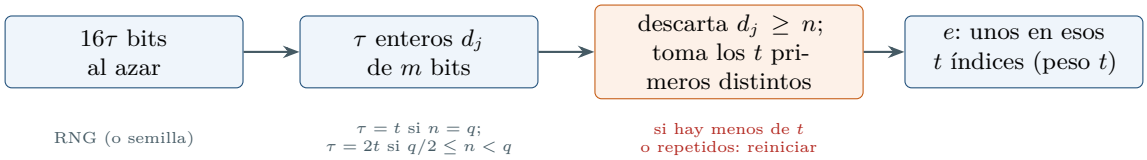


Figura 10.1: FixedWeight: de bits al azar a un soporte de tamaño t exacto. El lote de τ enteros abarata el muestreo y facilita la vectorización; el rechazo garantiza uniformidad exacta (proposición 10.1). Los valores τ son $128, 192, 256, 238, 128$ para los cinco conjuntos de la tabla 9.1.

10.2 Encode: el criptograma es un síndrome

Con e en la mano, encapsular es casi gratis:

$$C = (\mathrm{I}_{mt} \,|\, T)\, e \in \mathbb{F}_2^{mt}.$$

Recordemos el diccionario del capítulo 8: todo en columnas; T es la clave pública de $mt \times k$; escribir $e = (e_{\text{alto}}, e_{\text{bajo}})$ con $e_{\text{alto}} \in \mathbb{F}_2^{mt}$ y $e_{\text{bajo}} \in \mathbb{F}_2^{k}$ permite ver que la multiplicación real usa solo T:

$$C = e_{\text{alto}} + T\, e_{\text{bajo}}.$$

Ese es todo el cifrado: $mt \cdot k$ bits de multiplicación matriz–vector, unos 0,25 MB de operaciones de XOR para el conjunto más pequeño. La ecuación (10.2) es también la razón por la que C no se puede «descifrar» sin la clave secreta: encontrar e de peso t con $He = C$ es *exactamente* el problema duro de la parte IV (proposición 7.4).

10.3 Decode: Patterson con el soporte secreto

Descifrar es lo caro, y es donde vive la trampa de Goppa. La entrada es C y la clave secreta; Decode procede así:

1. Se extiende C con k ceros: $v = (C, \mathbf{0}, \dots, \mathbf{0}) \in \mathbb{F}_2^n$.
2. Se busca el único $c \in \mathbb{F}_2^n$ con (1) $Hc = 0$ y (2) la distancia de Hamming entre c y v es $\leq t$. Si no existe, se devuelve $\perp$.
3. Se define $e = v + c$.
4. Si $\mathrm{wt}(e) = t$ y $C = He$, se devuelve e; si no, $\perp$.

El paso 2 es el descifrado de Patterson del capítulo 4, aplicado al código Γ_0 que definen g y el ordenamiento $\alpha'_0, \ldots, \alpha'_{n-1}$ (la versión «ya permutada» que guarda la clave secreta). El truco del relleno de ceros que ya usamos en el capítulo 7 es justo el que permite aplicarlo tal cual: como $v + e$ es un codeword ($H(v+e) = C + C = 0$), el síndrome de Goppa de v coincide con el de e, y Patterson trabaja directamente sobre la información recibida. Para un criptograma legítimo, $c = v + e$ es un codeword a distancia t de v, y la unicidad del paso 2 se debe a la distancia mínima: si $c_1 \neq c_2$ estuvieran ambas a distancia $\leq t$ de v, entonces $\mathrm{wt}(c_1 + c_2) \leq 2t < d$, imposible en un código de distancia d.

El paso 4 no es decorativo, y merece un párrafo. Si C es un criptograma legítimo, el chequeo pasa siempre (lo comprobamos en los ejercicios). Pero C también admite valores «tramposos»: por ejemplo, $C = He'$ con $\mathrm{wt}(e') = t - 1$. Entonces $v + e'$ *sí* es un codeword, Patterson lo encuentra, y el paso 2 devuelve una distancia menor que t: exactamente lo que el chequeo rechaza. Sin él, el descifrador aceptaría mensajes de peso distinto de t, la imagen de Encode dejaría de ser el conjunto de los vectores de peso exacto —la uniformidad que exige el capítulo 8— y la «rigidez» que sustituye a la reencriptación dejaría de valer. Este chequeo es, en miniatura, toda la transformación FO: verificación exacta de que el criptograma es legítimo, y ningún fallo distinguible (el rechazo implícito del paso final).

10.4 Una vuelta completa

Juntando todo, la especificación ejecuta esto:

$\mathsf{Encap}(\mathrm{pk} = T)$	$\mathsf{Decap}(\mathrm{sk}, C)$
1. $e \leftarrow \mathsf{FixedWeight}()$	1. $b \leftarrow 1$
2. $C \leftarrow (\mathrm{I}_{mt} \mid T)e$	2. extraer s y $\Gamma_0 = (g, \alpha'_0, \ldots, \alpha'_{n-1})$ de la clave secreta
3. $K \leftarrow H(1, e, C)$	3. $e' \leftarrow \mathsf{Decode}(C, \Gamma_0)$; si $e' = \perp$: $e' \leftarrow s$, $b \leftarrow 0$
4. salida (C, K)	4. $K \leftarrow H(b, e', C)$
	5. salida K

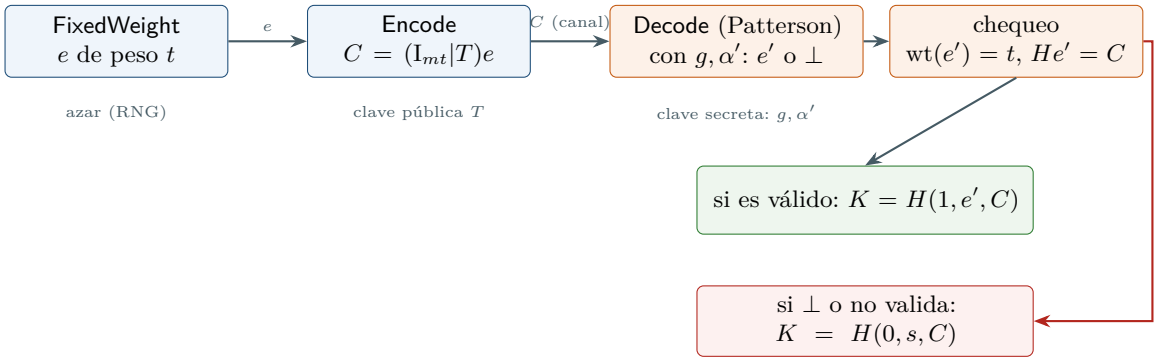


Figura 10.2: El recorrido completo. En azul, lo que hace Encap con la clave pública; en naranja, lo que hace Decap con la secreta; en verde y rojo, las dos formas de terminar. La especificación unifica el hash final como $K = H(b, v, C)$: con $b = 1$ y $v = e'$ si el descifrado valida, y con $b = 0$ y $v = s$ (los n bits secretos) si no.

Sobre el hash, la especificación es precisa:

$$H(b, v, C) = \text{primeros 256 bits de } \mathsf{SHAKE256}\big(\mathrm{byte}(b) \,\|\, \mathrm{bytes}(v) \,\|\, \mathrm{bytes}(C)\big),$$

con v codificado en $\lceil n/8 \rceil$ bytes y C en $\lceil mt/8 \rceil$ bytes. El byte inicial 0 o 1 es la separación de dominio del capítulo 8: las claves legítimas y las de rechazo viven en «mundos» distintos del hash, y ningún criptograma puede hacer colisionar ambos. Para `mceliece348864`, la entrada del hash ocupa $1 + \lceil 3488/8 \rceil + \lceil 768/8 \rceil = 533$ bytes; su salida, 32.

Dos detalles finos que el lector encontrará en los archivos de prueba de la especificación: (1) b es un bit, pero se escribe como byte entero; (2) en los conjuntos donde n o mt no son múltiplos de 8 (`mceliece6960119`: $mt = 1547$), los bits sobrantes del último byte son *relleno*. La especificación define dos modos de lectura: *Simply Decoded* (ignora el relleno) y *Narrowly Decoded* (rechaza criptogramas con relleno distinto de cero). Volveremos a esta distinción —que es una forma benigna de maleabilidad— en el capítulo 12.

10.5 ¿Cuánto cuesta?

Los costos, en orden de magnitud y por operación, son los siguientes.

- Encap: un muestreo de peso t y una multiplicación binaria $mt \times k$: para `mceliece348864`, unos 261120 bytes de lectura y XOR con un acumulador de 96 bytes; para `mceliece8192128`, unos 1,3 MB. Es comparable a una exponenciación modular pequeña, pero con operaciones de bits: decenas de microsegundos en hardware moderno.
- Decap: el síndrome en $\mathbb{F}_q$ cuesta del orden de tn operaciones en el cuerpo (unos 223 000 productos para `mceliece348864`); Patterson añade $O(t^2)$ y la búsqueda de raíces un coste similar. Sigue siendo, con diferencia, la operación más lenta —milisegundos o menos en implementaciones optimizadas—, pero muy lejos de la decodificación genérica de la parte IV: la trampa hace su trabajo [23].
- Memoria: Decap toca la clave secreta y el criptograma; nada más. La semilla δ no se usa en línea (solo para regenerar la clave), y T no se usa para descifrar: la eliminación de la clave pública que vimos en el capítulo 8.

Observación 10.2: Patterson no es opcional

El descifrado de un código de Goppa *no* es «buscar el codeword más cercano» de forma exhaustiva: es un algoritmo algebraico (Patterson, 1975) que usa el polinomio g y el soporte para resolver el problema con álgebra de polinomios. Ese algoritmo es la trampa, y por eso su descripción (la clave secreta) es lo único que no puede salir de la caja. La distancia mínima garantiza además que, cuando el descifrado existe, es único: la trampa no es solo eficiente, es inequívoca.

10.6 Ejercicios

1. **La ruta de v (laboratorio).** En el juguete del capítulo 5 ($n = 16$, $k = 8$, $t = 2$, soporte $\{10, 13\}$), escriba $v = (C, \mathbf{0})$ y compruebe con las matrices del laboratorio que $c = v + e$ satisface $Hc = 0$ y que su distancia a v es t. Repita el cálculo para el síndrome tramposo $C' = He'$ con $\mathrm{wt}(e') = 1$ y describa qué devolvería cada paso de Decode.
2. **Uniformidad de FixedWeight.** Repita la demostración de la proposición 10.1 señalando (a) dónde se usa que $16 \geq m$; (b) dónde se usa la independencia; (c) por qué el rechazo por repeticiones no sesga el resultado. Calcule $\prod_{i=1}^{t-1}(1 - i/n)$ para el juguete ($n = 16$, $t = 2$) y para `mceliece348864` (use $e^{-t(t-1)/(2n)}$ como aproximación).

3. **Unicidad del candidato.** Demuestre con detalle que en el paso 2 no puede haber dos codewords distintos c_1, c_2 con distancia $\leq t$ a v. (Pista: $\mathrm{wt}(c_1 + c_2) \leq 2t$ y la distancia mínima del código de Goppa es $\geq 2t + 1$.)

4. **El chequeo imprescindible.** Con el juguete, fabrique $C' = He'$ con $\mathrm{wt}(e') = 1$ y explique qué devolverían el paso 2 y el paso 4. ¿Qué pasaría con la uniformidad de los mensajes aceptados si suprimiéramos el chequeo? ¿Cómo se relaciona esto con la rigidez del capítulo 8?

5. **Cuentas de costos.** Para `mceliece8192128` ($m = 13$, $n = 8192$, $t = 128$, $k = 6528$): calcule los bits que toca Encode ($mt \cdot k$) y las operaciones de cuerpo de un síndrome de Decode (tn). ¿Cuál de las dos operaciones domina, y por qué eso es exactamente lo que busca un KEM (cifrar barato, descifrar con clave)?

6. **La entrada del hash.** Para `mceliece6960119`: calcule los bytes de la entrada $H(b, v, C)$ y explique para qué sirve cada pieza (etiqueta, vector, criptograma). ¿Por qué el hash no recibe la lista de posiciones del soporte de e en vez del vector entero?

Laboratorio 2: el KEM completo

En este capítulo

El juguete del capítulo 5 decodificaba; los capítulos 8 a 10 definieron el KEM. Este laboratorio enchufa las dos cosas: genera una clave del juguete desde una semilla (como el capítulo 9), encapsula, desencapsula y comprueba la igualdad exacta de las claves de 32 bytes. Después rompe el descifrado a propósito, cambiando un solo bit del criptograma, y comprueba que el rechazo implícito no deja rastro. Todos los números de este capítulo se reproducen ejecutando el programa del apéndice A.

Los vectores de prueba del NIST son esto

La especificación de Classic McEliece distribuye archivos *known-answer test* (KAT): para cada conjunto y cada semilla δ, dan la clave pública, la privada, el criptograma y la clave de sesión esperados. La razón de ser es exactamente la de este laboratorio: que cualquier implementación independiente reproduzca *bit a bit* los mismos resultados. Nuestra tabla del juguete es un KAT en miniatura, escrito a mano.

11.1 La clave del juguete (KeyGen en miniatura)

Fijamos, como el generador de claves, una semilla $\delta =$ `5a5a...5a` (el byte `5a` repetido 32 veces) y aplicamos la expansión del capítulo 9:

- $G = \mathsf{SHAKE256}(64 \| \delta)$. Sus primeros $n = 16$ bits son la clave de rechazo:

$$s = \texttt{0111 0101 1101 0011} \quad \text{(bit 0 a la izquierda)},$$

  esto es, $s =$ `CBAE` como entero, con bytes little-endian `AE CB`.
- El «FieldOrdering» del juguete es el orden natural: α_j es el elemento de $\mathbb{F}_{2^4}$ cuyo entero vale j, para $j = 0, \ldots, 15$ (nótese que s, en cambio, es una cadena de bits *uniforme*, no un vector de peso t: exactamente lo que ordena la especificación).
- El polinomio de Goppa es $g(x) = x^2 + x + z^3$, el irreducible del capítulo 5 (comprobado: $g(\alpha) \neq 0$ para todo $\alpha \in \mathbb{F}_{2^4}$).
- MatGen produce, como ya vimos, $(\mathrm{I}_8 \,|\, T)$ con la T de 8×8 bits del capítulo 5.

Así, la clave pública del juguete es T (¡8 bytes!) y la secreta es (δ, g, α, s). Todo lo demás del KEM son cuentas.

11.2 Encap: de la semilla al criptograma

FixedWeight sortea el error $e =$ `0000000000100100` (posiciones 10 y 13, como en todas las cuentas del juguete). Para cifrar, separamos e en dos mitades de 8 bits:

$$e_{\text{alto}} = \texttt{00000000}, \qquad e_{\text{bajo}} = \texttt{00100100},$$

y calculamos $C = e_{\text{alto}} + T\, e_{\text{bajo}}$. La mitad alta es cero, así que basta combinar las columnas 2 y 5 de T, que son las que selecciona e_{bajo}:

$$C = \text{col}_2(T) \oplus \text{col}_5(T) = \texttt{36} \oplus \texttt{AE} = \texttt{98} \quad \Longrightarrow \quad C = \texttt{00011001}.$$

El hash final recibe la etiqueta y las codificaciones en bytes little-endian de e y C: $e = 9216 = \texttt{2400}$ aporta los bytes $\texttt{00}\,\texttt{24}$, y C aporta $\texttt{98}$:

$$K = H(1, e, C) = \mathsf{SHAKE256}(\texttt{01} \,\|\, \texttt{00}\,\texttt{24} \,\|\, \texttt{98}),$$
$$K = \texttt{597f46fbba325c68f0b0b45c6ea984961461eb220d1f24024ed888912b336278}.$$

El criptograma es, entonces, $C = \texttt{98}$ (un solo byte) y la clave de sesión, 32 bytes.

11.3 Decap: la vuelta, paso a paso

Del otro lado, con la clave secreta, el descifrado repite el ejercicio del capítulo 5 pero sobre $v = (C, \mathbf{0}) = \texttt{0001100100000000}$ (el relleno de ceros del que hablaba el capítulo 10). Patterson calcula el localizador:

$$S(x) = \texttt{4} + \texttt{9}x, \qquad \sigma(x) = \texttt{5} + \texttt{e}x + \texttt{2}x^2, \qquad \text{raíces: } \{10, 13\},$$

las mismas cuentas que la traza del capítulo 5 (§ «Patterson, paso a paso»), como debe ser: el síndrome de $v = e + c$, con $c \in \Gamma$, es el de e, porque para todo codeword $\sum_j 1/(x+\alpha_j) \equiv 0 \pmod{g}$. Encontrado $e' = \texttt{0000000000100100} = e$, el chequeo final pasa ($\text{wt}(e') = 2$ y $(\text{I}_8|T)e' = C$) y la clave es la misma:

$$K' = H(1, e', C) = \mathsf{SHAKE256}(\texttt{01}\|\texttt{00}\,\texttt{24}\|\texttt{98}) = K.$$

La figura 11.1 resume la vuelta completa con todos los valores a la vista.

11.4 Un criptograma manipulado

Cambiemos un solo bit del criptograma: $\tilde{C} = \texttt{10011001}$, el resultado de $C \oplus \texttt{01}$. La especificación no responde con un error: intenta descifrar y, al fallar, recurre a la clave de rechazo. Los números son los siguientes:

- El síndrome cambia: $\tilde{S}(x) = \texttt{b} + \texttt{6}x$, y Patterson llega a $\tilde{\sigma}(x) = \texttt{5} + \texttt{8}x + \texttt{a}x^2$, que *no tiene raíces* en $\mathbb{F}_{2^4}$: no hay error de peso ≤ 2 que explique $\tilde{C}$.
- Con el fallo, Decap pone $b \leftarrow 0$ y $e \leftarrow s$, y devuelve

$$K'' = H(0, s, \tilde{C}) = \mathsf{SHAKE256}(\texttt{00} \,\|\, \texttt{AE}\,\texttt{CB} \,\|\, \texttt{99}),$$
$$K'' = \texttt{ba1a72ac26b0fcf20c55c108bb681e27aa9c270c60802a821601130b7f1e1494}.$$

- Nadie distingue K'' de una clave legítima: es la salida de un hash sobre entradas distintas, de 32 bytes como todas. Ese es el rechazo implícito del capítulo 8 en acción.

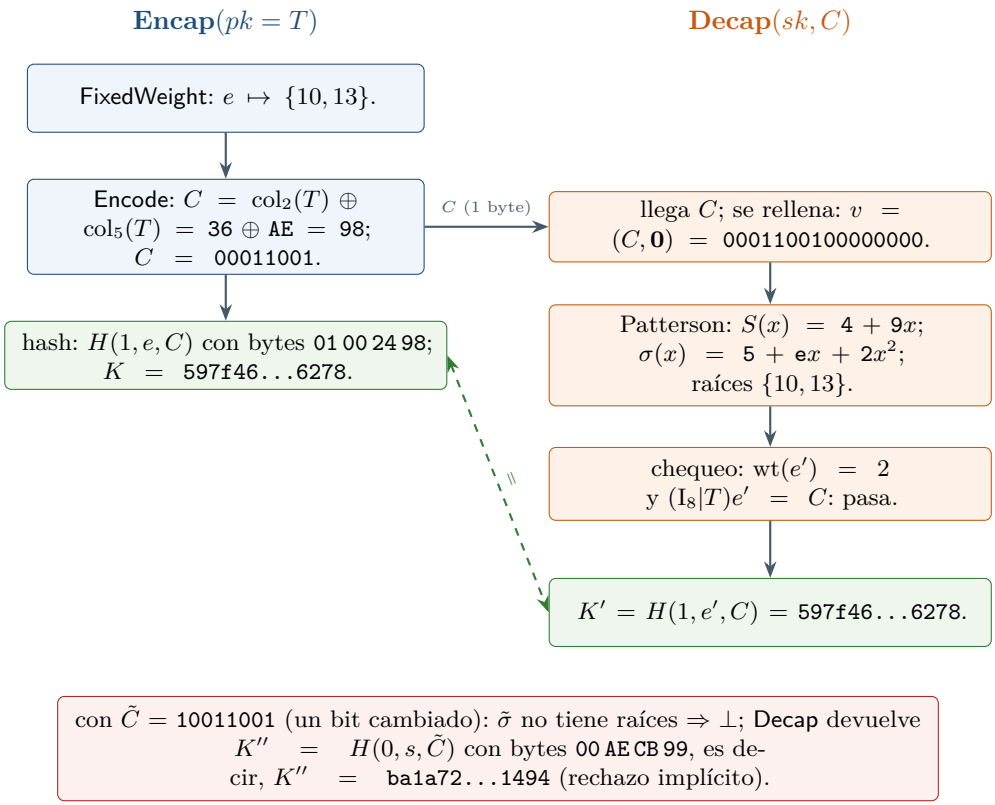


Figura 11.1: El KEM del juguete, con todos los números. A la izquierda, Encap: del error al criptograma y a la clave. A la derecha, Decap: del criptograma al error y a la misma clave (el signo = verde). Abajo, el caso manipulado: un bit distinto y una clave completamente distinta, indistinguible de una legítima.

Hay una estadística del juguete que conviene mirar de cerca: de los $2^8 = 256$ síndromes posibles, solo 137 son alcanzables por errores de peso ≤ 2:

$$1 + 16 + 120 = 137 \qquad (\text{peso } 0, 1, 2).$$

Patterson resuelve esos 137, pero Decap solo acepta los 120 de peso exactamente 2: los 17 de peso 0 o 1 se rechazan por el chequeo de peso, igual que los 119 síndromes inalcanzables. Es decir, un criptograma manipulado al azar se rechaza con probabilidad $136/256 \approx 53\,\%$: más de la mitad de las veces. Sin rechazo implícito, esa mayoría de fallos sería una señal detectable; con él, Decap es una función total que siempre devuelve 32 bytes.

11.5 Del juguete al estándar (y qué no prueba el juguete)

La tabla 11.1 pone los dos mundos al lado.

	juguete	`mceliece348864`
m (bits por elemento)	4	12
n (longitud)	16	3488
t (peso del error)	2	64
k (dimensión)	8	2720
clave pública T	8 bytes	261 120 bytes (255 KiB)
criptograma C	1 byte	96 bytes
semilla δ	32 bytes	32 bytes
clave de sesión K	32 bytes	32 bytes
seguridad de la decodificación	nula (2^{16} palabras)	≈ 141 bits (capítulo 13)

Cuadro 11.1: El juguete y el primer conjunto real. Los elementos criptográficos «gruesos» —semilla y clave de sesión— ya tienen el tamaño definitivo; lo que crece es la matemática: código, matriz pública y criptograma.

El juguete demuestra que la *tubería* funciona: las mismas etiquetas, el mismo relleno, el mismo rechazo implícito, las mismas cuentas de bytes. Lo que no demuestra —ni puede— es seguridad: con $n = 16$, cualquiera decodifica por fuerza bruta, y el problema «duro» no es duro. Esa parte vive en la parte IV, y es la única que no se puede simular en pequeño.

11.6 Ejercicios

1. **Reproducir la clave.** Ejecute el programa del apéndice A (o reimplemente sus tres pasos) y compruebe que SHAKE256(`01`$\|$`00 24`$\|$`98`) produce exactamente la K de la figura 11.1. Compruebe también $s =$ `CBAE` y sus bytes.

2. **Bits y bytes.** Explique por qué el entero $e = 9216 =$ `2400` se codifica como `00 24` (little-endian) y cómo se concilia con la convención «posición 0 a la izquierda». Repita el razonamiento para s.

3. **El síndrome que no cambia.** Demuestre que, si $v = (C, \mathbf{0})$ y $C = He$, entonces el síndrome polinomial de v coincide con el de e. (Pista: recuerde que $\sigma_j(x) = 1/(x + \alpha_j)$ cumple $\sum_j c_j \sigma_j \equiv 0 \pmod g$ para todo codeword c.) ¿Por qué este hecho *no* se cumple para $\tilde{C}$?

4. **¿Por qué no devolver $\perp$?** Suponga que Decap respondiera «error» cuando el descifrado falla. Describa cómo un atacante usaría el bit manipulado $\tilde{C}$ para distinguir respuestas, y qué papel juegan en ello los 119 síndromes inalcanzables de la sección anterior.

5. **Contar síndromes.** Verifique $1 + 16 + 120 = 137$ y $1 + 16 + 120 + \binom{16}{3} + \cdots$ (¿por qué los errores de peso 3 ya *no* están garantizados como únicos?). Relacione $137/256 \approx 53{,}5\,\%$ con la fracción de bytes que Patterson resuelve y calcule la probabilidad de que Decap rechace un byte manipulado al azar (pista: solo los errores de peso exactamente 2 pasan el chequeo).

6. **Escalar.** Con la tabla 11.1: calcule cuántas veces más grande es la clave pública real que la del juguete, y el mismo cociente para el criptograma. Enumere tres mecanismos del esquema real que el juguete omite (pistas: σ_1, σ_2, forma semi-sistemática, red de Benes) e indique en qué capítulo se estudia cada uno.

CAPÍTULO 12

Los conjuntos y las variantes

En este capítulo

El capítulo 9 mostró cómo nace una clave; los capítulos 10 y 11, cómo se encapsula y desencapsula. Falta el catálogo: qué conjuntos concretos define Classic McEliece, qué significan los sufijos `f`, `pc` y `pcf`, cómo se guardan claves y criptogramas byte a byte, y qué seguridad se les atribuye. Al terminar, el lector debería poder elegir un conjunto con criterio y saber cuántos bytes maneja en cada paso.

12.1 Los cinco tamaños

Las tablas 9.1 y 9.2 fijaron los cinco pares (m, n, t) y el desglose interno de las claves. La tabla 12.1 añade las dos cifras que faltaban para decidir: el criptograma y la categoría de seguridad.

Conjunto	categoría	ek (bytes)	ct (bytes)	sk (bytes)	¿ISO?
`mceliece348864`	1	261 120	96	6492	no
`mceliece460896`	3	524 160	156	13 608	sí
`mceliece6688128`	5	1 044 992	208	13 932	sí
`mceliece6960119`	5	1 047 319	194	13 948	sí
`mceliece8192128`	5	1 357 824	208	14 120	sí

Cuadro 12.1: Los cinco conjuntos base, con $ek = mt\lceil k/8 \rceil$, $ct = \lceil mt/8 \rceil$ y sk como en la tabla 9.2. La columna «¿ISO?» anticipa la sección siguiente: el estándar ISO incluye los cuatro tamaños mayores, cada uno con cuatro variantes; `mceliece348864` queda fuera.

¿De dónde salen estos números, a primera vista caprichosos? Los conjuntos se eligieron siempre *maximizando la seguridad sujeta a un límite de tamaño* [28]:

- `mceliece6960119`, con $(n, t) = (6960, 119)$, maximiza la seguridad para claves públicas que quepan en 2^{20} bytes. La elección de $n = 6960$ viene de la optimización clásica de 2008: para límites de 2^{16} a 2^{20} bytes, los códigos Goppa de longitudes 1744, 2480, 3408, 4624 y 6960, con t proporcionalmente creciente, maximizan la dificultad del ataque; la tasa óptima ronda 0,75. La única diferencia con aquella elección es que Classic McEliece añade 119 errores en lugar de 121: sin decodificación por listas no puede decodificar de forma única más allá de la capacidad del código, y añadir dos errores menos reduce marginalmente la seguridad.
- `mceliece6688128` también maximiza dentro de 2^{20} bytes, pero exigiendo que n y t sean múltiplos de 32; `mceliece460896` y `mceliece348864` hacen lo mismo dentro de 2^{19} y 2^{18} bytes.
- `mceliece8192128` es el conjunto «clásico»: n y t potencias de dos, como en el McEliece original de 1978, a costa de pasarse un poco de 2^{20} bytes (1 357 824 bytes = 1,29 MiB).

Maximizar bajo un límite es más robusto que apuntar a un nivel exacto: las métricas de seguridad varían entre modelos, y un conjunto clavado a un objetivo puede quedar por debajo en otro modelo distinto. Al maximizar por tamaño, la incertidumbre cambia poco el orden.

Observación 12.1: Múltiplos de 32: por qué

Cuando n y t son múltiplos de 32 —y por tanto mt y k también lo son, porque $k = n - mt$ y $q = 2^{13}$— no hay *bits de relleno* en las codificaciones de claves y criptogramas, y la distinción entre las dos formas de leer un criptograma (sección 12.5) desaparece. Es una pequeña ventaja de interoperabilidad a cambio de un pequeño empeoramiento en la relación tamaño-seguridad. Los conjuntos 3488/4608/6688 y el 8192 la tienen; el 6960, no.

12.2 Los dieciséis conjuntos de ISO

ISO estandarizó Classic McEliece en junio de 2026 (enmienda 2 de ISO/IEC 18033-2:2006 [45]), y el estándar incluye **dieciséis conjuntos**: los cuatro tamaños de la tabla 12.1 (sin `mceliece348864`) cruzados con cuatro variantes:

- el conjunto **base**, tal cual lo define el equipo;
- el sufijo `f`: forma semi-sistemática $(\mu, \nu) = (32{,}64)$ (sección 12.3);
- el sufijo `pc`: confirmación de texto claro (sección 12.4);
- el sufijo `pcf`: ambas cosas a la vez.

La tabla 12.2 y la figura 12.1 muestran el catálogo completo. Obsérvese lo que cada sufijo hace a los tamaños: `f` no cambia nada; `pc` suma exactamente $\lceil \ell/8 \rceil = 32$ bytes al criptograma.

tamaño	(m, n, t, k)	ek	sk	ct `base,f`	ct `pc,pcf`	cat.
`460896`	$(13, 4608, 96, 3360)$	524 160	13 608	156	188	3
`6688128`	$(13, 6688, 128, 5024)$	1 044 992	13 932	208	240	5
`6960119`	$(13, 6960, 119, 5413)$	1 047 319	13 948	194	226	5
`8192128`	$(13, 8192, 128, 6528)$	1 357 824	14 120	208	240	5

Cuadro 12.2: Los dieciséis conjuntos de ISO: cada fila, con los sufijos —, `f`, `pc`, `pcf`. El nombre completo es `mceliece` seguido del número y el sufijo, por ejemplo `mceliece6960119pcf`. Los tamaños de cada fila valen para las cuatro variantes salvo el criptograma de `pc`/`pcf`.

¿Por qué falta `mceliece348864`? Por el modelo cuántico de ISO. ISO pidió a los candidatos **128 bits de seguridad en su modelo cuántico**, que asume la aceleración de raíz cuadrada de la búsqueda de Grover; pedir 128 bits en ese modelo es mucho más exigente que pedir 128 bits clásicos [24]. Con la raíz cuadrada aplicada a los mejores ataques clásicos sin trucos de memoria:

conjunto	ataque clásico	mitad (modelo cuántico)	$\geq 2^{128}$?
`mceliece348864`	$2^{151,4}$	$2^{75,7}$	no
`mceliece460896`	$2^{193,3}$	$2^{96,7}$	no
`mceliece6688128`	$2^{268,0}$	$2^{134,0}$	sí
`mceliece6960119`	$2^{268,4}$	$2^{134,2}$	sí
`mceliece8192128`	$2^{303,2}$	$2^{151,6}$	sí

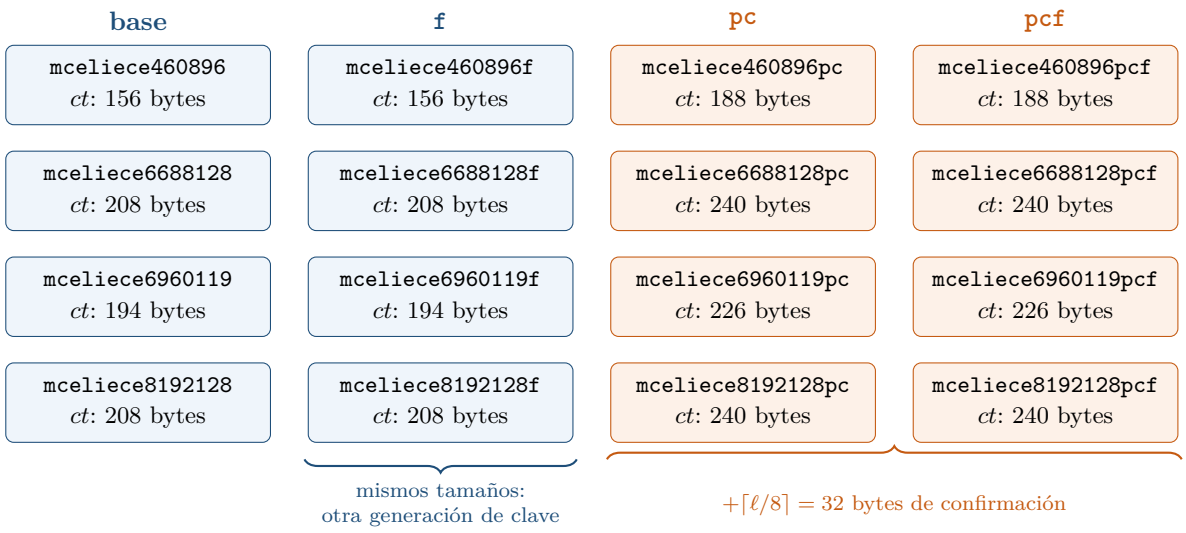


Figura 12.1: Los dieciséis conjuntos de ISO, como familia. Los cuatro tamaños (filas) y las cuatro variantes (columnas). La confirmación de texto claro (`pc`, `pcf`) añade 32 bytes al criptograma; la variante `f` no cambia ningún tamaño, solo la manera de generar la clave (forma semi-sistemática).

Los valores clásicos son los de Stern bajo memoria gratis (tabla 12.3), cuyos estimadores cuentan operaciones vectoriales como una sola operación, de modo que son cotas prudentes. Con esa vara, el borrador ISO que el equipo publicó en 2023 incluyó solo los tamaños 6688, 6960 y 8192 —todos $\geq 2^{134}$ cuánticos—. Pero la aceleración de Grover está, en la práctica, limitada por la latencia del ataque, así que ISO decidió estandarizar también `mceliece4*` (el conjunto 460896) a sabiendas de que no alcanza el listón cuántico [24]. El equipo recomienda los tamaños `mceliece6*` para seguridad a largo plazo.

Cómo llegó Classic McEliece a ISO

En octubre de 2022, unas diapositivas de Microsoft revelaron que el subcomité ISO/IEC JTC1/SC27/WG2 había solicitado comentarios sobre la inclusión de FrodoKEM, Classic McEliece y CRYSTALS-Kyber en sus estándares. El equipo de Classic McEliece preparó como borrador el documento `iso-mceliece-20230419` y lo publicó para recibir comentarios. Siguiendo la regla ISO de expresar los requisitos «en términos de rendimiento, no de diseño o características descriptivas», el borrador especificaba las *funciones matemáticas* a computar, sin constreñir cómo se computan. ISO estandarizó Classic McEliece en junio de 2026, junto a ML-KEM y FrodoKEM, como enmienda a ISO/IEC 18033-2 [24]. Mientras tanto, el NIST había *aplazado* la estandarización de McEliece en 2025, alegando que «la estandarización concurrente por parte del NIST y de ISO crea el riesgo de estándares incompatibles» y que «después de que concluya el proceso de ISO, el NIST podría considerar desarrollar un estándar basado en el de ISO» (traducción propia, [26]). El NIST, por tanto, no ha estandarizado McEliece; su única selección de cifrado post-cuántico adicional ha sido HQC [2].

12.3 La variante `f`: forma semi-sistemática

Recordemos cómo genera el capítulo 9 la clave pública (sección «MatGen»): se reduce la matriz binaria $\hat{H}$ a forma sistemática $(\mathrm{I}_{mt} \,|\, T)$ y, si la reducción falla, se empieza de nuevo con otra clave. Con $(\mu, \nu) = (0, 0)$ el intento tiene éxito con probabilidad $\approx 29\,\%$, así que en promedio

hacen falta unos 3,4 intentos [28].

La forma (μ,ν)-*semi-sistemática* relaja ese requisito. Una matriz R de rango r en forma escalonada reducida, con pivotes en las columnas $c_0 < \cdots < c_{r-1}$, está en forma (μ,ν)-semi-sistemática si no tiene filas nulas, $c_i = i$ para $0 \le i < r-\mu$, y cada pivote posterior admite un margen:

$$c_i \;\le\; i-\mu+\nu \qquad (0 \le i < r).$$

Para $(\mu,\nu) = (0,0)$ esto es, exactamente, la forma sistemática. Si $\nu > \mu$, hay muchas más matrices admisibles, y con $(\mu,\nu) = (32,64)$ basta con que R sea escalonada salvo por los últimos $\mu = 32$ pivotes, que pueden desplazarse dentro de una ventana de $\nu = 64$ columnas. MatGen (caso general) reduce $\hat{H}$ hasta esa forma, y luego intercambia las últimas columnas $i \leftrightarrow c_i$ —arrastrando los α_j— hasta dejar $(\mathrm{I}_{mt} \,|\, T)$. El «libro de permutaciones» $c = (c_{mt-\mu}, \ldots, c_{mt-1})$ se guarda en la clave privada (8 bytes; es la lista c de la tabla 9.2, presente también en los conjuntos $(\mu,\nu) = (0,0)$, donde codifica $2^{32}-1$). La figura 12.2 resume el mecanismo.

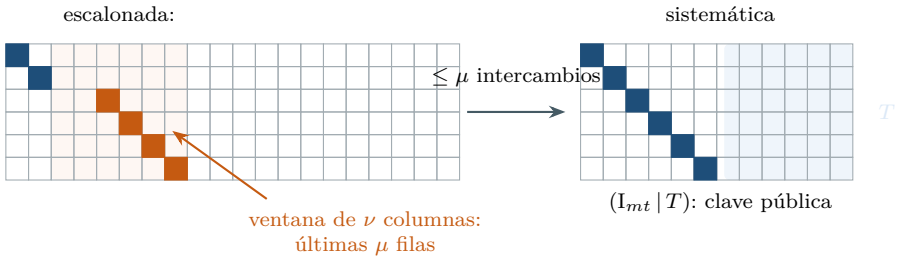


Figura 12.2: La forma (μ,ν)-semi-sistemática, en esquema reducido ($\mu = 4$, $\nu = 6$; en el estándar, $(32,64)$). Izquierda: la matriz escalonada; los primeros $mt-\mu$ pivotes ya están en la diagonal y los últimos μ viven dentro de una ventana de ν columnas. Basta con $\le \mu$ intercambios de columnas para alcanzar la forma sistemática de la derecha.

¿Qué se gana? Probabilidad de éxito. Con $(\mu,\nu) = (32,64)$ la probabilidad de fallo de cada intento cae por debajo de 2^{-30}: casi siempre basta *un* intento, frente a los $\approx 3{,}4$ de $(\mu,\nu) = (0,0)$. El precio: la computación en forma escalonada debe hacerse sin ramificaciones dependientes de secretos (en tiempo constante), y eso solo es razonable si queda confinada a una ventana pequeña —de ahí que ν se mantenga modesto—. La ganancia de holgura en la demostración de seguridad también se cuantifica: aceptar más matrices pierde como mucho 2 bits de tensión en la reducción [28].

¿Por qué existen entonces conjuntos sin `f`? Porque hay tres argumentos para mantener $(\mu,\nu) = (0,0)$ [28]: (1) el software optimizado de Classic McEliece ya acelera los reintentos saltándose casi toda la eliminación gaussiana en los casos de fallo; (2) muchas aplicaciones no tienen la generación de claves como cuello de botella; (3) la simplicidad tiene valor. ISO estandarizó las cuatro variantes para cada tamaño: quien quiera velocidad de generación elige `f`; quien prefiera el camino más simple y auditado, el conjunto base.

12.4 La variante pc: confirmación de texto claro

En la ronda 3, la transformación CCA de Classic McEliece usaba *dos* defensas a la vez: rechazo implícito *y* confirmación de texto claro. La confirmación añade un segundo componente al criptograma, $C_1 = H(2,e)$, y el descifrado lo comprueba [27]:

- Encap: $C_0 = \mathsf{Encode}(e,T)$, $C_1 = H(2,e)$; el criptograma es $C = (C_0, C_1)$ y la clave, $K = H(1,e,C)$.

- Decap: se parte $C = (C_0, C_1)$; se descifra C_0 como siempre; después se recalcula $C_1' = H(2, e)$ y, si no coincide con C_1, también se toma la rama de rechazo ($e \leftarrow s$, $b \leftarrow 0$). La clave sale de $K = H(b, e, C)$.

Nótese el detalle: la confirmación *no* introduce errores explícitos. Si la comprobación falla, Decap sigue devolviendo 32 bytes por la rama de rechazo implícito; el KEM continúa siendo una función total. Lo que añade es una *demostración*: C_1 solo puede calcularlo quien conoce e, así que el criptograma certifica que su emisor conocía el error. Quien posee la clave privada puede verificar esa demostración explícitamente (descifra C_0, recalcula $H(2, e')$ y compara con C_1), si su aplicación lo necesita.

El costo y la historia: C_1 ocupa $\lceil \ell/8 \rceil = 32$ bytes (los mismos que una clave de sesión), y el hash pasa a tener tres tipos de entrada, distinguidas por el primer byte: $(2, v)$, $(1, v, C)$ y $(0, v, C)$. En la ronda 4, el equipo *quitó* la confirmación del conjunto por defecto y la dejó como variante opcional [25], por tres razones: (1) separarse limpiamente de la patente estadounidense US 9.912.479 —que no se aplica literalmente a Classic McEliece, cuyo paquete de cifrado tiene un solo objeto, el error, y no «el vector de error y el valor del texto claro», pero cuya sombra legal se prefirió despejar, volviendo a la «forma tradicional de Niederreiter»—; (2) ahorrar los 32 bytes, que en un esquema de criptogramas diminutos se notan; y (3) simplificar especificación y software, ya que para las demostraciones ajustadas de IND-CCA2 en ROM y cuasi-ajustadas en QROM (teoremas 8.4 y 8.5) el rechazo implícito por sí solo basta. En el debe: quitar la confirmación abre un ataque eficiente en un modelo *más allá* de IND-CCA2 en el que un solo bit de la clave privada almacenada se invierte (por ejemplo, un fallo de DRAM); el ataque análogo contra NTRU-HRSS [8] motivó precisamente este cuidado. La mitigación recomendada no toca el KEM: codificar la clave privada almacenada con corrección de errores (los sistemas comerciales ya hacen SECDED sobre la DRAM). ISO, en fin, estandariza las dos familias de variantes: quien quiera las defensas duales de la ronda 3 usa `pc` o `pcf`.

12.5 Simply Decoded y Narrowly Decoded: el relleno

Los conjuntos de parámetros no eligen un múltiplo de 8 por casualidad: el criptograma ocupa $\lceil mt/8 \rceil$ bytes y la clave pública $mt\lceil k/8 \rceil$ bytes (una fila de T por línea, rellena cada una a byte completo). Cuando mt o k *no* son múltiplos de 8, la última posición contiene bits de relleno que la especificación deja a cero, y ahí nacen dos formas de leer una cadena de bytes [10]:

- **Simply Decoded**: los bits de relleno se ignoran al interpretar la entrada;
- **Narrowly Decoded**: se rechazan las entradas (criptogramas y claves públicas) con relleno distinto de cero, devolviendo $\perp$.

De los dieciséis conjuntos, **solo `mceliece6960119` tiene relleno**: allí $mt = 1547$ (el criptograma tiene 194 bytes, con 5 bits de relleno en el último) y $k = 5413$ (cada fila de la clave pública tiene 3 bits de relleno). En los demás, mt y k son múltiplos de 8 y las dos lecturas coinciden. La figura 12.3 ilustra el caso.

¿Por qué se llama maleabilidad benigna? Porque dos cadenas de bytes que difieran solo en el relleno representan el *mismo* criptograma (el mismo elemento de $\mathbb{F}_2^{mt}$), y por tanto el mismo K en modo Simply; en modo Narrowly una se acepta y la otra no. La distinción no rompe la seguridad —el teorema de IND-CCA2 cubre las cadenas codificadas—, pero es un contrato de interoperabilidad que las partes deben acordar.

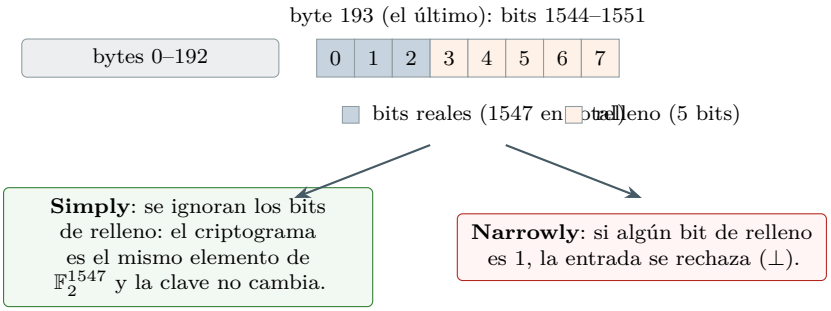


Figura 12.3: El relleno de `mceliece6960119`: el criptograma son 1547 bits dentro de 194 bytes, y a los 5 bits finales la especificación les da dos lecturas. Una implementación u otra debe elegirse de forma consistente entre las dos partes del protocolo; mezclarlas convierte en divergencia lo que es una maleabilidad benigna.

Tres bits por fila, 580 bytes

Un implementador que empaquete la clave pública «apretada», como una tira continua de kmt bits, obtendrá para `mceliece6960119` una clave de $\lceil kmt/8 \rceil = 1\,046\,739$ bytes. La cadena estándar tiene 1 047 319: la diferencia, 580 bytes, son los 3 bits de relleno de cada una de las 1547 filas ($1547 \times 3/8 = 580{,}125$). Ambos empaquetados describen la misma matriz T, pero solo el segundo es válido frente a otra implementación —y frente a los vectores de prueba del estándar—. Con T en la mano el error se detecta enseguida, pero es un clásico de los primeros días de una implementación.

12.6 ¿Cuánta seguridad?

La tabla 12.3 resume las estimaciones de la guía de seguridad [29]: el mejor ataque de decodificación por conjuntos de información (capítulo 14) según el estimador de Esser–Bellini, en dos de sus modelos de memoria, junto a la categoría que cada conjunto reclama.

conjunto	Prange	MO (mem. libre)	MO (mem. con costo)	categoría (mem. realista)	categoría (mem. gratis)
`mceliece348864`	173,4	140,8	158,6	1	1
`mceliece460896`	216,8	179,8	201,0	3	2
`mceliece6688128`	295,7	246,2	278,2	5	4
`mceliece6960119`	296,8	245,7	279,2	5	4
`mceliece8192128`	334,1	275,6	315,6	5	5

Cuadro 12.3: Costos en bits ($\log_2$) de los mejores ataques conocidos según el estimador de Esser–Bellini: contra Prange (1962) y contra May–Ozerov (MO), con memoria de acceso gratis y con costo de acceso en raíz cuadrada. Las dos últimas columnas: la categoría que el equipo asigna contando costos realistas de memoria, y la que correspondería si la memoria se contara como gratis.

Tres advertencias sobre esa tabla, todas de la propia guía:

1. **Los números subestiman.** El estimador cuenta cada operación vectorial como *una* operación, sea cual sea su longitud. El 140,8 de `mceliece348864`, por ejemplo, cuenta más de 2^{143} operaciones de bits, y el ataque requiere memoria $2^{86{,}6}$ [29].

2. **La memoria no es gratis.** La columna «mem. libre» supone acceso libre a memoria arbitrariamente grande, físicamente irreal; ahí los algoritmos con «representaciones» (BJMM, MO) brillan. Cuando se cobra la memoria a su costo plausible, la ventaja se estrecha.

3. **Las categorías dependen del modelo.** Contando memoria realista, el equipo asigna $1, 3, 5, 5, 5$; contando memoria gratis y operaciones de bits, $1, 2, 4, 4, 5$. Ambas tablas son sostenibles sobre los mismos datos; el número clave es que `mceliece6960119` es categoría 5 si se cuentan los accesos a memoria ($2^{279,2} > 2^{272}$, la referencia de AES-256) y categoría 4 si no ($2^{245,7} < 2^{256}$).

Las referencias del NIST para las categorías 1, 3 y 5 son búsquedas exhaustivas de AES-128, AES-192 y AES-256, estimadas en 2^{143}, 2^{207} y 2^{272} operaciones de bits. De ahí las tensiones numéricas reseñadas: 140,8, el peor valor, roza por abajo la referencia de la categoría 1; 201,0 cae bajo 2^{207}; y 245,7, bajo 2^{256}. El equipo responde, con razón documentada, que estas comparaciones mezclan métricas: no es lo mismo «operaciones de bits contadas por un estimador que trata un XOR de mil bytes como una operación» que operaciones reales. La discusión completa, y por qué el récord de estabilidad del esquema es el argumento más fuerte, llega en los capítulos 13 y 14.

12.7 Elegir un conjunto

Con todo lo anterior, la guía práctica es corta:

- **Tamaño.** Para seguridad a largo plazo, el equipo recomienda los tamaños `mceliece6*` (6688128, 6960119) [24]; añaden un margen generoso sobre la categoría 5 clásica. Si el entorno aprieta (clave pública de 512 KiB), el candidato es `mceliece460896`; `mceliece348864` existe en la especificación, pero queda fuera de ISO y no es la elección para décadas.
- **Variante `f`.** Úsese cuando la generación de claves sea cuello de botella o deba ser rápida en tiempo constante; añade la forma semi-sistemática sin cambiar ningún tamaño.
- **Variante `pc`/`pcf`.** Úsese cuando el protocolo quiera la confirmación de texto claro de la ronda 3 (por ejemplo, para verificar el conocimiento de e o para defensas en modelos con fallos de clave privada); cuesta 32 bytes por criptograma.

Para calibrar los compromisos, la tabla 12.4 compara los tamaños con los otros KEM que un despliegue real barajará. La silueta de Classic McEliece es inconfundible: frente a ML-KEM-768, una clave pública entre cuatrocientas y mil doscientas veces mayor (según el conjunto), un criptograma entre cinco y ocho veces menor, y una clave privada moderada. La clave de HQC, en cambio, es de 40 bytes: es una semilla, regenerada al vuelo, la misma filosofía que la δ de Classic McEliece pero llevada al secreto entero.

¿Y en un protocolo real?

La clave pública de Classic McEliece no viaja en cada conexión: viaja una vez y se cachea (es estable durante años). El criptograma, en cambio, viaja en cada intercambio: ahí el tamaño importa, y es donde los 96–240 bytes juegan a favor. En TLS 1.3, por ejemplo, ML-KEM y McEliece pueden convivir como grupos híbridos; la página de implementación del equipo documenta los detalles [23].

esquema	ek (bytes)	ct (bytes)	sk (bytes)
ML-KEM-768	1184	1088	2400
ML-KEM-1024	1568	1568	3168
HQC-128	2249	4497	40
HQC-256	7245	14485	40
`mceliece460896`	524 160	156	13 608
`mceliece6960119`	1 047 319	194	13 948

Cuadro 12.4: Tamaños de los KEM post-cuánticos relevantes (ML-KEM según FIPS 203 [34]; HQC según el informe del NIST [2]; Classic McEliece según la tabla 12.1). El criptograma de McEliece es el más pequeño con diferencia; la clave pública, la más grande.

12.8 Ejercicios

1. **Aritmética de bytes.** Con $ct = \lceil mt/8 \rceil$ y $ek = mt\lceil k/8 \rceil$, verifique las columnas de la tabla 12.1. ¿Por qué para cuatro conjuntos $ek = \lceil kmt/8 \rceil$ pero para `mceliece6960119` no? Calcule la diferencia exacta.

2. **La variante `pc`.** Para `mceliece460896pc`, complete la tabla 12.2 para los dieciséis conjuntos y calcule después cuántos bytes recibe cada hash: $H(2, e)$ por un lado, y $H(1, e, C)$ y $H(0, s, C)$ por otro.

3. **El listón cuántico de ISO.** Usando el modelo de raíz cuadrada, calcule la mitad del logaritmo del ataque de Stern (tabla 12.3) para los cinco conjuntos y compruebe qué conjuntos superan 2^{128}. ¿Coincide con los que eligió el borrador ISO? ¿Y con lo que ISO decidió estandarizar al final? Explique la diferencia.

4. **Intentos de generación.** Con $(\mu, \nu) = (0, 0)$ el éxito de MatGen ronda el 29 %: calcule el número esperado de intentos y la probabilidad de necesitar más de dos. Con $(\mu, \nu) = (32, 64)$ el fallo por intento es $< 2^{-30}$: calcule el número esperado de intentos. ¿Cuánto trabajo de generación se ahorra la variante `f`?

5. **Relleno benigno.** Para `mceliece6960119`, construya dos cadenas de 194 bytes que difieran solo en los 5 bits de relleno. ¿Cuántos K distintos producen en modo Simply? ¿Y en modo Narrowly? ¿Por qué esta diferencia debe resolverse por acuerdo de protocolo y no dejarse al azar de la implementación?

6. **Elegir con criterio.** Un servicio de mensajería cachea claves públicas de servidor y envía millones de criptogramas pequeños; una etiqueta IoT de bajo consumo genera su propia clave una vez y no la cachea. ¿Qué conjunto y qué variante propondría para cada caso? Justifique con las tablas 12.1 y 12.4.

Cierre de la parte III. Los cuatro capítulos de esta parte recorrieron la especificación completa: la generación determinista de claves (capítulo 9), la encapsulación y desencapsulación con Patterson y rechazo implícito (capítulos 10 y 11) y el catálogo de los dieciséis conjuntos ISO con sus variantes (capítulo 12). El mecanismo está entero sobre la mesa; queda la pregunta que decide si merece la pena: su seguridad. La parte IV la analiza sin adornos.

PARTE IV

Seguridad

CAPÍTULO 13

¿Qué significa romper McEliece?

En este capítulo

La especificación ya está contada: sabemos generar claves, encapsular y desencapsular. Falta la pregunta que lo decide todo: ¿cuánto cuesta de verdad romper un conjunto de parámetros? Esta parte la responde en cuatro capítulos. Este fija el vocabulario: qué objetivos puede perseguir un atacante (el mensaje, la clave, distinguir), qué está demostrado y qué se conjetura. Los capítulos 14 y 15 recorren las dos grandes familias de criptoanálisis —decodificación y ataques estructurales— y el 16 fecha el estado del arte: los resultados de 2025 y 2026 que obligaron a revisar supuestos.

13.1 Tres objetivos (y no todos rompen)

Un atacante contra Classic McEliece ve una clave pública pk y, en el modelo activo (definición 8.2), puede además preguntar al oráculo de desencapsulación. A grandes rasgos puede perseguir tres objetivos, de dificultad y significado muy distintos:

1. **Romper el mensaje.** Dados pk $= T$ y un criptograma C, encontrar la semilla e con la que se encapsuló: la única de peso t con $\mathbf{H}e = C$, donde $\mathbf{H} = (\mathrm{I}_{mt} \,|\, \mathbf{T})$.
2. **Romper la clave.** Encontrar *alguna* descripción eficiente del código público: el par (g, α) secreto, o más en general cualquier algoritmo que decodifique t errores para el código de pk (una *clave equivalente*, definición 13.1). Con ella se descifra todo, hoy y en el futuro de esa clave.
3. **Distinguir.** Decidir si pk es una clave legítima o una matriz aleatoria del mismo tamaño. Es información *estructural*: revela que el código lleva huellas de Goppa, aunque no descifra nada por sí sola.

Definición 13.1: Clave equivalente

Sea pk $= T$ con $\mathbf{H} = (\mathrm{I}_{mt} \,|\, \mathbf{T})$. Una *clave equivalente* es cualquier algoritmo $\mathcal{D}$ que, dado $C \in \mathbb{F}_2^{mt}$, devuelve un $e \in \mathbb{F}_2^n$ con $\mathrm{wt}(e) \leq t$ y $\mathbf{H}e = C$ siempre que tal e exista, y el símbolo de fallo en caso contrario.

El nombre es exacto: quien tiene una clave equivalente descifra directamente el criptograma C —basta pedirle $\mathcal{D}(C)$: eso es exactamente lo que hace Decode— sin saber nada de g ni del orden de las α_i. Los ataques estructurales del capítulo 15 buscan exactamente esto, o más: a veces se conforman con distinguir.

La figura 13.1 ordena los tres objetivos y sus relaciones. Recuperar la clave implica romper el mensaje: una clave equivalente decodifica cualquier criptograma. El recíproco no se conoce: un algoritmo que resuelva *el juego* del objetivo 1 solo tiene que ganar con los criptogramas que el juego le entrega, y nadie ha reducido a partir de ahí un decodificador general. Y distinguir está, hoy por hoy, al margen: un buen distinguidor de claves públicas no se convierte —con lo que se sabe— en un ataque contra el mensaje. Cada flecha de esa figura tiene, por tanto, una dirección demostrada o un interrogante, y la parte IV entera se organiza alrededor de ellas.

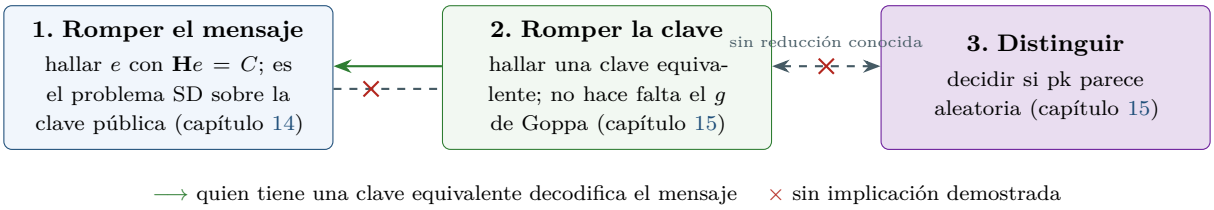


Figura 13.1: Los tres objetivos de un atacante, con la dirección demostrada (clave ⇒ mensaje) y los interrogantes abiertos. La seguridad del KEM descansa, en último término, en el objetivo 1: los teoremas del capítulo 8 reducen su IND-CCA2 a la OW-CPA del esquema desnudo, que es exactamente este problema.

13.2 El juego OW-CPA y el problema SD

Para medir el primer objetivo se usa la noción clásica de las PKE deterministas.

Definición 13.2: One-wayness bajo ataque de texto claro elegido (OW-CPA)

Sea $(\mathsf{Gen}', \mathsf{Enc}, \mathsf{Dec})$ una PKE determinista y correcta, con espacio de mensajes P. El juego:

1. el retador genera $(\mathrm{pk}', \mathrm{sk}') \leftarrow \mathsf{Gen}'$ y sortea $e \leftarrow P$ uniforme;
2. calcula $C = \mathsf{Enc}(\mathrm{pk}', e)$ y entrega (pk', C) al adversario $\mathcal{A}$;
3. $\mathcal{A}$ termina con e' y gana si $e' = e$.

La PKE es OW-CPA si todo adversario eficiente gana con probabilidad despreciable.

En Classic McEliece, Enc es $\mathsf{Encode}(\cdot, T)$ y P son los vectores de peso exactamente t —que $\mathsf{FixedWeight}$ recorre de forma uniforme (proposición 10.1)—. El lema siguiente traduce el juego al lenguaje de la teoría de códigos.

Lema 13.3: OW-CPA de Classic McEliece es SD sobre la clave pública

Fijemos un conjunto de parámetros y sea $(\mathsf{KeyGen}, \mathsf{Encode}, \mathsf{Decode})$ el esquema subyacente. Ganar el juego OW-CPA de la definición 13.2 es, exactamente, resolver SD (definición 2.14, versión de búsqueda) para el código público: dados $\mathbf{T}$ y $C = \mathsf{Encode}(e, T)$, hallar la palabra de error e de peso t. En particular, un atacante OW-CPA es un algoritmo de decodificación por síndromes para la distribución de matrices que genera KeyGen, y todo algoritmo así gana el juego.

Demostración. Por linealidad de Encode: $\mathsf{Encode}(e', T) = \mathbf{H}e'$ con $\mathbf{H} = (\mathrm{I}_{mt} \mid \mathbf{T})$. Si $\mathcal{A}$ gana con e', entonces $\mathbf{H}(e + e') = 0$: el vector $e + e'$ pertenece al código público, que tiene distancia mínima $\geq 2t + 1$ (teorema 4.4), y su peso es a lo sumo $2t$. Por tanto $e + e' = 0$, es decir, $e' = e$: no hay soluciones «falsas» de peso $\leq t$, y el juego coincide tarea a tarea con SD. En sentido contrario, quien resuelve SD para $C = \mathbf{H}e$ devuelve e y gana el juego. □

Obsérvese lo que el lema *no* dice: nada sobre la dificultad de resolver SD en la distribución de matrices que produce KeyGen. Traducir un problema a otro no lo vuelve difícil; solo identifica el enemigo con nombre y apellidos. Eso permite medir la seguridad con las herramientas del capítulo 14, pero también recuerda que la confianza última es una conjetura sobre esa distribución concreta, no sobre «SD en general».

13.3 La cadena de reducciones y la conjetura central

La figura 13.2 ordena lo que sabemos. La seguridad IND-CCA2 del KEM está demostrada (capítulo 8) a partir de dos insumos: la OW-CPA del esquema desnudo y suposiciones simétricas (el hash y la pseudoaleatoriedad de la clave de rechazo). La OW-CPA es, por el lema 13.3, la dificultad de SD sobre la clave pública. Y ahí se detiene la cadena demostrativa: la dificultad de SD *en esa distribución* es una conjetura.

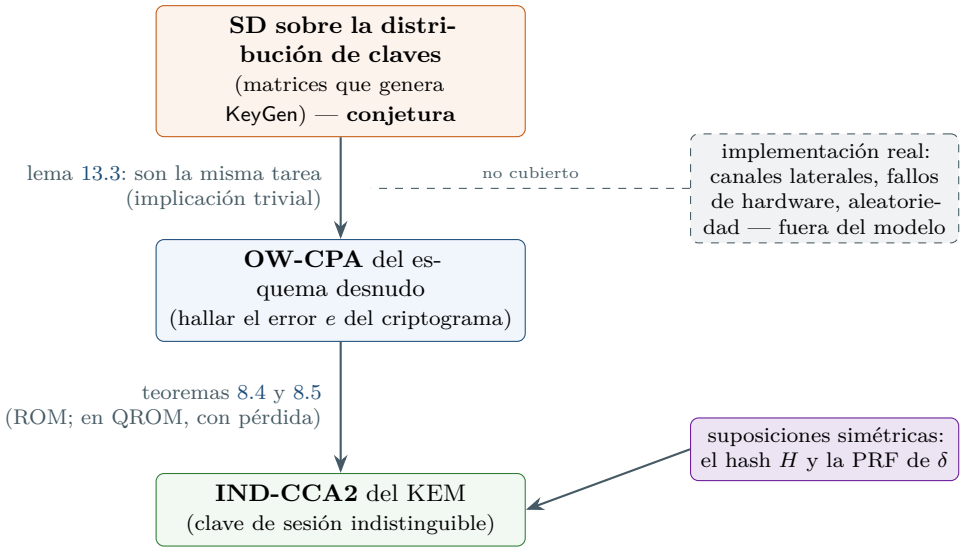


Figura 13.2: La cadena de reducciones de Classic McEliece. Arriba, la conjetura dura; en medio, la propiedad que se mide con el criptoanálisis (capítulos 14 y 15); abajo, la garantía del KEM, que además necesita el hash y la PRF. La implementación real queda fuera del modelo, y se vigila aparte.

La conjetura central, sin adornos

Para cada conjunto de parámetros, la conjetura es esta: no existirá un algoritmo esencialmente mejor que los mejores conocidos hoy para encontrar e a partir de $(\mathbf{T}, \mathsf{Encode}(e, T))$ con e uniforme de peso t. La historia admite lectura doble. Es la conjetura que ha sostenido casi medio siglo de escrutinio, desde 1978, y ninguna de las mejoras publicadas ha pasado de recortes del exponente (capítulo 14). Pero también es una apuesta inductiva: nada demuestra que la curva de ataques se vaya a quedar quieta, y los parámetros se eligen con margen —y se revisan cuando el margen se estrecha— precisamente por eso. Lo que sí está demostrado: la corrección del esquema (proposiciones 6.2 y 7.2), la equivalencia del lema 13.3, la NP-dureza de SD en el peor caso [6] y las reducciones del capítulo 8. Lo conjetural es solo —y nada menos que— la dureza en la distribución real.

13.4 Las familias de ataque de esta parte

El criptoanálisis de McEliece se organiza en dos grandes familias, más su versión cuántica; la tabla 13.1 las sitúa y las reparte entre los capítulos siguientes.

Las fronteras entre familias son porosas, en las dos direcciones. Un decodificador *es* una clave equivalente —por definición— y un ataque estructural que recupere una descripción decodificante

familia	objetivo	herramienta principal	dónde
decodificación clásica	el mensaje (y, con trabajo extra, la clave)	conjuntos de información: de Prange (1962) a May–Ozerov, con estimadores modernos	cap. 14
decodificación cuántica	el mensaje	Grover y paseos cuánticos: a lo sumo, la raíz del exponente clásico	cap. 14
estructural	la clave o distinguir	particiones de soporte, álgebra sobre $\mathbb{F}_{2^m}$, distinguidores	cap. 15

Cuadro 13.1: Las familias de ataque contra Classic McEliece. El capítulo 16 añade la cuarta dimensión: el tiempo, con los resultados de 2025 y 2026 y las respuestas del equipo.

rompe el mensaje. Y en sentido inverso: un algoritmo que resolviera SD para síndromes *elegidos* por el atacante —no solo los que le da el juego OW-CPA— podría reconstruir una matriz de paridad del código público; informalmente, pidiendo la solución del síndrome $\mathbf{H}_j + \mathbf{H}_k$, que es la palabra de peso $2 \le t$ única $e_j + e_k$, y despejando columnas. Por eso los ataques de decodificación también producen, con trabajo adicional, recuperación de clave, y las dos familias se vigilan juntas.

«Distinguen McEliece»: titular y matiz

Cada pocos años aparece un titular que distingue las claves públicas de McEliece de matrices aleatorias. Es un resultado sobre la *estructura* del código, y en sí no descifra nada: no se conoce ninguna reducción de un distinguidor a un ataque contra el mensaje. Lo que un buen distinguidor sí hace es erosionar el argumento —informal— de que «la clave pública no revela nada», y obliga a revisar qué garantías dependían de ese parecido. El episodio de 2025 y 2026 se cuenta con números en los capítulos 15 y 16.

13.5 Cómo se leen las cifras de seguridad

Todas las estimaciones de esta parte son *cotas inferiores del esfuerzo del mejor ataque conocido*: nada más, y nada menos. Un «2^b» significa siempre 2^b unidades de un modelo de coste declarado, en una fecha, contra un objetivo declarado. Los tres lenguajes de coste más frecuentes se resumen en la tabla 13.2.

lenguaje	una unidad es…	se usa para… y su peligro al comparar
operaciones de bits	un XOR, un AND… sobre un bit	comparar con las referencias tipo NIST (AES-128 $\approx 2^{143}$); engaña si el ataque trabaja con vectores: un XOR de vectores son miles de operaciones de bits
operaciones vectoriales	un XOR entre vectores completos	seguir los estimadores de decodificación (las unidades de la tabla 12.3); engaña a la baja: es «vender barato» el trabajo real
tiempo en hardware	un segundo medido	validar implementaciones y ataques reales; no extrapola a décadas ni a máquinas futuras

Cuadro 13.2: Tres lenguajes de coste. Toda comparación honesta de cifras exige fijar las unidades y el modelo de memoria; la sección 12.6 ya dio las advertencias para la tabla 12.3.

Un ejemplo de lectura, con la tabla 12.3 delante. Para `mceliece348864`, el mejor ataque clásico contra el mensaje se cotiza en $2^{140{,}8}$ operaciones *vectoriales* con memoria $2^{86{,}6}$ (modelo «memoria gratis»), o $2^{158{,}6}$ si la memoria se paga; el más simple, Prange, cuesta $2^{173{,}4}$. La

referencia NIST de categoría 1 es AES-128, estimado en 2^{143} operaciones de bits: comparar 140,8 con 143 sin más sería un error de unidades (la propia guía señala que ese ataque cuenta más de 2^{143} operaciones de bits, precisamente por eso). Del lado cuántico, el mensaje es corto: los algoritmos conocidos ofrecen, en el mejor caso, la raíz del exponente clásico —con sobrecostes enormes y supuestos de hardware lejanos [39]—, y los parámetros se eligen con ese margen en cuenta (capítulo 12); la sección 14.6 lo cuantifica.

Y cada cifra lleva fecha. Las de la tabla 12.3 proceden de la guía de seguridad del equipo (2022) [29]; el capítulo 16 actualiza el estado del arte a octubre de 2026.

13.6 Fuera del modelo: la implementación

El adversario también mira el código

Los teoremas y las cifras de esta parte hablan del adversario matemático. El adversario real, además: mide tiempos de respuesta (las implementaciones de referencia de Classic McEliece están escritas para no filtrar secretos por el tiempo [23]); provoca fallos de hardware (un bit invertido en la clave privada almacenada puede ser fatal en modelos más allá de IND-CCA2 —el capítulo 12 contó el episodio, las variantes `pc` y las mitigaciones con corrección de errores—); y ataca la aleatoriedad (una semilla δ o un e sesgados rompen cualquier KEM por muy dura que sea su matemática). Ninguna de las tres cosas la cubre una reducción, y ninguna se arregla con parámetros más grandes.

13.7 Ejercicios

1. **Direcciones de implicación.** Con la figura 13.1 delante: (a) explique por qué una clave equivalente rompe el mensaje; (b) explique por qué un rompemensajes «de juego» no se convierte en decodificador general con lo que se sabe; (c) ¿qué implicaciones tiene un distinguidor de claves públicas? ¿Por qué su papel es erosionar supuestos y no descifrar?

2. **SD en el juguete, a mano.** En el juguete del capítulo 5, $n = 16$ y $t = 2$. Enumere las $1 + 16 + 120 = 137$ palabras de peso ≤ 2 y localice la única cuyo síndrome —en el sentido del capítulo 5— es $(2, \mathtt{b})$. ¿Por qué la unicidad estaba garantizada de antemano por la distancia mínima? Después compare escalas: enumerar palabras de peso 64 para `mceliece348864` ronda 2^{456} candidatos, mientras el ataque de Prange se cotiza en $2^{173{,}4}$. ¿De qué vive esa diferencia?

3. **Los cuantificadores de la conjetura.** Escriba la conjetura del recuadro «Rigor» con cuantificadores explícitos: qué algoritmos, sobre qué distribución, con qué éxito, con qué recursos. Explique después por qué estos tres enunciados *no* son equivalentes a ella: (a) «SD es NP-duro»; (b) «no existe un algoritmo polinómico»; (c) «el mejor ataque conocido cuesta $2^{140{,}8}$». ¿Cuál de los tres es, en la práctica, el que sostiene las decisiones?

4. **Leer las columnas.** Explique, sin inventar cifras nuevas, por qué $2^{140{,}8}$ (tabla 12.3) no contradice la categoría 1 pese a que AES-128 se estima en 2^{143}: unidades, papel de la memoria $2^{86{,}6}$ y significado de «operaciones vectoriales». Calcule además el factor entre $2^{140{,}8}$ y $2^{158{,}6}$.

5. **La mitad cuántica.** Para `mceliece348864`, el mejor ataque clásico sin trucos de memoria se cotiza en $2^{151{,}4}$. Calcule su raíz *en el exponente* (la «mitad»), compárela con el listón de

ISO del capítulo 12 (2^{128} en su modelo cuántico) y explique, con la historia de ese capítulo, por qué ISO estandarizó igualmente el conjunto `460896`.

6. **Taxonomía de titulares.** Clasifique cada titular en su familia y capítulo, y diga qué rompe y qué no: (a) «Un algoritmo distingue las claves públicas de McEliece de matrices aleatorias»; (b) «Mejora de un factor 2^{10} en el coste de los ataques de decodificación»; (c) «Se recupera una clave de un despliegue real midiendo tiempos de respuesta».

CAPÍTULO 14

Decodificación por conjuntos de información

En este capítulo

El lema 13.3 del capítulo anterior redujo el objetivo principal de un atacante —recuperar la semilla e— al problema SD sobre la clave pública. Este capítulo mide ese problema. La familia dominante de ataques son los *decodificadores por conjuntos de información*: de Prange (1962) a May–Ozerov (2015), sesenta años de mejoras combinatorias que han bajado el exponente de $2^{173,4}$ a $2^{140,8}$ para `mceliece348864`. Veremos la idea de cada algoritmo, la tabla de costes de la guía de seguridad (2022) y, con lupa, el debate que la rodea: qué cuenta un 2^b, qué papel juega la memoria y qué significa exactamente "la mitad" en el modelo cuántico. Es el material con el que se asignan las categorías de seguridad de la tabla 12.3.

14.1 La idea: un conjunto de información

Recordemos el montaje exacto. La clave pública de Classic McEliece es pk $= \mathbf{T}$ con $\mathbf{H} = (\mathrm{I}_{mt} \,|\, \mathbf{T})$, el criptograma es $C = \mathbf{H}e$ y el objetivo es hallar e de peso t (lema 13.3). Todos los ataques de este capítulo tratan $\mathbf{T}$ —o su matriz generadora equivalente— como una matriz *sin estructura*, como si fuese aleatoria. Esa apuesta da a los algoritmos su enorme generalidad: si mañana alguien encuentra una grieta en las matrices de Goppa concretas (capítulo 15), estos ataques no se dan por enterados; su coste depende solo de n, k y t.

El algoritmo más viejo sigue siendo el más fácil de contar, y data de 1962 [62]. En la forma del McEliece de 1978 —clave pública G de $k \times n$, texto cifrado $y = Ga + e$— funciona así:

1. se eligen k columnas de G al azar —el *conjunto de información*— y se comprueba que formen una matriz invertible G_I;
2. se calcula $a = G_I^{-1} y_I$. Si los t errores de e cayeran *todos* fuera de I, esta a es la buena;
3. se verifica: si $y - Ga$ tiene peso t, el ataque termina.

La figura 14.1 cuenta la aritmética. En cada intento, la probabilidad de que las t posiciones del error caigan fuera del conjunto de información es

$$\prod_{i=0}^{t-1}\Big(1 - \frac{k}{n-i}\Big) \;\approx\; \Big(1 - \frac{k}{n}\Big)^t \;=\; \Big(\frac{n-k}{n}\Big)^t,$$

y cada intento cuesta la eliminación gaussiana de una matriz de $k \times k$ —un orden de k^3 operaciones— más la verificación. Para `mceliece348864`, con $k = 2720$ y $t = 64$:

$$\Big(\frac{3488 - 2720}{3488}\Big)^{64} = (0{,}2202\ldots)^{64} \approx 2^{-139{,}7}, \qquad k^3 \approx 2^{34{,}2},$$

de modo que el coste total ronda $2^{139{,}7} \cdot 2^{34} \approx 2^{173}$: ahí está el 173,4 de la tabla 12.3. Prange, con dos multiplicaciones y una eliminación gaussiana, fija la vara de medir de todo lo que sigue.

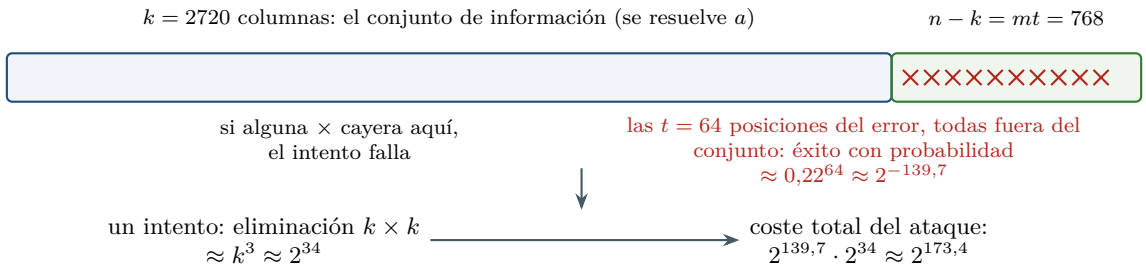


Figura 14.1: Prange (1962): elegir k columnas, resolver el sistema y repetir hasta que el azar deje el error fuera del conjunto de información. Los números son los de `mceliece348864` ($n = 3488$, $k = 2720$, $t = 64$) y reproducen el $2^{173{,}4}$ de la guía de seguridad [29].

1962 y 1978: el ataque ya tenía dieciséis años

Prange publicó su decodificación por conjuntos de información en 1962, dieciséis años antes de que McEliece propusiera usar SD como candado [62, 56]. La consecuencia es una virtud didáctica poco común: cuando McEliece presentó su esquema, el mejor ataque genérico ya existía y estaba estudiado, y todo lo que la comunidad ha hecho desde entonces ha sido *afilar* ese ataque. Cada mejora —Stern 1988, Dumer 1991, ball-collision 2011, BJMM 2012, May–Ozerov 2015— es una página de esa historia, y las páginas que siguen las recorren en orden.

14.2 Sesenta años de mejoras

Las mejoras a Prange se entienden mejor como ataques a la condición «hay que acertar que *ningún* error cae en el conjunto de información», que es carísima. Todas aflojan esa condición de una manera u otra:

- **Lee–Brickell (1988) y Leon (1988).** En vez de exigir cero errores en las k columnas elegidas, se toleran unos pocos: se resuelve el sistema con un pequeño conjunto de posiciones presuntamente erróneas y se busca que el vector residual tenga el peso correcto [49, 50].
- **Stern (1988/89): cruzar listas.** El ataque se convierte en una búsqueda de tipo *meet-in-the-middle*: se parte el conjunto de información en dos mitades, se enumeran errores parciales en cada una y se cruzan las listas por su síndrome en una ventana fija (figura 14.2) [70]. El cruce de dos listas de tamaño L cuesta del orden de $2L$ operaciones *por iteración*, y a cambio la probabilidad de éxito por iteración sube muchísimo.
- **Dumer (1991).** Reutiliza las listas de Stern entre permutaciones en lugar de recomputarlas: mismo coste, menos trabajo por intento [31].
- **Ball-collision (2011).** Bernstein, Lange y Peters convierten el cruce de listas en un problema de colisiones de bolas con radios optimizables [11].
- **Representaciones: MMT (2011) y BJMM (2012).** La idea "$1 + 1 = 0$": hay muchas maneras de descomponer un error en suma de vectores parciales, y se puede pedir que las listas contengan solo *representantes*, reduciendo su tamaño sin perder soluciones [54, 5].
- **May–Ozerov (2015).** Sustituye el cruce exacto de listas por la búsqueda eficiente de *vecinos cercanos* en métrica de Hamming, con las estructuras de datos de Indyk–Motwani; es la mejora que fija los mejores números de la tabla 14.1 [55].

- **Both–May (2018).** Afina el régimen de tasas de error altas y las estructuras de vecinos cercanos; su variante con peso parcialmente disjunto es una de las columnas de la tabla 14.1 [15].

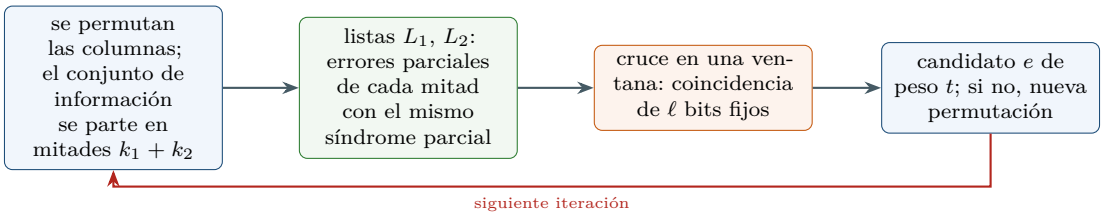


Figura 14.2: La idea de Stern y de toda la familia moderna: en vez de exigir un conjunto de información perfecto, enumerar errores parciales en dos listas y cruzarlas. Las mejoras de 1991–2018 refinan cada caja: cómo se construyen las listas, cómo se cruzan y qué se reutiliza entre permutaciones.

Reproducir la tabla en una tarde

El estimador que produce todos los números de este capítulo es público: el *syndrome decoding estimator* de Esser y Bellini, publicado en PKC 2022 [32] y disponible (con un sucesor más amplio) en el repositorio `Crypto-TII/syndrome_decoding_estimator`. Con él y los parámetros $(n, k, w) = (3488, 2720, 64)$ —y análogos para los otros cuatro conjuntos— se reproducen la tabla 14.1 y sus variantes de memoria. Es la mejor forma de comprobar que estos números son *cotas del mejor algoritmo conocido*, no teoremas: viven en una herramienta, con fecha y versión.

14.3 Los números: memoria libre y memoria con coste

La tabla 14.1 reproduce la tabla principal de la guía de seguridad del equipo [29]: la salida del estimador de Esser–Bellini para los cinco conjuntos, con el modelo de *memoria libre* (acceso a memoria arbitrariamente grande sin coste). La segunda tabla añade los dos modelos con coste de acceso, que discutiremos en la sección siguiente.

conjunto	Prange	Stern	Dumer	BC	BJMM	pdw	MO	BM
`mceliece348864`	173,4	151,4	151,4	151,5	141,9	143,4	**140,8**	142,7
`mceliece460896`	216,8	193,3	193,2	193,3	180,8	182,6	**179,8**	182,1
`mceliece6688128`	295,7	268,0	267,9	268,0	247,3	248,8	**246,2**	249,6
`mceliece6960119`	296,8	268,4	268,3	268,4	246,6	248,4	**245,7**	249,2
`mceliece8192128`	334,1	303,2	307,4	303,4	277,7	279,1	**275,6**	281,1

Cuadro 14.1: El mejor ataque conocido contra cada conjunto (en bits, modelo de memoria libre), según el estimador de Esser–Bellini, en la tabla 1 de la guía de seguridad [29]. BC: ball-collision (Bernstein–Lange–Peters, 2011); pdw: variante de BJMM con peso parcialmente disjunto; MO: May–Ozerov, 2015; BM: Both–May, 2018. El mínimo de cada fila está en negrita: May–Ozerov gana en los cinco conjuntos.

La figura 14.3 cuenta la historia de esos números para `mceliece348864` como una escalera. Prange cotiza en $2^{173,4}$; el primer salto —Stern y su cruce de listas— baja hasta $2^{151,4}$, y treinta años de refinamientos posteriores rascan otros 10,6 bits hasta el $2^{140,8}$ de May–Ozerov. Con la memoria pagada, la escalera se aplana: de 178,3 a 158,6, donde las mejoras modernas aportan solo 4 bits sobre Stern. Leer las dos escaleras juntas es leer toda la sección siguiente.

conjunto	mem	Prange	Stern	BJMM	MO
`mceliece348864`	1/3	176,7	159,3	159,1	156,4
	1/2	178,3	162,8	162,2	**158,6**
`mceliece460896`	1/3	220,4	201,7	201,6	198,6
	1/2	222,1	205,3	204,8	**201,0**
`mceliece6688128`	1/3	299,4	279,2	278,9	274,9
	1/2	301,2	283,0	282,3	**278,2**
`mceliece6960119`	1/3	300,4	280,1	279,4	275,8
	1/2	302,2	283,9	283,3	**279,2**
`mceliece8192128`	1/3	337,7	316,8	316,3	311,9
	1/2	339,5	320,7	320,0	**315,6**

Cuadro 14.2: Los mismos ataques con *costes de acceso a memoria* (modelo 1/3: coste de raíz cúbica; modelo 1/2: coste de raíz cuadrada), tabla 1 de [29]. Compárese una fila con la otra: las mejoras combinatorias valen mucho menos cuando la memoria se paga (para `mceliece348864`, de 178,3 a 158,6 frente a de 173,4 a 140,8).

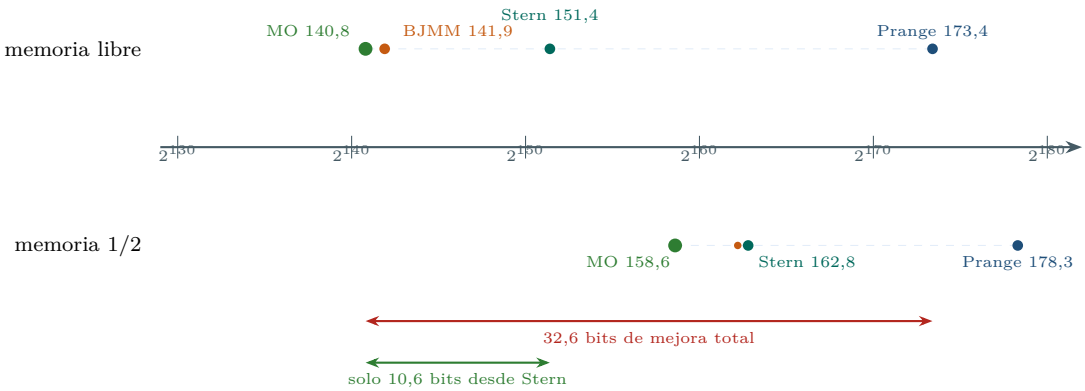


Figura 14.3: La escalera de los ataques contra `mceliece348864`: de Prange (1962) al mejor ataque de 2022, con memoria libre (arriba) y pagando la memoria con raíz cuadrada (abajo). El primer salto —el cruce de listas de Stern— es el grande; el resto es afinar. BJMM aparece en ambos carriles para mostrar el aplanamiento: 141,9 con memoria libre se convierte en 162,2 pagándola.

14.4 Lo que las mejoras no cambian

Hay una manera engañosa y una manera honesta de contar estas seis décadas de ataques, y la guía de seguridad las separa con cuidado. La manera asintótica: si $k = Rn$ con $0 < R < 1$ y t crece despacio (como en McEliece, $t \approx n/\log_2 n$), el coste de Prange es

$$\Big(\frac{1}{1-R} + o(1)\Big)^t,$$

y —esto es lo notable— *ninguna* de las mejoras desde 1962 ha cambiado el término principal: solo el $o(1)$ [29]. En el sentido de los exponentes asintóticos, Stern, BJMM y May–Ozerov no han hecho mejor que Prange; han reescrito las constantes. Cuando se leen artículos que reportan ganancias asintóticas, casi siempre están midiendo otro régimen —valores de t del orden de n, como media distancia de Gilbert–Varshamov; por ejemplo, allí May–Ozerov reportan $O(2^{0{,}0473n})$ frente a $O(2^{0{,}0576n})$ de Prange [55]— que no es el régimen de McEliece.

⚖ $o(1)$ no es 0

De $(1/(1-R))^t$ no se sigue que «Prange cuesta 5^t»: para $R = 0{,}8$, Prange cuesta $(5 + o(1))^t$ y las mejoras también, pero reemplazar $o(1)$ por 0 produce dos afirmaciones contradictorias —que Prange cuesta exactamente 5^t y que las mejoras cuestan exactamente 5^t—, mientras que en los números concretos las mejoras reducen el coste de verdad (de 173,4 a 140,8 con memoria libre). La resolución es que el $o(1)$ esconde factores que a tamaño criptográfico no son pequeños, y que difieren entre algoritmos: el análisis concreto *es* el análisis del $o(1)$. Por eso la tabla 14.1 se calcula con estimadores, no con teoremas: los exponentes asintóticos dicen qué problema es difícil; los estimadores dicen cuánto cuesta hoy.

Observación 14.1: ¿Por qué $n = 4608$ no es sospechosamente corto?

El conjunto `mceliece460896` usa $n = 4608$ mientras que su cuerpo tiene 8192 elementos: $n < q$, por debajo del $n = q$ original de 1978. La duda —¿y si el caso corto fuera más fácil?— la resuelve una reducción de acortamiento: cualquier ataque contra (q, n, t), con $n \le q$, se transporta a un ataque contra (q, q, t) —el sistema original, con cuarenta años de estudio— y su probabilidad de éxito se multiplica exactamente por $\binom{n}{t}/\binom{q}{t}$. Para $(4608, 96)$ y $(8192, 96)$, la pérdida de seguridad de este traslado es de 0,8 bits en el modelo 1/2: analizar el sistema original de McEliece es, a efectos prácticos, tan exigente como analizar el acortado. La reducción completa está en la sección 4.1 de [29].

14.5 El debate metodológico: qué cuenta un 2^b

La tabla 14.2 introduce el matiz más importante de toda la parte: *el modelo de memoria cambia las conclusiones*. Los modelos oficiales del estimador son tres [32]:

- **mem $= 0$: memoria libre.** Los accesos a memoria arbitrariamente grande no cuestan. Es el modelo de la tabla 14.1; la propia guía lo llama «físicamente implausible» [29].
- **mem $= 1/3$ y $1/2$: costes de raíz cúbica y raíz cuadrada.** Cada modelo asigna al acceso a memoria un coste plausible en función de su tamaño. En estos modelos, las mejoras modernas valen mucho menos: «la mayor parte de la aceleración que reivindican estos algoritmos a tamaño criptográfico depende de asignar costes irrealmente bajos a los accesos a memoria» [29].

El segundo matiz es la *unidad*. El estimador cuenta cada operación vectorial —un XOR entre vectores de n bits, digamos— como una operación: encontrar C colisiones entre dos listas de L vectores cuesta $2L + C$, «sin importar la longitud del vector» [29]. Es una unidad cómoda para comparar algoritmos entre sí, pero no es una operación de bits. De ahí la paradoja aparente que ya señaló el capítulo 12: el 140,8 de `mceliece348864` «cae por debajo» del 143 de AES-128 solo si se comparan unidades distintas; en operaciones de bits, ese ataque cuenta bastante más de 2^{143}. Y hay una tercera capa de realismo, la única que valida implementaciones: el tiempo medido en hardware. La guía lo resume con las dos formas de comparar `mceliece6960119` con AES-256: con memoria libre, «las variantes modernas de ISD han reducido el número de operaciones de bits considerablemente por debajo de 2^{256}»; con costes realistas, «se espera que este conjunto sea más caro de romper que AES-256 pre-cuántico y mucho más caro que AES-256 post-cuántico» [29]. Por eso las categorías del equipo son $1, 3, 5, 5, 5$: están calculadas con la memoria pagada.

⚠ Comparar el número equivocado con la referencia equivocada

Tres errores de lectura, frecuentes incluso en publicaciones. (1) Comparar una cifra de memoria libre con las referencias AES: en 1/2, para `mceliece6960119`, May–Ozerov cuesta 279,2 bits, por encima de AES-256 (272): la «tensión» con la categoría 5 que algunos artículos señalan nace de comparar 245,7 (libre) con 272. (2) Olvidar que los estimadores subestiman las operaciones de bits: la guía espera que los números «sean sustituidos por números mayores procedentes de estimadores que cuenten operaciones de bits» [29]. (3) Creer que un número bajo implica un ataque ejecutable: $2^{140,8}$ operaciones vectoriales con memoria $2^{86,6}$ describen un atacante hipotético con acceso perfecto a más memoria de la que ningún computador real puede ofrecer (`mceliece348864` no está amenazado en la práctica; el propio estimador la asume gratuita y perfectamente direccionable).

14.6 Criptoanálisis cuántico

Los ataques de esta sección admiten aceleración cuántica, y el resultado es más modesto de lo que la palabra «cuántico» sugiere. Grover —y los paseos cuánticos, su generalización— ofrece una raíz cuadrada: si un ataque requiere 2^{λ} operaciones clásicas, la versión cuántica ronda $2^{\lambda/2}$, pero solo del bucle externo, y con sobrecostes grandes en el bucle interno [41, 7, 46, 47]. La propia guía lo formula así: «está claro estructuralmente que los ataques cuánticos conocidos producen un efecto menor que en, por ejemplo, la búsqueda de claves AES», porque Grover aplicado a la decodificación por conjuntos de información sufre un sobrecoste mucho mayor en el bucle interior [29].

Con esa advertencia, el modelo estándar para asignar seguridad post-cuántica (el de ISO, capítulo 12) es simple: tomar el mejor ataque clásico sin memoria y partir su exponente. Para Stern sin memoria [29], y con la mitad:

	`348864`	`460896`	`6688128`	`6960119`	`8192128`
clásico (Stern, mem. 0)	151,4	193,3	268,0	268,4	303,2
mitad (modelo cuántico)	75,7	96,7	134,0	134,2	151,6

Dos consecuencias. La primera: en ese modelo pesimista, los dos conjuntos pequeños caen por debajo de 2^{128}, y por eso la guía solo recomienda los tres mayores para horizontes largos (la historia completa, incluida la decisión de ISO de estandarizar la familia entera, está en el capítulo 12). La segunda: el mecanismo de rechazo implícito añade una defensa independiente. La semilla maestra δ (que genera la clave de rechazo s) tiene 32 bytes —256 bits—, y el término

correspondiente del teorema QROM (teorema 8.5) es la raíz de su espacio: un atacante cuántico necesitaría del orden de 2^{128} consultas al hash solo para explotar esa vía [29]. El cuello de botella sigue siendo SD, pero el diseño no deja flancos más baratos.

14.7 Ejercicios

1. **Prange a mano.** Para `mceliece348864` ($n = 3488$, $k = 2720$, $t = 64$): (a) calcule el exponente de $((n-k)/n)^t$ con dos decimales; (b) estime el coste de una iteración como k^3 y componga el total; (c) compare con el 173,4 de la tabla 14.1 y explique el medio bit de diferencia (pista: invertibilidad y constantes de la eliminación).

2. **El primer salto.** La escalera de `mceliece348864` va de 173,4 (Prange) a 151,4 (Stern) y de ahí a 140,8 (May–Ozerov). Calcule cuántos bits y cuántos factores de 2^{20} aporta cada salto. ¿Por qué decimos que «el primer salto es el grande»?

3. **La paradoja del 5^t.** Explique con el recuadro de rigor: ¿cómo pueden Prange y May–Ozerov tener el mismo exponente asintótico $(1/(1-R))^t$ y, a la vez, costes concretos que difieren en 32,6 bits para `mceliece348864`?

4. **Las dos referencias.** Con las tablas 14.1 y 14.2: (a) ¿cuántos bits cuesta pagar la memoria en el mejor ataque de cada modelo para `mceliece6960119`? (b) ¿Cómo se compara cada cifra con AES-256 (272 operaciones de bits) y por qué la categoría del equipo es la 5?

5. **El acortamiento.** Con la observación 14.1: escriba en cinco líneas por qué saber que $(8192, 96)$ es duro «basta» para creer que $(4608, 96)$ lo es, y en qué se pierde el 0,8 de bit.

6. **Optimismo cuántico.** La tabla cuántica de esta sección parte el exponente de Stern, pero la guía avisa de sobrecostes del bucle interno. Explique por qué la mitad es una *cota optimista* para el atacante, y qué papel juegan en el balance los 256 bits de la semilla maestra δ (que genera la clave de rechazo s).

CAPÍTULO 15

Ataques estructurales: recuperar la clave privada

En este capítulo

Los ataques de decodificación del capítulo 14 tratan la clave pública como una matriz aleatoria y van a por el error e. Esta otra familia hace lo contrario: explota que detrás de la matriz hay *estructura* —un polinomio de Goppa g y un soporte ordenado $\alpha_0, \ldots, \alpha_{n-1}$— y ataca directamente la clave privada. Es la familia que durante casi medio siglo pareció una promesa rota: soportes que se dividen, distinguidores que reconocen a Goppa, y aun así un muro de $2^{9116,8}$ donde el mejor ataque genérico cuesta $2^{140,8}$. Veremos por qué el diseño se blindó contra ella ($n < q$), mediremos el abismo con una figura, y dejaremos el terreno preparado para el terremoto de 2025–2026 del capítulo 16. Este es el capítulo que responde a la pregunta «¿y si atacan la clave en vez del mensaje?».

15.1 Dos cajas fuertes: la anatomía de la clave privada

Recordemos qué hay dentro de una clave privada (capítulo 4): un polinomio de Goppa

$$g(x) \;=\; g_t x^t + g_{t-1}x^{t-1} + \cdots + g_1 x + g_0 \;\in\; \mathbb{F}_q[x],$$

sin raíces en $\mathbb{F}_q$, y un soporte ordenado $\alpha_0, \ldots, \alpha_{n-1}$ de elementos distintos de $\mathbb{F}_q$ —las raíces del polinomio $\prod_i (x - \alpha_i)$—. Con la clave privada $(g, \alpha_0, \ldots, \alpha_{n-1})$ uno descifra exactamente igual que el receptor: cualquier atacante que la recupere rompe la tarea OW-CPA sin tocar el problema SD. La pregunta estructural es, por tanto: *¿cuánto cuesta enumerar las claves privadas posibles?*

Dos hechos de conteo elemental, que la guía y la rationale del equipo subrayan [29, 28]:

- Los polinomios g candidatos (mónicos, de grado t sobre $\mathbb{F}_q$, sin raíces en $\mathbb{F}_q$) son del orden de q^t/t. Para el conjunto clásico $m = 12$, $t = 64$: $q^t/t = 4096^{64}/64 = 2^{768-6} = 2^{762}$. Para los parámetros originales de 1978, $2^{494,4}$.
- Las tuplas ordenadas $(\alpha_0, \ldots, \alpha_{n-1})$ de elementos distintos de $\mathbb{F}_q$ son $q(q-1)\cdots(q-n+1)$: para `mceliece348864`, unos 38 499 bits; para los parámetros originales ($n = q$), $1024! \approx 2^{8\,769}$.

A primera vista ambos números son abrumadores, y ahí está el primer malentendido: *los ataques estructurales no enumeran a lo bruto*. Dividen el trabajo. La figura 15.1 muestra las dos rutas conocidas hacia la clave, y la caja negra en la que se apoyan: el algoritmo de división del soporte de Sendrier (2000) [67].

15.2 La división del soporte

El algoritmo de Sendrier merece presentación pausada, porque es la pieza que hace pensables los ataques estructurales. Su tesis: *dados g y el desordenado conjunto $\{\alpha_0, \ldots, \alpha_{n-1}\}$ —no las*

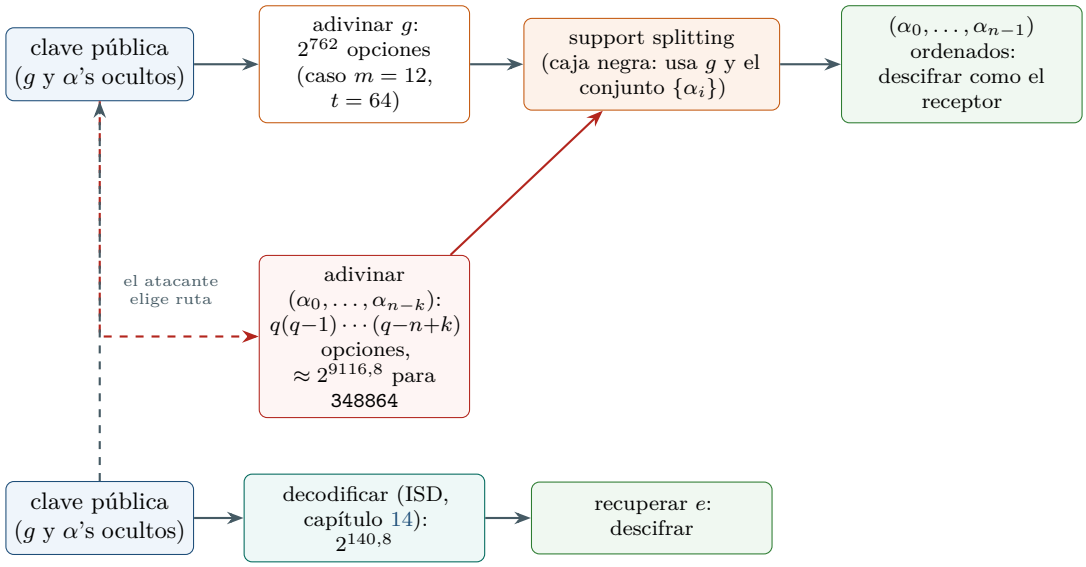


Figura 15.1: Las rutas conocidas hacia la clave privada, a fecha de 2022. Arriba, la ruta estructural: adivinar g y dividir el soporte con el algoritmo de Sendrier [67]; con $n = q$ basta adivinar g, y con $n < q$ la puerta exige además las posiciones $(\alpha_0, \ldots, \alpha_{n-k})$ (ataque de Kirshanova–May, 2022, con dato parcial [48]). Abajo, la ruta genérica de decodificación, que no usa estructura alguna. Los números son los de las fuentes: 2^{762} y $2^{9116,8}$ para `mceliece348864`, $2^{140,8}$ de la tabla 14.1 [29].

posiciones, solo el conjunto—, el polinomio de Goppa permite reconstruir el orden $\alpha_0, \ldots, \alpha_{n-1}$ *en tiempo rápido.* El nombre viene de ahí: el algoritmo «divide» el soporte en las partes que g distingue, y de esas divisiones deduce qué columna del código corresponde a qué elemento del cuerpo [67]. La guía lo enuncia así: recupera $(\alpha_0, \ldots, \alpha_{n-1})$ «a partir de la clave pública, g y $\{\alpha_0, \ldots, \alpha_{n-1}\}$, bajo supuestos menores» [29]. Los detalles finos viven en el artículo original; para contar el ataque completo basta tratarlo como caja negra.

La consecuencia para el McEliece original es inmediata. Su diseño tomaba $n = q$: el soporte es *todo* el cuerpo $\mathbb{F}_q$, un conjunto público, y por tanto el ataque estructural se reduce a enumerar el polinomio g —$q^t/t \approx 2^{494,4}$ posibilidades para los parámetros de 1978— y dejar que la división del soporte haga el resto. En 2000 esto no rompió a McEliece, pero convirtió lo que parecía un espacio de claves de $1024! \approx 2^{8\,769}$ (las tuplas ordenadas completas) en una enumeración de $q^t/t \approx 2^{494,4}$ polinomios g —el orden del soporte lo aporta el propio algoritmo—, y dejó claro cuál era la pieza a defender.

La defensa, ya lo habrá adivinado el lector, es $n < q$: si el cuerpo tiene q elementos y solo se usan n, el conjunto $\{\alpha_0, \ldots, \alpha_{n-1}\}$ *deja de ser público.* Aparecen $\binom{q}{n}$ conjuntos posibles, y la caja negra ya no tiene una de sus entradas. La rationale del equipo lo dice explícitamente: mantener n por debajo de q es «una defensa extra contra la división del soporte»; para `mceliece6960119` hay $\binom{8192}{6960} \approx 2^{4997}$ conjuntos posibles —«aunque podría haber maneras de fusionar el trabajo entre posibilidades» [28]. (Merece la pena notar el doble filo: contra los ataques de decodificación, $n < q$ no ayuda ni perjudica; el capítulo 14 cuantificó ese efecto en menos de un bit. La asimetría es real: el mismo truco que es neutro contra ISD es el que sostiene el muro contra lo estructural.)

En 2022, Kirshanova y May añadieron una pieza más al rompecabezas [48]: dado un *dato parcial* —las posiciones $(\alpha_0, \ldots, \alpha_{n-k})$ de la clave ordenada—, se recupera el resto de la clave privada con un algoritmo rápido. Combinado con una búsqueda a lo bruto sobre ese dato parcial, esto da la cota superior estándar para la recuperación de clave: hay $q(q-1)\cdots(q-n+k)$ tuplas posibles, «aproximadamente $2^{9116,797}$ posibilidades para $(q, n, k, t) = (4096, 3488, 2720, 64)$», los parámetros de `mceliece348864` [29]. La tabla 15.1 resume todos los espacios de búsqueda en

juego.

ruta	espacio a enumerar	coste (ejemplo)	fuente
adivinar g; soporte conocido ($n = q$)	q^t/t	2^{762} ($q = 4096$, $t = 64$)	[29]
adivinar g; con $n < q$	$\times \binom{q}{n}$	2^{4997} ($\binom{8192}{6960}$)	[28]
dato parcial $(\alpha_0, \ldots, \alpha_{n-k})$	$q \cdots (q-n+k)$	$2^{9116{,}8}$ (348864)	[29, 48]
parámetros originales (1978)	q^t/t y $q \cdots (q-n+k)$	$2^{494{,}4}$ y $2^{4794{,}7}$	[28]
referencia: decodificación genérica	—	$2^{140{,}8}$ (348864)	[29]

Cuadro 15.1: Los espacios de búsqueda de los ataques estructurales, con los ejemplos que citan las fuentes. La última fila no es un ataque estructural: es el rival genérico del capítulo 14, incluido para que la comparación de la figura 15.2 tenga a mano sus dos extremos.

Sendrier y la tradición de la equivalencia de códigos

Los ataques estructurales no nacieron contra McEliece: nacieron de una pregunta más vieja de la teoría de códigos, la *equivalencia*: ¿cuándo dos matrices generadoras distintas describen el mismo código con las posiciones permutadas? Sendrier llevaba años construyendo herramientas para responderla cuando, en 2000, publicó la división del soporte [67]. Desde entonces la criptografía basada en códigos la adoptó como parte estándar del oficio; el capítulo de Overbeck–Sendrier del libro clásico de criptografía post-cuántica la presenta junto a los ataques de decodificación como la segunda gran familia [59]. La lista de trabajos de criptoanálisis que la rationale del equipo recopila —treinta publicaciones entre 1981 y 2022, con hitos como Canteaut–Sendrier (1998) o la reciente Carrier–Debris-Alazard–Meyer-Hilfiger–Tillich (2022)— muestra las dos tradiciones entrelazadas durante cuatro décadas [28, 18, 19].

Observación 15.1: El doble filo de $n < q$, en una línea

Para ISD, elegir $n < q$ cuesta a lo sumo fracciones de bit (observación 14.1). Para lo estructural, elegir $n < q$ multiplica el trabajo de la división del soporte por el número de conjuntos posibles ($\approx 2^{4997}$ para 6960119, según [28]). Un mismo parámetro, dos efectos de signo opuesto: por eso los conjuntos se eligen con el estimador delante, no a ojo.

15.3 Distinguidores: saben que es Goppa, ¿y qué?

Hay una familia de resultados estructurales que no recupera la clave ni decodifica: *distingue*. Un distinguidor es un algoritmo que, mirando la clave pública, decide si proviene de una Goppa binaria o de una matriz aleatoria con probabilidad de acierto apreciable, y con un coste que en los parámetros reales sería ridículo (2^{40}, $2^{60}\ldots$). Como en el capítulo 13 adelantamos en forma de trampa, un distinguidor viola el supuesto «pk parece aleatoria», pero no implica por sí solo ni recuperar e ni recuperar la clave. La historia de estos ataques es la historia de sus *márgenes*: todos funcionan en regímenes de parámetros que McEliece no usa, y todos mueren mucho antes de llegar a los conjuntos reales. La tabla 15.2 los recoge.

Distinguir no es romper

La lógica, para que no queden cabos sueltos. (1) El objetivo criptográfico del capítulo 13 es OW-CPA, y el lema 13.3 lo redujo a SD: recuperar e. (2) Un distinguidor devuelve un

ataque	criterio	funciona en	McEliece usa
Faugère et al. 2013 [33]	pocos errores	≤ 8 ($n = 1024$); ≤ 20 ($n = 8192$)	50; 96–128
Carrier et al. 2022 [19]	tasa de código baja	$< 0{,}3$	0,7–0,8
Couvreur et al. 2017 [30]	grado pequeño	$m \leq 2$	$m = 12, 13$

Cuadro 15.2: Los distinguidores conocidos y sus márgenes, según la rationale del equipo [28]. La segunda columna dice qué región del espacio de parámetros explota cada ataque; la tercera, hasta dónde llega; la cuarta, dónde viven los conjuntos de Classic McEliece. El ataque de 2013 es el llamado «distinguidor de alta tasa» de Faugère–Gauthier-Umaña–Otmani–Perret–Tillich; el de Couvreur–Otmani–Tillich ataca las variantes «wild».

bit —«¿es Goppa?»— y nada más; no produce ni e ni $(g, \alpha_0, \dots)$. (3) No existe reducción conocida del primero al segundo: que la clave pública sea reconocible como Goppa no da, hoy por hoy, ninguna vía directa para invertir SD más rápido que los ataques genéricos. (4) El «caso peor lógico» está identificado desde el propio distinguidor de 2013: que alguien encadene el distinguidor con un ataque de inversión. Los autores del distinguidor y el equipo de Classic McEliece han vigilado esa posibilidad durante una década —y los márgenes de la tabla 15.2 son precisamente la distancia que la cadena tendría que cubrir. (5) Nada de esto es un teorema de imposibilidad: es un historial. El capítulo 16 contará cómo en 2025 un distinguidor directo —no una cadena— cambió el tono de la conversación.

«Si su familia cayó, la mía también»

Cuando una propuesta basada en otra familia de códigos se rompe —y ha ocurrido muchas veces, como recuerda la rationale [28]—, los autores de los ataques suelen probarlos contra Goppa binaria. La rationale recoge el resultado invariable: «estos estudios muestran consistentemente que el sistema de McEliece está mucho más allá de todos los ataques conocidos» (cada golpe contra Goppa queda en la situación de la tabla 15.2: funcionan en regiones que los parámetros evitan). La lección no es que Goppa sea invulnerable: es que cada familia tiene su anatomía, y el criptoanálisis se trasplanta entre familias con mucha menos facilidad de la que sugieren los titulares. El capítulo 16 mostrará la otra cara del mismo argumento: cuando el trasplante *sí* prendió.

15.4 El balance de 2022

Con todo lo anterior, la foto a fecha de la guía de seguridad es nítida: el mejor ataque de recuperación de clave contra `mceliece348864` cuesta $2^{9116{,}8}$ (tabla 15.1) y el mejor ataque genérico, $2^{140{,}8}$ (tabla 14.1). La figura 15.2 pone las tres barras relevantes en una misma escala logarítmica: decodificar, adivinar g, adivinar los α's.

La lectura del abismo es obligada en este punto del libro. Durante cuarenta y cinco años, todos los intentos de explotar la estructura de Goppa se estrellaron contra distancias como esa: la cita de la rationale es tajante —«los ataques de recuperación de clave del estado del arte son enormemente más lentos que la decodificación por conjuntos de información» [28]— y la conclusión estratégica del equipo fue doble. Primera: conservar los códigos de Goppa binarios, «la elección tradicional, conservadora y bien estudiada» [28], pese a que existen alternativas con claves más pequeñas. Segunda: documentar cada margen, porque los márgenes *son* la seguridad en esta familia de ataques: no existe un teorema que diga «Goppa es rígida»; existe la cuenta de todos los ataques que se han quedado cortos.

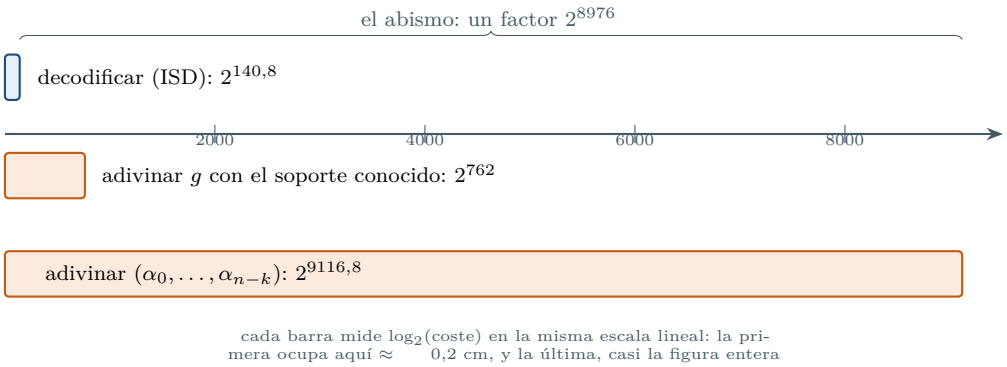


Figura 15.2: El balance de 2022 para `mceliece348864`, en escala de exponentes (bits). Decodificar cuesta $2^{140,8}$; recuperar la clave por la vía de g cuesta 2^{762} (y esta vía requiere haber resuelto antes el conjunto, o $n = q$); recuperarla por la vía del dato parcial cuesta $2^{9116,8}$. Datos: tablas 15.1 y 14.1.

Ese es el muro tal como se veía en 2022. El capítulo 16 cuenta cómo, entre 2025 y 2026, aparecieron las primeras grietas —primero un distinguidor directo, después ataques de recuperación de clave— y cómo reaccionaron el equipo, ISO, BSI y NIST.

15.5 Ejercicios

1. **Aritmética del muro.** Verifique con una calculadora: (a) $4096^{64}/64 = 2^{762}$; (b) $1024^{50}/50 \approx 2^{494,4}$; (c) $\log_2(q(q-1)\cdots(q-769+1)) \approx 9116{,}8$ para $q = 4096$ (puede usar la identidad $\log_2 P(q,r) = \sum_{j=0}^{r-1} \log_2(q-j)$, una hoja de cálculo, o la aproximación de Stirling).

2. **Conjuntos y tuplas.** Estime $\log_2 \binom{4096}{3488}$ con la fórmula de Stirling para entropías ($\approx 2^{2476}$) y compárela con $2^{9116,8}$: ¿por qué una va con n y la otra con $n-k$? (pista: la segunda pide posiciones *concretas* del soporte ordenado, no solo el conjunto).

3. **El abismo en potencias.** (a) Calcule $9116{,}8 - 140{,}8$ y exprese el cociente de costes como potencia de 2. (b) Compare esa potencia con 2^{256} (el margen de AES-256) y comente en dos líneas por qué el titular «recuperar claves de McEliece cuesta 2^{9116}» es, además de correcto en 2022, casi cómico como amenaza práctica.

4. **Distinguir no es romper.** Use el recuadro de rigor: escriba la cadena lógica que convertiría un distinguidor en un ataque de inversión de SD, señale el eslabón que falta y explique qué papel juegan los márgenes de la tabla 15.2.

5. **El doble filo de $n < q$.** Con la observación 14.1 (capítulo 14) y la observación 15.1: explique en un párrafo por qué los conjuntos de Classic McEliece pueden tomar $n < q$ sin pagar seguridad contra ISD y cobrar, en cambio, ventaja contra la división del soporte.

6. **Tabla de supervivencia.** Ordene los tres distinguidores de la tabla 15.2 por «holgura» (distancia relativa entre su límite y los parámetros reales) y discuta cuál de los tres le parece el candidato más peligroso a escalar. (No hay solución única: se trata de razonar sobre los regímenes.)

CAPÍTULO 16

El terremoto: 2025–2026

En este capítulo

El capítulo 15 cerró con un muro: recuperar la clave privada costaba, en 2022, unos $2^{9116,8}$ frente a los $2^{140,8}$ de la decodificación genérica. Este capítulo cuenta cómo ese muro se resquebrajó en tres actos. Primero la grieta (2024–2025): un distinguidor subexponencial y su réplica formal. Después la primavera de 2026: ataques algebraicos que rompen retos públicos y acercan las estimaciones, aún por encima de los niveles declarados. Y por fin el verano: el método *holdout*, un distinguidor cuasipolinomial de origen insospechado, y una cascada de estimaciones de recuperación de clave que caen a 2^{89}–2^{149} *por debajo* de los niveles declarados de todos los conjuntos —con supuestos heurísticos y sin ataque práctico alguno—. Cerraremos con el debate técnico (¿cuánto cuesta mover los bits?), la respuesta institucional (el equipo, el BSI, ISO, NIST) y una lectura sobria de lo que un lector de este libro debe llevarse. La regla del capítulo: separar a cada paso lo demostrado de lo heurístico, y lo heurístico de lo titulado.

16.1 La primera grieta (2024–2025)

La tradición del capítulo 15 tenía una moraleja estadística: cada intento estructural se quedaba corto por miles de bits. Esa estadística empezó a romperse en 2024, cuando Hugues Randriambololona publicó el *distinguidor de sizigias* («syzygy distinguisher»)[1] [63], publicado después en EUROCRYPT 2025. Su hallazgo: las claves públicas de Goppa binaria se pueden distinguir de matrices aleatorias con complejidad *subexponencial* en la capacidad de corrección del código. Para `mceliece348864`, la estimación rondaba los 2^{529}; para los conjuntos recomendados por el BSI, al menos 2^{683} (véase su evaluación de 2026 [17]). Frente a los 2^{9000} de la tabla 15.1, la grieta era visible; frente a los $2^{140,8}$ del mejor ataque genérico, seguía siendo abisal.

La respuesta del equipo llegó el 17 de abril de 2025, en una nota fechada titulada precisamente *Notes on a recent claim that a* `mceliece348864` *distinguisher uses only* 2^{529} *operations* [20]. Es una pieza ejemplar de metodología, y conviene resumir sus tres argumentos porque enmarcan todo lo que viene después.

1. **La seguridad no descansa en la indistinguibilidad.** La cadena del capítulo 13 va de arriba abajo: IND-CCA2 en el QROM se apoya en OW-CPA del cifrado, y OW-CPA en la dureza de la decodificación. Que la clave pública sea reconocible como Goppa no toca ningún eslabón: el equipo lo ilustra con RSA, cuyas claves se distinguen de enteros aleatorios de forma trivial sin que ello amenace la factorización [20].

2. **Hay un suelo trivial de 2^{256}.** La clave privada se genera a partir de una semilla de 256 bits; un atacante puede, en el peor caso, enumerar semillas. Cualquier ataque de recuperación de clave por debajo de 2^{256} sería una noticia de primera magnitud; los 2^{529} no lo son. Esta observación, casi trivial, se convertirá en la vara de medir del capítulo.

[1] En álgebra conmutativa, una *sizigia* es una relación lineal entre un conjunto de polinomios o generadores ($\sum a_i p_i = 0$). En el criptoanálisis de códigos, permite buscar relaciones algebraicas no visibles a simple vista calculando el núcleo de matrices de productos, es decir, mediante álgebra lineal pura.

3. **El historial manda prudentemente.** La nota repasa la larga lista de «avances» algebraicos previos —desde los ataques de 2002 que se anunciaban como amenaza para AES hasta un trabajo de 2010 que se describía como «breakthrough»— todos ellos digeridos sin consecuencias: el método muere al comparar constantes, o no escala, o confunde distinguir con romper.

Añadamos dos piezas de contexto. En marzo de 2025, NIST cerró su cuarta ronda eligiendo HQC como respaldo basado en códigos y dejó a Classic McEliece fuera de la estandarización, anotando en el informe IR 8545 que el criptoanálisis reciente «debilita en cierta medida el argumento de tratarlo como una opción especialmente conservadora» y señalando que `mceliece460896` ya quedaba corto frente a su categoría [2]. Y en noviembre de 2025, el equipo publicó una nota sobre *You Only Decapsulate Once*, un ataque de canal lateral que explota variaciones de tiempo introducidas por compiladores modernos en software criptográfico: una advertencia de que la superficie de ataque de una implementación no termina en su algoritmo [22].

De 2002 a 2024: el largo cementerio algebraico

La nota del equipo de abril de 2025 [20] y la de junio de 2026 [21] enumeran, con paciencia de archivero, las promesas algebraicas caídas. La de 2026 abre con un ejemplo de 2002 que anunciaba coste «subexponencial» contra cifrados de bloque y resultó inofensivo; sigue con la lista de trabajos contra McEliece —2010: «breakthrough in code-based cryptography»; 2014: reducción de Wild–Goppa; 2016: el método de claves compactas de Faugère–Otmani–Perret–de Portzamparc–Tillich; 2023: el modelo Pfaffiano de Couvreur–Mora–Tillich; 2024: las sizigias de Randriambololona—. La lección del cementerio no es que los papers fueran malos: es que durante dos décadas cada funeral corporativo se celebraba antes de la autopsia. En 2026, por primera vez, el ritual se invertirá: primero las réplicas, después los títulos.

16.2 La primavera de 2026: de CFS a los retos TII

El 3 de marzo de 2026 apareció el primer golpe con *víctima real* fuera de los parámetros de Classic McEliece: Bardet, Lemoine y Tillich rompieron el esquema de firmas CFS, de 25 años de antigüedad, construido sobre códigos de Goppa binarios de tasa muy alta, aprovechando el modelo Pfaffiano de 2023[2] y una búsqueda de cuadrados en el ideal correspondiente. El mismo documento recuperó las claves de retos de TII «con seguridad declarada de hasta 210 bits» [4]. La letra pequeña salvó a Classic McEliece: el ataque vive de tasas de código extremas, y los conjuntos del esquema tienen tasas de 0,7–0,8. Pero el cementerio acababa de enterrar un esquema entero.

En junio llegaron dos piezas más. El 10 de junio, Briaud, Lemoine, Randriambololona y Tillich (BLRT) publicaron un ataque algebraico *subexponencial heurístico* basado en encontrar matrices de rango 2 en el código de relaciones cuadráticas asociado a la Goppa; con ellas se recupera la estructura secreta y, de ahí, una clave equivalente. Rompieron retos TII de 83, 89, 119, 166, 210 y 248 bits declarados, y una clave CFS en 14 horas y 24 GB de RAM [16]. Para los parámetros de Classic McEliece nivel 1, los propios autores cifran el ataque en «a lo sumo del orden de 2^{454}». El 23 de junio, el equipo respondió con una nota fechada [21]: auditando el cómputo, el coste varía con las suposiciones algebraicas entre 2^{454} (Wiedemann), 2^{513}, 2^{515} (con exponente de multiplicación 2,373), 2^{607} y 2^{610}; ninguna variante baja del suelo trivial 2^{256} (ninguno de esos trabajos lo compara, observa el equipo con retintín); los retos TII rotos son

[2] Un *Pfaffiano* es un polinomio que calcula la raíz cuadrada del determinante de una matriz antisimétrica. El modelo Pfaffiano expresa las relaciones algebraicas del código alternante subyacente como un sistema de ecuaciones polinómicas (un ideal algebraico) sobre los menores antisimétricos de una matriz asociada.

juguetes ($n \leq 1409$) que «omiten una de las defensas centrales» al reducir t y con él las opciones de g; y la narrativa del «ataque que rompe el esquema» repite, palabra por palabra, la de 1991.

El 29 de junio, Tobias Hemmert —del propio BSI— convirtió en recuperación de clave el distinguidor de «anulación de orden superior» (*higher-order vanishing*) que él y Andreas Wiemers habían presentado en CRYPTO 2026 [42, 43]: los polinomios homogéneos que se anulan con multiplicidad alta en las columnas de la matriz de paridad no solo distinguen Goppa de aleatorio; su estructura permite reconstruir la clave. Conjetura del autor: el coste sería «aproximadamente el mismo que el del distinguidor», que el BSI evalúa en al menos 2^{550} para los conjuntos que recomienda [17]. Markku-Juhani Saarinen extendió el método (HOVER) sin mejora significativa para estos parámetros [66]. Balance de la primavera: las estimaciones estructurales habían bajado de miles de bits a 2^{454}–2^{742} —inmensas, pero ya del orden de la decena de miles de millones de veces «solo» por encima de los objetivos, y con la dirección de la flecha en contra.

1991, 1997, 2026: la misma obra, el mismo papel

La nota de junio de 2026 [21] recuerda el precedente: un «criptoanálisis del criptosistema de McEliece» de 1991, que resultó ser un ataque sobre un juguete con $n = 63$; la historia la contó Berson en 1997, y este libro la citó en el capítulo 8 como anécdota de época. El patrón se repite: una demostración en tamaño de juguete, una extrapolación en el titular, una espera de quince años hasta saber si el ataque es un embrión o un espejismo. La diferencia de 2026 es la velocidad: lo que en 1991 tardó un año en replicarse, aquí tarda semanas —y la propia respuesta del equipo está fechada con precisión notarial.

16.3 El método *holdout* (agosto de 2026)

El 7 de agosto de 2026, Ashrujit Ghoshal, Yuval Ishai, Aayush Jain y Nuozhou Sun (GIJS) publicaron un distinguidor *cuasipolinomial* para Goppa–McEliece en el régimen asintótico del esquema ($m = \Theta(\log n)$, $t = \Theta(n/\log n)$), con tiempo $n^{\mathcal{O}(\log n)}$ y ventaja $1 - o(1)$. El origen del método es delicioso: nació de un intento *fallido* de construir protocolos de recuperación privada de información (PIR) a partir de códigos localmente decodificables. El título del artículo [38] lo confiesa: «PIR Meets McEliece».

La idea, en la figura 16.1. Las columnas de la clave pública son evaluaciones de polinomios secretos: yacen —salvo reescalado— sobre una curva algebraica oculta. El test *retira* (hold out) una columna y busca polinomios de bajo grado que se anulen con multiplicidad alta en todas las demás. En una clave real, esos polinomios también se anulan en la columna retirada: es la estructura de curva forzándolo. En una matriz aleatoria, solo ocurriría por azar. Ese sesgo, amplificado sobre muchas retiradas, es el distinguidor. Así descrito parece poco, pero trae la propiedad que ningún distinguidor previo tenía: la complejidad crece *cuasipolinomialmente* en el régimen de los parámetros, y el método aplica a los cinco conjuntos, con estimaciones de 2^{114}–2^{124} operaciones de bits para el test (frente a los 2^{151}–2^{287} de la decodificación genérica), aunque con almacenamiento de exabytes (2^{66}–2^{71} bits) que los autores no intentaron optimizar [38].

El 27 de agosto, GIJS añadieron a su artículo los detalles de un ataque de *recuperación de clave* heurístico basado en la misma maquinaria, con estimaciones de $2^{130,4}$ a $2^{149,0}$ según el conjunto, y de un ataque de recuperación de texto plano que el equipo evaluó en unos 2^{266} para `mceliece6960119` [38, 21]. La revisión también declaraba, con honestidad ejemplar, que el ataque «no es todavía concretamente eficiente» pero que «puede ser relevante para los niveles de seguridad de NIST». En paralelo, la réplica pública del equipo (21 de agosto) recordaba que un distinguidor no ataca los objetivos de seguridad del esquema —el argumento de abril de 2025— y que la maquinaria no produce, por ahora, ningún ataque ejecutable [9, 20].

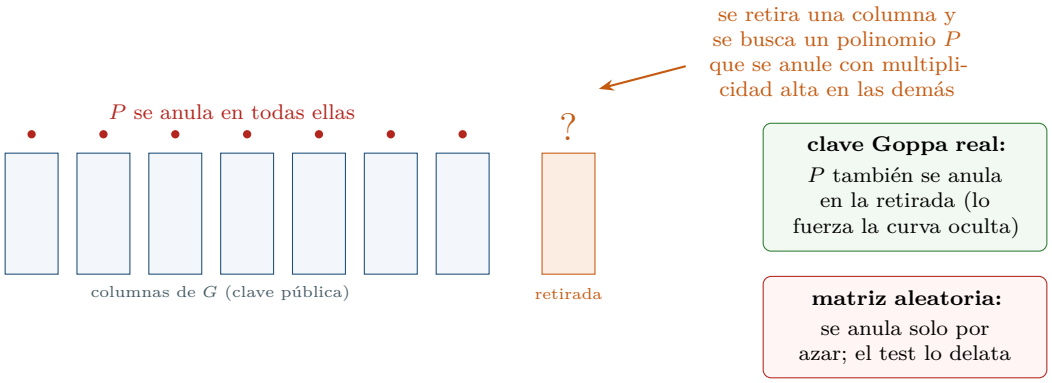


Figura 16.1: El test *holdout* de GIJS [38]: retirar una columna y buscar polinomios de bajo grado que se anulen con multiplicidad alta en todas las demás. Sobre una clave real el polinomio se anula también en la retirada (estructura de curva); sobre una matriz aleatoria, no. La extensión a recuperación de clave usa la geometría local de la curva en la columna retirada [72, 73].

16.4 La cascada de septiembre

Lo que ocurrió después fue una carrera de tres semanas entre grupos —con préstamos cruzados, porque todos parten del test de GIJS— para llevar el *holdout* de distinguidor a recuperación de clave. Conviene registrar los hitos con fechas, porque la velocidad es parte de la historia.

- **Vedenev (20 de agosto; revisión de octubre).** Extiende el método a la recuperación de clave para subcódigos de GRS (y por tanto alternantes y Goppa)[3]: un *único* tipo de ejecución del test recupera la clave, con coste estimado de $2^{89,9}$ a $2^{98,6}$ operaciones de bits para los parámetros de Classic McEliece. Demostración: claves de los retos TII-173, TII-249 y TII-253, «los dos primeros no resueltos hasta donde sabemos» [72].
- **Saarinen (24 de agosto; revisiones).** Publica una contabilidad de costes del método («Holdout»), de $2^{126,8}$ a $2^{145,2}$ según la revisión de septiembre, y resuelve el reto TII-254 —el más difícil resuelto hasta la fecha en su métrica original, 2^{254}— con una variante de dos anclas: dos núcleos de relaciones de dimensión 121, un núcleo común de 80, una geometría proyectiva sobre $\mathbb{F}_{2^8}$, y una secuencia de Krylov final[4] de 27,2 horas-GPU en aceleradoras NVIDIA [64, 65].
- **Weis (11 de septiembre; versión 0.5 de octubre).** Stephen Weis, de Anthropic, mejora el ataque en dos frentes. Primero, el coste de una ejecución baja a 2^{89}–2^{98} operaciones de bits (sin cargo de memoria); pagando cada operación por la memoria que direcciona, a coste de raíz cuadrada, 2^{107}–2^{117}; bajo la regla contable de la guía de seguridad —que carga el coste de toda la memoria en cada operación—, 2^{117}–2^{128}, contra 2^{159}–2^{316} de la decodificación genérica en ese mismo modelo. Segundo, dos rutas de recuperación de clave: una apoyada en un teorema que demuestra y que exige entre 100 y 1400 ejecuciones; otra, de una sola ejecución, que construye sobre el key recovery de GIJS. «Nada de esto está cerca de lo práctico —escribe—, y una ejecución a tamaño completo no puede probarse» [73].

[3] Los códigos Reed–Solomon generalizados (GRS) y los códigos alternantes son familias más amplias que engloban a los códigos de Goppa: un código de Goppa binario es el subcódigo subcampo sobre $\mathbb{F}_2$ de un código GRS.

[4] Un subespacio de Krylov $\mathcal{K}_r(M, v) = \mathrm{span}\{v, Mv, \ldots, M^{r-1}v\}$ permite calcular vectores del núcleo o resolver sistemas lineales dispersos gigantescos (como en los algoritmos de Lanczos o Wiedemann) mediante aplicaciones sucesivas de la matriz, sin requerir su inversión explícita.

- **Apon (26–28 de agosto) y la réplica.** Daniel Apon publica una cota inferior de la ruta de Vedenev: por razones algebraico-geométricas, el número de columnas retiradas que su paso crítico necesita es mayor que $2t + 3$; para `mceliece8192128`, eso implica un coste superior a 2^{1500} operaciones de bits. Vedenev respondió con una revisión que añade una segunda ruta («código menor») y sostiene que las objeciones ya no aplican a la versión nueva [3, 72].

La tabla 16.1 reúne las estimaciones publicadas con el mismo protocolo: operaciones de bits, sin cargo de memoria, para los cinco conjuntos. Es la tabla que en 2022 habría sido ciencia ficción: la columna estructural ya no está en los miles de bits.

conjunto	decodificación (ISD, cap. 12)	GIJS (27-ago)	Saarinen (15-sep)	Weis (v0.5)	AES ref. (NIST)
`mceliece348864`	$2^{151,4}$	$2^{130,4}$	$2^{126,8}$	$2^{94,3}$	2^{143}
`mceliece460896`	$2^{193,3}$	$2^{149,0}$	$2^{145,2}$	$2^{99,3}$	2^{207}
`mceliece6688128`	$2^{268,0}$	$2^{141,3}$	$2^{137,5}$	$2^{102,4}$	2^{272}
`mceliece6960119`	$2^{268,4}$	$2^{140,5}$	$2^{136,7}$	$2^{101,6}$	2^{272}
`mceliece8192128`	$2^{303,2}$	$2^{141,6}$	$2^{137,5}$	$2^{102,4}$	2^{272}

Cuadro 16.1: La fotografía de octubre de 2026, en operaciones de bits y sin cargo de memoria. La columna ISD es el mejor ataque genérico sin trucos de memoria (guía de seguridad; capítulos 12 y 14). Las columnas de 2026 son estimaciones *condicionales*, apoyadas en supuestos heurísticos, de ataques de recuperación de clave: GIJS (revisión del 27 de agosto, su tabla 3), Saarinen (revisión del 15 de septiembre) y Weis (versión 0.5 de octubre, tabla 1) [38, 64, 73]. La última columna es la referencia de NIST para las categorías 1/3/5 (búsqueda de clave sobre el AES correspondiente). Ninguna de las cifras de 2026 corresponde a un ataque ejecutado ni ejecutable en hardware real.

Un matiz imprescindible, del propio equipo: los retos TII resueltos —$n = 214$, $n = 223$— tienen t diminutos y por tanto pocas opciones de polinomio g; «omiten una de las defensas centrales» del esquema [21]. Que la maquinaria digiera juguetes no prueba que digiera los conjuntos reales: eso lo dirá el tiempo, o una ejecución a escala que nadie ha hecho.

16.5 El debate de los costes

Los números de la tabla 16.1 comparten un supuesto silencioso: que una operación de bits cuesta lo mismo que cualquier otra. Daniel Bernstein lo atacó de frente en el foro de NIST. Un *run* del ataque es una cadena de 2^{32} a 2^{36} pasadas mutuamente dependientes sobre un array de 28 a 540 TiB ($2^{47,8}$–$2^{52,1}$ bits); el ataque contra `mceliece348864` movería del orden de 2^{93} bits entre memoria y unidades aritméticas. Enrutar esos bits a la velocidad de interconexión de los superordenadores actuales, calculaba Bernstein, llevaría unos 90 000 años, solo para esa componente; y las cotas de área-tiempo de Brent–Kung sugieren que ningún trazado de circuito elimina el peaje. La misma asimetría se esconde en la comparación con AES: la búsqueda de claves no toca memoria, se amortiza entre muchos objetivos y gana con Grover; el *holdout* no hace nada de eso [9].

Weis, lejos de esquivar el golpe, lo incorpora: su artículo admite que no conoce distribución de datos que haga local el acceso, y ofrece la escalera completa de modelos contables. Con cargo de raíz cuadrada —un modelo con precedente en la propia guía—, una ejecución cuesta 2^{107}–2^{117}: aún por debajo de la decodificación genérica con memoria libre. Con la regla más severa de la guía, 2^{117}–2^{128}. La discusión no es bizantina: es *el mismo* debate metodológico del capítulo 14, ahora aplicado a la otra orilla. Y tiene precedente ilustre: en 2020, el propio Bernstein argumentó

que NIST aplicó a Kyber-512 la vara de las compuertas clásicas (2^{143}), y NIST respondió que tiende a dar el beneficio de la duda a un conjunto cuyos ataques quedan a pocos bits de la categoría. Aquí la distancia no es de pocos bits: bajo el modelo más severo, Weis estima 25 bits bajo la categoría 1 para `mceliece348864` y 82–145 bits bajo las categorías del resto de conjuntos; y Apon declaró el 1 de octubre tener un análisis no publicado que sitúa el nivel 1 en 2^{89}–2^{92} *incluyendo* costes de memoria, pero decidió no publicarlo «para no perjudicar a los usuarios actuales» [73, 9].

⚖ Demostrado, heurístico, estimado: el mapa de 2026

La contabilidad honesta de esta historia tiene tres columnas. (1) **Demostrado**: el distinguidor de GIJS está probado (en su régimen asintótico), y Weis prueba varios enunciados sobre Goppa binaria que sostienen su ruta. (2) **Heurístico**: la recuperación de clave a tamaño real depende de supuestos de rango y genericidad —Weis enumera cinco, los prueba en claves pequeñas y en retos TII ($n = 214$ frente a $n \geq 3488$) y argumenta por qué las desviaciones observadas no ocurrirían a escala—; el grado del ataque es 7; el paso caro (el cómputo de núcleos) no se ha ejecutado contra ningún conjunto real. (3) **Estimado**: todas las cifras de la tabla 16.1 dependen de un modelo de costes —el de las operaciones de bits desnudas— que el propio equipo disputa, y que mueve las cuentas 20–30 bits según el cargo. Nada de la columna (2) (3) convierte a 2026 en «McEliece roto»; nada de la columna (1) permite ya decir «McEliece inmune». El estado del arte es exactamente eso: un arte, con supuestos declarados y réplicas en curso.

16.6 La respuesta institucional

El equipo de Classic McEliece sostuvo durante todo el otoño una posición pública en dos capas. Como equipo: la réplica del 21 de agosto al distinguidor —no ataca los objetivos, no hay ataque ejecutable— y la evaluación del ataque de texto plano ($\approx 2^{266}$ para `mceliece6960119`). Y la respuesta fechada de junio [21]. Como individuos: Bernstein mantuvo, el 30 de septiembre, que ninguna nación-estado tiene recursos para ejecutar los ataques, que estos no ganan nada con múltiples objetivos ni con computación cuántica, y que su evaluación relativa —ML-KEM con más riesgo que McEliece— no había cambiado. La pregunta directa de Filippo Valsorda —¿sigue el equipo recomendando sus conjuntos estandarizados?— no había recibido respuesta etiquetada como del equipo a fecha de cierre de este libro [9].

El movimiento decisivo fue del BSI, el 1 de octubre de 2026, en una nota de cuatro páginas fechada el 29 de septiembre [17]. Su evaluación, en orden: los distinguidores previos no amenazaban los supuestos (el de sizigias, $\geq 2^{683}$; el de Hemmert–Wiemers, $\geq 2^{550}$; BLRT, $\geq 2^{742}$); los métodos de agosto –octubre sí merecen acción, con cifras de 2^{120}–2^{124} (distinguidor GIJS) y 2^{140}–2^{148} (recuperación GIJS) para los conjuntos de su guía, y 2^{99}–2^{102} en la contabilidad de Weis; los resultados «no permiten todavía un ataque práctico», pero la mejora estructural «es significativa» y se esperan más. Conclusión operativa: Classic McEliece «no debería seguir usándose en nuevos desarrollos»; para intercambio de claves, preferir ML-KEM o FrodoKEM, y HQC cuando su estándar aparezca; los despliegues híbridos existentes conservan al menos la seguridad de la mitad clásica; y la recomendación formal se revisará en TR-02102-1 a principios de 2027. El BSI añadió una lección de migración: los resultados «subrayan la importancia de la *crypto-agility*», y ninguno de los ataques toca a los otros esquemas recomendados —ni a HQC, que no usa códigos de Goppa ni la estructura oculta que estos ataques explotan [17].

En paralelo, el ecosistema: NIST dejó la puerta abierta a estandarizar McEliece más adelante, pero su IR 8545 ya había elegido HQC y anotado los recortes de `mceliece460896`; ISO publicó

en junio de 2026 su enmienda con los cuatro conjuntos grandes —sin `mceliece348864`, el más expuesto a los nuevos métodos— y cada uno de esos cuatro tiene ahora estimaciones por debajo de su nivel declarado; las agencias europeas divergen (ANSSI ya lo situaba por debajo de ML-KEM, FrodoKEM y HQC; el manual holandés lo tenía por conservador; la NCSC sueca solo bendice estándares NIST); y los despliegues reales —túneles VPN de Mullvad, Rosenpass, PQConnect—, todos en modo híbrido, activaron sus protocolos de revisión [9, 17, 2].

⚠ Los dos titulares fáciles

Desde agosto de 2026 circulan dos frases gemelas, y ambas son falsas. «*McEliece ha caído*»: nadie ha recuperado una clave ni descifrado un texto de ningún conjunto real; los propios autores de los ataques lo repiten; el ataque más caro de ejecutar —el cómputo de núcleos— no se ha corrido a escala; los retos rotos son juguetes. «*McEliece es inmune*»: las estimaciones de recuperación de clave de tres grupos independientes están por debajo de los niveles que el propio esquema declaró, el BSI —que lo recomendaba desde 2020— ha pedido detener nuevos despliegues, y la incógnita no es si los supuestos heurísticos fallarán, sino cuándo y cuánto. La versión utilizable de la verdad tiene tres cláusulas y ninguna es un titular: nadie ha roto nada, muchas estimaciones condicionales han bajado de los niveles declarados, y la comunidad tarda años en digerir cuánto pesa cada supuesto. Quien lea una sola de las dos frases, que relea este recuadro.

16.7 Lectura final

¿Qué debe llevarse el lector de este libro de todo el terremoto? Cuatro cosas, ninguna grandilocuente.

Primera: la criptografía finita se juega en los márgenes. Los conjuntos de Classic McEliece llevaban cuarenta y ocho años fuera de rango de todos los ataques estructurales conocidos, y estaban elegidos contra *un modelo* de ataque (la decodificación genérica) con márgenes que la propia guía declaraba para el otro modelo en miles de bits. El método *holdout* apareció fuera de ese modelo, traído de la teoría de PIR, y la lección no es «los márgenes eran falsos» sino la del capítulo 13: *seguridad es la mejor estimación disponible más los supuestos, y los supuestos envejecen.*

Segunda: el eje manda. Weis midió que el coste del *holdout* crece unos ocho bits por cada unidad del grado m del cuerpo, y apenas depende de n: por eso sus conjuntos recomendados (todos con $m = 12$ o 13) se comportan en bloque. Sus ilustraciones —$m = 18$, 26, 34, con claves de 0,6, 2,1 y 7,1 megabytes— y la instancia de Bernstein ($m = 18$, $n = 6176$, $t = 112$, clave de 1 MB, texto cifrado de 252 bytes, ejecución estimada en 2^{142}) dibujan la dirección de una futura re-parametrización: cuerpos más grandes, n bien por debajo de 2^m, claves más grandes. Nadie ha propuesto tal conjunto para estandarización, y Weis lo etiqueta «ilustraciones» [73].

Tercera: la asimetría de las pruebas. El distinguidor está probado; la recuperación de clave, no. Los experimentos existen solo en tamaño de juguete. Quien compare las cotas del capítulo con las de este debe recordar la dirección del rigor: una estimación heurística baja no convierte a un esquema en roto —ni una superioridad histórica lo convierte en intacto. La única moneda válida es la misma que usa este libro desde la página 1: enunciar el modelo, citar la fuente, y separar lo demostrado de lo conjeturado.

Cuarta: esto es historia viva. A la fecha de este libro (octubre de 2026), las réplicas siguen circulando (la última revisión de Vedenev es del 7 de octubre), el BSI revisará su guía a principios de 2027, ISO mantiene publicado su estándar, y las estimaciones de los distintos grupos —dentro del mismo marco— difieren en decenas de bits. Cualquier lector que llegue a

estas páginas años después debería buscar la actualización de la guía [29], las respuestas fechadas del equipo —la serie de notas continuará— y las réplicas de las revistas que aún están por llegar. El resto del libro no depende de la resolución del terremoto: el mecanismo, los conjuntos, la ISO, el laboratorio, todo lo anterior sigue siendo exactamente lo que las páginas describen. Lo que cambia es la conversación sobre si conviene usarlo, y esa conversación, por primera vez en la historia del esquema, está abierta.

16.8 Ejercicios

1. **El muro de 2022 contra el suelo de 2026.** (a) Compare la mejor estimación estructural de la tabla 15.1 para `348864` ($2^{9116,8}$) con la de Weis ($2^{94,3}$): calcule la diferencia de exponentes y comente en dos líneas por qué *no* es el «mismo ataque mejorado» (pista: ¿qué familia de métodos estaba agotada y cuál es nueva?). (b) Compare ahora $2^{94,3}$ con el suelo trivial 2^{256} de la respuesta del equipo [20]: ¿qué implica que la estimación esté por debajo?

2. **Una ejecución, cien ejecuciones.** Con la ruta de Weis que exige entre 100 y 1400 ejecuciones, calcule el coste total si una ejecución cuesta 2^{89} (sin memoria) y si cuesta 2^{117} (con cargo severo). Encaje ambos resultados con la afirmación «esta es nuestra mejor estimación; con el cargo de la guía queda por debajo de la decodificación genérica en ese modelo».

3. **El eje m.** Partiendo de $2^{94,3}$ ($m = 12$) y de la regla de Weis (unos 8 bits por unidad de m), estime el coste para $m = 18$ y compárelo con la instancia de Bernstein ($n = 6176$, $t = 112$), cifrada en $\approx 2^{142}$. ¿Cuánto sube la clave pública (en MB) frente a las actuales?

4. **Distinguir, recuperar, romper.** Clasifique cada afirmación como demostrada, heurística o estimada, usando el recuadro de rigor: (a) «las claves de Goppa binaria se distinguen de matrices aleatorias con ventaja $1 - o(1)$»; (b) «recuperar la clave de `8192128` cuesta $2^{102,4}$»; (c) «el ataque de Vedenev cuesta esencialmente una ejecución»; (d) «nadie ha ejecutado el cómputo de núcleos contra un conjunto real».

5. **El debate de la memoria, aplicado.** Use la analogía de AES del capítulo 14 (tabla de equivalencias) y explique, en cinco líneas, por qué comparar $2^{94,3}$ con 2^{143} (AES-128) favorece al ataque, mientras que comparar 2^{117}–2^{128} con 2^{143} favorece al esquema. ¿Qué información falta para cerrar la comparación en un sentido o en otro?

6. **Los titulares.** Escriba su propio titular de longitud máquina (menos de 80 caracteres) para la noticia del 1 de octubre de 2026, que sea *verdadero* según el recuadro de trampa, y otro deliberadamente falso en cada dirección. Intercámbielos con un colega y discutan qué cláusula falta en cada uno.

Cierre de la parte IV. Cuatro capítulos para responder a la pregunta «¿qué significa romper McEliece?»: los objetivos y la cadena de reducciones (capítulo 13); la trinchera de la decodificación genérica y sus pliegos de costes (capítulo 14); el muro estructural y su anatomía (capítulo 15); y el terremoto de 2025–2026 con su balance aún abierto (capítulo 16). Nadie ha roto Classic McEliece —no existe un ataque práctico, y cada estimación baja de los niveles declarados viaja con supuestos explícitos— pero las cifras de esa orilla han cambiado de siglo en cuatro años, y con ellas la conversación pública. La parte final del libro vuelve al suelo: las implementaciones, el ecosistema que ya existe, y el balance de lo que este esquema significa para quien lee.

PARTE V

Cierre

CAPÍTULO 17

Preguntas frecuentes

En este capítulo

Catorce capítulos de teoría, dos laboratorios y dos años de literatura convulsa dejan, al final, las preguntas de siempre: ¿qué es esto?, ¿por qué así?, ¿está roto?, ¿lo uso? Este capítulo las responde en formato breve y con punteros exactos a las partes donde cada respuesta está desarrollada. Si solo se va a leer una sección del libro después de la introducción, que sea esta; si algo de aquí resulta sorprendente o insuficiente, los punteros llevan al capítulo correspondiente.

17.1 El esquema, en preguntas cortas

1. **¿Qué es Classic McEliece, en una frase?** Un KEM post-cuántico: el criptosistema de McEliece (1978) en la forma dual de Niederreiter, con códigos de Goppa binarios, estandarizado por ISO en 2026. Es famoso por dos extremos: las claves públicas más grandes de la criptografía moderna—de 261 120 a 1 357 824 bytes—y algunos de los criptogramas más pequeños (96–208 bytes). Su seguridad se ancla en la dureza conjetural de la decodificación por síndromes, un problema bajo escrutinio desde 1962 [10, 29].

2. **¿Es «el McEliece de 1978» o es otra cosa?** El mismo esqueleto (códigos de Goppa, trampa de Patterson), con tres piezas reingenierizadas: la forma dual, donde el mensaje es un vector de error de peso fijo y no un texto; la forma sistemática con semilla de 32 bytes, que reduce el almacenamiento y hace reproducible la generación (capítulo 9); y el transformado a KEM con rechazo implícito (capítulos 8 y 10). La trampa—decodificar es fácil con g y α y es el problema duro sin ellos—es exactamente la de [56].

3. **¿Por qué las claves públicas son gigantescas?** Porque la clave pública es, precisamente, una matriz binaria aleatoria de $mt \times k$ bits (la T de $(\mathrm{I}_{mt}|T)$). No hay anillo ni estructura que la comprima, y esa ausencia de estructura es parte del argumento de seguridad: menos álgebra visible, menos superficie para el criptoanálisis estructural (capítulo 15). El contraste está en la tabla 12.4: frente a ML-KEM-768, entre 400 y 1 200 veces las claves (según el conjunto); a cambio, criptogramas entre 5 y 8 veces más pequeños [29].

4. **¿Cifra mensajes o claves?** Es un KEM: acuerda una clave de sesión de 32 bytes, que después cifra el mensaje en un esquema simétrico (AES-GCM, ChaCha20). En un protocolo real se combina en híbrido con un intercambio clásico (X25519): si un lado cae, el otro sostiene; los despliegues conocidos usan todos el modo híbrido [23, 9].

5. **¿Qué tan rápido es?** En los términos de conteo del capítulo 10: la encapsulación es una multiplicación matriz-vector sobre la clave pública (XORs; $\sim 2^{23{,}4}$ operaciones de bit en el conjunto mayor) más los hashes; la desencapsulación añade Patterson. Lo caro es la *generación* (reintentos de MatGen: $\sim 3{,}4$ de media en la forma base, una sola en las variantes `f`), pero ocurre una vez por clave [28].

6. **¿Por qué el «mensaje» es un vector de error?** Porque el problema duro de un código es el síndrome: dada $C = He$, recuperar e es exactamente decodificar (capítulo 14). La especificación no disimula el corazón del problema: se lo pide al atacante en su forma desnuda, y restaura la uniformidad con el transformado a KEM (capítulos 8 y 10).

7. **¿Qué hay dentro de una clave secreta?** En el conjunto mayor, 14 120 bytes: los 12 800 que codifican el orden de los elementos α (red de Benes), los 1 024 del vector de rechazo s, los 256 del polinomio g y las semillas (40). El orden de α, no el código—que es público—, es el secreto (capítulo 9).

17.2 Ecosistema y estándares

8. **¿Está estandarizado?** Sí, por ISO/IEC: la Enmienda 2 a la 18033-2 (junio de 2026) incluye Classic McEliece con cuatro tamaños y cuatro variantes—16 perfiles—junto a ML-KEM y FrodoKEM [45, 24]. El NIST *no* lo estandarizó: su proceso de KEMs cerró en marzo de 2025 con HQC, y en 2025 aplazó la decisión sobre McEliece citando la estandarización ISO en curso [2, 26]. Norma ISO hay; FIPS, no.

9. **¿Dónde se usa hoy?** En despliegues ligados a la transición post-cuántica: túneles VPN (Mullvad), Rosenpass, PQConnect, y en el ecosistema de implementaciones liboqs/PQClean y SUPERCOP [9, 23]. Todos en modo híbrido.

10. **¿Por qué el NIST lo dejó fuera?** Por ingeniería, no por criptoanálisis: cientos de kilobytes de clave pública no encajan en sus casos de uso objetivo (TLS de alto volumen), y en 2025 pesó la coexistencia con ISO. Es el patrón de toda la historia del esquema: seguridad cómoda, ergonomía incómoda (capítulo 12).

11. **¿Qué fue «You Only Decapsulate Once»?** Un trabajo de 2025 que afirmaba recuperar la clave privada mediante un ataque de canal lateral de tiempo con una única traza de desencapsulación, explotando que optimizaciones de compiladores modernos (como clang) convertían código de enmascaramiento en tiempo constante en bifurcaciones condicionales. El equipo respondió el 20 de noviembre de 2025 [22] señalando que el análisis apuntaba a una versión obsoleta de PQClean, que ellos mismos habían alertado públicamente de ese problema en 2024 y que el software oficial ya utilizaba la biblioteca `cryptoint` y las pruebas automáticas de TIMECOP para prevenirlo (capítulo 16).

17.3 Seguridad: qué se sabe y qué no

12. **¿Está roto?** No. A octubre de 2026 no hay ataque práctico contra ningún conjunto, ni recuperación de clave ejecutada contra una clave real, ni rotura de los retos de la especificación. Pero el tono cambió: aparecieron distinguidores que alcanzan el régimen real (GIJS, BLRT, Hemmert–Wiemers) y estimaciones de recuperación muy por debajo de ciertas cifras declaradas—de 2^{99}–2^{102} en la contabilidad de una sola ejecución de Weis a 2^{140}–2^{148} en las estimaciones del BSI—frente a los $2^{140{,}8}$–$2^{275{,}6}$ del modelo clásico de memoria libre según el conjunto [38, 73, 17].

13. **¿Qué significa «distinguir», exactamente?** Que un algoritmo decide si una matriz pública es de Goppa o aleatoria. No recupera e ni la clave: es una propiedad más débil (figura 13.2). Durante décadas los distinguidores vivieron en regímenes que McEliece no usa;

los de 2026 alcanzan los parámetros reales, aunque con costes aún fuera de alcance y sin reducción conocida de distinguir a romper [38]. Capítulos 15 y 16.

14. **¿Y la amenaza cuántica?** No hay rotura cuántica conocida: Shor no aplica (no hay grupo abeliano oculto), y los mejores ataques cuánticos ganan a lo sumo la mitad del exponente [46]. ISO partió esos exponentes y exigió $\geq 2^{128}$: excluyó a `mceliece348864` y admitió los demás con matices [45, 24] (capítulo 12).

15. **¿Qué dice el BSI?** Su nota del 29 de septiembre de 2026 es la advertencia institucional más dura recibida: los métodos de agosto–octubre merecen tomarse en serio; «no debería seguir usándose en nuevos desarrollos»; los despliegues híbridos existentes conservan la mitad clásica; recomienda ML-KEM, FrodoKEM y HQC, y revisará su TR-02102-1 [17].

16. **¿Y qué responde el equipo?** Que ningún ataque completo está verificado; que las cifras agresivas dependen de contabilidades discutibles (operaciones de bits, memoria gratis, pasos heurísticos no ejecutados); y que el suelo trivial de recuperación de clave es 2^{256}, la búsqueda exhaustiva de la semilla de 32 bytes [20, 21, 9]. El desacuerdo es técnico, público y está abierto (capítulo 16).

17.4 Práctica: elegir, desplegar, verificar

17. **¿Qué conjunto y qué variante elegir?** La recomendación del equipo es simétrica y simple: los seis dígitos para uso a largo plazo (`mceliece6688128`, `mceliece6960119`); tamaños intermedios solo con presupuesto ajustado; `f` si el coste de generación importa ($< 2^{-30}$ de reintento, sin efecto apreciable en seguridad); `pc`/`pcf` si se quiere confirmación de conocimiento del error (+32 bytes) [29, 24]. La guía completa está en el capítulo 12.

18. **¿Cómo verifico una implementación?** Con los vectores de prueba de la especificación: la clave entera es reproducible bit a bit desde la semilla de 32 bytes, así que cualquiera regenera clave y criptograma y compara. Para C portable están PQClean y liboqs; para medidas comparables, SUPERCOP [23].

19. **¿Y si tengo que decidir hoy?** Tres reglas prudentes, coherentes con la nota del BSI [17]: (1) en entornos con guía BSI, no iniciar despliegues nuevos y preferir ML-KEM, FrodoKEM o HQC; (2) donde ya haya Classic McEliece, mantener el modo híbrido y monitorizar la literatura (la mitad clásica cubre la eventualidad de una degradación del componente post-cuántico); (3) si el diseño exige diversidad de familias, tratar los tamaños `6*` como el mínimo razonable y revisar los resultados de 2025–2026 antes de congelar parámetros [21, 29].

Observación 17.1: La respuesta honesta a «¿lo uso?»

Ningún documento de 2026—ni los ataques, ni la nota del BSI, ni las réplicas del equipo—afirma que Classic McEliece esté roto (octubre de 2026). Y ninguno autoriza a ignorar la tendencia. La cifra que mejor resume el momento es la del capítulo 16: las estimaciones agresivas bajaron de 2^{9116} a 2^{94}–2^{102} en *una parte* de la contabilidad, mientras la otra sigue en 2^{142}–2^{150}; la distancia entre ambas contabilidades es, literalmente, un problema abierto. Un usuario racional hace lo que hacen los usuarios reales: híbrido, tamaños grandes, seguimiento activo, y cero apuestas a que el desacuerdo se resuelva a su favor.

CAPÍTULO 18

Repaso y soluciones

En este capítulo

El libro termina como empezó: con lápiz y papel. La sección 18.1 propone doce preguntas de repaso que recorren el argumento completo de las partes I–IV: quien las responda de memoria puede considerar que entendió el esquema. La sección 18.2 da las respuestas concisas. Y la sección 18.3 resuelve, capítulo a capítulo, los 94 ejercicios del libro, con las cifras ya calculadas y verificadas. En los laboratorios, el programa del apéndice A reproduce todos los números.

18.1 Repaso: doce preguntas

1. Defina el problema SD en su versión de búsqueda y el juego OW-CPA del cifrado de Niederreiter, y enuncie su equivalencia (capítulo 13).

2. ¿Por qué los conjuntos de Classic McEliece pueden tomar $n < q$? Dé las dos caras: el acortamiento y la división del soporte.

3. Escriba las fórmulas de tamaño del criptograma, la clave pública y la clave secreta de un conjunto (m, t, n, k), y diga qué componente domina la clave secreta.

4. Enumere los cuatro pasos de Decode y explique por qué los dos chequeos finales (peso y síndrome) son imprescindibles para el KEM.

5. ¿Con qué probabilidad tiene éxito MatGen con $(\mu, \nu) = (0, 0)$? ¿Qué compra y qué cuesta la forma semi-sistemática $(32, 64)$?

6. Describa la tubería Encap/Decap con rechazo implícito y diga qué es exactamente s.

7. Enuncie los tres términos de la cota ROM del teorema 8.4 y evalúelos para `mceliece348864` con $q_D = q_H = 2^{64}$.

8. Distinga las tres familias de ataque que aparecen en el libro (ISD, estructurales, distinguidores) con un método y un capítulo para cada una.

9. Escriba la probabilidad de éxito de una iteración de Prange, evalúela para $(n, k, t) = (3488, 2720, 64)$ y componga el coste total. ¿De dónde sale el medio bit de diferencia con la guía?

10. Explique «$o(1)$ no es 0» y su consecuencia para comparar Prange con May–Ozerov.

11. ¿Qué cuenta exactamente el «muro» $2^{9116{,}8}$ y qué *no* es? ¿A qué distancia quedó de él la familia de 2025–2026?

12. Resuma el año 2025–2026 en cuatro cifras y una moraleja (capítulo 16).

18.2 Soluciones del repaso

1. **SD y OW-CPA.** SD (búsqueda): dadas $H \in \mathbb{F}_2^{mt \times n}$, $C \in \mathbb{F}_2^{mt}$ y t, hallar e de peso $\leq t$ con $He = C$; encontrar el criptograma que genera una clave pública. OW-CPA: dados $(T, C = \mathsf{Encode}(e, T))$ con e uniforme de peso t, hallar e. La equivalencia (lema del capítulo 13): el cifrado es una biyección lineal en e, así que invertirlo es exactamente decodificar el síndrome; sin decodificador no hay inversión y viceversa.

2. $n < q$. (a) Acortamiento: cualquier ataque contra (q, n, t) se transporta a un ataque contra (q, q, t) con el éxito multiplicado por $\binom{n}{t}/\binom{q}{t}$; la pérdida de seguridad del traslado es de fracciones de bit (unos 0,8 bits para 460896 en el modelo 1/2): bajar n no regala seguridad contra ISD (capítulo 14). (b) División del soporte: los ataques que adivinan los elementos α deben elegir n de q columnas, y pagan $\binom{q}{n}$: tomar $n < q$ los encarece (capítulo 15). El diseño cobra en el segundo frente lo que no pierde en el primero.

3. **Tamaños.** $ct = \lceil mt/8 \rceil$ bytes; $ek = mt\lceil k/8 \rceil$ (la T de $(\mathrm{I}_{mt}|T)$, una fila por byte redondeado); $sk = 32 + 8 + t\lceil m/8 \rceil + \lceil (2m-1)2^{m-4} \rceil + \lceil n/8 \rceil$ (semilla, contador, g, orden de α en red de Benes, s). En `8192128`: 14120 bytes, dominados por los 12800 de α; la pública es toda T (capítulo 9).

4. **Decode.** (1) $v = (C, \mathbf{0})$; (2) el único $c \in \Gamma_2$ con $Hc = 0$ y $d(v, c) \leq t$ (Patterson con g y el orden de α); si no existe, $\perp$; (3) $e = v + c$; (4) aceptar si y solo si $\mathrm{wt}(e) = t$ y $He = C$. Los chequeos caracterizan la imagen de $\mathsf{FixedWeight}$: sin ellos, entrarían errores de peso menor y el «mensaje» aceptado dejaría de ser uniforme en el conjunto de peso t que exige la reducción.

5. **MatGen.** Con $(0, 0)$ el éxito es $\prod_{i=0}^{mt-1}(1 - 2^{i-mt}) \approx 0{,}289$: unos 3,4 intentos de media. Con $(32, 64)$, el fallo por intento cae bajo 2^{-30} (≈ 1 intento), al precio de ≤ 2 bits de tensión en la reducción y de cómputo en tiempo constante dentro de la ventana (capítulos 9 y 12).

6. **Encap/Decap.** Encap: $e \leftarrow \mathsf{FixedWeight}$, $C = \mathsf{Encode}(e, T)$, $K = H(1, e, C)$. Decap: $e' \leftarrow \mathsf{Decode}(C)$; si $e' = \perp$ entonces $e' \leftarrow s$, $b \leftarrow 0$, si no $b \leftarrow 1$; $K = H(b, e', C)$. s es un vector uniforme de n bits guardado en la clave secreta: la clave de la rama de rechazo. El rechazo implícito nunca devuelve error: dos criptogramas (válido e inválido) producen siempre 32 bytes pseudoaleatorios (capítulo 8).

7. **Cota ROM.** $\mathrm{Adv} \leq \Pr[\text{OW-CPA}] + \dfrac{q_D}{|P|} + \dfrac{2q_H}{|S|}$. Para `348864`: $2^{64}/2^{456} = 2^{-392}$ y $2^{65}/2^{256} = 2^{-191}$: el tercer término domina (acotado por los 256 bits de entropía de la semilla δ que genera a $s \in \mathbb{F}_2^{3488}$) y aun así es despreciable; el verdadero supuesto es la dureza OW-CPA, la conjetura de SD.

8. **Tres familias.** (1) Decodificación genérica: Prange, Stern, May–Ozerov… (capítulo 14): atacan el problema SD con álgebra lineal y listas. (2) Estructurales: división del soporte, álgebra sobre el código, distinguidores con inversión (Faugère, Carrier, Kirshanova, el *holdout* de GIJS…) (capítulos 15 y 16): buscan g y el orden de α, o recuperan e estructuralmente. (3) Distinguidores (capítulo 15): deciden si la clave es de Goppa; no rompen por sí solos, pero erosionan supuestos y, desde 2026, operan en el régimen real.

9. **Prange.** Una iteración elige k columnas y espera que las t posiciones del error caigan fuera: $((n-k)/n)^t = (768/3488)^{64} = 2^{-139{,}73}$; con coste por iteración $\approx k^3 = 2^{34{,}23}$, el total es

$2^{173,96} \approx 2^{174,0}$. La guía cotiza 173,4: el medio bit restante son las constantes de la eliminación (pivotes, invertibilidad) que el estimador fino sí cuenta (capítulo 14).

10. $o(1)$ **no es** 0**.** Asintóticamente, Prange y las mejoras comparten exponente $(1/(1-R))^t$: difieren solo en el $o(1)$. En concreto, ese $o(1)$ no es pequeño: para 348864, Prange cuesta $2^{173,4}$ y May–Ozerov $2^{140,8}$ (32,6 bits, un factor $6{,}5 \times 10^9$). Las dos afirmaciones son ciertas a la vez: misma pendiente asintótica, constantes muy distintas; de ahí que las tablas se construyan con estimadores concretos (capítulo 14).

11. **El muro.** Es $\log_2$ del número de datos parciales $q(q-1)\cdots(q-769+1) \approx 2^{9116,8}$ que la familia de ataques de 2022 necesitaría inspeccionar para `348864`: un muro *de su modelo*, no de la criptografía. La familia de 2025–2026 (el *holdout* y sus derivados) no lo escala: lo evita, cambiando el objeto de la búsqueda, y sus estimaciones bajan a 2^{94}–2^{150} según la contabilidad (capítulos 15 y 16).

12. **Cuatro cifras y una moraleja.** (i) $2^{9116,8} \to 2^{94,3}$: la caída del modelo estructural de 2022; (ii) 2^{120}–2^{124}: el coste cotizado de los distinguidores de 2026 en el régimen real; (iii) 2^{99} –2^{102} frente a 2^{140}–2^{148}: las dos contabilidades que no acuerdan; (iv) 2^{256}: el suelo trivial (buscar la semilla). Moraleja: a octubre de 2026 nadie ha roto Classic McEliece —ni un ataque práctico, ni una clave real recuperada— y nadie puede ya decir «inmune»: hay conjeturas con supuestos, contabilidades en disputa y vigías institucionales (capítulo 16).

18.3 Soluciones de los ejercicios de los capítulos 1–16

Las páginas que siguen resuelven, capítulo a capítulo, los 94 ejercicios del libro. Las cifras están calculadas con la aritmética exacta de los apéndices (y de `toy.py` cuando el ejercicio es de laboratorio); los argumentos completos están en el texto del capítulo correspondiente.

Capítulo 1 — La amenaza cuántica y el giro post-cuántico (capítulo 1)

1. Grover busca la clave en $2^{k/2}$ evaluaciones: AES-256 cae a 2^{128} (dentro del listón) y AES-128 a 2^{64} (muy por debajo). La aceleración cuántica es cuadrática en el exponente, no total: basta doblar el tamaño de clave.

2. *Criptogramas*, de menor a mayor: `mceliece460896` (156 B) < ML-KEM-768 (1088 B) < HQC-128 ($\approx$ 4497 B). *Claves públicas*: ML-KEM-768 (1184 B) < HQC-128 ($\approx$ 2249 B) < `mceliece460896` (524 160 B). McEliece intercambia los extremos: el criptograma más pequeño y la clave más grande.

3. Porque el tráfico cifrado *hoy* ya se graba: cuando exista el ordenador cuántico, se descifrará retroactivamente. Los datos de vida útil larga están expuestos desde el momento en que se capturan; migrar después no repara lo ya grabado, de ahí actuar ahora (híbridos incluidos).

4. (Investigación.) La nota del BSI (1 de octubre de 2026, [17]) sostiene, en esencia: (a) los métodos de agosto–octubre (distinguidores de 2^{120}–2^{124}; recuperaciones estimadas de 2^{140}–2^{148}; el cálculo de una sola ejecución de Weis, 2^{99}–2^{102}) estrechan el margen lo bastante para tomarse en serio, aunque nada esté roto; (b) principio de precaución: no iniciar despliegues nuevos, preferir ML-KEM, FrodoKEM y HQC, y conservar la mitad clásica de los híbridos existentes, revisando la TR-02102-1. (Respuesta abierta: lo evaluable es distinguir lo demostrado de lo estimado y leer la recomendación como decisión de política.)

Capítulo 2 — Códigos lineales (capítulo 2)

1. Si $\mathbf{x}$ y $\mathbf{z}$ difieren en la posición j, entonces $\mathbf{x}_j \neq \mathbf{y}_j$ o $\mathbf{y}_j \neq \mathbf{z}_j$ (si ambas igualdades se dieran, $\mathbf{x}_j = \mathbf{z}_j$). Así $\operatorname{supp}(\mathbf{x}+\mathbf{z}) \subseteq \operatorname{supp}(\mathbf{x}+\mathbf{y}) \cup \operatorname{supp}(\mathbf{y}+\mathbf{z})$, y tomando cardinales se obtiene la desigualdad triangular.

2. Una base: las filas 1100 y 0011 sirven como G y también como $\mathbf{H}$ (las comprobaciones son $x_1+x_2=0$ y $x_3+x_4=0$). Las palabras no nulas son 1100, 0011 y 1111, de pesos 2, 2 y 4: $d=2$.

3. Para $y=0101010$: síndrome $= \mathbf{H}_2+\mathbf{H}_4+\mathbf{H}_6 = 011+101+010 = 100 = \mathbf{H}_5$: se corrige la posición 5 (0-indexada 4): $\hat{y}=0101110$, mensaje 0101. Para $y'=1100110$: síndrome $= \mathbf{H}_1+\mathbf{H}_2+\mathbf{H}_5+\mathbf{H}_6 = 110+011+100+010 = 011 = \mathbf{H}_2$: se corrige la posición 2: $\hat{y}'=1000110$, mensaje 1000.

4. Hay $\binom{n}{t}$ palabras de peso t: $\binom{7}{1}=7$ para el Hamming; $\binom{3488}{64} \approx 2^{456{,}3}$ para `348864`: unos 10^{137} candidatos.

5. Sea e'' de peso mínimo con $\operatorname{Synd}(e'')=\operatorname{Synd}(y)$ (existe: al menos e). Entonces $y+e''$ es un codeword (su síndrome es 0) y todo codeword c cumple $\operatorname{wt}(y+c) \geq \operatorname{wt}(e'')$, porque $y+c$ tiene el mismo síndrome que e'' y e'' es el mínimo. Así el codeword más cercano dista de y exactamente $\operatorname{wt}(e'')$, que es a lo sumo $\operatorname{wt}(e)$.

6. Un código sirve para «publicar $G'=SGP$» si tiene (a) decodificación eficiente hasta t (sin la descripción secreta no; con ella, Patterson) y (b) ocultación: el disfraz hace que G' parezca una matriz aleatoria. Los dos recursos de quien descifra son el computacional (Patterson) y el algebraico (la descripción g y el orden de α, que convierten G' en un código con decodificador).

Capítulo 3 — Cuerpos finitos (capítulo 3)

1. $z^5 = z\cdot z^4 = z(z+1) = z^2+z$; y $z^{-5} = z^{15-5} = z^{10} = z^2+z+1 = \mathtt{7}$.

2. $\operatorname{Tr}(1) = 1+1+1+1 = 0$; $\operatorname{Tr}(z) = z+z^2+z^4+z^8 = z+z^2+(z+1)+(z^2+1) = 0$. Tienen traza 1 exactamente la mitad: 8 elementos (el núcleo de Tr es un subespacio de dimensión $m-1=3$).

3. $\sqrt{z^3+z^2}$: como $z^3+z^2=z^6$ (pues $z^6=z^4z^2=(z+1)z^2=z^3+z^2$), la raíz es z^3. $\sqrt{z^2+z+1}$: como $z^2+z+1=z^{10}$, la raíz es $z^5=z^2+z=\mathtt{6}$. En general, $\sqrt{\beta}=\beta^{2^{m-1}}$ (para $m=4$, elevar al exponente 8).

4. $\operatorname{Tr}(\beta^2)=\beta^2+\beta^4+\beta^8+\beta^{16}=\beta^2+\beta^4+\beta^8+\beta=\operatorname{Tr}(\beta)$ (en $\mathbb{F}_{2^4}$, $\beta^{16}=\beta$). La linealidad sobre $\mathbb{F}_2$: Frobenius da $(\beta+\gamma)^{2^i}=\beta^{2^i}+\gamma^{2^i}$ para cada i, y sumar conserva la aditividad; $\operatorname{Tr}(a\beta)=a\operatorname{Tr}(\beta)$ para $a\in\mathbb{F}_2$ es inmediato.

5. x^2+x+1 no tiene raíces en $\mathbb{F}_2$ ($P(0)=P(1)=1$), luego es irreducible; $x^2+x=x(x+1)$ es reducible. El criterio de la traza (x^2+x+a irreducible $\iff \operatorname{Tr}(a)=1$) coincide: $\operatorname{Tr}(1)=1$ y $\operatorname{Tr}(0)=0$. (Sobre $\mathbb{F}_{2^4}$ ambos se parten: x^2+x+1 tiene las raíces de orden 3.)

6. Suma: un XOR por bit. Producto: red de AND+XOR de ancho $\approx m^2$. Raíz cuadrada: casi gratis, es $\mathbb{F}_2$-lineal (una matriz de XORs). Inversa: la más cara (cadena de elevar-al-cuadrado y multiplicar, $x\mapsto x^{q-2}$; o Itoh–Tsujii; o tablas), tanto por profundidad como por dependencia de datos. Patterson la usa $O(t)$ veces.

Capítulo 4 — Códigos de Goppa (capítulo 4)

1. Si $g(\alpha_i) = 0$, entonces $(x + \alpha_i)$ divide a g y $1/(x + \alpha_i)$ no existe módulo g (es un divisor de cero); la columna de síndrome $\sigma_i = (x+\alpha_i+1)/g(\alpha_i)$ no se puede calcular. Equivalente: todas las condiciones se anulan en la posición i (el término $c_i/(x + \alpha_i)$ queda libre): aparecerían palabras de peso 1 y la distancia se derrumba.

2. $A(x)/(x - \alpha_i) = \prod_{j\neq i}(x - \alpha_j)$: evaluado en α_i da $\prod_{j\neq i}(\alpha_i - \alpha_j)$. Y $A'(x)$ es la suma de los productos que omiten un factor; al evaluar en α_i, los términos que conservan el factor $(x - \alpha_i)$ se anulan, y queda exactamente el mismo producto. Es la identidad que traduce la suma de fracciones a la fórmula polinomial de la especificación.

3. Con el ordenamiento natural del juguete ($\alpha_0 = 0$, $\alpha_1 = 1$): $g(0) = g(1) = z^3$, luego $1/g(0) = z^{-3} = z^{12} = \mathtt{f}$; la columna 0 es $(\mathtt{f}, 0)$ y la 1 es $(\mathtt{f}, \mathtt{f})$. (La tabla 5.1 del laboratorio muestra el resto.)

4. $d \geq 2t + 1$ dice exactamente que la bola de radio $2t$ alrededor de 0 solo contiene al codeword nulo: ningún peso $\leq 2t$. No contradice $d > 2t + 1$ porque es una cota *inferior*: el lema garantiza al menos $2t + 1$; la distancia real puede ser mayor (en el juguete es $5 = 2t + 1$, pero la cota no prohíbe 6 o más en otros códigos).

5. Con $g = h^2$: la hipótesis del teorema 4.4 (libre de cuadrados) falla, pero la respuesta de fondo es la identidad de van Lint $\Gamma_2(L, g) = \Gamma_2(L, g^2)$: tomando $g = h$ queda $\Gamma_2(L, h^2) = \Gamma_2(L, h)$. El código con $g = h^2$ es *el mismo* que con h: el «t efectivo» sigue siendo $\deg h$, no se gana distancia ni dimensión. McEliece nunca lo intentó porque duplicar el grado nominal no cambia el código: promesa vacía.

6. Semillas → g (con Irreducible) y orden de α (con FieldOrdering) → el código $\Gamma_2(L, g) = \{c : \sum_j c_j/(x + \alpha_j) \equiv 0 \pmod g\}$ → la tabla de síndromes $\tilde{\mathbf{H}}$ ($2 \times n$ sobre $\mathbb{F}_{2^4}$, columnas $\alpha_i^j/g(\alpha_i)$) → la matriz binaria $\hat{\mathbf{H}}$ (8×16) → eliminación gaussiana → $(\mathrm{I}_8 \,|\, T)$. Se publica solo T (junto con t); se mantienen secretos g, el orden α (y las semillas, y s en el KEM). El código $\ker(\mathrm{I}_{mt}|T)$ es público.

Capítulo 5 — Laboratorio 1 (capítulo 5)

1. $g(z) = z^2 + z + z^3 = z^3 + z^2 + z = \mathtt{e}$; $1/\mathtt{e} = z^{-11} = z^4 = z + 1 = \mathtt{3}$; $z \cdot \mathtt{3} = z^2 + z = \mathtt{6}$. La columna $j = 2$ de la tabla 5.1 es exactamente $(\mathtt{3}, \mathtt{6})$: coinciden.

2. $\tilde{\mathbf{H}}e_5 = \tilde{\mathbf{H}}_5 = (\mathtt{8}, \mathtt{e})$ (pues $g(z^2 + 1) = \mathtt{f}$, $1/\mathtt{f} = z^3 = \mathtt{8}$ y $(z^2 + 1)z^3 = \mathtt{e}$); $\tilde{\mathbf{H}}e_9 = (\mathtt{a}, \mathtt{5})$ (pues $g(z^3 + 1) = z^6$, $1/z^6 = z^9 = \mathtt{a}$ y $(z^3 + 1)z^9 = \mathtt{5}$). La suma es $(\mathtt{8} + \mathtt{a}, \mathtt{e} + \mathtt{5}) = (\mathtt{2}, \mathtt{b})$. $e_5 + e_9$ se decodifica de forma única porque dos palabras de peso ≤ 2 con el mismo síndrome diferirían en un codeword de peso $\leq 4 < d = 5$: imposible.

3. Multiplicando y usando $z^4 = z + 1$ en cada coeficiente: constante $4 \cdot \mathtt{a} = z^2(z^3 + z) = z^3 + z^2 + z = \mathtt{e}$; lineal $4 \cdot 7 + 9 \cdot \mathtt{a} = \mathtt{f} + \mathtt{5} = \mathtt{a}$; cuadrático $9 \cdot 7 = z^3 + z = \mathtt{a}$. En el lado derecho, $1 + (z + z^3)g(x)$ tiene esa misma forma: $z + z^3 = \mathtt{a}$, $1 + \mathtt{a}z^3 = \mathtt{e}$. Es el certificado de Patterson.

4. Con $E = \{j\}$: $\sigma(x) = \prod_{i\in E}(x + \alpha_i) = x + \alpha_j$ (grado $1 \leq t$), y $\sigma' = 1$: la ecuación clave se satisface con $\sigma = x + \alpha_j$. Patterson devuelve el localizador; su raíz α_j señala la posición del error.

5. $(\mathbf{x}P^{-1})_1 = \mathbf{x}_{\sigma^{-1}(1)} = \mathbf{x}_{15} = 1$ y $(\mathbf{x}P^{-1})_3 = \mathbf{x}_{\sigma^{-1}(3)} = \mathbf{x}_{13} = 0$: la misma permutación envía los dos errores de $\mathbf{z}$ a las posiciones 0 y 10 ($\mathrm{supp}(\mathbf{z}P^{-1}) = \{\sigma(3), \sigma(12)\} = \{10, 0\}$). Es exactamente lo que el laboratorio decodifica con Patterson.

6. Dos ataques: (a) enumerar las 137 palabras de peso ≤ 2 y quedarse con la que comparte síndrome con C (120 de peso 2, 16 de peso 1, la nula); (b) tabla completa síndrome→palabra mínima (256 entradas, 137 no triviales): un oráculo de descifrado instantáneo. Con $n \approx 8000$ el espacio de síndromes pasa a $\geq 2^{64}$ y $\binom{n}{t}$ es astronómico: mueren la tabla y la enumeración, y quedan los algoritmos de la parte IV (ISD, métodos estructurales).

Capítulo 6 — El McEliece de 1978 (capítulo 6)

1. La cadena es $\mathbf{x}P^{-1} = (\mathbf{u}S)G + \mathbf{z}P^{-1}$: $\mathrm{wt}(\mathbf{z}P^{-1}) = \mathrm{wt}(\mathbf{z})$ (permutar no cambia el peso), y $\mathrm{wt}(\mathbf{z}) \leq t$ es lo que permite aplicar el corolario de corrección al codeword $(\mathbf{u}S)G$. El paso fino: para otro codeword c', $\mathrm{d}(\mathbf{x}P^{-1}, c') \geq d - \mathrm{wt}(\mathbf{z}) \geq 2t + 1 - t = t + 1 > t$: el codeword verdadero está *estrictamente* más cerca. Con errores de peso $t + 1$ «a veces», esa desigualdad se rompe (podría haber dos codewords a distancia $t + 1$): Patterson podría devolver otro codeword y el descifrado fallaría o mentiría.

2. G' para $(1024, 524)$: $524 \cdot 1024 = 536\,576$ bits ($\approx 65{,}5$ KiB). Para `348864`: $2720 \cdot 3488 = 9\,487\,360$ bits ($\approx 1{,}13$ MB). La parte sistemática $(\mathrm{I}_{2720}|T)$ ocupa $mt\lceil k/8 \rceil = 768 \cdot 340 = 261\,120$ B (la clave pública): el ahorro frente a G' completa es $1185920/261120 = 4{,}54$ veces.

3. Si $(\mathbf{u}', \mathbf{z}') \neq (\mathbf{u}, \mathbf{z})$ diera el mismo $\mathbf{x}$, entonces $(\mathbf{u} + \mathbf{u}')G' = \mathbf{z} + \mathbf{z}'$: un codeword de peso $\leq 2t < 2t + 1 \leq d$, imposible salvo que sea 0; y si es 0, $\mathbf{u} = \mathbf{u}'$ y $\mathbf{z} = \mathbf{z}'$.

4. $\binom{974}{524}/\binom{1024}{524} = 2^{-53{,}61}$: es la probabilidad de que un conjunto de k coordenadas elegido al azar evite las $t = 50$ posiciones del error; con $2^{53{,}61}$ intentos de coste $k^3 \approx 2^{27{,}1}$ sale $2^{80{,}7} \approx 2 \cdot 10^{24}$ (el «ataque de las k coordenadas», cotizado en 10^{19} en el artículo: el recálculo favorece al defensor). $\sum_{i \leq 50} \binom{1024}{i} = 2^{284{,}11}$, así que un síndrome al azar se decodifica con probabilidad $2^{284{,}11 - 500} = 2^{-215{,}89}$ (el artículo decía $2^{-215{,}59}$: coincide en lo esencial).

5. Porque la NP-dureza es un enunciado *del peor caso*, *asintótico* y sobre *instancias arbitrarias*: (a) no dice nada de las instancias aleatorias de una familia estructurada (los códigos de Goppa); (b) para $n = 3488$ fijo no acota nada concreto; (c) y la seguridad del esquema es una conjetura de dureza *en media* sobre su distribución (capítulo 13).

6. Con el oráculo que avisa: se envía $\mathbf{x} \oplus \mathbf{e}_j$ para cada j. Si j está en el soporte del error, el peso baja a $t - 1$ y el descifrado «funciona»; si no, sube a $t + 1$ y «falla». Bastan $n = 1024$ consultas para aprender el soporte de $\mathbf{z}$ y, con él, $\mathbf{u}$. En el KEM real cada consulta devuelve una clave pseudoaleatoria (rechazo implícito): no hay aviso que aprender (capítulo 10).

Capítulo 7 — La reformulación de Niederreiter (capítulo 7)

1. Con $H_0 = (\mathrm{I}_{n-k} \,|\, T_0)$, la palabra $v = (\gamma, \mathbf{0})$ tiene $vH_0^\top = \mathrm{I} \cdot \gamma + 0 = \gamma$. El paso (2) del descifrado pide el único $\mathbf{f}$ de peso $\leq t$ con $\mathbf{f}H_0^\top = \gamma$: exactamente el problema de decodificación del capítulo 4 (encontrar el vector de menor peso con un síndrome dado), y Patterson lo resuelve operando sobre el polinomio síndrome del vector rellenado.

2. Si $\mathbf{f}H_0^\top = \mathbf{f}'H_0^\top$, entonces $(\mathbf{f} + \mathbf{f}')H_0^\top = 0$: $\mathbf{f} + \mathbf{f}'$ es un codeword de Γ de peso $\leq 2t < d$, luego $\mathbf{f} + \mathbf{f}' = 0$ y $\mathbf{f} = \mathbf{f}'$.

3. La demostración: primero $G'(H')^\top = S_1 G P P^\top H_0^\top S_2^\top$: aquí se usa $PP^\top = \mathrm{I}_n$; después $S_1(GH_0^\top)S_2^\top = 0$: aquí, la ortogonalidad $GH_0^\top = 0$ (los codewords tienen síndrome nulo). Con permutaciones distintas quedaría $P_1 P_2^\top \neq \mathrm{I}_n$ en medio y la ortogonalidad no se seguiría: sin ella no hay reducción (el síndrome de $\mathbf{x}$ ya no es el de $\mathbf{z}$).

4. Factores $k/(n-k)$: juguete $8/8 = 1$; `348864`, $2720/768 = 3{,}54$; `460896`, $3360/1248 = 2{,}69$; `6688128`, $5024/1664 = 3{,}02$; `6960119`, $5413/1547 = 3{,}50$; `8192128`, $6528/1664 = 3{,}92$. El

original $(1024, 524)$: $k/(n-k) = 524/500 = 1{,}05$: con tasa $\approx 1/2$, la matriz dual era casi tan grande como la generadora; el ahorro, mísero (del orden del 5 %).

5. Tamaño del espacio de mensajes: el descifrado debe funcionar sobre la imagen de FixedWeight; admitir pesos menores agrandaría la imagen con preimágenes «fáciles» (pesos pequeños, búsqueda más ligera) sin ganar entropía. Uniformidad: los teoremas de las transformaciones a KEM (capítulo 8) y la proposición 10.1 exigen $\mathbf{e}$ uniforme en *el* conjunto de vectores de peso t exacto; mezclar pesos rompería el modelo y la simetría del reto.

6. Los dos problemas de inversión: $\mathbf{x} = \mathbf{u}G' + \mathbf{z} \mapsto \mathbf{u}$ (McEliece) y $\mathbf{c} = \mathbf{e}(H')^\top \mapsto \mathbf{e}$ (Niederreiter). Son «el mismo problema» por la proposición 7.4: el síndrome público de $\mathbf{x}$ es el de $\mathbf{z}$ (ortogonalidad $G'(H')^\top = 0$), y recuperar $\mathbf{e}$ equivale a recuperar $\mathbf{z}$; en ambos casos el fondo es el problema de decodificación por síndromes: dada la imagen de un error de peso $\leq t$, encontrarlo.

Capítulo 8 — Del cifrado a un KEM (capítulo 8)

1. En la transformación, Encap genera e, publica $C = \mathsf{Enc}(e)$ (aquí $C = (\mathrm{I}_{mt}|T)e$) y deriva $K = H(1, e, C)$; Decap calcula $e' = \mathsf{Decode}(C)$ y $K' = H(1, e', C)$. Las dos partes coinciden si y solo si $\mathsf{Decode}(\mathsf{Enc}(e)) = e$ con probabilidad 1: el único punto donde entra la corrección de la PKE (el $(\varepsilon_{\mathrm{corr}})$ de los teoremas).

2. Con $K = H(e)$, la consulta al oráculo con la segunda codificación $\tilde{C}$ de C^* (distinta como bytes, idéntica como elemento, por el relleno del capítulo 12) devuelve $H(e^*) = K^*$: el reto cae con una consulta que no es C^*. Con $K = H(1, e, C)$, el criptograma entra en el hash: la respuesta del oráculo es $H(1, e^*, \tilde{C})$, una clave distinta (ligada a la codificación exacta) e inútil para el ataque; la etiqueta 1 además separa el dominio de la rama de rechazo $H(0, s, C)$.

3. Devolver $\perp$ obliga a los protocolos a reaccionar (alertas, reintentos) y esa reacción es observable: es un oráculo de validez. Devolver siempre una clave pseudoaleatoria elimina la señal: el atacante no distingue «no decodificable» de «decodificable con valor improbable». La clave s aparece como secreto de la rama de rechazo: en el teorema 8.4 en el término $2q_H/|S|$ (hay que adivinar s para simular) y en el teorema 8.5 en los términos de PRF sobre s.

4. Segundo término: $q_D/|P| = 2^{64}/2^{456} = 2^{-392}$. Tercero: $2q_H/|S| \leq 2 \cdot 2^{64}/2^{256} = 2^{-191}$ (acotando la clave $s \in \mathbb{F}_2^{3488}$ por la entropía de los 256 bits de la semilla δ). Para que el tercero dejara de ser despreciable haría falta $q_H \approx 2^{255}$: un número de consultas imposible (y entonces fallarían antes los demás términos del modelo).

5. (a) Con ventaja $< 1/1000$: $2\sqrt{\varepsilon_{\mathrm{OW}}} < 1/1000$ exige $\varepsilon_{\mathrm{OW}} < (1/2000)^2 \approx 2^{-21{,}9}$. (b) Con ventaja $< 2^{-128}$: $\varepsilon_{\mathrm{OW}} < 2^{-258}$. Es decir: el teorema convierte el objetivo CCA de 128 bits en un requisito OW-CPA de $\approx 2^{-258}$: los parámetros deben tener margen doble, no nulo (es la raíz cuadrada del QROM, que el modelo de costes realista suaviza pero no elimina).

6. La tabla 8.2 lo dice: (1) sin reencriptar: la verificación $\mathsf{Enc}(\mathrm{pk}, e') = C$ no hace falta porque Decode ya exige $\mathrm{wt}(e') = t$ y $(\mathrm{I}_{mt}|T)e' = C$; (2) sin pk' dentro de la clave secreta (no hay reencriptación que lo use); (3) s de n bits en vez de 256; (4) $\mathbf{e}$ de FixedWeight en vez de uniforme-genérico. La verificación de Decode basta porque, para un criptograma, «ser imagen de un e' de peso t» y «cumplir $\mathrm{wt}(e') = t \ \wedge \ (\mathrm{I}_{mt}|T)e' = C$» son la misma condición: la primera se comprueba sin reencriptar.

Capítulo 9 — La generación de claves (capítulo 9)

1. Todo el proceso, incluidos los reintentos, es una función determinista de la semilla δ: al fallar FieldOrdering, se toma $\delta' \leftarrow$ (últimos 256 bits de E) y se repite sin pedir azar nuevo. Las pruebas de vectores fijan δ y el resultado esperado: cada implementación consume el mismo flujo E y reproduce los mismos reintentos, comparables byte a byte.

2. Con $(7,1,9,3)$: al ordenar los pares (a_i, i) queda $1 < 3 < 7 < 9$, es decir $\pi = (1,3,0,2)$; leyendo los bits de $\pi(i)$ como $\alpha_i = \sum_{j=0}^{m-1} \pi(i)_j z^{m-1-j}$ resulta $(\alpha_0, \ldots, \alpha_3) = (z,\ z+1,\ 0,\ 1)$. Con $(0,1,2,3)$: π es la identidad y $(\alpha_0, \ldots, \alpha_3) = (0,\ z,\ 1,\ z+1)$. Con $(0,0,0,0)$ los enteros no son distintos: FieldOrdering devuelve $\perp$ y MatGen reintenta con $\delta \leftarrow \delta'$ (determinista).

3. β recorre uniformemente $\mathbb{F}_{q^t}$ (los bloques de $\sigma_1 = 16$ bits por coeficiente; solo se usan los primeros m de cada bloque). El polinomio mínimo de β sobre $\mathbb{F}_q$ es mónico, irreducible, y de grado t exactamente cuando β no cae en un subcuerpo propio $\mathbb{F}_{q^s}$, $s \mid t$, $s < t$ (si cae, el grado es s y Irreducible devuelve $\perp$). Cada mónico irreducible de grado t divide a $x^{q^t} - x$ y tiene exactamente t raíces, todas simples, en $\mathbb{F}_{q^t}$; sus t raíces son exactamente las β cuyo mínimo es él. Así cada g admite t de las q^t elecciones: β uniforme $\Rightarrow$ g uniforme. La masa rechazada —elementos que caen en subcuerpos propios— está dominada por $\mathbb{F}_{q^{t/2}}$: $\approx q^{t/2}/q^t = q^{-t/2}$, despreciable para $t \geq 16$.

4. La probabilidad de que las primeras mt columnas de una matriz binaria aleatoria $mt \times n$ sean independientes es $\prod_{i=0}^{mt-1}(1 - 2^{i-mt}) = \prod_{j=1}^{mt}(1 - 2^{-j}) \approx 0{,}289$: 3,46 intentos de media, y $P(\geq 3) \approx (1 - 0{,}289)^2 \approx 0{,}5055$. Con $(\mu, \nu) = (32, 64)$ el fallo por intento cae bajo 2^{-30}: un solo intento en la práctica, a cambio de ≤ 2 bits de tensión en la reducción y una ventana de intercambios de tamaño fijo (tiempo constante).

5. $pk = mt\lceil k/8 \rceil = 1664 \cdot 816 = 1\,357\,824$ B; $sk = 32 + 8 + t\lceil m/8 \rceil + \lceil (2m-1)2^{m-4} \rceil + \lceil n/8 \rceil = 40 + 256 + 12\,800 + 1024 = 14\,120$ B. En la clave secreta domina α (los 12 800 bytes de la red de Benes, el 91 %); en la pública, la matriz T entera.

6. El código $\ker(\mathrm{I}_{mt}|T)$ es público: dimensión $k = 2720$ y 2^{2720} codewords en 348864 (cualquiera con la clave pública comprueba membresías). Quien descubra g y el orden α obtiene la descripción de Goppa del código público: un decodificador eficiente, es decir, una clave equivalente: descifrar todo (observación 9.3 y proposición 7.4: atacar el esquema es decodificar el código público; el secreto es su descripción).

Capítulo 10 — Cifrado y descifrado (capítulo 10)

1. $v = (C, \mathbf{0})$; como $C = \hat{\mathbf{H}}e$, para $c = v + e$ vale $\hat{\mathbf{H}}c = \hat{\mathbf{H}}v + \hat{\mathbf{H}}e = C + C = 0$: es un codeword, y $d(c, v) = \mathrm{wt}(e) = 2 = t$. Con $C' = \hat{\mathbf{H}}e'$ y $\mathrm{wt}(e') = 1$: el paso 2 de Decode encuentra el único codeword c' a distancia ≤ 2 (que es $v' + e'$); el paso 3 devuelve $e'' = e'$ (peso 1); el paso 4 *rechaza* ($\mathrm{wt}(e') \neq t$) y Decap toma la rama de rechazo.

2. (a) Los trozos de 16 bits bastan para nombrar cualquier posición porque todo índice es $< n \leq q = 2^m \leq 2^{16}$ (en los conjuntos reales $m \leq 13$; el juguete usa $m = 4$); (b) cada elección usa un trozo fresco: las elecciones son independientes; (c) descartar los trozos fuera de rango y repetir no sesga: condicionado a ser aceptado, cada residuo es equiprobable. Productos: juguete: $1 - 1/16 = 0{,}9375$; 348864: $e^{-t(t-1)/(2n)} = e^{-64 \cdot 63/6976} \approx 0{,}561$.

3. Si $c_1 \neq c_2$ estuvieran ambos a distancia $\leq t$ de v, entonces $\mathrm{wt}(c_1 + c_2) \leq 2t < 2t + 1 \leq d$: la diferencia sería un codeword no nulo de peso bajo la distancia mínima. Imposible: el candidato es único.

4. Con $C' = \hat{\mathbf{H}}e'$, $\mathrm{wt}(e') = 1$: el paso 2 devuelve e' (la corrección funciona: $1 \le t$); el paso 4 lo rechaza por el chequeo de peso. Sin ese chequeo, Decap aceptaría todo síndrome con preimagen de peso $\le t$, y los «mensajes» aceptados serían uniformes en $\{\text{peso} \le t\}$ en vez de en $\{\text{peso} = t\}$: se romperían la distribución y la rigidez que las pruebas del capítulo 8 usan (la aceptación debe caracterizar exactamente los criptogramas de FixedWeight).

5. Encode: $mt \cdot k = 1664 \cdot 6528 = 10\,862\,592$ bits $\approx 2^{23{,}4}$; síndrome de Decode: $tn = 128 \cdot 8192 = 1\,048\,576 \approx 2^{20}$ operaciones de cuerpo. Encode toca unos 10 veces más bits, pero son XORs sobre datos públicos (vectorizables, sin secretos); las de Decode son multiplicaciones en $\mathbb{F}_{2^{13}}$, más Patterson (t^2 operaciones) y la verificación: en coste efectivo domina Decode. Y esa asimetría es justo la que quiere un KEM: el lado sin clave (cifrar) por el camino barato y paralelo, y el lado con la clave (descifrar) pagando el álgebra fuerte.

6. Para `6960119`: $H(b, v, C)$ recibe $1 + \lceil n/8 \rceil + \lceil mt/8 \rceil = 1 + 870 + 194 = 1065$ bytes. Piezas: la etiqueta (0/1/2) separa dominios (rechazo, éxito, confirmación `pc`); $v = e$ fija el error; C ata la clave a la codificación exacta del criptograma (el escenario del capítulo 8). No se usa la lista de posiciones del soporte porque el vector ya es una codificación de longitud fija y canónica ($\lceil n/8 \rceil$ bytes): una lista exigiría convenios de orden, tendría longitud variable y no aportaría nada.

Capítulo 11 — Laboratorio 2 (capítulo 11)

1. Ejecutando el programa del apéndice A: SHAKE256(`01` $\|$ `00 24` $\|$ `98`) da exactamente la K de la figura 11.1 (`597f...6278`), y $s =$ `CBAE` se serializa como los bytes `AE CB` (little-endian por grupos de 8).

2. $e = 9216 = 2^{10} + 2^{13}$: las posiciones 10 y 13; cada byte empaqueta 8 posiciones con la posición j en el bit j mód 8 del byte $j/8$: el primer byte (posiciones 0–7) es `00` y el segundo (posiciones 8–15) es $2^2 + 2^5 =$ `24`: los bytes `00 24`. La convención «posición 0 a la izquierda» describe cómo se *imprimen* los vectores; la serialización empaqueta en little-endian. Igual para s: `CBAE` escribe `AE CB`.

3. Si $c = v + e$ es un codeword y $S_c = \sum_j c_j \sigma_j \equiv 0 \pmod g$, entonces $S_v = \sum_j v_j \sigma_j \equiv S_c + S_e \equiv S_e \pmod g$ (en $\mathbb{F}_2$, $v_j + e_j = c_j$). Para $\tilde{C}$ el vector $\tilde{v} + e$ ya *no* es un codeword: su síndrome es la columna del bit volteado ($\neq 0$), la cancelación falla y el síndrome cambia (en el juguete, $S = \mathtt{4} + \mathtt{9}x$ pasa a $\mathtt{b} + \mathtt{6}x$).

4. Si Decap dijera «error», el atacante voltearía bits en C y aprendería de la respuesta si $\tilde{C}$ decodifica: un oráculo de validez sobre los 256 bytes (los 119 inalcanzables marcan los que siempre fallan; cada consulta revelaría clases). En el KEM real no hay señal: la rama de rechazo devuelve $H(0, s, \tilde{C})$, indistinguible de una K válida; los 119 síndromes inalcanzables se tratan como cualquier otro fracaso.

5. $1 + 16 + 120 = 137$ síndromes alcanzables con peso ≤ 2; los de peso 3 ya no están garantizados como únicos porque la diferencia de dos palabras de peso 3 puede pesar hasta $6 > d = 5$ (solo los pesos con sumas $< d$ son únicos). $137/256 \approx 53{,}5\,\%$ es la fracción de bytes alcanzables por algún error de peso ≤ 2; un byte manipulado al azar falla tras el chequeo de peso con probabilidad $(256 - 120)/256 = 136/256 \approx 53{,}1\,\%$: casi la misma cifra (los 17 síndromes de peso 0/1, rechazados por el chequeo, reparten la diferencia).

6. Clave pública: $261\,120/8 = 32\,640$ veces la del juguete; criptograma: $96/1 = 96$ veces. Mecanismos omitidos: $\sigma_1 = 16$ (bits sobrantes por coeficiente en Irreducible), $\sigma_2 = 32$ (bits por elemento en FieldOrdering), la forma semi-sistemática (μ, ν) y la red de Benes (compresión del orden α): los cuatro, en el capítulo 9 (y el tercero también en el 12).

Capítulo 12 — Los conjuntos y las variantes (capítulo 12)

1. Con $ct = \lceil mt/8 \rceil$ y $ek = mt\lceil k/8 \rceil$: `348864`: 96 y $768 \cdot 340 = 261\,120$; `460896`: 156 y $1248 \cdot 420 = 524\,160$; `6688128`: 208 y $1664 \cdot 628 = 1\,044\,992$; `6960119`: 194 y $1547 \cdot 677 = 1\,047\,319$; `8192128`: 208 y $1664 \cdot 816 = 1\,357\,824$. En todos menos `6960119`, k es múltiplo de 8 y $mt\lceil k/8 \rceil = \lceil kmt/8 \rceil$. En `6960119`, $k = 5413$: $\lceil kmt/8 \rceil = \lceil 8\,373\,911/8 \rceil = 1\,046\,739$, frente a 1 047 319: la fila se redondea a byte y se pierden 580 B (el relleno de la última fila, 3 bits por fila).

2. Con $H(2, e)$: $1 + \lceil n/8 \rceil$ bytes; $H(1, e, C)$ y $H(0, s, C)$: $1 + \lceil n/8 \rceil + \lceil mt/8 \rceil + 32$ en las variantes `pc` (el 32 es la confirmación). Para `460896pc`: 577 y 765 B; la serie: 437/577/837/871/1025 (para $H(2, e)$) y, en `pc`, 565/765/1077/1097/1265 (para los otros dos).

3. Mitades de Stern: 75,7, 96,7, 134,0, 134,2, 151,6. Superan 2^{128} los tres conjuntos `6*`: exactamente el borrador ISO de 2023. ISO estandarizó *además* `460896`, sabiendo que no alcanza el listón ingenuo, porque la raíz cuadrada es optimista para el atacante (Grover limitada por latencia): el borrador lo dejó fuera y la versión final lo incluyó con esa salvedad (la historia del capítulo 12).

4. $p \approx 0{,}289$: 3,46 intentos de media y $P(> 2) = (1-p)^2 \approx 0{,}5055$. Con $(\mu, \nu) = (32, 64)$ el fallo es $< 2^{-30}$: ≈ 1 intento. La variante `f` se ahorra $\approx 3{,}5$ generaciones por clave (y elimina la cola y la varianza).

5. Dos cadenas de 194 bytes que difieran solo en los 5 bits de relleno representan el mismo elemento de $\mathbb{F}_2^{1547}$. En modo Simply el relleno se ignora: ambas producen la *misma* K (un solo valor distinto). En modo Narrowly, la cadena con relleno no nulo se rechaza: una produce la K legítima y la otra la clave de la rama de rechazo. Debe fijarlo el protocolo porque una implementación Simply y una Narrowly, mezcladas, discreparían en qué clave corresponde a los mismos bytes: es la maleabilidad benigna de la figura 12.3.

6. Mensajería: `mceliece6960119` (el criptograma más pequeño entre los `6*`: 194 B; `pc` si se quieren las defensas de fallo, +32 B): su clave pública de ≈ 1 MB se cachea una vez y se amortiza entre millones de mensajes. IoT: `mceliece460896f` (o `6960119f` si se exige largo plazo): la variante `f` porque la generación con reintentos es el punto débil de un dispositivo lento, y el tamaño de la clave pública allí importa menos que poder generarla de una vez.

Capítulo 13 — ¿Qué significa romper McEliece? (capítulo 13)

1. (a) Con una clave equivalente (g y el orden α) se decodifica el código público: todo criptograma se abre, y con él todo mensaje. (b) Un rompemensajes «de juego» triunfa sobre la *distribución* del reto (por ejemplo con conocimiento del reparto uniforme de errores) y no tiene por qué resolver el problema de decodificación sobre síndromes arbitrarios: no hay reducción conocida de juego a decodificador general, y hay instancias estructuradas débiles. (c) Un distinguidor solo decide «¿Goppa o aleatoria?»: no descifra; su papel es erosionar el supuesto de que la clave pública es pseudoaleatoria (el modelo que usan las reducciones) y, a veces, ser la primera etapa de la recuperación estructural; la reducción distinguidor→rotura es justamente la que no se conoce.

2. Las 137 palabras de peso ≤ 2 y el síndrome $(\mathtt{2}, \mathtt{b})$: el único candidato es $e_5 + e_9$, porque $\tilde{\mathbf{H}}e_5 + \tilde{\mathbf{H}}e_9 = (\mathtt{8}, \mathtt{e}) + (\mathtt{a}, \mathtt{5}) = (\mathtt{2}, \mathtt{b})$ (la unicidad estaba garantizada por $d = 5$: dos candidatos diferirían en un codeword de peso $\leq 4 < d$). Enumerar las palabras de peso 64 de `348864` cuesta $\approx 2^{456}$; Prange se cotiza en $2^{173{,}4}$: la diferencia (≈ 283 bits) vive en el algoritmo: la estructura lineal convierte el «gran catálogo» en una probabilidad por iteración con iteraciones baratas (el acortamiento del capítulo 14).

3. La conjetura, con cuantificadores: *para todo* algoritmo A de tiempo subexponencial, *para* la instancia $(H,\, C = He)$ generada por la distribución del esquema (e uniforme de peso t), *con éxito* $= \Pr[A(H,C) = e]$ despreciable (con los mejores ataques conocidos $\geq 2^{140}$–2^{150} en los modelos serios). No equivalen: (a) «NP-duro» habla del peor caso, asintótico y de instancias arbitrarias: ni media, ni distribución, ni parámetros; (b) «no existe algoritmo polinómico» ni fija la distribución ni cuantifica el éxito (y no implica la dureza concreta de las instancias del esquema); (c) «el mejor ataque cuesta $2^{140,8}$» es una cota *superior* empírica: evidencia, no teorema, y puede mejorar (2025–2026 lo mostró). En la práctica, el que sostiene las decisiones es (c): las categorías y los márgenes se calculan con el mejor ataque conocido; la conjetura es el marco de la promesa.

4. $2^{140,8}$ son operaciones *vectoriales* en el modelo de memoria libre, donde una operación de vector grande cuenta como una: no es directamente comparable con las 2^{143} operaciones de bits de buscar una clave AES-128; con el cargo realista de la memoria ($2^{86,6}$), el mismo ataque sube a $2^{158,6}$: factor $2^{158,6-140,8} = 2^{17,8} \approx 228\,210$. La categoría 1 se certifica con el modelo que carga la memoria ($158{,}6 \geq 143$): de ahí que no haya contradicción.

5. Mitad en el exponente: $151{,}4/2 = 75{,}7$, muy por debajo del listón 2^{128}: `348864` no pasa el test cuántico ingenuo. ISO estandarizó igualmente `460896` (96,7, también por debajo) porque la raíz cuadrada sobreestima al atacante: Grover está limitada por la latencia del bucle interno, y el coste cuántico real es mayor; con esa salvedad documentada, ISO prefirió incluir el conjunto de claves pequeñas antes que dejar el perfil sin cobertura.

6. (a) Distinguidor (familia estructural, capítulos 13 y 15): rompe supuestos de indistinguibilidad; no descifra. (b) Mejora de ISD (capítulo 14): mueve la frontera de la decodificación genérica; sola no rompe el esquema: cambia la estimación. (c) Canal lateral (capítulos 8–12 y 16): rompe un despliegue real (recupera la clave de una instalación concreta midiendo tiempos), no el esquema abstracto.

Capítulo 14 — Decodificación por conjuntos de información (capítulo 14)

1. (a) $64 \log_2(768/3488) = -139{,}73$; (b) $k^3 = 2720^3$: $3 \log_2 2720 = 34{,}23$; total 173,96; (c) la guía cotiza 173,4: el medio bit son las constantes finas del estimador (trabajo de la eliminación, pivotaje, normalización del éxito) que nuestro producto grueso sobrecuenta. Diferencia de constantes, no de modelo.

2. Primer salto: 22,0 bits $= 2^{22} \approx 4{,}19$ millones ≈ 4 factores de 2^{20}; segundo salto: 10,6 bits $\approx 1\,552 = 1{,}5 \cdot 2^{10}$. El primero es el grande: el cruce de listas de Stern aporta $\approx 4 \cdot 2^{20}$ sobre Prange; la búsqueda de vecinos de May–Ozerov añade «solo» $1{,}5 \cdot 2^{10}$.

3. Porque ambos son $\left(1/(1-R)\right)^t$ por $2^{o(t)}$: con $R \approx 0{,}78$, la base es $1/(1-R) \approx 4{,}55$ ($2^{2{,}18t}$; para $t = 64$, $2^{139{,}8}$). Prange arrastra un coste polinómico ($k^3 = 2^{34{,}2}$): de 173,96; May–Ozerov mete el trabajo en las listas y los vecinos y se queda cerca del término desnudo (140,8). El «5^t» es taquigrafía del término dominante ($5 \approx 1/(1-R)$ redondeado): mismo exponente, $o(1)$ distinto, y a $t = 64$ ese $o(1)$ vale 33 bits.

4. (a) Memoria gratis vs realista para `6960119`: $279{,}2 - 245{,}7 = 33{,}5$ bits; (b) $245{,}7 < 272$ (el ataque con memoria gratis sale «más barato» que agotar AES-256), pero $279{,}2 > 272$ (con el cargo de la memoria ya no): la categoría del equipo es la 5 porque certifica con el presupuesto realista.

5. (8192,96) cubre a (4608,96) por acortamiento: cualquier ataque contra el acortado se transporta al original (q, q, t) —el sistema de McEliece, con su historia de estudio— con el éxito

multiplicado por $\binom{4608}{96}/\binom{8192}{96}$; la pérdida de seguridad del traslado es de solo $\approx 0{,}8$ bits en el modelo 1/2. Por eso basta analizar el original: bajar n no regala seguridad contra ISD.

6. La mitad del exponente es una cota optimista porque supone que cada iteración de Grover cuesta lo mismo que su versión clásica y que el paralelismo no tiene penalización: en la práctica el bucle interno (accesos a memoria, la búsqueda en listas) domina y la ventaja cuántica real es menor que la cuadrática. Los 256 bits de δ fijan el suelo complementario: la búsqueda cuántica de la semilla cuesta 2^{128} (Grover sobre 256 bits, con una generación por comprobación), de modo que la seguridad cuántica de un conjunto es mín(mitad-ISD, 2^{128}): por eso el listón de ISO es exactamente 2^{128} y por eso los conjuntos por debajo (75,7, 96,7) son el punto débil, aunque el modelo de la mitad sea optimista.

Capítulo 15 — Ataques estructurales: recuperar la clave privada (capítulo 15)

1. (a) $4096^{64}/64 = 2^{768-6} = 2^{762}$; (b) $1024^{50}/50 \approx 2^{500-5{,}6} = 2^{494{,}4}$; (c) $\log_2 P(4096, 769) = \sum_{j=0}^{768} \log_2(4096 - j) \approx 9116{,}8$.

2. $\binom{4096}{3488}$ cuenta *conjuntos* de n posiciones: $\approx 2^{2476}$ (entropía de «qué elementos forman el soporte»); $P(4096, 769)$ cuenta *tuplas ordenadas* de 769 posiciones concretas: el dato parcial que el modelo de 2022 necesitaba reconstruir. La diferencia (≈ 6640 bits) es el precio del orden y de las posiciones concretas: una va con el tamaño del soporte (n) y la otra con los elementos libres ($n - k + 1$).

3. (a) $9116{,}8 - 140{,}8 = 8976$ bits; el cociente de costes es 2^{8976} (¡2^{8720} veces el margen entero de AES-256!). (b) Comparada con 2^{256}, la cifra de 2022 es inalcanzable por cualquier medio físico imaginable: el titular era «correcto» dentro de un modelo de datos parciales que estaba a punto de agotarse, y como amenaza práctica era casi cómico (nadie ejecutaría jamás tal cómputo).

4. La cadena sería: distinguidor con ventaja $1 - o(1) \to$ estadístico que expone estructura en las claves reales $\to$ (eslabón que falta) inversión eficiente en recuperación de clave $\to$ descifrado. Los márgenes de la tabla 15.2 miden la distancia del régimen de cada método a los parámetros reales: Faugère ($t \leq 20$ frente a 64–128) tiene holgura enorme; Carrier (tasa $< 0{,}3$ frente a 0,7) moderada; el *holdout*/anulación de orden superior ya alcanza el t real: es el candidato natural a escalar, y el eslabón que falta es la barrera que hoy separa el estadístico de la clave.

5. $n < q$ es doblemente bueno: contra ISD no se paga nada (el acortamiento transporta la dureza con 0,8 bits de pérdida: observación 14.1), y contra la división del soporte el atacante debe adivinar qué n de las q columnas forman L y en qué orden: paga $\binom{q}{n}$ (y tuplas ordenadas: $2^{9116{,}8}$); con $n = q$ ese frente se desvanece (observación 15.1). El diseño cobra en un frente lo que no pierde en el otro.

6. Holguras (menor = más peligroso): (1) anulación de orden superior / *holdout* (su régimen alcanza el t real: la frontera está pegada a los parámetros, aunque sus costes aún no escalen); (2) Carrier (necesita tasa baja, $< 0{,}3$, y los conjuntos reales van a 0,7–0,8: margen moderado en el otro lado de la frontera); (3) Faugère ($t \leq 20$ frente a ≥ 64: holgura enorme). El más peligroso a escalar es el primero: es el único cuyo *régimen* ya contiene a los conjuntos reales. (Ejercicio de razonamiento: la respuesta admite matices, pero no intercambiar los tres regímenes.)

Capítulo 16 — El terremoto: 2025–2026 (capítulo 16)

1. (a) $9116{,}8 - 94{,}3 = 9022{,}5$ bits de diferencia. No es el «mismo ataque mejorado»: la familia de 2022 (datos parciales y reconstrucción del soporte) estaba agotada contra su muro; el *holdout* (GIJS/Weis) es una familia nueva, importada de la teoría de la información privada: no busca el soporte, evalúa el código con familias de funciones correlacionadas. (b) $94{,}3 < 256$: por debajo del suelo trivial (enumerar la semilla de 256 bits, con una generación por candidato). Una estimación así solo es compatible con (i) una contabilidad que no cobra el trabajo real por candidato/paso (memoria gratis, «operaciones de bits», heurísticas no ejecutadas) o (ii) una rotura histórica. De ahí la exigencia de verificación pública: a octubre de 2026, ninguna de las dos cosas está demostrada.

2. Total con 100–1400 ejecuciones: sin memoria, $2^{89} \cdot [100, 1400] = 2^{95{,}6}$–$2^{99{,}5}$; con cargo severo, $2^{117} \cdot [100, 1400] = 2^{123{,}6}$–$2^{127{,}5}$. La decodificación genérica en ese modelo para `348864` está en $2^{158{,}6}$: ambos rangos quedan por debajo: «nuestra mejor estimación; con el cargo de la guía queda por debajo de la decodificación genérica en ese modelo». Es exactamente la afirmación del equipo, y la razón del sobresalto: incluso el rango con cargo severo ($\leq 2^{127{,}5}$) roza el listón ISO de 2^{128}.

3. $94{,}3 + 8 \cdot 6 = 142{,}3 \approx 2^{142}$: la regla de Weis (unos 8 bits por unidad de m) reproduce la instancia de Bernstein ($n = 6176$, $t = 112$, $m = 18$), cifrada en $\approx 2^{142}$, con error de 0,3 bits. Clave pública allí: $mt\lceil k/8 \rceil = 2016 \cdot 520 = 1\,048\,320$ B $\approx 1{,}0$ MB: cuatro veces la de `348864` (0,25 MB) y del orden de la mayor actual (`8192128`: 1,36 MB). El eje m compra seguridad exponencial a precio lineal en la clave.

4. (a) Demostrada (en su régimen asintótico: es un teorema con ventaja $1 - o(1)$). (b) Estimada: el método de recuperación es heurístico (cinco supuestos enumerados por Weis, probados solo en claves pequeñas y retos TII) y la cifra concreta $2^{102{,}4}$ depende de la contabilidad. (c) Heurística y disputada: la afirmación de Vedenev «esencialmente una ejecución» no está verificada de forma independiente (hay réplicas y contra-réplicas). (d) Hecho (negativo y verificable): nadie ha ejecutado el cómputo de núcleos contra un conjunto real; los retos TII tienen $n = 214$.

5. $2^{94{,}3}$ frente a 2^{143} (AES-128): en el mismo registro de «operaciones de bits», el ataque saldría 48,7 bits más barato: alarmante. Pero ese registro no cobra la memoria ni la latencia; con el cargo severo (2^{117}–2^{128}), el ataque cuesta tanto o más que agotar AES-128: favorable al esquema. Falta: (i) un modelo de costes consensuado (cómo cobrar memoria y accesos) y (ii) una ejecución de extremo a extremo (nadie ha corrido el método contra un conjunto real): sin ambas, la comparación está abierta (BSI elige la precaución; el equipo, su contabilidad clásica).

6. Ejemplos (límite de 80 caracteres): *verdadero* — «McEliece: ataques de 2026 estimados, ninguno ejecutado ni verificado»; *falso por alarma* — «McEliece roto: un ataque de 2^{94} desarma el estándar ISO» (le falta: estimación, modelo de costes disputado, no ejecutado); *falso por complacencia* — «McEliece intacto: nada ha cambiado» (le falta: las estimaciones bajaron 9000 bits en cuatro años y el BSI desaconseja despliegues nuevos). (Ejercicio abierto: intercambiar con un colega y discutir la cláusula ausente.)

CAPÍTULO 19

Glosario

En este capítulo

Este glosario reúne, en orden alfabético, los términos que el libro usa con un significado técnico preciso. Cada entrada recuerda en una o dos líneas la definición y remite al capítulo donde se desarrolla. Hace además las veces de índice: quien busque «¿dónde se habló de X?» puede venir aquí primero.

Los nombres de algoritmos y los parámetros aparecen con su notación del texto; los capítulos se citan por su número de referencia.

Acortamiento (*shortening*)
Técnica que fija a cero s coordenadas de un código (n, k); en ISD transporta la dureza entre instancias con pérdida $\leq 0{,}8$ bits, y es la cara amable de tomar $n < q$ (caps. 14 y 15).

AES-128 / AES-256
Cifrado simétrico de referencia. Grover reduce su búsqueda de clave a 2^{64} y 2^{128}: el «listón de 128 bits» que traducen las categorías (caps. 1 y 14).

Alfa (α; orden del soporte)
Lista ordenada $\alpha_0, \ldots, \alpha_{q-1}$ de todos los elementos de $\mathbb{F}_q$; el soporte del código es $L = \{\alpha_0, \ldots, \alpha_{n-1}\}$. Su *orden* es secreto y se sortea con FieldOrdering (cap. 9).

Anulación de orden superior (*higher-order vanishing*)
Familia de distinguidores y ataques de 2025–2026 basada en hallar polinomios homogéneos multivariados de grado creciente que se anulan con alta multiplicidad en las palabras del código; de ella surgen el *holdout* y las estimaciones de GIJS/Weis (caps. 15 y 16).

Benes, red de
Representación de la permutación π (el orden α) con $2m-1$ etapas de 2^{m-1} conmutadores: $\lceil (2m-1)2^{m-4} \rceil$ bytes (5 888 para $m = 12$); es como la clave secreta guarda α (cap. 9).

Berson, ataque de reacción de
Con un descifrado que avisa del fallo, n consultas $x \oplus e_j$ revelan el soporte del error. El rechazo implícito del KEM lo desactiva (cap. 6).

BSI (*Bundesamt für Sicherheit in der Informationstechnik*)
Oficina federal alemana de seguridad. Su nota del 1-oct-2026 desaconseja Classic McEliece en nuevos despliegues tras las estimaciones de 2026 y recomienda ML-KEM, FrodoKEM o HQC (cap. 16).

Canal lateral
Ataque a la implementación (tiempos, cachés, accesos) que no rompe el esquema abstracto; el ataque de tiempos de Shoup contra la decodificación es el ejemplo clásico (caps. 8 y 16).

Categorías (NIST)
Escala que traduce «romper el esquema» a costes tipo AES: categoría 1 $\approx 2^{143}$ operaciones de puerta (AES-128), categoría 5 $\approx 2^{272}$ (AES-256). Classic McEliece se certifica con el modelo de costes que carga la memoria (caps. 12 y 14).

Clase lateral
Conjunto $y + C$ de vectores con el mismo síndrome que y; decodificar es elegir en la clase el representante de peso mínimo (cap. 2).

Clave equivalente
: Objeto (por ejemplo g y el orden α, o un decodificador del código público) con el que se descifra todo sin ser la clave secreta literal; su obtención es una rotura total (caps. 9, 13 y 15).

Código de Goppa ($\Gamma_2(L, g)$)
: Código $\{c : \sum_j c_j/(x + \alpha_j) \equiv 0 \pmod{g}\}$, con g de grado t sin raíces en L; la versión binaria (subcuerpo) con g irreducible tiene distancia $\geq 2t + 1$: es el código que usa Classic McEliece (cap. 4).

Código lineal
: Subespacio de $\mathbb{F}_2^n$ descrito por una matriz generadora G o una de paridad $\mathbf{H}$; su distancia mínima d es el peso del codeword no nulo más ligero (cap. 2).

Conjetura de dureza (SD en media)
: «Para todo algoritmo subexponencial, el éxito sobre instancias de la distribución del esquema es despreciable»: el supuesto en el que descansa la seguridad (caps. 13 y 14).

Cuerpo finito ($\mathbb{F}_q$ con $q = 2^m$; $\mathbb{F}_2$)
: El terreno donde viven g, α y los síndromes; $\mathbb{F}_2$ es el cuerpo base y $\mathbb{F}_q$ el de extensión (cap. 3).

Delta (δ; semilla)
: Los 32 bytes que siembran toda la generación: de ella se derivan deterministamente el código, el orden y el secreto de rechazo s; la clave secreta completa es regenerable desde δ (cap. 9).

Decode / Encode
: Primitivas del KEM: $\mathsf{Encode}(e) = \hat{\mathbf{H}}e$ y $\mathsf{Decode}(C)$, que corrige hasta t errores y solo acepta vectores de peso exactamente t (caps. 10 y 11).

Despliegues
: Implementaciones reales y usos del esquema: SUPERCOP y `cmimpl`, liboqs y PQClean, y los despliegues híbridos (Mullvad, Rosenpass, PQConnect), todos anteriores al aviso del BSI de 2026 (cap. 16).

Distancia de Hamming; distancia mínima (d)
: El peso de la diferencia entre dos vectores; d es el mínimo peso no nulo de un codeword. El teorema de Goppa da $d \geq 2t + 1$ y de ahí la decodificación única (caps. 2 y 4).

Distinguidor
: Algoritmo que decide «¿Goppa de verdad o matriz aleatoria?» con ventaja no despreciable. Erosiona el supuesto de indistinguibilidad de la clave pública, pero no descifra por sí solo (caps. 13–16).

Dumer, algoritmo de
: Mejora de Prange (1991) que precomputa columnas y amplifica la probabilidad de éxito por iteración; primer escalón relevante tras Prange (cap. 14).

e (vector de error)
: Vector de peso exactamente t que cifra: $C = \hat{\mathbf{H}}e$. Su soporte es el «mensaje-error» del esquema (caps. 6–10).

Escalera de ataques
: La sucesión que baja la cotización de ISD para cada conjunto: Prange (1962) → Lee–Brickell/Leon (1988) → Stern/Dumer (1989/91) → MMT/BJMM (2011/12) → May–Ozerov (2015) → Both–May (2018) → Esser–Bellini y Carrier (2022) → las revisiones de 2026 (cap. 14).

Espacio de criptogramas (P)
: La imagen de los errores de peso t bajo Encode ($\binom{n}{t}$ elementos; $|P| > 2^{456}$ en `348864`);

aparece en la cota ROM (cap. 8).

Faugère, ataque de
Recuperación algebraica de la clave por sistemas polinómicos con interrupción temprana (XLS de Faugère et al.); hoy limitado a $t \leq 20$: holgura enorme frente a los t reales (caps. 15 y 16).

FieldOrdering
Algoritmo de la especificación que convierte $32q$ bits en el orden uniforme α; el choque de enteros provoca reintento determinista (cap. 9; proposición 9.1).

FIPS 203/204/205
Los estándares post-cuánticos del NIST (ML-KEM, ML-DSA, SLH-DSA); el ecosistema del que Classic McEliece es alternativa y complemento (caps. 1 y 16).

FixedWeight
Algoritmo que muestrea errores uniformes de peso exacto t con trozos de 16 bits y rechazo; su uniformidad es una proposición (cap. 10; proposición 10.1).

FO (Fujisaki–Okamoto)
Transformación genérica de PKE a KEM en el ROM; Classic McEliece usa una versión simplificada (sin reencriptación, con rechazo implícito y secretos más pequeños) (cap. 8).

FrodoKEM
KEM de retículos (LWE) considerado conservador; una de las alternativas que el BSI recomienda mientras se revisa la TR-02102-1 (cap. 16).

G' (matriz pública de McEliece)
La generadora disfrazada $G' = SGP$; su versión sistemática se reduce a la clave pública T (caps. 6 y 7).

g (polinomio de Goppa)
Polinomio mónico de grado t sobre $\mathbb{F}_q$ sin raíces en L; el secreto central del código, sorteado con Irreducible (caps. 4 y 9).

Gaussiana (eliminación)
Procedimiento para llevar $\hat{\mathbf{H}}$ a la forma $(\mathrm{I}_{mt} \,|\, T)$; también el corazón de las iteraciones de ISD (caps. 5 y 14).

Grover, algoritmo de
Búsqueda cuántica con aceleración cuadrática en el exponente: AES-256 $\rightarrow 2^{128}$; en ISD, la «mitad» del exponente, con la latencia como límite realista (caps. 1 y 12).

Hamming, código de ($[7, 4, 3]$)
El ejemplo recurrente para síndromes y corrección: matriz de paridad con las siete columnas no nulas y decodificación de un error (cap. 2).

Harvest now, decrypt later
Grabar hoy el tráfico cifrado para descifrarlo cuando exista el ordenador cuántico; la razón de migrar antes de que sea urgente (cap. 1).

Híbrido
Combinación de un KEM post-cuántico con uno clásico (por ejemplo X25519); los despliegues reales de Classic McEliece son híbridos, lo que conserva la mitad clásica ante roturas (caps. 1 y 16).

Holgura (*tightness*)
Pérdida entre una cota de seguridad y el juego real: el QROM pierde la raíz cuadrada del éxito OW-CPA (cap. 8).

HQC
KEM de códigos cuasi-cíclicos seleccionado por el NIST en 2025; la otra alternativa que el BSI pone por delante de McEliece (caps. 12 y 16).

Irreducible
: Algoritmo de la especificación que convierte $16t$ bits en un g mónico irreducible uniforme; su demostración es la proposición de uniformidad (cap. 9).

ISD (decodificación por conjuntos de información)
: Familia de algoritmos de decodificación genérica (Prange, Stern, MMT, BJMM, May–Ozerov, ...) que fija la seguridad clásica de los conjuntos (cap. 14).

ISO/IEC 18033-2 (Amd.2)
: Norma con los 4 conjuntos × 4 variantes = 16 perfiles de Classic McEliece (jun-2026); descarta `348864` (caps. 12 y 16).

Juguete (el)
: La instancia diminuta usada en los laboratorios: $m = 4$ ($q = 16$), $n = 16$, $t = 2$, elaborada paso a paso en el capítulo 5 y completa en el 11; `toy.py` (apéndice A) reproduce sus números.

K (clave del KEM)
: Los 32 bytes que encapsula y desencapsula el KEM: $H(1, e, C)$ en el éxito y $H(0, s, C)$ en el rechazo (caps. 8 y 11).

KAT (vectores de prueba)
: Archivos de la especificación con claves, criptogramas y secretos de referencia; comparables byte a byte porque todo el proceso (incluidos los reintentos) es determinista (cap. 9).

KEM (encapsulamiento de claves)
: Primitiva Encap/Decap que produce una clave compartida a partir de una clave pública; la que Classic McEliece deriva de su PKE (cap. 8).

Latencia (cuántica)
: Límite práctico de Grover: las iteraciones se serializan y la raíz cuadrada sobreestima al atacante real; de ahí que la «mitad» del exponente se compare con cautela (caps. 12 y 16).

Lee–Brickell
: Primer algoritmo de ISD con lista (1988): fija p posiciones «con error» y busca en ellas; semilla de la línea Stern (cap. 14).

Leon
: Variante de lista por información canónica (1988), hermana de Lee–Brickell (cap. 14).

Localizador (polinomio σ)
: Polinomio cuyas raíces señalan las posiciones del error: $\sigma(x) = \prod_{j \in \mathrm{supp}(e)} (x + \alpha_j)$; Patterson lo recupera del síndrome (caps. 4 y 5).

m
: Exponente del cuerpo: $q = 2^m$. Classic McEliece usa $m = 12$ ($q = 4096$) o $m = 13$ ($q = 8192$); la seguridad estructural crece ≈ 8 bits por unidad de m (caps. 12 y 15).

MatGen
: Subalgoritmo de generación de claves: forma semi-sistemática de las mt primeras columnas, con reintentos deterministas ($\delta \leftarrow \delta'$) (cap. 9).

May–Ozerov / Both–May
: Ataques que llevan MMT/BJMM y la búsqueda de vecinos a ISD; May–Ozerov (2015) cotiza `348864` en $2^{140{,}8}$ (memoria libre) y Both–May (2018) afinan el régimen de tasas de error altas (cap. 14).

McEliece (criptosistema)
: El cifrado de 1978: $x = uG' + z$ con la generadora disfrazada y error de peso t; quien conoce el código lo decodifica con Patterson (cap. 6).

Memoria (coste de la)
: En ISD, las tablas grandes dominan el coste real: cargarla añade decenas de bits (hasta 33,5 en `6960119`) y decide las categorías (caps. 14 y 16).

ML-KEM
El KEM post-cuántico estandarizado por el NIST (FIPS 203); referencia de comparación para la criptografía de códigos (caps. 1 y 16).

Modelo de costes
La convención con la que se cuenta el trabajo de un ataque (puertas, bits, operaciones vectoriales, memoria, latencia); las columnas «libre» y «con cargo» de las tablas (caps. 14, 15 y 16).

NCP (decodificación / palabra más cercana)
El problema hermano de SD: dado y, hallar el codeword de peso mínimo a distancia de y; el descifrado de McEliece es una instancia (caps. 2 y 13).

Niederreiter (reformulación)
Versión dual basada en síndromes: $C = e(H')^\top$; equivalente a McEliece por la reducción de la proposición 7.4 (cap. 7).

NIST
Organizador de la competición post-cuántica (2016–2024); estandarizó ML-KEM/ML-DSA/SLH-DSA, aplazó su decisión sobre McEliece y seleccionó HQC como KEM extra (caps. 1 y 16).

OW-CPA
Juego de seguridad del PKE: recuperar e dado C; la raíz que el QROM divide por dos en su cota (caps. 8 y 13).

Parámetros (conjuntos de)
Los cinco del estándar: `348864`, `460896`, `6688128`, `6960119`, `8192128` ($m = 12$ o 13, $t = 64$–128); sus tablas de tamaños y seguridad (caps. 12–16).

Paridad (matriz de, H)
Matriz cuyas filas ortogonalizan el código ($GH^\top = 0$); en forma sistemática $\hat{\mathbf{H}} = (\mathrm{I}_{mt} \,|\, T)$ (caps. 2 y 9).

Patterson (algoritmo de)
Decodificador de los códigos de Goppa: forma el síndrome S, resuelve la ecuación clave (Euclides/Berlekamp) y halla las raíces del localizador; $O(t^2)$ operaciones en $\mathbb{F}_q$ más inversiones (caps. 4, 5 y 10).

pcf / pc / f (variantes)
Sufijos de los conjuntos: `f` = generación sin reintentos visibles (forma $(32, 64)$); `pc` = confirmación de texto claro ($C = (C_0, C_1)$ con $C_1 = H(2, e)$); `pcf` = ambas (cap. 12).

Permutación (P, π)
El disfraz de coordenadas, $(\mathbf{v}P)_j = \mathbf{v}_{\sigma(j)}$; su orden es el secreto que FieldOrdering sortea (caps. 5 y 9).

Peso (wt)
Número de posiciones no nulas. Los parámetros del esquema son pesos: $\mathrm{wt}(e) = t$ y $\mathrm{wt}(z) \le t$ (caps. 2 y 6).

Pi (π)
La permutación que resulta de ordenar los enteros de FieldOrdering; se guarda comprimida en la red de Benes (cap. 9).

PKE (cifrado de clave pública)
El cifrado subyacente (McEliece/Niederreiter) del que el KEM se deriva mediante la transformación del capítulo 8.

PQC (criptografía post-cuántica)
El campo entero: algoritmos diseñados para resistir atacantes con ordenador cuántico (caps. 1 y 16).

Prange
: ISD original (1962): elegir k columnas y comprobar si el error cae fuera de ellas; iteración barata y probabilidad pequeña. La referencia contra la que se mide todo lo demás (cap. 14).

QROM
: Modelo del oráculo aleatorio cuántico: la cota de FO pierde la raíz cuadrada del éxito OW-CPA (caps. 8 y 15).

Recuperación de clave (*key recovery*)
: El objetivo más fuerte de la taxonomía estructural: obtener g, el orden α o un decodificador; a él apuntan las cifras de 2026 (2^{140}–2^{148}, 2^{99}–2^{102} según contabilidad) (caps. 15 y 16).

Rechazo implícito
: Decap nunca devuelve $\perp$: entrega una clave pseudoaleatoria $H(0, s, C)$; elimina el oráculo observacional de los protocolos (caps. 8 y 10).

ROM
: Modelo del oráculo aleatorio clásico; la cota aditiva de la transformación de Fujisaki–Okamoto (cap. 8).

s (secreto de rechazo)
: Cadena uniforme de n bits dentro de la clave secreta; la clave de la rama de rechazo es $H(0, s, C)$ (caps. 9 y 10).

SD (Syndrome Decoding)
: El problema: dado (H, y) con $y = He$, hallar e de peso $\leq t$. NP-duro en el peor caso; duro en media (conjetura) sobre la distribución del esquema (caps. 13 y 14).

Semi-sistemática (forma, (μ, ν))
: Truco de MatGen: permutar hasta que las mt primeras columnas sean independientes; $(\mu, \nu) = (32, 64)$ en las variantes `f` (cap. 9).

SHAKE256
: El XOF de la especificación: expande semillas, genera g y α, y deriva las claves K y de rechazo (caps. 9 y 11).

Shor, algoritmo de
: Rompe RSA y ECC en tiempo polinómico: la razón de ser de toda la migración post-cuántica (cap. 1).

Shoup, ataque de tiempos de
: Ataque de canal lateral clásico contra implementaciones de decodificación; la motivación histórica de decodificar en tiempo constante (cap. 8).

Síndrome
: $S(y) = \mathbf{H}y^\top$; en el código de Goppa también $S(x) = \sum_j c_j/(x + \alpha_j)$ con columnas $\sigma_j = (x + \alpha_j + 1)/g(\alpha_j)$; su espacio tiene dimensión $n - k$ (caps. 2 y 5).

Soporte (supp)
: El conjunto de posiciones no nulas de un vector. El error del KEM es uniforme sobre los $\binom{n}{t}$ soportes de tamaño t (caps. 2 y 10).

Stern (algoritmo de)
: ISD con cruce de listas (1989): el primer gran salto de la escalera (22 bits sobre Prange en `348864`) (cap. 14).

t
: Grado del polinomio de Goppa y peso exacto del error ($t = 64$–128 en los conjuntos); fija la distancia $\geq 2t + 1$ y el trabajo de Patterson (caps. 4 y 12).

T (clave pública)
: La mitad derecha de $\hat{\mathbf{H}} = (\mathrm{I}_{mt}|T)$: la clave pública, de $mt\lceil k/8 \rceil$ bytes (261 120 a 1 357 824) (caps. 9 y 12).

Terremoto (2025–2026)
: El periodo de distinguidores, estimaciones y contabilidades en disputa que movió la conversación pública sin romper el esquema (caps. 15 y 16).

TII
: Retos públicos de instancias Goppa pequeñas ($n = 214$) usados para probar heurísticas estructurales; no existe (a octubre de 2026) un cómputo de núcleos contra un conjunto real (cap. 16).

Traza (Tr)
: El funcional $\mathrm{Tr}(\beta) = \sum_{i=0}^{m-1} \beta^{2^i}$; criterios de irreducibilidad, raíces cuadradas y linealidad sobre $\mathbb{F}_2$ (cap. 3).

Vectores (notación)
: Las cadenas binarias se imprimen con la posición 0 a la izquierda; al serializar se empaqueta en little-endian (posición j en el bit j mód 8 del byte $j/8$) (caps. 11 y A).

Weis (ataques de 2026)
: Recuperación heurística de clave vía *holdout*, con cifras entre 2^{89} y 2^{128} según la contabilidad, y la regla «≈ 8 bits por unidad de m» (caps. 15 y 16).

Fin del glosario. Los apéndices que siguen cierran el libro: el código comentado del juguete (apéndice A), la cronología de medio siglo de historia y las tablas del cuerpo $\mathbb{F}_{2^4}$.

CAPÍTULO A

El programa del juguete: `toy.py`

En este capítulo

Este apéndice reproduce, íntegro y ejecutable, el programa que generó las cifras de los laboratorios de los capítulos 5 y 11: el McEliece de juguete sobre $\mathbb{F}_{2^4}$ con $n = 16$ y $t = 2$. El listado no es decorativo: cada número impreso en esos capítulos está aquí calculado, y cada afirmación delicada está protegida por una aserción que detiene el programa si algo no cuadra.

A.1 Qué es y cómo se ejecuta

`toy.py` es un único archivo de Python 3, sin más dependencia que la biblioteca estándar (`hashlib.shake_256`); ocupa unas 450 líneas. Se ejecuta con `python3 toy.py` y termina imprimiendo `TODAS LAS COMPROBACIONES OK` si todo cuadra; si una aserción falla, se detiene en la primera discrepancia. Reproduce, en orden:

1. el cuerpo $\mathbb{F}_{2^4}$ y las 16 columnas de la matriz $\tilde{H}$ de la tabla 5.1 (capítulo 5);
2. la forma sistemática $(\mathrm{I}_8 \,|\, T)$, la matriz $G = (T^\top \,|\, \mathrm{I}_8)$ y la permutación σ del laboratorio (figura 5.4);
3. el síndrome del error e con soporte $\{10, 13\}$ y la traza completa de Patterson, incluidos sus certificados;
4. la comprobación de tipo van Lint–MacWilliams: las 2^{16} palabras del código, contadas dos veces y comparadas;
5. los 120 errores de peso ≤ 2, los 137 síndromes alcanzables y la lista completa de síndromes;
6. la cadena del capítulo 11: la semilla s, el error e, el criptograma C, la clave K y el criptograma manipulado $\tilde{C}$ con su rechazo implícito K''.

A.2 El listado completo

```
#!/usr/bin/env python3
# ---------------------------------------------------------------------
# toy.py - el juguete de bolsillo de los capitulos 4, 5 y 11
# Reproduce, paso a paso, todas las cifras impresas del libro.
# Uso:  python3 toy.py
# ---------------------------------------------------------------------

from hashlib import shake_256

# =====================================================================
# 1. GF(16) = F2[z]/(z^4+z+1)
# Cada elemento es un entero 0..15 (un digito hexadecimal): el bit l
# es el coeficiente de z^l.  z = 2, y z^4+1 = z.
# =====================================================================
```

```
15
16  def _tablas():
17      exp, log = [0] * 15, [0] * 16
18      v = 1
19      for e in range(15):
20          exp[e] = v
21          log[v] = e
22          v <<= 1
23          if v & 0x10:
24              v ^= 0x13                       # z^4 = z+1
25      return exp, log
26
27  EXP, LOG = _tablas()
28
29  def mul(a, b):
30      return 0 if not a or not b else EXP[(LOG[a] + LOG[b]) % 15]
31
32  def inv(a):
33      return EXP[(-LOG[a]) % 15]
34
35  def pw(a, k):
36      return 0 if not a else EXP[(LOG[a] * k) % 15]
37
38  # ======================================================================
39  # 2. Polinomios sobre GF(16): listas de coeficientes, indice = grado
40  # ======================================================================
41
42  def trim(p):
43      p = p[:]
44      while len(p) > 1 and p[-1] == 0:
45          p.pop()
46      return p
47
48  def p_add(a, b):
49      r = [0] * max(len(a), len(b))
50      for i, c in enumerate(a):
51          r[i] ^= c
52      for i, c in enumerate(b):
53          r[i] ^= c
54      return trim(r)
55
56  def p_esc(c, p):                            # multiplica por el escalar c
57      return trim([mul(c, x) for x in p])
58
59  def p_mul(a, b):
60      r = [0] * (len(a) + len(b) - 1)
61      for i, x in enumerate(a):
62          if x:
63              for j, y in enumerate(b):
64                  if y:
65                      r[i + j] ^= mul(x, y)
66      return trim(r)
67
68  def p_divmod(a, b):                         # a = q*b + r,  deg r < deg b
69      a = trim(a)
70      q = [0] * (max(0, len(a) - len(b)) + 1)
71      while a != [0] and len(a) >= len(b):
72          c = mul(a[-1], inv(b[-1]))
73          k = len(a) - len(b)
74          q[k] ^= c
75          for i, y in enumerate(b):
76              a[i + k] ^= mul(c, y)
77          a = trim(a)
78      return trim(q), a
79
80  def p_mod(a, m):
81      return p_divmod(a, m)[1]
82
83  def p_eval(p, x):
84      r = 0
85      for c in reversed(p):
86          r = mul(r, x) ^ c
```

```
    return r

def p_inv_mod(a, m):                        # a^{-1} mod m  (gcd(a,m)=1)
    r0, r1 = trim(m), p_mod(a, m)
    u0, u1 = [0], [1]                       # invariante: r_i = u_i*a mod m
    while len(r1) > 1:                      # hasta llegar a una constante
        assert r1 != [0], "sin inverso: gcd != 1"
        q, r2 = p_divmod(r0, r1)
        r0, r1 = r1, r2
        u0, u1 = u1, p_add(u0, p_mul(q, u1))
    assert r1 != [0], "sin inverso: gcd != 1"
    lam = r1[0]                             # r1 = lam == u1*a  ->  1 = (u1/lam)*a
    return p_mod(p_esc(inv(lam), u1), m)

def p_sqrt_mod(v, g):                       # deg g = 2: anillo GF(256)
    w = p_mod(v, g)                         # raiz = v^(2^7)
    for _ in range(7):
        w = p_mod(p_mul(w, w), g)
    return w

def p_hex(p):
    def t(i):
        return "" if i == 0 else ("x" if i == 1 else f"x^{i}")
    s = '+'.join(f"{c:x}{t(i)}" for i, c in enumerate(p) if c)
    return s or '0'

# ======================================================================
# 3. El codigo de Goppa del juguete (m = 4, n = 16, t = 2)
# ======================================================================

N, M4 = 16, 4
ALPHA = list(range(N))                      # L = (0, 1, z, 1+z, ...)
G_POLY = [0x8, 0x1, 0x1]                    # g(x) = x^2 + x + z^3
INV_G = [inv(p_eval(G_POLY, a)) for a in ALPHA]
COLS = [(INV_G[j], mul(ALPHA[j], INV_G[j])) for j in range(N)]

def sbits(v, n):
    return ''.join(str((v >> i) & 1) for i in range(n))

def vbits(bs):
    return sum(b << i for i, b in enumerate(bs))

def syndrome(word):
    # S(x) = suma_j palabra_j * (x + a_j + 1)/g(a_j)   (mod g)
    S = [0]
    for j, b in enumerate(word):
        if b:
            S = p_add(S, p_mod(p_mul([ALPHA[j] ^ 1, 1], [INV_G[j]]), G_POLY))
    return S

def patterson(S, traza=None):
    R = p_inv_mod(S, G_POLY)                    # R = S^{-1}
    W = p_sqrt_mod(p_add(R, [0, 1]), G_POLY)    # W^2 = x + R
    tau = p_inv_mod(W, G_POLY)                  # tau = W^{-1}
    A, B = p_divmod(G_POLY, tau)                # g = tau*A + B
    sigma = p_add(p_mul(A, A), p_mul([0, 1], p_mul(B, B)))
    if traza is not None:
        traza.update(R=R, W=W, tau=tau, A=A, B=B)
    return sigma

def raices(sigma):
    return [j for j in range(N) if p_eval(sigma, ALPHA[j]) == 0]

# ---- expansion a bits, forma sistematica (I8|T) y G = (T^t|I8) -------

def hhat_variante(variante):
    # variante: (intercambio, invertir_bits)
    swap, msb = variante
    u = lambda j: COLS[j][1] if swap else COLS[j][0]   # filas 0-3
    v = lambda j: COLS[j][0] if swap else COLS[j][1]   # filas 4-7
    H = []
    for r in range(4):
```

```
159         H.append([(u(j) >> (3 - r if msb else r)) & 1 for j in range(N)])
160     for r in range(4):
161         H.append([(v(j) >> (3 - r if msb else r)) & 1 for j in range(N)])
162     return H
163
164 def sistematica(H):
165     A = [row[:] for row in H]
166     nr = len(A)
167     for c in range(nr):
168         piv = next((r for r in range(c, nr) if A[r][c]), None)
169         if piv is None:
170             return None
171         A[c], A[piv] = A[piv], A[c]
172         for r in range(nr):
173             if r != c and A[r][c]:
174                 A[r] = [x ^ y for x, y in zip(A[r], A[c])]
175     return [row[nr:] for row in A]
176
177 EXPECT_G = ["1100010110000000", "1100101001000000", "0110110000100000",
178             "1001110000010000", "1011101000001000", "0111010100000100",
179             "0110101100000010", "1001011100000001"]
180
181 def elige_convencion():
182     for variante in ((0, 0), (0, 1), (1, 0), (1, 1)):
183         H = hhat_variante(variante)
184         T = sistematica(H)
185         if T is None:
186             continue
187         G = [sbits(sum(T[a][i] << a for a in range(8)), 8)
188              + sbits(1 << i, 8) for i in range(8)]
189         if G == EXPECT_G:
190             return H, T, G, variante
191     raise SystemExit("ninguna convencion reproduce la G del libro")
192
193 HHAT, T, G, VARIANTE = elige_convencion()
194
195 print("=" * 70)
196 print("0. GF(16) = F2[z]/(z^4+z+1);  1/g y a/g se calculan en GF(16).")
197 print("   Potencias de z:", ' '.join(f"z^{e}={EXP[e]:x}" for e in range(15)))
198
199 # ---------------------------------------------------------------------
200 print()
201 print("=" * 70)
202 print("1. Las 16 columnas de H~ (tabla del libro):  (1/g(a_j), a_j/g(a_j))")
203 EXPECT_COLS = {0: (0xf, 0x0), 1: (0xf, 0xf), 2: (0x3, 0x6), 3: (0x3, 0x5),
204                4: (0x8, 0x6), 5: (0x8, 0xe), 6: (0x2, 0xc), 7: (0x2, 0xe),
205                8: (0xa, 0xf), 9: (0xa, 0x5), 10: (0xc, 0x1), 11: (0xc, 0xd),
206                12: (0x5, 0x9), 13: (0x5, 0xc), 14: (0x4, 0xd), 15: (0x4, 0x9)}
207 for j in range(N):
208     a, b = COLS[j]
209     assert (a, b) == EXPECT_COLS[j], f"columna {j} discrepa"
210     print(f"   j={j:2d}  a_j={ALPHA[j]:x}  g(a_j)={p_eval(G_POLY, ALPHA[j]):x}"
211           f"  (1/g, a/g)=({a:x},{b:x})")
212
213 # ---------------------------------------------------------------------
214 print()
215 print("=" * 70)
216 print("2. Forma sistematica: filas de G = (T^t | I8)  [T unica]")
217 for i, row in enumerate(G):
218     assert row == EXPECT_G[i]
219     print(f"   G_{i} = {row}")
220 print("   columnas de T (hex):",
221       ' '.join(f"{sum(T[a][i] << a for a in range(8)):02x}" for i in range(8)))
222
223 # ---------------------------------------------------------------------
224 print()
225 print("=" * 70)
226 print("3. Sindrome del error e = posiciones {10, 13}")
227 e2 = [0] * N
228 e2[10] = e2[13] = 1
229 s0 = 0
230 s1 = 0
```

```
for j in (10, 13):
    s0 ^= COLS[j][0]
    s1 ^= COLS[j][1]
assert (s0, s1) == (0x9, 0xd)              # H~ e = (9, d)
S_e = syndrome(e2)
print(f"   H~ e = (s0, s1) = ({s0:x}, {s1:x})")
print(f"   S(x) = s0*x + (s0+s1) = {p_hex(S_e)}   con S_e = [{S_e[0]:x}, {S_e[1]:x}]")
assert S_e == [0x4, 0x9]

# ---------------------------------------------------------------------
print()
print("=" * 70)
print("4. Patterson paso a paso para S(x) = 4 + 9x (con certificados)")
tr = {}
sig49 = patterson([0x4, 0x9], traza=tr)
R, W, tau, A, B = tr["R"], tr["W"], tr["tau"], tr["A"], tr["B"]
assert R == [0xa, 0x7]
assert p_mul([0x4, 0x9], R) == p_add([1], p_esc(0xa, G_POLY))
assert p_add(R, [0, 1]) == [0xa, 0x6]
assert W == [0xc, 0x7]
assert p_mul(W, W) == p_add([0xa, 0x6], p_esc(0x6, G_POLY))
assert tau == [0x3, 0xb]
assert p_mul(W, tau) == p_add([1], p_esc(0x4, G_POLY))
assert A == [0x3, 0x5] and B == [0xd]
assert p_add(p_mul(tau, A), B) == G_POLY
assert sig49 == [0x5, 0xe, 0x2]
assert raices(sig49) == [10, 13]
q1, r1 = p_divmod(sig49, [0xa, 1])
q2, r2 = p_divmod(sig49, [0xd, 1])
assert r1 == [0] and r2 == [0] and p_add(q1, q2) == [0xe]
assert p_mul(sig49, [0x4, 0x9]) == p_add([0xe], p_mul([0xe, 1], G_POLY))
print(f"   R = S^-1 = {p_hex(R)}        [certificado: S*R = 1 + (z+z^3)*g]")
print(f"   x + R = {p_hex(p_add(R, [0,1]))}")
print(f"   W = raiz(x+R) = {p_hex(W)}   [certificado: W^2 = (x+R) + (z+z^2)*g]")
print(f"   tau = W^-1 = {p_hex(tau)}    [certificado: W*tau = 1 + z^2*g]")
print(f"   g = tau*A + B con A = {p_hex(A)}, B = {p_hex(B)}  (division exacta)")
print(f"   sigma = A^2 + x*B^2 = {p_hex(sig49)}")
print(f"   raices de sigma: {raices(sig49)} = posiciones del error")
print(f"   sigma' = e (0xe) y sigma*S = sigma' + (x+e)*g  [OK]")

# ---------------------------------------------------------------------
print()
print("=" * 70)
print("5. Identidad de van Lint: Gamma2(L,g) = Gamma2(L,g^2)")

def nucleo(H):
    nr = len(H)
    colmask = [sum(H[r][j] << r for r in range(nr)) for j in range(N)]
    tabL, tabH = [0] * 256, [0] * 256
    for b in range(256):
        sL = sH = 0
        for j in range(8):
            if (b >> j) & 1:
                sL ^= colmask[j]
                sH ^= colmask[8 + j]
        tabL[b], tabH[b] = sL, sH
    return frozenset(w for w in range(1 << N)
                     if tabL[w & 255] ^ tabH[w >> 8] == 0)

H4 = []
g2inv = [inv(mul(p_eval(G_POLY, a), p_eval(G_POLY, a))) for a in ALPHA]
for i in range(4):
    for l in range(4):
        H4.append([(mul(1 if i == 0 else pw(ALPHA[j], i), g2inv[j]) >> l) & 1
                   for j in range(N)])
C1, C2 = nucleo(HHAT), nucleo(H4)
assert len(C1) == len(C2) == 256 and C1 == C2
print(f"   |nucleo(H~)| = {len(C1)},  |nucleo(4 filas a^i/g^2)| = {len(C2)}")
print("   las 65536 palabras: coinciden palabra a palabra  [OK]  (y son 256)")

# ---------------------------------------------------------------------
print()
```

```
print("=" * 70)
print("6. Correccion: los 120 errores de peso 2 y los 137 sindromes")
ok = 0
for i in range(N):
    for j in range(i + 1, N):
        w = [0] * N
        w[i] = w[j] = 1
        if raices(patterson(syndrome(w))) == [i, j]:
            ok += 1
assert ok == 120
print(f"   Patterson corrige {ok}/120 errores de peso 2  [OK]")
sind = {0}
for j in range(N):
    sind.add((COLS[j][0] << 4) | COLS[j][1])
for i in range(N):
    for j in range(i + 1, N):
        sind.add((COLS[i][0] ^ COLS[j][0]) << 4 | (COLS[i][1] ^ COLS[j][1]))
assert len(sind) == 1 + 16 + 120 == 137
print(f"   de los 2^8 = 256 sindromes, alcanzables con peso <= 2: {len(sind)}"
      "  (1 + 16 + 120)  [OK]")

# ---------------------------------------------------------------------
print()
print("=" * 70)
print("7. McEliece 1978 en miniatura (capitulo 5): G' = S*G*P")

def mmul(A, B):
    return [[sum(A[i][k] * B[k][j] for k in range(len(B))) % 2
             for j in range(len(B[0]))] for i in range(len(A))]

def perm_pull(M, perm):                    # (M P)[i][j] = M[i][perm[j]]
    return [[row[perm[j]] for j in range(len(row))] for row in M]

def inv_f2(M):
    n = len(M)
    A = [row[:] + [1 if i == j else 0 for j in range(n)]
         for i, row in enumerate(M)]
    for c in range(n):
        piv = next(r for r in range(c, n) if A[r][c])
        A[c], A[piv] = A[piv], A[c]
        for r in range(n):
            if r != c and A[r][c]:
                A[r] = [x ^ y for x, y in zip(A[r], A[c])]
    return [row[n:] for row in A]

S8 = [[int(b) for b in row] for row in
      ("01101010", "10010001", "01010001", "01010110",
       "01110110", "00111110", "10011100", "00010101")]
SIG = [14, 2, 7, 10, 4, 8, 12, 6, 9, 15, 11, 5, 0, 3, 13, 1]
SIGINV = [SIG.index(j) for j in range(N)]
Gm = [[int(b) for b in row] for row in EXPECT_G]
GP = mmul(S8, perm_pull(Gm, SIG))
GP_fig = ["1111001110010101", "0000110101111001", "0010000011101001",
          "1000100010100011", "1101000010110010", "1001001000110111",
          "0000011100111110", "0100100101110111"]
for i, row in enumerate(GP):
    txt = sbits(vbits(row), 16)
    assert txt == GP_fig[i], f"G' fila {i} != figura"
print("   G' = S*G*P reproduce la figura del libro  [OK]")

u = [int(c) for c in "01001101"]
z = [int(c) for c in "0001000000001000"]
uGP = [sum(u[i] * GP[i][j] for i in range(8)) % 2 for j in range(N)]
assert sbits(vbits(uGP), 16) == "0000011010001011"
x = [a ^ b for a, b in zip(uGP, z)]
assert sbits(vbits(x), 16) == "0001011010000011"
xPinv = [x[SIGINV[j]] for j in range(N)]
zPinv = [z[SIGINV[j]] for j in range(N)]
assert sbits(vbits(xPinv), 16) == "0100000011101100"
assert sbits(vbits(zPinv), 16) == "1000000000100000"
c = [a ^ b for a, b in zip(xPinv, zPinv)]
assert sbits(vbits(c), 16) == "1100000011001100"
```

```
print(f"   u = {sbits(vbits(u), 8)}  (4 unos)   z = {sbits(vbits(z), 16)}")
print(f"   x = u*G' + z = {sbits(vbits(x), 16)}")
print(f"   x*P^-1 = {sbits(vbits(xPinv), 16)}   z*P^-1 = {sbits(vbits(zPinv), 16)}")
print(f"   c = x*P^-1 + z*P^-1 = {sbits(vbits(c), 16)}")
S_x = syndrome(xPinv)
assert S_x == [0x2, 0x3]                    # S(x) = 2 + 3x
sig = patterson(S_x)
assert sig == [0, 0xc, 0xf]                 # c x + f x^2
r = raices(sig)
assert r == [0, 10]
print(f"   sindrome: {p_hex(S_x)};  sigma = {p_hex(sig)} (multiplo escalar del localizador)")
print(f"   raices: {r} -> z' recuperado = {sbits(vbits(zPinv), 16)}  [OK]")
uS = c[8:]
u_rec = [sum(uS[i] * inv_f2(S8)[i][k] for i in range(8)) % 2 for k in range(8)]
assert sbits(vbits(uS), 8) == "11001100"
assert u_rec == u
print(f"   u*S = {sbits(vbits(uS), 8)};  u = (u*S)*S^-1 = {sbits(vbits(u_rec), 8)}"
      "  -> mensaje recuperado  [OK]")

# ---------------------------------------------------------------------
print()
print("=" * 70)
print("8. KEM del juguete (capitulo 11)")

DELTA = bytes([0x5a]) * 32
flujo = shake_256(bytes([0x40]) + DELTA).digest(8)   # etiqueta 64 = 0x40
s_val = flujo[0] | (flujo[1] << 8)
assert s_val == 0xCBAE
s_str = sbits(s_val, 16)
print(f"   delta = 5a repetido 32 veces")
print(f"   s = {s_str[:4]} {s_str[4:8]} {s_str[8:12]} {s_str[12:]}"
      f" = {s_val:04x}  (bytes AE CB, little-endian)")

e_alto, e_bajo = 0x00, 0x24                 # e = (e_alto, e_bajo), posiciones 10 y 13
C_bits = [(e_alto >> r) & 1 ^
          sum(T[r][j] * ((e_bajo >> j) & 1) for j in range(8)) % 2
          for r in range(8)]
C_val = vbits(C_bits)
assert C_val == 0x98
print(f"   e = 0000000000100100 (posiciones 10 y 13)")
print(f"   C = e_alto + T*(e_bajo) = col2(T) + col5(T) = 36 ^ AE = {C_val:02x}"
      f" = {sbits(C_val, 8)}")

K = shake_256(bytes([0x01]) + bytes([e_alto, e_bajo]) + bytes([C_val])).digest(32)
assert K.hex() == \
    "597f46fbba325c68f0b0b45c6ea984961461eb220d1f24024ed888912b336278"
print(f"   K  = SHAKE256(01|00 24|98) = {K.hex()}")

v = [(C_val >> r) & 1 for r in range(8)] + [0] * 8
S_v = syndrome(v)
assert S_v == [0x4, 0x9]
sig_v = patterson(S_v)
assert sig_v == [0x5, 0xe, 0x2]
r_v = raices(sig_v)
assert r_v == [10, 13]
e_dec = [0] * N
for p in r_v:
    e_dec[p] = 1
assert e_dec == e2
K2 = shake_256(bytes([0x01]) + bytes([e_alto, e_bajo]) + bytes([C_val])).digest(32)
assert K2 == K
print(f"   decap: v = (C, 0); S(v) = {p_hex(S_v)}; sigma = {p_hex(sig_v)}")
print(f"   raices {r_v} -> e correcto;  K' = K  [OK]")

C_t = 0x99
v_t = [(C_t >> r) & 1 for r in range(8)] + [0] * 8
S_t = syndrome(v_t)
sig_t = patterson(S_t)
assert raices(sig_t) == []
K2t = shake_256(bytes([0x00]) + bytes([flujo[0], flujo[1]]) + bytes([C_t])).digest(32)
assert K2t.hex() == \
    "ba1a72ac26b0fcf20c55c108bb681e27aa9c270c60802a821601130b7f1e1494"
```

```
print(f"   C~ = C+1 = {sbits(C_t, 8)}: S~ = {p_hex(S_t)}, sigma~ = {p_hex(sig_t)},"
      " sin raices")
print(f"   K'' = SHAKE256(00|AE CB|99) = {K2t.hex()}  (rechazo)")

print()
print("=" * 70)
print("TODAS LAS COMPROBACIONES OK")
```

A.3 La salida

La salida siguiente es la de una ejecución real (octubre de 2026), sin más recorte que los espacios en blanco finales de línea:

```
======================================================================
0. GF(16) = F2[z]/(z^4+z+1);  1/g y a/g se calculan en GF(16).
   Potencias de z: z^0=1 z^1=2 z^2=4 z^3=8 z^4=3 z^5=6 z^6=c z^7=b z^8=5 z^9=a z^10=7 z^11=e z^12=f z^13=d
     z^14=9

======================================================================
1. Las 16 columnas de H~ (tabla del libro):  (1/g(a_j), a_j/g(a_j))
   j= 0  a_j=0  g(a_j)=8  (1/g, a/g)=(f,0)
   j= 1  a_j=1  g(a_j)=8  (1/g, a/g)=(f,f)
   j= 2  a_j=2  g(a_j)=e  (1/g, a/g)=(3,6)
   j= 3  a_j=3  g(a_j)=e  (1/g, a/g)=(3,5)
   j= 4  a_j=4  g(a_j)=f  (1/g, a/g)=(8,6)
   j= 5  a_j=5  g(a_j)=f  (1/g, a/g)=(8,e)
   j= 6  a_j=6  g(a_j)=9  (1/g, a/g)=(2,c)
   j= 7  a_j=7  g(a_j)=9  (1/g, a/g)=(2,e)
   j= 8  a_j=8  g(a_j)=c  (1/g, a/g)=(a,f)
   j= 9  a_j=9  g(a_j)=c  (1/g, a/g)=(a,5)
   j=10  a_j=a  g(a_j)=a  (1/g, a/g)=(c,1)
   j=11  a_j=b  g(a_j)=a  (1/g, a/g)=(c,d)
   j=12  a_j=c  g(a_j)=b  (1/g, a/g)=(5,9)
   j=13  a_j=d  g(a_j)=b  (1/g, a/g)=(5,c)
   j=14  a_j=e  g(a_j)=d  (1/g, a/g)=(4,d)
   j=15  a_j=f  g(a_j)=d  (1/g, a/g)=(4,9)

======================================================================
2. Forma sistematica: filas de G = (T^t | I8)  [T unica]
   G_0 = 1100010110000000
   G_1 = 1100101001000000
   G_2 = 0110110000100000
   G_3 = 1001110000010000
   G_4 = 1011101000001000
   G_5 = 0111010100000100
   G_6 = 0110101100000010
   G_7 = 1001011100000001
   columnas de T (hex): a3 53 36 39 5d ae d6 e9

======================================================================
3. Sindrome del error e = posiciones {10, 13}
   H~ e = (s0, s1) = (9, d)
   S(x) = s0*x + (s0+s1) = 4+9x   con S_e = [4, 9]

======================================================================
4. Patterson paso a paso para S(x) = 4 + 9x (con certificados)
   R = S^-1 = a+7x        [certificado: S*R = 1 + (z+z^3)*g]
   x + R = a+6x
   W = raiz(x+R) = c+7x   [certificado: W^2 = (x+R) + (z+z^2)*g]
   tau = W^-1 = 3+bx    [certificado: W*tau = 1 + z^2*g]
   g = tau*A + B con A = 3+5x, B = d  (division exacta)
   sigma = A^2 + x*B^2 = 5+ex+2x^2
   raices de sigma: [10, 13] = posiciones del error
   sigma' = e (0xe) y sigma*S = sigma' + (x+e)*g  [OK]

======================================================================
5. Identidad de van Lint: Gamma2(L,g) = Gamma2(L,g^2)
```

```
   |nucleo(H~)| = 256,  |nucleo(4 filas a^i/g^2)| = 256
   las 65536 palabras: coinciden palabra a palabra  [OK]  (y son 256)

======================================================================
6. Correccion: los 120 errores de peso 2 y los 137 sindromes
   Patterson corrige 120/120 errores de peso 2  [OK]
   de los 2^8 = 256 sindromes, alcanzables con peso <= 2: 137  (1 + 16 + 120)  [OK]

======================================================================
7. McEliece 1978 en miniatura (capitulo 5): G' = S*G*P
   G' = S*G*P reproduce la figura del libro  [OK]
   u = 01001101  (4 unos)   z = 0001000000001000
   x = u*G' + z = 0001011010000011
   x*P^-1 = 0100000011101100   z*P^-1 = 1000000000100000
   c = x*P^-1 + z*P^-1 = 1100000011001100
   sindrome: 2+3x;  sigma = cx+fx^2 (multiplo escalar del localizador)
   raices: [0, 10] -> z' recuperado = 1000000000100000  [OK]
   u*S = 11001100;  u = (u*S)*S^-1 = 01001101  -> mensaje recuperado  [OK]

======================================================================
8. KEM del juguete (capitulo 11)
   delta = 5a repetido 32 veces
   s = 0111 0101 1101 0011 = cbae  (bytes AE CB, little-endian)
   e = 0000000000100100 (posiciones 10 y 13)
   C = e_alto + T*(e_bajo) = col2(T) + col5(T) = 36 ^ AE = 98 = 00011001
   K  = SHAKE256(01|00 24|98) = 597f46fbba325c68f0b0b45c6ea984961461eb220d1f24024ed888912b336278
   decap: v = (C, 0); S(v) = 4+9x; sigma = 5+ex+2x^2
   raices [10, 13] -> e correcto;  K' = K  [OK]
   C~ = C+1 = 10011001: S~ = b+6x, sigma~ = 5+8x+ax^2, sin raices
   K'' = SHAKE256(00|AE CB|99) = ba1a72ac26b0fcf20c55c108bb681e27aa9c270c60802a821601130b7f1e1494  (rechazo
     )

======================================================================
TODAS LAS COMPROBACIONES OK
```

A.4 Simplificaciones del juguete

El juguete no implementa la especificación real: la ilustra. Sus diferencias con Classic McEliece, todas deliberadas:

- **El error es fijo.** FixedWeight devuelve siempre el error de soporte $\{10, 13\}$: alcanza para verificar la tubería, pero la especificación lo sortea uniformemente entre las $\binom{n}{t}$ posibilidades (proposición 10.1).
- **No hay semillas, ni reintentos, ni variantes.** El soporte, g y σ están fijados a mano: no aparecen σ_1 y σ_2, ni la forma semi-sistemática $(\mu, \nu) = (32, 64)$, ni la red de Benes que comprime α en la clave secreta (todo el capítulo 9); tampoco las variantes `f`, `pc` y `pcf` (capítulo 12).
- **La seguridad es nula.** Con $n = 16$, enumerar las 2^{16} palabras es un parpadeo: el juguete verifica la corrección, nunca la dificultad. El problema «duro» empieza en los capítulos 14 y 15.
- **Las etiquetas del hash sí son las de la especificación**: `0x40` para el generador $G(\delta)$, `0x01` y `0x00` para las dos ramas de la clave de sesión (capítulo 11), con la convención de bits LSB-primero y bytes little-endian de la sección 6.2 de [10].

Para las cuentas a mano, el apéndice C tiene las tablas de $\mathbb{F}_{2^4}$ en formato de bolsillo.

CAPÍTULO B

Cronología: de Goppa (1970) a octubre de 2026

En este capítulo

Medio siglo de fechas, en orden: los cimientos (códigos de Goppa, decodificación, el criptosistema de 1978), las seis décadas de criptoanálisis clásico y cuántico, los procesos de estandarización y, con más detalle, el año que cambió el tono (2025–2026). Las referencias apuntan a la bibliografía; los hitos de 2026 citan los trabajos originales, no las crónicas.

fecha	hito
Fundaciones (1970–1991)	
1970	Valeri Goppa introduce los códigos algebraico-geométricos sobre cuerpos finitos que hoy llevan su nombre [40].
1975	Patterson publica el algoritmo de decodificación que hace utilizables los códigos de Goppa [60].
1978	Robert J. McEliece propone el primer criptosistema de clave pública basado en códigos [56]; Berlekamp, McEliece y van Tilborg demuestran que la decodificación por síndromes es NP-completa [6].
1986	Niederreiter reformula el esquema en forma dual, con vectores de error de peso fijo como mensajes [58].
1988	Primeros ataques por conjuntos de información: Lee–Brickell y Leon toleran unos pocos errores en las k columnas elegidas [49, 50].
1989	Stern convierte el ataque en un cruce de listas (*meet-in-the-middle*) [70].
1991	Dumer reutiliza y optimiza las listas de Stern [31].
La amenaza cuántica y el criptoanálisis clásico (1994–2015)	
1994	Shor: factorización y logaritmo discreto en tiempo polinómico en un ordenador cuántico [68]; nace la agenda post-cuántica.
1996	Grover: búsqueda cuántica con aceleración cuadrática [41].
1997	Berson: ataques de reacción contra McEliece [13].
1998	Canteaut y Sendrier: el ataque por división del soporte [18].
2000	Sendrier consolida la división del soporte como la segunda gran familia de criptoanálisis estructural [67].
2001	Shoup: ataques de temporización contra la decodificación [69].
2009	Overbeck y Sendrier publican el panorama del criptoanálisis basado en códigos [59].
2010	Bernstein: «Grover vs. McEliece»; los costes de puerta limitan la ventaja cuántica incluso en el peor caso [7].
2011	May–Meurer–Thomae [54] y Bernstein–Lange–Peters (colisión de bolas) [11]: representaciones y radios óptimos.
2012	Becker–Joux–May–Meurer refinan las representaciones [5].
2013	Faugère et al.: primer distinguidor de claves de Goppa, en el régimen de tasa alta [33].

fecha	hito
2015	May–Ozerov: la búsqueda de vecinos cercanos que fija los mejores números del estimador [55].
Estandarización (2016–2024)	
2016	El NIST convoca su proceso de estandarización post-cuántica.
2017	Classic McEliece se presenta al concurso (llegará a la ronda final de KEMs) [10]; Kachigar–Tillich: la decodificación por síndromes en el modelo cuántico [46].
2018	Both–May mejoran las representaciones [15]; Bernstein–Persichetti prueban la conversión a KEM en el modelo de oráculo aleatorio [12].
2019	Bindel et al.: cotas más finas de seguridad CCA en el modelo de oráculo aleatorio cuántico [14].
2020	Classic McEliece pasa a la cuarta ronda del NIST, junto a BIKE, HQC y SIKE.
2022	El NIST selecciona Kyber (con Dilithium, Falcon y SPHINCS+) y cierra la ronda 3; SIKE cae ante el ataque de Castryck–Decru; Esser–Bellini publican el estimador de ISD que sustentará las tablas del equipo [32]; Carrier et al. y el ataque de fallos contra NTRU [19, 8]; el equipo documenta los cambios de la ronda 4 [25].
2023	ISO/IEC abre la enmienda con Classic McEliece: borrador del 19 de abril [24].
2024	El NIST finaliza FIPS 203 (ML-KEM), 204 (ML-DSA) y 205 (SLH-DSA) [34], y publica el plan de transición IR 8547 [57].
2025: la primera sacudida	
17 abr 2025	El equipo responde a la afirmación de un distinguidor de `mceliece348864` «de solo 2^{529} operaciones» [20].
mar 2025	El NIST cierra su proceso de KEMs con HQC; Classic McEliece queda fuera de la selección [2].
20 nov 2025	El equipo se pronuncia sobre «You Only Decapsulate Once» [22].
2026: el año del terremoto	
3 mar 2026	Bardet et al.: ataques estructurales sobre CFS y los retos TII [4].
jun 2026	ISO/IEC publica la Enmienda 2 a la 18033-2 con Classic McEliece: cuatro tamaños, cuatro variantes [45].
10 jun 2026	Briaud et al. (BLRT): un nuevo distinguidor algebraico [16].
23 jun 2026	El equipo responde a la especulación de una recuperación de clave «de 2^{610} operaciones» [21].
29 jun 2026	Hemmert–Wiemers: distinguidor por anulación de orden superior (CRYPTO 2026) [43, 42].
7 ago 2026	Ghoshal–Ishai–Jain–Sun (GIJS): el test *holdout*, con préstamos de la teoría de la información privada [38].
20 ago 2026	Vedenev extiende el método a la recuperación de clave; resuelve retos TII [72].
21 ago 2026	El equipo publica su réplica a los nuevos métodos [9].
24 ago 2026	Saarinen: contabilidad de costes del *holdout* y desafíos resueltos (tres anclas; TII-254 con GPU dedicada durante 27,2 horas) [64, 65, 66].
26–28 ago 2026	Apon et al. publican una cota inferior (2^{1500} operaciones de bits) para el paso crítico del método de GIJS; Vedenev responde con una ruta alternativa [3, 72].
27 ago 2026	GIJS añaden a su artículo el método de recuperación de clave aplicado a los conjuntos reales [38].
11 sep 2026	Weis: el *holdout* aplicado directamente; el equipo estimará después que las cuentas de una sola ejecución quedan en 2^{99}–2^{102} [73].
15 sep 2026	Saarinen revisa sus cifras a $2^{126,8}$–$2^{145,2}$ [65].

fecha	hito
30 sep 2026	Bernstein defiende la contabilidad clásica del equipo [9].
1 oct 2026	El BSI publica su postura, fechada el 29 de septiembre: los métodos de agosto–octubre merecen tomarse en serio; «no debería seguir usándose en nuevos desarrollos» [17].
7 oct 2026	Vedenev publica la revisión de su ataque [72].
16 oct 2026	Cierre de estos apuntes.

Observación B.1: Cómo leer una cronología en movimiento

Tres convenciones, para no confundir categorías: (1) las fechas de 2026 son las de las *versiones públicas* de los trabajos en el ePrint del IACR, no necesariamente las de su concepción; (2) «resuelve el reto TII-x» describe desafíos de instancias de laboratorio, no la rotura de ningún conjunto de Classic McEliece [4, 65]; (3) las cifras de coste de esta cronología son las que cada autor reclama, con su contabilidad: el debate del capítulo 16 precisamente es sobre la contabilidad.

CAPÍTULO C

GF(16) de bolsillo

El cuerpo $\mathbb{F}_{2^4} = \mathbb{F}_2[z]/(z^4 + z + 1)$ es el banco de pruebas de todo el libro (ejemplo 3.4 y los laboratorios de los capítulos 5 y 11). Este apéndice reúne, para consulta rápida, las tablas que los ejercicios y las cuentas a mano necesitan: potencias, logaritmos, inversos, raíces cuadradas, traza y la tabla de multiplicar completa. Convenio: cada elemento se escribe como dígito hexadecimal con el bit i-ésimo igual al coeficiente de z^i (en particular $z = 2$ y $z^4 = z + 1$); los dígitos hex van en minúsculas, como en el programa del apéndice A.

C.1 Potencias y logaritmos

i	hex	polinomio	i	hex	polinomio
0	1	1	8	5	$1 + z^2$
1	2	z	9	a	$z + z^3$
2	4	z^2	10	7	$1 + z + z^2$
3	8	z^3	11	e	$z + z^2 + z^3$
4	3	$1 + z$	12	f	$1 + z + z^2 + z^3$
5	6	$z + z^2$	13	d	$1 + z^2 + z^3$
6	c	$z^2 + z^3$	14	9	$1 + z^3$
7	b	$1 + z + z^3$			

Cuadro C.1: Las potencias z^i, $i = 0, \ldots, 14$: el valor de z^i es el dígito hex y su expresión polinómica, la de al lado. El cuerpo tiene 15 elementos no nulos: z es primitivo y su ciclo los agota todos (figura 3.1).

a	$\log_z a$	a	$\log_z a$
1	0	9	e
2	1	a	9
3	4	b	7
4	2	c	6
5	8	d	d
6	5	e	b
7	a	f	c
8	3	0	—

Cuadro C.2: Logaritmos en base z (los exponentes, también en hex: por ejemplo $\log_z 6 = 5$, pues $z^5 = z + z^2$, y el 0 no tiene logaritmo). Para multiplicar: $ab = z^{\log a + \log b}$ con el exponente módulo 15.

C.2 Inversos, cuadrados, raíces y traza

a	a^{-1}	a	a^{-1}
1	1	9	2
2	9	a	c
3	e	b	5
4	d	c	a
5	b	d	4
6	7	e	3
7	6	f	8
8	f	0	—

Cuadro C.3: Inversos multiplicativos. Comprobación rápida: $3 \cdot e = z^4 \cdot z^{11} = z^{15} = 1$.

x	x^2	$\sqrt{x}$	$\mathrm{Tr}(x)$	x	x^2	$\sqrt{x}$	$\mathrm{Tr}(x)$
0	0	0	0	8	c	a	1
1	1	1	0	9	d	b	1
2	4	5	0	a	8	f	1
3	5	4	0	b	9	e	1
4	3	2	0	c	f	8	1
5	2	3	0	d	e	9	1
6	7	7	0	e	b	d	1
7	6	6	0	f	a	c	1

Cuadro C.4: Cuadrados, raíces cuadradas ($\sqrt{x} = x^8$, la mitad de 2^{m-1} porque la característica es 2) y traza absoluta ($\mathrm{Tr}(x) = x + x^2 + x^4 + x^8 \in \mathbb{F}_2$). Exactamente 8 elementos tienen traza 1: el núcleo de Tr es un subespacio de dimensión 3.

C.3 La tabla de multiplicar

C.4 Cómo usar estas tablas

- **Sumar** es XOR sin acarreo, bit a bit: $6 \oplus 5 = 3$, es decir $(z + z^2) + (1 + z^2) = 1 + z$.
- **Multiplicar** pasa por logaritmos: $a\,b = z^{\log_z a + \log_z b \pmod{15}}$. Ejemplo: $6 \cdot 5 = z^5 z^8 = z^{13} = d$; la versión polinómica es $(z + z^2)(1 + z^2) = 1 + z^2 + z^3$.
- **Invertir** es cambiar de signo el logaritmo. Con eso, la división es una resta de exponentes, y los $1/g(\alpha_i)$ del capítulo 5 se calculan sin escribir una sola división polinómica.
- **La raíz cuadrada** no necesita algoritmos iterativos: en característica 2, $\sqrt{x} = x^{2^{m-1}} = x^8$; Patterson la usa constantemente (capítulo 4).
- **La traza** decide irreducibilidad y pertenencia a subcuerpos; en el juguete, $\mathrm{Tr}(\beta) = \mathrm{Tr}(\beta^2)$ para todo β (ejercicio 4).

·	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
2	0	2	4	6	8	a	c	e	3	1	7	5	b	9	f	d
3	0	3	6	5	c	f	a	9	b	8	d	e	7	4	1	2
4	0	4	8	c	3	7	b	f	6	2	e	a	5	1	d	9
5	0	5	a	f	7	2	d	8	e	b	4	1	9	c	3	6
6	0	6	c	a	b	d	7	1	5	3	9	f	e	8	2	4
7	0	7	e	9	f	8	1	6	d	a	3	4	2	5	c	b
8	0	8	3	b	6	e	5	d	c	4	f	7	a	2	9	1
9	0	9	1	8	2	b	3	a	4	d	5	c	6	f	7	e
a	0	a	7	d	e	4	9	3	f	5	8	2	1	b	6	c
b	0	b	5	e	a	1	f	4	7	c	2	9	d	6	8	3
c	0	c	b	7	5	9	e	2	a	6	1	d	f	3	4	8
d	0	d	9	4	1	c	8	5	2	f	b	6	3	e	a	7
e	0	e	f	1	d	3	2	c	9	7	6	8	4	a	5	b
f	0	f	d	2	9	6	b	1	e	c	3	8	a	7	5	4

Cuadro C.5: Producto $a \cdot b$ en $\mathbb{F}_{2^4}$, con a en la fila y b en la columna (como en todas las tablas de este apéndice). Cada fila no nula es una permutación de $\{1, \ldots, \mathtt{f}\}$: la tabla lo muestra y la estructura de cuerpo lo garantiza.

- **Verificación cruzada.** El programa del apéndice A construye estas mismas tablas en una función de dos líneas y las usa para comprobar, número a número, todas las cuentas de los capítulos 5 y 11.

Bibliografía

[1] C. Aguilar-Melchor, N. Aragon, S. Bettaieb, L. Bidoux, O. Blazy, et al. *Hamming Quasi-Cyclic (HQC): specification*, submission to the NIST post-quantum cryptography standardization process, round 4, 2023.

[2] G. Alagic et al. *Status report on the fourth round of the NIST post-quantum cryptography standardization process*, NIST Internal Report 8545, March 2025.

[3] D. Apon. An algebraic-geometry lower bound against the ePrint 2026/1747 McEliece key-recovery attack, Cryptology ePrint Archive, Report 2026/1810, 2026.

[4] M. Bardet, A. Lemoine, J.-P. Tillich. An attack on the CFS scheme and on TII McEliece challenges. In *Advances in Cryptology — CRYPTO 2026*, Springer, 2026.

[5] A. Becker, A. Joux, A. May, A. Meurer. Decoding random binary linear codes in $2^{n/20}$: How $1 + 1 = 0$ improves information set decoding. In *Advances in Cryptology — EUROCRYPT 2012*, LNCS 7237, pp. 520–536, Springer, 2012.

[6] E. R. Berlekamp, R. J. McEliece, H. C. A. van Tilborg. On the inherent intractability of certain coding problems. *IEEE Transactions on Information Theory* 24(3):384–386, 1978.

[7] D. J. Bernstein. Grover vs. McEliece. In *Post-Quantum Cryptography — PQCrypto 2010*, LNCS 6061, pp. 73–80, Springer, 2010.

[8] D. J. Bernstein. A one-time single-bit fault leaks all previous NTRU-HRSS session keys to a chosen-ciphertext attack, Cryptology ePrint Archive, Report 2022/1125, 2022.

[9] D. J. Bernstein et al. Posturas y respuestas del equipo Classic McEliece, lista de correo del *pqc-forum* del NIST, agosto–octubre de 2026, https://groups.google.com/a/list.nist.gov/g/pqc-forum.

[10] D. J. Bernstein, T. Chou, T. Lange, et al. *Classic McEliece: conservative code-based cryptography* (specification), 23 de octubre de 2022, https://classic.mceliece.org/mceliece-spec-20221023.pdf.

[11] D. J. Bernstein, T. Lange, C. Peters. Smaller decoding exponents: ball-collision decoding. In *Advances in Cryptology — CRYPTO 2011*, LNCS 6841, pp. 743–760, Springer, 2011.

[12] D. J. Bernstein, E. Persichetti. Towards KEM unification, Cryptology ePrint Archive, Report 2018/526, 2018.

[13] T. A. Berson. Failure of the McEliece public-key cryptosystem under message-resend and related-message attack. In *Advances in Cryptology — CRYPTO '97*, LNCS 1294, pp. 213–220, Springer, 1997.

[14] N. Bindel, M. Hamburg, K. Hövelmanns, A. Hülsing, E. Persichetti. Tighter proofs of CCA security in the quantum random oracle model. In *Theory of Cryptography — TCC 2019*, LNCS 11892, pp. 61–90, Springer, 2019.

[15] L. Both, A. May. Decoding linear codes with high error rate and its impact for LPN security. In *Post-Quantum Cryptography — PQCrypto 2018*, LNCS 10786, pp. 25–46, Springer, 2018.

[16] P. Briaud, A. Lemoine, H. Randriambololona, J.-P. Tillich. A heuristic subexponential attack on the McEliece cryptosystem. In *Advances in Cryptology — ASIACRYPT 2026*, Springer, 2026.

[17] Bundesamt für Sicherheit in der Informationstechnik. *Notes on recent developments concerning Classic McEliece*, 29 de septiembre de 2026, https://www.bsi.bund.de.

[18] A. Canteaut, N. Sendrier. Cryptanalysis of the original McEliece cryptosystem. In *Advances in Cryptology — ASIACRYPT '98*, LNCS 1514, pp. 187–199, Springer, 1998.

[19] K. Carrier, T. Debris-Alazard, C. Meyer-Hilfiger, J.-P. Tillich. Statistical decoding 2.0: reducing decoding to LPN, arXiv:2208.02201, 2022.

[20] Classic McEliece Team. *Notes on a recent claim that a* `mceliece348864` *distinguisher uses only* 2^{529} *operations*, 17 de abril de 2025, https://classic.mceliece.org/mceliece-529-20250417.pdf.

[21] Classic McEliece Team. *Notes on recent speculation that a* `mceliece348864` *key-recovery attack uses only* 2^{610} *operations*, 23 de junio de 2026, https://classic.mceliece.org/mceliece-610-20260623.pdf.

[22] Classic McEliece Team. *Statement regarding the paper "You Only Decapsulate Once"*, 20 de noviembre de 2025, https://classic.mceliece.org/mceliece-20251120.txt.

[23] Classic McEliece Team. *Classic McEliece: conservative code-based cryptography: guide for implementors*, 23 de octubre de 2022, https://classic.mceliece.org/mceliece-impl-20221023.pdf.

[24] Classic McEliece Team. Página de estandarización ISO, https://classic.mceliece.org/iso.html, consultada en octubre de 2026.

[25] Classic McEliece Team. *Classic McEliece: conservative code-based cryptography: description of modifications from round 3 to round 4*, 23 de octubre de 2022, https://classic.mceliece.org/mceliece-mods3-20221023.pdf.

[26] Classic McEliece Team. Página de estandarización NIST, https://classic.mceliece.org/nist.html, consultada en octubre de 2026.

[27] Classic McEliece Team. *What plaintext confirmation means*, 23 de octubre de 2022, https://classic.mceliece.org/mceliece-pc-20221023.pdf.

[28] Classic McEliece Team. *Classic McEliece: conservative code-based cryptography: design rationale*, 23 de octubre de 2022, https://classic.mceliece.org/mceliece-rationale-20221023.pdf.

[29] Classic McEliece Team. *Classic McEliece: conservative code-based cryptography: guide for security reviewers*, 23 de octubre de 2022, https://classic.mceliece.org/mceliece-security-20221023.pdf.

[30] A. Couvreur, A. Otmani, J.-P. Tillich. Polynomial time attack on wild McEliece over quadratic extensions. *IEEE Transactions on Information Theory* 63(1):404–427, 2017.

[31] I. Dumer. On minimum distance decoding of linear codes. En *Proc. 5th Joint Soviet-Swedish International Workshop on Information Theory*, pp. 50–52, 1991.

[32] A. Esser, E. Bellini. Syndrome decoding estimator. In *Public-Key Cryptography — PKC 2022*, LNCS 13177, pp. 112–141, Springer, 2022.

[33] J.-C. Faugère, V. Gauthier-Umaña, A. Otmani, L. Perret, J.-P. Tillich. A distinguisher for high-rate McEliece cryptosystems. *IEEE Transactions on Information Theory* 59(10):6830–6844, 2013.

[34] National Institute of Standards and Technology. *Module-lattice-based key-encapsulation mechanism standard*, FIPS 203, agosto de 2024.

[35] National Institute of Standards and Technology. *Module-lattice-based digital signature standard*, FIPS 204, agosto de 2024.

[36] National Institute of Standards and Technology. *Stateless hash-based digital signature standard*, FIPS 205, agosto de 2024.

[37] E. Fujisaki, T. Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In *Advances in Cryptology — CRYPTO '99*, LNCS 1666, pp. 537–554, Springer, 1999.

[38] A. Ghoshal, Y. Ishai, A. Jain, N. Sun. Quasipolynomial cryptanalysis of the McEliece cryptosystem (or: PIR meets McEliece), Cryptology ePrint Archive, Report 2026/1630, 2026.

[39] C. Gidney, M. Ekerå. How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits. *Quantum* 5:433, 2021.

[40] V. D. Goppa. A new class of linear correcting codes. *Problemy Peredachi Informatsii* 6(3):24–30, 1970.

[41] L. K. Grover. A fast quantum mechanical algorithm for database search. En *Proc. 28th ACM Symposium on Theory of Computing (STOC)*, pp. 212–219, 1996.

[42] T. Hemmert. Key recovery for the McEliece cryptosystem using higher-order vanishing, Cryptology ePrint Archive, Report 2026/1339, 2026.

[43] T. Hemmert, A. Wiemers. Distinguishing Goppa codes using higher-order vanishing. In *Advances in Cryptology — CRYPTO 2026*, pp. 191–223, Springer, 2026.

[44] D. Hofheinz, K. Hövelmanns, E. Kiltz. A modular analysis of the Fujisaki–Okamoto transformation. In *Theory of Cryptography — TCC 2017*, LNCS 10677, pp. 341–371, Springer, 2017.

[45] ISO/IEC. *Information technology — Security techniques — Encryption algorithms — Part 2: Asymmetric ciphers — Amendment 2: Classic McEliece*, ISO/IEC 18033-2:2006/Amd 2:2026, junio de 2026.

[46] G. Kachigar, J.-P. Tillich. Quantum information set decoding algorithms. In *Post-Quantum Cryptography — PQCrypto 2017*, LNCS 10346, pp. 69–89, Springer, 2017.

[47] E. Kirshanova. Improved quantum information set decoding. In *Post-Quantum Cryptography — PQCrypto 2018*, LNCS 10786, pp. 507–527, Springer, 2018.

[48] E. Kirshanova, A. May. Decoding McEliece with a hint — secret Goppa key parts reveal everything. In *Security and Cryptography for Networks — SCN 2022*, LNCS 13409, Springer, 2022.

[49] P. J. Lee, E. F. Brickell. An observation on the security of McEliece's public-key cryptosystem. In *Advances in Cryptology — EUROCRYPT '88*, LNCS 330, pp. 275–280, Springer, 1988.

[50] J. S. Leon. A probabilistic algorithm for computing minimum weights of large error-correcting codes. *IEEE Transactions on Information Theory* 34(5):1354–1359, 1988.

[51] Y. X. Li, R. H. Deng, X. M. Wang. On the equivalence of McEliece's and Niederreiter's public-key cryptosystems. *IEEE Transactions on Information Theory* 40(1):271–273, 1994.

[52] R. Lidl, H. Niederreiter. *Finite Fields*, Encyclopedia of Mathematics and its Applications 20, Cambridge University Press, 2.ª edición, 1997.

[53] F. J. MacWilliams, N. J. A. Sloane. *The Theory of Error-Correcting Codes*, North-Holland, 1977.

[54] A. May, A. Meurer, E. Thomae. Decoding random linear codes in $\tilde{\mathcal{O}}(2^{0.054n})$. In *Advances in Cryptology — ASIACRYPT 2011*, LNCS 7073, pp. 107–124, Springer, 2011.

[55] A. May, I. Ozerov. On computing nearest neighbors with applications to decoding of binary linear codes. In *Advances in Cryptology — EUROCRYPT 2015*, LNCS 9056, pp. 203–228, Springer, 2015.

[56] R. J. McEliece. A public-key cryptosystem based on algebraic coding theory. *DSN Progress Report 42-44*, Jet Propulsion Laboratory, pp. 114–116, 1978.

[57] D. Moody et al. *Transition to post-quantum cryptography standards*, NIST Internal Report 8547 (borrador público inicial), noviembre de 2024.

[58] H. Niederreiter. Knapsack-type cryptosystems and algebraic coding theory. *Problems of Control and Information Theory* 15(2):159–166, 1986.

[59] R. Overbeck, N. Sendrier. Code-based cryptography. En *Post-Quantum Cryptography* (D. J. Bernstein, J. Buchmann, E. Dahmen, eds.), pp. 95–145, Springer, 2009.

[60] N. J. Patterson. The algebraic decoding of Goppa codes. *IEEE Transactions on Information Theory* 21(2):203–207, 1975.

[61] E. Persichetti. *Improving the efficiency of code-based cryptography*, tesis doctoral, University of Auckland, 2012.

[62] E. Prange. The use of information sets in decoding cyclic codes. *IRE Transactions on Information Theory* 8(5):5–9, 1962.

[63] H. Randriambololona. The syzygy distinguisher. In *Advances in Cryptology — EUROCRYPT 2025*, LNCS 15606, Springer, 2025 (revisión ampliada: Cryptology ePrint Archive, Report 2024/1193).

[64] M.-J. O. Saarinen. Bit operation cost of "holdout" key-recovery attacks against Classic McEliece, Cryptology ePrint Archive, Report 2026/1786, 2026.

[65] M.-J. O. Saarinen. Two-anchor holdout/Hermite: solving the TII-254 McEliece key recovery challenge, Cryptology ePrint Archive, Report 2026/1986, 2026.

[66] M.-J. O. Saarinen. HOVER: Higher-order vanishing endomorphism recovery, Cryptology ePrint Archive, Report 2026/1778, 2026 (NORDSEC 2026, LNCS, en prensa).

[67] N. Sendrier. Finding the permutation between equivalent linear codes: the support splitting algorithm. *IEEE Transactions on Information Theory* 46(4):1193–1203, 2000.

[68] P. W. Shor. Algorithms for quantum computation: discrete logarithms and factoring. En *Proc. 35th IEEE Symposium on Foundations of Computer Science (FOCS)*, pp. 124–134, 1994.

[69] V. Shoup. A proposal for an ISO standard for public key encryption, Cryptology ePrint Archive, Report 2001/112, 2001.

[70] J. Stern. A method for finding codewords of small weight. En *Coding Theory and Applications*, LNCS 388, pp. 106–113, Springer, 1989.

[71] J. H. van Lint. *Introduction to Coding Theory*, Graduate Texts in Mathematics 86, Springer, 3.ª edición, 1999.

[72] K. Vedenev. Extending distinguishing to key recovery for subfield subcodes of GRS codes, Cryptology ePrint Archive, Report 2026/1747, 2026.

[73] S. A. Weis. Improving GIJS key recovery for Classic McEliece, Cryptology ePrint Archive, Report 2026/1984, 2026.